

Trabajo fin de grado

Herramientas de visualización de eventos en
excavaciones arqueológicas



Javier San Andrés de Pedro

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Herramientas de visualización de eventos en
excavaciones arqueológicas**

Autor: Javier San Andrés de Pedro

Tutor: Sacha Gómez Moñivas

junio 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© 10 de Mayo de 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

Javier San Andrés de Pedro
Herramientas de visualización de eventos en
excavaciones arqueológicas

Javier San Andrés de Pedro
C\ Francisco Tomás y Valiente Nº 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

AGRADECIMIENTOS

La verdad que este trabajo ha sido muy difícil de llevar a cabo. En ocasiones, llegué a pensar que no sería capaz de realizarlo, porque veía que la complejidad del proyecto era demasiado alta y no tenía los conocimientos necesarios para llevarlo a cabo. Sin embargo, gracias a mi fuerza de voluntad y al apoyo de mi familia y amigos, que siempre confiaron en mí, he conseguido superar los obstáculos y completar este proyecto. Por ello, quiero agradecer a todas las personas que me han dado ánimos, porque gracias a ellos he podido seguir adelante incluso en los momentos más difíciles.

También quiero agradecer al cliente, Dr. Cristo Manuel Hernández Gómez, por mostrarse abierto y receptivo a responder mis preguntas y por darme la oportunidad de trabajar en un proyecto tan interesante y desafiante. Por último, quiero agradecer a mi tutor, porque ha sido de gran ayuda a la hora de gestionar el proyecto, facilitando la comunicación con el cliente y siendo flexible a la hora de adaptar el proyecto a mis necesidades y circunstancias personales.

RESUMEN

Este trabajo presenta el desarrollo de una aplicación de escritorio para la creación y visualización de esquemas Matrix Harris en el contexto de excavaciones arqueológicas. Para abordar este proyecto, primero ha sido necesario traducirlo en términos informáticos, lo que se ha conseguido mediante la concepción de los esquemas Matrix Harris como hipergrafos jerárquicos con enrutamiento ortogonal.

Una vez identificado el problema, esta memoria supone un recorrido detallado por el proceso de desarrollo de la aplicación, desde el análisis del problema y el diseño de la solución, hasta la implementación y validación de la misma.

En este sentido, este documento presenta una naturaleza dual. Por una parte, atiende todos los aspectos relacionados con el desarrollo de una aplicación software, incluyendo un análisis de tecnologías similares y de los requerimientos del sistema; así como el diseño de la arquitectura y el ciclo de desarrollo.

Por otra parte, que de hecho supone el eje central, se centra en el problema a resolver desde el punto de vista de la Teoría de Grafos Computacional. En este ejercicio, se han explorado algoritmos de minimización de cruces en DAGs k -estratificados, algoritmos de asignación de coordenadas horizontales y algoritmos de enrutamiento ortogonal, entre otros. La solución se apoya en heurísticas y métodos clásicos adaptados al contexto interactivo de la aplicación, con el objetivo de mantener la coherencia visual del esquema durante las actualizaciones realizadas por el usuario.

El resultado es una herramienta modular y válida para construir y editar esquemas Matrix Harris con claridad, legibilidad y posibilidad de ampliación futura.

PALABRAS CLAVE

Matrix Harris, hipergrafos jerárquicos, DAG, representación ortogonal, visualización de información, diagramas de Hasse, Teoría de Grafos Computacional

ABSTRACT

This thesis presents the development of a desktop application for the creation and visualisation of Harris Matrix diagrams in the context of archaeological excavations. To tackle this project, it was first necessary to translate it into computer science terms, which was achieved by conceptualising Harris Matrix diagrams as hierarchical hypergraphs with orthogonal routing.

Having identified the problem, this thesis provides a detailed account of the application's development process, from problem analysis and solution design to implementation and validation.

In this sense, this document has a dual nature. On the one hand, it addresses all aspects related to the development of a software application, including an analysis of similar technologies and system requirements, as well as the design of the architecture and the development cycle.

On the other hand, which in fact constitutes the central focus, it concentrates on the problem to be solved from the perspective of Computational Graph Theory. In this exercise, algorithms for minimising crossings in k -layered DAGs, algorithms for assigning horizontal coordinates, and orthogonal routing algorithms, amongst others have been explored. The solution relies on heuristics and classical methods adapted to the interactive context of the application, with the aim of maintaining the visual consistency of the diagram during updates made by the user.

The result is a modular tool which can be useful for creating and editing Harris Matrix diagrams with clarity and readability, whilst allowing for future expansion.

KEYWORDS

Matrix Harris, hierarchical hypergraphs, DAG, orthogonal representation, information visualisation, Hasse diagrams, Computational Graph Theory

ÍNDICE

1	Introducción	1
1.1	Marco Institucional	1
1.2	Problema a resolver	1
1.3	Necesidad del Sistema	3
1.4	Alcance	3
1.5	Estructura del documento	4
2	Estado del Arte	6
2.1	Marco teórico	6
2.1.1	Relaciones de orden parcial	6
2.1.2	Grafos, DAGs, Diagramas de Hasse e Hipergrafos	7
2.1.3	Hipergrafos jerárquicos con enrutamiento ortogonal	9
2.2	Magnitud del problema	10
2.2.1	Representación de árboles	10
2.2.2	Representación de DAGs	10
2.2.3	Representación de diagramas de Hasse	12
2.2.4	Representación de grafos	12
2.2.5	Representación de esquemas de Matrix Harris	12
2.3	Análisis comparativo de la competencia	13
2.4	Análisis comparativo de las tecnologías	14
2.5	Entorno de desarrollo y de control de versiones	14
3	Análisis del Sistema	15
4	Diseño del Sistema	16
4.1	Maquetas de la interfaz de usuario	16
4.2	Arquitectura del sistema	16
5	Desarrollo	18
5.1	Entorno de Desarrollo y Herramientas	18
5.2	Ciclo de vida del desarrollo	18
6	Implementación: Representación Ortogonal de Hipergrafos Jerárquicos	20
6.1	La estructura de datos del Hipergrafo	21
6.2	Minimización de cruces entre hiperaristas	23
6.3	Asignación de coordenadas horizontales	26

6.4	Ordenación de hiperaristas	28
6.5	Asignación de puertos	29
6.6	Asignación de coordenadas verticales	32
6.7	Lógica transaccional: hacer y deshacer	34
7	Validación y Resultados	35
7.1	Testing	35
7.2	Pruebas de validación algorítmica y rendimiento	35
7.3	Encuesta de usabilidad	38
8	Conclusiones	40
	Bibliografía	43
	Apéndices	44
A	Estado del arte	46
A.1	Análisis comparativo de la competencia	46
A.2	Análisis comparativo de las Tecnologías	50
A.2.1	Librería gráfica y lenguaje de programación	50
A.2.2	Librerías de dibujo de grafos y <i>solvers</i> MIP	52
B	Análisis del sistema	54
B.1	Catálogo de Requisitos funcionales	54
B.2	Catálogo de Requisitos no funcionales	57
B.2.1	Requisitos de Rendimiento	58
B.2.2	Requisitos de Fiabilidad	58
B.2.3	Requisitos de Usabilidad	58
B.2.4	Requisitos de Portabilidad y Compatibilidad	59
B.2.5	Requisitos de Mantenibilidad	59
B.2.6	Requisitos de Persistencia	59
B.2.7	Requisitos de Documentación	59
B.3	Diagrama de casos de uso	60
B.4	Catálogo de casos de uso	60
B.4.1	CU-1: Crear nuevo proyecto	60
B.4.2	CU-2: Guardar proyecto	61
B.4.3	CU-3: Abrir proyecto	62
B.4.4	CU-4: Añadir diagrama	63
B.4.5	CU-5: Eliminar diagrama	64
B.4.6	CU-6: Renombrar diagrama	65
B.4.7	CU-7: Navegar entre diagramas	66

B.4.8 CU-8: Crear nodo raíz	66
B.4.9 CU-9: Crear nodo hijo	67
B.4.10 CU-10: Crear nodo padre	68
B.4.11 CU-11: Crear nodo dentro de una hiperarista	69
B.4.12 CU-12: Crear origen de una hiperarista	70
B.4.13 CU-13: Crear destino de una hiperarista	71
B.4.14 CU-14: Añadir un nuevo padre a nodo	72
B.4.15 CU-15: Añadir un nuevo hijo a nodo	73
B.4.16 CU-16: Añadir nodo origen a hiperarista	75
B.4.17 CU-17: Añadir nodo destino a hiperarista	76
B.4.18 CU-18: Eliminar nodo	77
B.4.19 CU-19: Eliminar conexión entre nodos	77
B.4.20 CU-20: Eliminar origen de una hiperarista	78
B.4.21 CU-21: Eliminar destino de una hiperarista	79
B.4.22 CU-22: Eliminar hiperarista completa	80
B.4.23 CU-23: Fusionar nodos	81
B.4.24 CU-24: Renombrar nodo	82
B.4.25 CU-25: Reordenar nodo en su capa	83
B.4.26 CU-26: Minimizar cruces manualmente	84
B.4.27 CU-27: Ajustar zoom del diagrama	85
B.4.28 CU-28: Arrastrarse a lo largo del diagrama	85
B.4.29 CU-29: Incorporar diagrama al diagrama conjunto	86
B.4.30 CU-30: Calcular layout	87
B.4.31 CU-31: Deshacer operación	89
B.4.32 CU-32: Rehacer operación	90
B.5 Catálogo de diagramas de secuencia	92
B.5.1 SD-01: Crear nuevo proyecto	92
B.5.2 SD-02: Guardar proyecto	93
B.5.3 SD-03: Abrir proyecto	94
B.5.4 SD-04: Añadir diagrama	95
B.5.5 SD-05: Eliminar diagrama	96
B.5.6 SD-06: Renombrar diagrama	97
B.5.7 SD-07: Navegar entre diagramas	98
B.5.8 SD-08: Crear nodo raíz	99
B.5.9 SD-09: Crear nodo hijo	100
B.5.10 SD-10: Crear nodo padre	101

B.5.11 SD-11: Crear nodo dentro de una hiperarista	102
B.5.12 SD-12: Crear origen de una hiperarista	103
B.5.13 SD-13: Crear destino de una hiperarista	104
B.5.14 SD-14: Añadir un nuevo padre a nodo	105
B.5.15 SD-15: Añadir un nuevo hijo a nodo	106
B.5.16 SD-16: Añadir nodo origen a hiperarista	107
B.5.17 SD-17: Añadir nodo destino a hiperarista	108
B.5.18 SD-18: Eliminar nodo	109
B.5.19 SD-19: Eliminar conexión entre nodos	110
B.5.20 SD-20: Eliminar origen de una hiperarista	111
B.5.21 SD-21: Eliminar destino de una hiperarista	112
B.5.22 SD-22: Eliminar hiperarista completa	113
B.5.23 SD-23: Fusionar nodos	114
B.5.24 SD-24: Renombrar nodo	115
B.5.25 SD-25: Reordenar nodo en su capa	116
B.5.26 SD-26: Minimizar cruces manualmente	117
B.5.27 SD-27: Ajustar zoom del diagrama	118
B.5.28 SD-28: Arrastrarse a lo largo del diagrama	119
B.5.29 SD-29: Incorporar diagrama al diagrama conjunto	120
B.5.30 SD-30: Calcular layout	121
B.5.31 SD-31: Deshacer operación	122
B.5.32 SD-32: Rehacer operación	123
C Diseño del sistema	124
C.1 Catálogo de Maquetas de la interfaz de usuario	124
C.2 Arquitectura y patrones de diseño	127
C.2.1 Diseño de la aplicación	127
D Representación ortogonal de hipergrafos jerárquicos	130
D.1 Nodos, Hiperaristas y el Hipergrafo	130
D.1.1 Nodo	130
D.1.2 Hiperarista	131
D.1.3 Hipergrafo	132
D.2 Minimización de cruces	157
D.3 Asignación de coordenadas horizontales	166
D.4 Asignación de puertos	169
D.4.1 Detección y resolución de conflictos de solapamiento	170
E Resultados y Validación	176

E.1	Desglose de pruebas unitarias	176
E.2	Pruebas de validación algorítmica y rendimiento	177
E.3	Encuesta de usabilidad	185

LISTAS

Lista de algoritmos

6.1	Subdivisión de hiperarista larga	22
6.2	Asignación de coordenadas verticales	33
D.1	Disolución de segmentos	136
D.2	Determinación de reubicaciones	137
D.3	Aplicación de reubicaciones y propagación	138
D.4	Detección de ancestro	140
D.5	Detección de ciclos	141
D.6	Eliminación de conexiones transitivas	143
D.7	Crear un nodo real con padre opcional	145
D.8	Crear un nodo que se inserte como padre de un nodo existente	145
D.9	Crear un nodo intercalado sobre una arista existente	146
D.10	Crear un nodo fuente o destino para una arista existente	147
D.11	Añadir una conexión binaria entre dos nodos existentes	148
D.12	Añadir una fuente o destino a una arista existente (Parte I)	150
D.13	Añadir una fuente o destino a una arista existente (Parte II)	151
D.14	Eliminar un nodo existente	152
D.15	Eliminar una conexión entre dos nodos existentes	153
D.16	Eliminar fuentes o destinos de una arista existente	155
D.17	Fusionar dos nodos existentes	156
D.18	Sifting Step	159
D.19	Ordenación de la capa inferior	161
D.20	Ordenación de los <i>hubs</i> del gap	161
D.21	Construcción del orden inicial de bloques	162
D.22	Búsqueda de posición óptima para bloques móviles	164
D.23	Posicionamiento de bloques móviles	165
D.24	Reducción de codos horizontales	171
D.25	Detección de posibles conflictos entre segmentos verticales	172
D.26	Resolución de conflictos entre puertos	174
E.1	Generación aleatoria de hipergrafo bicapa sin conexiones redundantes	184

Lista de códigos

D.1	Atributos de Node	130
D.2	Atributos de Hyperedge	131
D.3	Atributos de Hypergraph	132
D.4	addNodeToLayer	133
D.5	removeNodeFromLayer	134
D.6	getAllAncestors / getAllDescendants	134
D.7	createHyperedge	134
D.8	addHyperedgeToLayer	135
D.9	removeHyperedgeFromLayer	135
D.10	edgeIsShort	135

Lista de figuras

1.1	Ejemplo de esquema Matrix Harris	2
2.1	Representación de un DAG y su correspondiente diagrama de Hasse	8
2.2	Comparativo entre un DAG y un hipergrafo dirigido	9
2.3	Diferentes representaciones de un mismo DAG	11
2.4	Otro ejemplo más complejo de Matrix Harris	14
4.1	Diagrama de clases de la arquitectura general del sistema	17
6.1	Contar cruces correctamente en un hipergrafo	24
6.2	Transformar hipergrafo en un DAG para contar cruces	25
6.3	Segmentos interiores	25
6.4	Transformar hipergrafo en un DAG para asignar coordenadas horizontales	27
6.5	Ordenación de hiperarista afecta en los cruces	28
6.6	Puertos origen y puertos destino	30
6.7	Solapamientos entre segmentos verticales	30
6.8	Tipos de puertos	31
6.9	Solapamientos entre segmentos verticales de nodos distintos	32
7.1	Resultados de la implementación de Global Sifting	36
7.2	Resultados de la implementación de Brandes-Köpf	37
7.3	Implementación de ordenación de hiperaristas	38
B.1	Diagrama global de casos de uso de la aplicación Matrix Harris	60
B.2	Diagrama de secuencia del B.1: Crear nuevo proyecto.	92

B.3	Diagrama de secuencia del B.2: Guardar proyecto.	93
B.4	Diagrama de secuencia del B.3: Abrir proyecto.	94
B.5	Diagrama de secuencia del B.4: Añadir diagrama.	95
B.6	Diagrama de secuencia del B.5: Eliminar diagrama.	96
B.7	Diagrama de secuencia del B.6: Renombrar diagrama.	97
B.8	Diagrama de secuencia del B.7: Navegar entre diagramas.	98
B.9	Diagrama de secuencia del B.8: Crear nodo raíz.	99
B.10	Diagrama de secuencia del B.9: Crear nodo hijo.	100
B.11	Diagrama de secuencia del B.10: Crear nodo padre.	101
B.12	Diagrama de secuencia del B.11: Crear nodo dentro de una hiperarista.	102
B.13	Diagrama de secuencia del B.12: Crear origen de una hiperarista.	103
B.14	Diagrama de secuencia del B.13: Crear destino de una hiperarista.	104
B.15	Diagrama de secuencia del B.14: Añadir un nuevo padre a nodo.	105
B.16	Diagrama de secuencia del B.15: Añadir un nuevo hijo a nodo.	106
B.17	Diagrama de secuencia del B.16: Añadir nodo origen a hiperarista.	107
B.18	Diagrama de secuencia del B.17: Añadir nodo destino a hiperarista.	108
B.19	Diagrama de secuencia del B.18: Eliminar nodo.	109
B.20	Diagrama de secuencia del B.19: Eliminar conexión entre nodos.	110
B.21	Diagrama de secuencia del B.20: Eliminar origen de una hiperarista.	111
B.22	Diagrama de secuencia del B.21: Eliminar destino de una hiperarista.	112
B.23	Diagrama de secuencia del B.22: Eliminar hiperarista completa.	113
B.24	Diagrama de secuencia del B.23: Fusionar nodos.	114
B.25	Diagrama de secuencia del B.24: Renombrar nodo.	115
B.26	Diagrama de secuencia del B.25: Reordenar nodo en su capa.	116
B.27	Diagrama de secuencia del B.26: Minimizar cruces manualmente.	117
B.28	Diagrama de secuencia del B.27: Ajustar zoom del diagrama.	118
B.29	Diagrama de secuencia del B.28: Arrastrarse a lo largo del diagrama.	119
B.30	Diagrama de secuencia del B.29: Incorporar diagrama al diagrama conjunto.	120
B.31	Diagrama de secuencia del B.30: Calcular layout.	121
B.32	Diagrama de secuencia del B.31: Deshacer operación.	122
B.33	Diagrama de secuencia del B.32: Rehacer operación.	123
C.1	Maqueta de creación de nuevos nodos.	124
C.2	Maqueta de asignación de nombre a nodos nuevos.	125
C.3	Maqueta de menú de despliegue de un nodo.	125
C.4	Maqueta de menú de despliegue de una arista.	126
C.5	Capturas de pantalla de la reunión de validación con el cliente.	126
C.6	Diagrama de clases de la capa de dominio (<code>hypergraph_logic</code>).	128
C.7	Diagrama de clases de la capa de aplicación (editores y proyecto).	129

D.1	Ejemplo de hipergrafo donde BFS falla	139
D.2	DFS normal no basta para detectar ciclos	140
D.3	Ejemplo de eliminación de conexión transitiva	142
D.4	Bloques en <i>Global Sifting</i>	158
D.5	Inicialización pobre tras actualización	160
D.6	Posición óptima para nuevos nodos <i>dummy</i>	163
D.7	Representación del mismo grafo para los cuatro sesgos	168
D.8	Resolución de codos horizontales	169
D.9	Elección de proporción baricéntrica	175
E.1	Visualización de instancia aleatoria de prueba de Global Sifting	181
E.2	SUS ítem 1	187
E.3	SUS ítem 2	187
E.4	SUS ítem 3	188
E.5	SUS ítem 4	188
E.6	SUS ítem 5	188
E.7	SUS ítem 6	189
E.8	SUS ítem 7	189
E.9	SUS ítem 8	189
E.10	SUS ítem 9	190
E.11	SUS ítem 10	190
E.12	SUS ítem 11	190
E.13	SUS ítem 12	191
E.14	Ítem adicional 1	191
E.15	Ítem adicional 2	192
E.16	Ítem adicional 3	192
E.17	Ítem adicional 4	192
E.18	Ítem adicional 5	193

Lista de tablas

A.1	Análisis de diagrams.net (draw.io)	47
A.2	Análisis de Stratify	48
A.3	Análisis de Harris Matrix Composer (HMC)	49
A.4	Análisis de Gephi	50
B.1	Especificación del caso de uso CU-1: Crear nuevo proyecto	61
B.2	Especificación del caso de uso CU-2: Guardar proyecto	62
B.3	Especificación del caso de uso CU-3: Abrir proyecto	63

B.4	Especificación del caso de uso CU-4: Añadir diagrama	64
B.5	Especificación del caso de uso CU-5: Eliminar diagrama	65
B.6	Especificación del caso de uso CU-6: Renombrar diagrama	66
B.7	Especificación del caso de uso CU-7: Navegar entre diagramas	66
B.8	Especificación del caso de uso CU-8: Crear nodo raíz	67
B.9	Especificación del caso de uso CU-9: Crear nodo hijo	68
B.10	Especificación del caso de uso CU-10: Crear nodo padre	69
B.11	Especificación del caso de uso CU-11: Crear nodo dentro de una hiperarista	70
B.12	Especificación del caso de uso CU-12: Crear origen de una hiperarista	71
B.13	Especificación del caso de uso CU-13: Crear destino de una hiperarista	72
B.14	Especificación del caso de uso CU-14: Añadir un nuevo padre a nodo	73
B.15	Especificación del caso de uso CU-15: Añadir un nuevo hijo a nodo	74
B.16	Especificación del caso de uso CU-16: Añadir nodo origen a hiperarista	75
B.17	Especificación del caso de uso CU-17: Añadir nodo destino a hiperarista	76
B.18	Especificación del caso de uso CU-18: Eliminar nodo	77
B.19	Especificación del caso de uso CU-19: Eliminar conexión entre nodos	78
B.20	Especificación del caso de uso CU-20: Eliminar origen de una hiperarista	79
B.21	Especificación del caso de uso CU-21: Eliminar destino de una hiperarista	80
B.22	Especificación del caso de uso CU-22: Eliminar hiperarista completa	81
B.23	Especificación del caso de uso CU-23: Fusionar nodos	82
B.24	Especificación del caso de uso CU-24: Renombrar nodo	83
B.25	Especificación del caso de uso CU-25: Reordenar nodo en su capa	84
B.26	Especificación del caso de uso CU-26: Minimizar cruces manualmente	84
B.27	Especificación del caso de uso CU-27: Ajustar zoom del diagrama	85
B.28	Especificación del caso de uso CU-28: Arrastrarse a lo largo del diagrama	86
B.29	Especificación del caso de uso CU-29: Incorporar diagrama al diagrama conjunto	87
B.30	Especificación del caso de uso CU-30: Calcular layout	89
B.31	Especificación del caso de uso CU-31: Deshacer operación	90
B.32	Especificación del caso de uso CU-32: Rehacer operación	91
E.1	Pruebas unitarias de la capa de dominio (<code>hypergraph_logic</code>).	176
E.2	Pruebas unitarias de la capa de aplicación (<code>editors</code>).	177
E.3	Análisis detallado de la optimización de cruces (Todas las instancias)	177
E.4	Comparativa de resultados en ordenación de hiperaristas: Implementación MIP vs. Fuerza Bruta (BF)	181
E.5	Impresiones de los participantes tras la ejecución de la tarea.	186
E.6	Respuesta a la pregunta de aspecto más útil o destacable.	193
E.7	Sugerencias de mejora	194

INTRODUCCIÓN

En este capítulo, se relatará el contexto en el que se sitúa este proyecto, así como los objetivos que se pretenden alcanzar con el mismo. Entre otras cosas, se comentará el problema a resolver, y se expondrá la necesidad de llevar a cabo este proyecto. Finalmente, se describirá la estructura del documento, para que el lector pueda orientarse a lo largo de la lectura del mismo.

1.1. Marco Institucional

Este Trabajo de Fin de Grado se desarrolla como un proyecto de colaboración entre la Fundación de la Universidad Autónoma de Madrid y el Instituto Universitario de Bio-Organica Antonio González. En concreto, con el grupo de investigación de PAST (Prehistoria y Arqueología del Espacio y el Tiempo), uno de cuyos miembros es el Dr. Cristo Manuel Hernández Gómez, principal cliente de este proyecto. El objetivo de esta colaboración es desarrollar un software que permita automatizar la generación de esquemas Matrix Harris, que son una herramienta empleada para describir la sucesión temporal de los contextos arqueológicos. A este efecto, se desarrolla la aplicación expuesta en este documento.

1.2. Problema a resolver

Para entender el problema, primero es necesario hablar en relativo detalle de lo que es un esquema Matrix Harris. El esquema Matrix Harris es un método de registro estratigráfico desarrollado por el arqueólogo Edward Cecil Harris en 1973 y publicado de forma sistemática en su obra *Principles of Archaeological Stratigraphy* (1979). Se trata de una representación gráfica que permite organizar y visualizar las relaciones temporales y deposicionales entre las distintas unidades estratigráficas identificadas durante una excavación arqueológica.

Más allá de la terminología pomposa y de la aparente complejidad, el razonamiento detrás de estos esquemas es simple. Es bien sabido que, a medida que pasa el tiempo, los materiales se van depositando unos sobre otros, formando capas o estratos. Estas capas pueden ser de diferentes tipos (sedimentarias, culturales, naturales, etc.) y pueden contener restos arqueológicos que nos permiten

entender la historia de un sitio. Cuando se excava un sitio arqueológico, si encontramos un sedimento por encima de otro, podemos inferir que el sedimento inferior es más antiguo que el superior. En este sentido, el esquema Matrix Harris es una herramienta que nos permite organizar esta información de manera visual, mostrando las relaciones de superposición entre los distintos estratos y ayudándonos a entender la secuencia temporal de los eventos que ocurrieron en el sitio. En la Figura 1.1 se muestra un ejemplo de esquema Matrix Harris, donde se pueden observar las distintas unidades estratigráficas y sus relaciones de superposición.

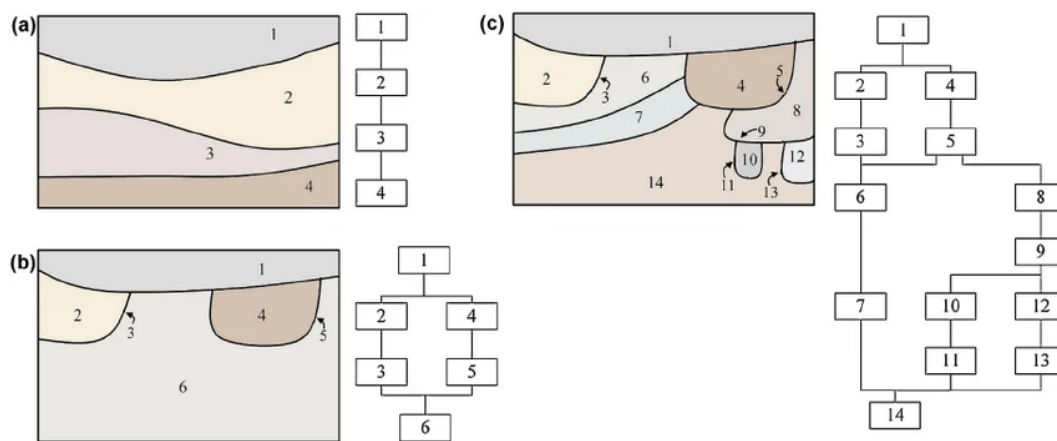


Figura 1.1: Ejemplo de esquema Matrix Harris.

El problema que se plantea en este proyecto es que, a pesar de la utilidad de los esquemas Matrix Harris, su generación manual puede ser un proceso tedioso y propenso a errores, especialmente cuando se trabaja con grandes cantidades de datos o con sitios arqueológicos complejos. En concreto, en lo que a este grupo de investigación respecta, el problema se vuelve más notorio cuando excavan en diversas zonas de un mismo sitio, lo que hace que generen un esquema para cada zona y luego quieran fusionarlos para obtener una visión global del sitio.

Por ejemplo, si hubo una erupción volcánica que esparció ceniza sobre el sitio y se encuentra ceniza en varias zonas, es razonable pensar que esa ceniza se depositó en un mismo evento, lo que implica que los estratos que contienen esa ceniza en distintas zonas deberían estar relacionados entre sí. Por tanto, habría que fusionar los esquemas de cada zona para reflejar esta relación.

Actualmente, en el proceso de generación de los esquemas de cada zona de excavación, el equipo de investigación cuenta con un software que dinamiza este proceso, pues los esquemas a generar no presentan demasiada dificultad. Sin embargo, en el proceso de fusión de los esquemas, el equipo no ha encontrado ninguna herramienta que se adapte a sus necesidades, por lo que actualmente están desarrollando esta fusión de forma manual, con todo el tedio y lentitud que esto conlleva.

Por tanto, el objetivo de este proyecto es desarrollar una herramienta que permita automatizar la fusión de esquemas Matrix Harris. No obstante, es inmediato observar que eso es equivalente a desarrollar una herramienta que permita automatizar la generación de esquemas Matrix Harris, pues

la fusión de esquemas no es más que un proceso de generación de un nuevo esquema a partir de los esquemas originales. Por tanto, el objetivo de este proyecto se puede reformular como el desarrollo de una herramienta que permita automatizar la generación de esquemas Matrix Harris, con especial atención a la fusión de esquemas.

En términos informáticos, el problema a resolver consiste en la **Representación Dinámica de Hipergrafos Jerárquicos con Enrutamiento Ortogonal**; aunque sobre esto incidiremos en el capítulo siguiente, explicando con detalle cada una de las partes que constituyen el nombre. En términos más generales, hay quien prefiere diagramar en términos de Grafos Dirigidos Acíclicos (DAGs); pero, por requerimientos explícitos del cliente, la aplicación tendrá que representar hipergrafos.

1.3. Necesidad del Sistema

La necesidad de crear este sistema no solo se ve reflejada en la existencia de un cliente que lo demanda, sino también hay muchos otros clientes potenciales que podrían beneficiarse y necesitar esta herramienta. En concreto, hemos encontrado diferentes foros y blogs [5], [9], [31] donde se pregunta abiertamente por herramientas que permitan generar esquemas Matrix Harris.

Según parece, existe una herramienta de pago que permite generar estos esquemas, pero el coste parece demasiado elevado; por lo que muchos arqueólogos se ven obligados a recurrir a programas no especializados y no automatizados, tal y como queda reflejado en [9]. Además, en [5] y [31] se detallan algunos otros programas gratuitos, pero ellos mismos reflejan que muchos están anticuados y no pueden desplegarse en sistemas operativos modernos y otros no son manejables cuando la complejidad de los esquemas a generar aumenta.

Por tanto, en este paradigma, surge la necesidad de desarrollar una herramienta que se adapte tanto a las necesidades y gustos personales de nuestro cliente, como a las necesidades de otros clientes potenciales.

1.4. Alcance

Este Trabajo de Fin de Grado tiene como objetivo el diseño, desarrollo y validación de una herramienta de escritorio para la construcción y edición dinámica de hipergrafos jerárquicos dirigidos, con aplicación directa a la representación de esquemas Matrix Harris en contextos arqueológicos.

A continuación, se detallan los objetivos a cumplir:

- **Análisis y especificación de requisitos:** Captura, análisis y formalización de los requisitos funcionales y no funcionales del sistema, incluyendo las restricciones estructurales del modelo de

hipergrafo (aciclicidad, propiedad de Hasse, gestión de aristas largas).

- **Diseño e implementación del motor de *layout*:** Implementación de un motor de cálculo automático de las coordenadas visuales del diagrama, que se ajuste a cómo el cliente espera que se representen los esquemas Matrix Harris y además muestre una buena legibilidad y estética.
- **Diseño de la interfaz de usuario:** Diseño e implementación de una interfaz de escritorio moderna que permita a los usuarios crear, editar y visualizar esquemas Matrix Harris de forma intuitiva.
- **Persistencia:** Diseño e implementación de un mecanismo de serialización y deserialización del estado completo del proyecto.
- **Pruebas y validación:** Diseño y ejecución de un plan de pruebas unitarias e de integración que cubra los componentes críticos del dominio, así como la realización de pruebas de usabilidad con usuarios reales.
- **Documentación:** Elaboración de la memoria del Trabajo de Fin de Grado, así como tutoriales de uso para el cliente y documentación técnica de la aplicación.

Por último, a continuación se detallan los aspectos que quedan fuera del alcance de este TFG:

- Exportación del diagrama a formatos de imagen o vectoriales (PNG, SVG, PDF), ni integración con herramientas de publicación arqueológica.
- Desarrollo de versiones para plataformas móviles o web.
- Integración con bases de datos de yacimientos arqueológicos o sistemas externos de gestión de excavaciones.
- Soporte colaborativo en tiempo real entre múltiples usuarios sobre un mismo proyecto.

1.5. Estructura del documento

El presente documento se organiza en siete capítulos y cinco apéndices, cuyo contenido se resume a continuación.

El Capítulo 1 sitúa el proyecto en su contexto institucional y arqueológico, describe el problema a resolver, justifica la necesidad del sistema y delimita su alcance.

El Capítulo 2 establece el marco teórico necesario para comprender la solución: la teoría de conjuntos parcialmente ordenados, los diagramas de Hasse, los grafos dirigidos acíclicos y los hipergrafos jerárquicos. A continuación, analiza la magnitud del problema a resolver y evalúa comparativamente las herramientas existentes y las tecnologías disponibles, justificando las finalmente adoptadas para el desarrollo de la aplicación.

El Capítulo 3 recoge el análisis del sistema: los requisitos funcionales y no funcionales, los casos de uso y los diagramas de secuencia que describen las interacciones principales entre el usuario y la aplicación.

El Capítulo 4 presenta las decisiones de diseño arquitectónico y de interfaz, incluyendo los patrones aplicados y las maquetas de la interfaz de usuario.

El Capítulo 5 describe el proceso de desarrollo seguido.

El Capítulo 6 constituye el núcleo técnico del documento. Detalla el modelo de datos del hipergrafo, así como todos los algoritmos necesarios para su correcta representación.

El Capítulo 7 presenta los resultados de las pruebas unitarias, de integración y de usabilidad. Además, se incluyen resultados de rendimiento y comparativas para los algoritmos implementados con respecto a los artículos originales en los que se basan.

Finalmente, los Apéndices recogen, de forma extensa y detallada, el material de soporte de cada capítulo: el análisis comparativo completo del estado del arte, los catálogos de requisitos y casos de uso, los diagramas de diseño, y la descripción pormenorizada de los algoritmos implementados en el motor de *layout*. Los Apéndices se han incluido para no interrumpir la lectura del documento principal, pero constituyen una parte integral del mismo y se recomienda su consulta para una comprensión completa del proyecto.

ESTADO DEL ARTE

En este capítulo, comentaremos el trasfondo teórico en el que se engloba este proyecto dentro del ámbito de la informática y las matemáticas. Además, también se efectuará un análisis comparativo de las aplicaciones ya existentes que ofrecen funcionalidades similares a las que se implementan en este proyecto, con el fin de obtener una visión más amplia de las posibles características que se podrían implementar. Finalmente, se comentarán las tecnologías empleadas para el desarrollo de esta aplicación, así como las razones para su elección.

2.1. Marco teórico

2.1.1. Relaciones de orden parcial

En el ámbito de las matemáticas, se define lo siguiente:

Definición 2.1 (Relación). *Dado un conjunto X , una relación R es un subconjunto de $X \times X$. Es decir, $R \subseteq X \times X$.*

Definición 2.2 (Relación de orden parcial). *Dado un conjunto X , se define una relación de orden parcial como una relación binaria R que es reflexiva, antisimétrica y transitiva. Es decir, para todo $a, b, c \in X$:*

- *Reflexiva: aRa*
- *Antisimétrica: Si aRb y bRa , entonces $a = b$*
- *Transitiva: Si aRb y bRc , entonces aRc*

En ocasiones, se denota $a \preceq b$ para indicar que aRb en una relación de orden parcial. En este caso, se dice que a es menor o igual que b según la relación de orden parcial R . La particularidad de las relaciones de orden parcial es que no todos los elementos del conjunto X son comparables entre sí. Es decir, puede haber elementos a y b en X para los cuales ni $a \preceq b$ ni $b \preceq a$ se cumple.

Nótese que esto se adhiere como un guante a la realidad de los estratos arqueológicos, donde no todos los estratos son comparables entre sí debido a que no se ha podido encontrar uno encima de otro. En este caso, si denotamos X como el conjunto de estratos, entonces $a \preceq b$ si el estrato a se encuentra debajo del estrato b de manera física o por medio de una sucesión de estratos físicamente conectados entre sí. En este contexto, esta relación de orden satisface trivialmente los tres axiomas.

2.1.2. Grafos, DAGs, Diagramas de Hasse e Hipergrafos

Antes de valorar las diferentes formas de representar las relaciones de orden parcial, es necesario introducir conceptos fundamentales de teoría de grafos. Para empezar, un grafo es una estructura matemática que permite modelar relaciones entre objetos.

Definición 2.3 (Grafo dirigido). Un **grafo dirigido** (o **dígrafo**) es un par $G = (V, E)$, donde:

- V es un conjunto finito y no vacío de **vértices** (o **nodos**).
- $E \subseteq V \times V$ es un conjunto de **aristas dirigidas**, donde cada arista $(u, v) \in E$ representa una conexión orientada del vértice u al vértice v .

Definición 2.4 (Camino y ciclo). Dado un grafo dirigido $G = (V, E)$, un **camino** de longitud k es una secuencia de vértices $v_0, v_1, \dots, v_k$ tal que $(v_i, v_{i+1}) \in E$ para todo $0 \leq i < k$. Un camino es un **ciclo** si $v_0 = v_k$ y $k \geq 1$.

Definición 2.5 (Grafo acíclico dirigido). Un **grafo acíclico dirigido** (DAG, del inglés *Directed Acyclic Graph*) es un grafo dirigido $G = (V, E)$ que no contiene ningún ciclo. Es decir, no existe ninguna secuencia de vértices $v_0, v_1, \dots, v_k$ con $v_0 = v_k$, $k \geq 1$ y $(v_i, v_{i+1}) \in E$ para todo $0 \leq i < k$.

Todo DAG induce de forma natural una relación de orden parcial sobre su conjunto de vértices. Si $(u, v) \in E$, se dice que u **precede** a v , lo que permite definir una jerarquía entre los nodos del grafo.

Para representar visualmente esta relación de orden parcial, se utilizan los diagramas de Hasse, que son una simplificación de los DAGs eliminando aristas transitivas y reflexivas. Esto queda formalizado de la siguiente manera:

Definición 2.6 (Diagrama de Hasse). Dado un conjunto parcialmente ordenado $(X, \preceq)$, su **diagrama de Hasse** es una representación gráfica del orden parcial en la que:

- Cada elemento de X se representa como un vértice.
- Se dibuja una arista de u a v si y solo si $u \preceq v$, $u \neq v$, y no existe ningún $w \in X$ tal que $u \preceq w \preceq v$ con $w \neq u$ y $w \neq v$.
- Los elementos mayores se sitúan visualmente por encima de los menores, de modo que la dirección de las aristas queda implícita y no es necesario dibujar las flechas.

Para entender mejor esta representación, en la Figura 2.1 se muestra un ejemplo de un DAG y su correspondiente diagrama de Hasse. El conjunto X es $\{a, b, c, d\}$ con la relación de orden parcial dada por $a \preceq b$, $a \preceq c$, $a \preceq d$, $b \preceq c$ y $b \preceq d$. En el DAG (Figura 2.1(a)), se representan todas las relaciones, incluyendo las transitivas y reflexivas. En el diagrama de Hasse (Figura 2.1(b)), se eliminan las aristas transitivas y reflexivas, mostrando únicamente las relaciones directas entre los elementos.

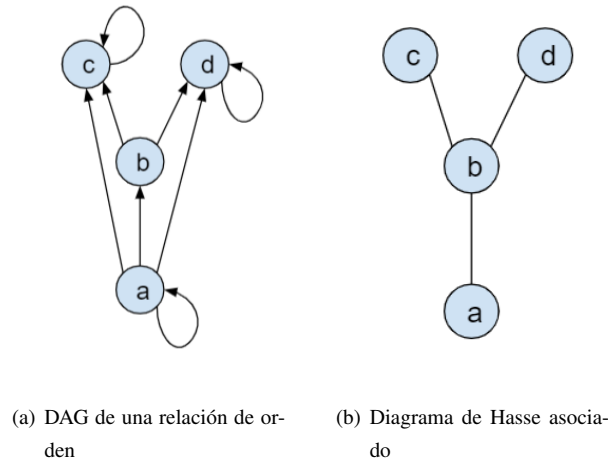


Figura 2.1: Representación de un DAG y su correspondiente diagrama de Hasse

Enlazando este marco con la aplicación, el esquema de Matrix Harris es un ejemplo de diagrama de Hasse utilizado en el ámbito de la arqueología para representar las relaciones cronológicas entre los estratos. No obstante, por requerimientos del cliente, la representación que nosotros realizamos no corresponde a la de un grafo, sino a la de un hipergrafo. Por ejemplo, en la Figura 1.1 de la sección anterior, si atendemos al ejemplo c), los estratos 3 y 5 tienen por debajo al estrato 6 a través de la misma conexión, esto no se puede representar con un grafo tradicional (donde las conexiones son binarias), sino que se requiere de un hipergrafo. Ello, nos lleva a la siguiente definición:

Definición 2.7 (Hipergrafo). Un **hipergrafo** es un par $H = (V, \mathcal{E})$, donde:

- V es un conjunto finito y no vacío de **vértices**.
- $\mathcal{E} = \{e_1, e_2, \dots, e_m\}$ es un conjunto de **hiperaristas**, donde cada hiperarista $e_i \subseteq V$ puede conectar un número arbitrario de vértices, a diferencia de las aristas convencionales o binarias, que solo conectan exactamente dos.

Definición 2.8 (Hipergrafo dirigido). Un **hipergrafo dirigido** es un par $H = (V, \mathcal{E})$ donde cada hiperarista $e \in \mathcal{E}$ es un par ordenado $e = (S, T)$ con $S, T \subseteq V$, $S \neq \emptyset$, $T \neq \emptyset$, siendo S el conjunto de **vértices origen** y T el conjunto de **vértices destino** de la hiperarista.

En la Figura 2.2 se muestra una comparativa entre un DAG y un hipergrafo dirigido. En el DAG (Figura 2.2(a)), cada arista conecta exactamente dos vértices, mientras que en el hipergrafo dirigido (Figura 2.2(b)), una hiperarista puede conectar múltiples vértices de origen con múltiples vértices de

destino, lo que permite representar relaciones más complejas como las que se encuentran en los estratos arqueológicos. Un hipergrafo generaliza el concepto de grafo, en particular, la Figura 2.2(a) puede ser vista como un hipergrafo dirigido con dos hiperaristas binarias.

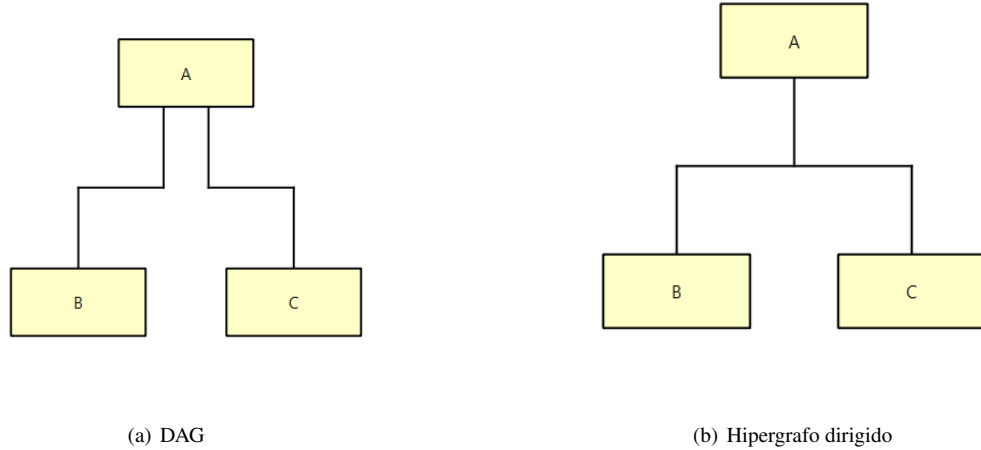


Figura 2.2: Comparativa entre un DAG y un hipergrafo dirigido

2.1.3. Hipergrafos jerárquicos con enrutamiento ortogonal

Estamos ya cerca de poder describir de manera formal el esquema de Matrix Harris que necesitamos representar. En concreto, hay dos detalles que no podemos obviar sobre la Figura 1.1: los nodos se agrupan en capas y las conexiones entre los nodos siguen ángulos rectos y siempre parten de la parte inferior de un nodo hacia la parte superior de otro. La primera propiedad va ligada a la siguiente definición:

Definición 2.9 (Hipergrafo jerárquico). Un **hipergrafo jerárquico** es un hipergrafo dirigido $H = (V, \mathcal{E})$ en el que los vértices están organizados en **capas** $L_0, L_1, \dots, L_k$, con $V = L_0 \cup L_1 \cup \dots \cup L_k$ y $L_i \cap L_j = \emptyset$ para $i \neq j$, de forma que toda hiperarista $e = (S, T) \in \mathcal{E}$ satisface:

- Todos los vértices origen pertenecen a la misma capa: $\exists i$ tal que $S \subseteq L_i$.
- Todos los vértices destino pertenecen a capas superiores: $T \subseteq \bigcup_{j=i+1}^k L_j$.

La segunda propiedad, se denota como enrutamiento ortogonal con puertos y se describe de la siguiente forma:

Definición 2.10 (Requisitos de enrutamiento ortogonal con puertos). Dado un hipergrafo jerárquico $H = (V, \mathcal{E})$, se dice que H cumple los requisitos de **enrutamiento ortogonal con puertos** si se cumplen las siguientes condiciones:

1. Cada hiperarista $e \in \mathcal{E}$ se compone de tres elementos: los **segmentos verticales superiores** $S_U(e)$, un **segmento horizontal** $S_H(e)$ y los **segmentos verticales inferiores** $S_L(e)$.
2. Los vértices o nodos se representan mediante rectángulos de tamaño fijo con un conjunto de **puertos** en sus lados superior e inferior. Cada puerto es el punto común entre un vértice y una hiperarista incidente.
3. Cada puerto de un vértice se corresponde con exactamente un segmento vertical, bien de $S_U(e)$ o bien de $S_L(e)$.
4. Para todo par de hiperaristas $e_1, e_2 \in \mathcal{E}$, se dice que e_1 y e_2 se **cruzan** si y solo si algún segmento horizontal $S_H(e_1)$ tiene puntos de cruce con algún segmento de $S_U(e_2)$ o de $S_L(e_2)$, o viceversa. En particular, el solapamiento de segmentos queda prohibido.

Nótese cómo las Figuras 1.1 y 2.2 satisfacen todos estos requisitos. En conclusión, esta es la definición formal matemática del esquema de Matrix Harris que el cliente nos ha pedido representar.

2.2. Magnitud del problema

En este apartado, vamos a analizar la dificultad y complejidad de representar el esquema de Matrix Harris. Para ello, pondremos en comparativa diferentes representaciones de modelos de datos.

2.2.1. Representación de árboles

Representar un árbol es una tarea relativamente sencilla, ya que se trata de una estructura de datos jerárquica en la que cada nodo tiene un único padre (excepto la raíz) y cero o más hijos. En concreto, basta con aplicar un algoritmo de recorrido en profundidad (DFS) o en anchura (BFS) para visitar todos los nodos del árbol y representarlos gráficamente.

No obstante, pudiera parecer que un esquema de Matrix Harris es parecido a un árbol, pero en realidad está muy lejos de serlo y los algoritmos de visualización necesarios son enormemente más complejos. Esto lo ilustraremos a lo largo de las siguientes subsecciones.

2.2.2. Representación de DAGs

El salto de complejidad entre árboles y DAGs es enorme, ya que en un DAG cada nodo puede tener múltiples padres, lo que hace que sea mucho más complejo de representar gráficamente. Existen múltiples formas de representar un mismo DAG, cada cual con sus ventajas e inconvenientes. Además, en función de la representación elegida, los datos se vuelven más o menos legibles. Esto se evidencia

en la siguiente imagen, donde se muestra el mismo DAG (que de hecho es también un diagrama de Hasse) representado de diferentes formas:

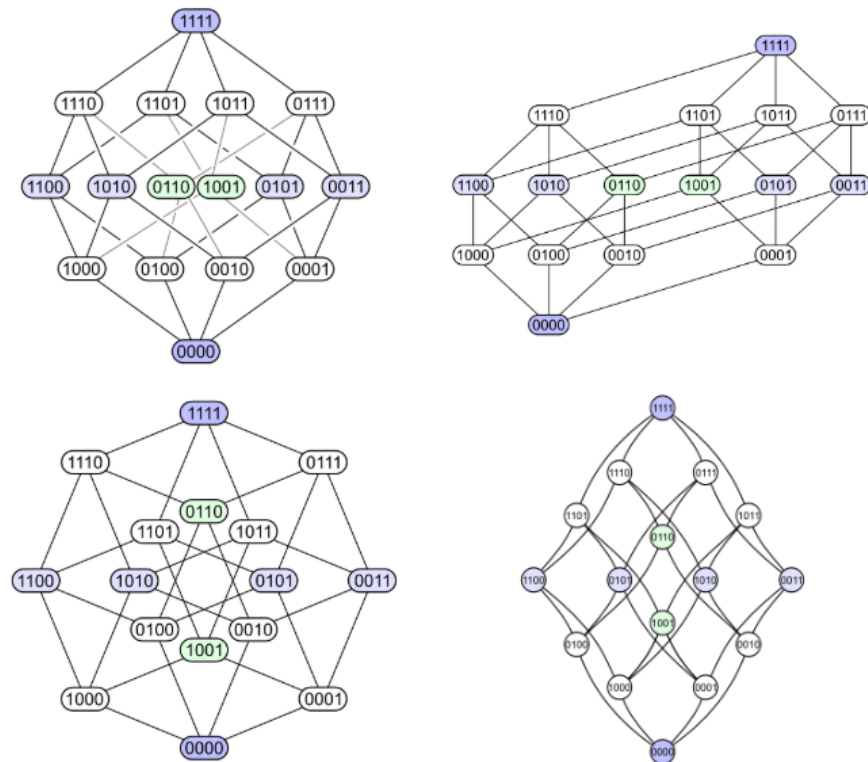


Figura 2.3: Diferentes formas de representar un mismo DAG.

A título personal, considero que son este tipo de ejemplos los que ponen de manifiesto la gran complejidad de representar un DAG de forma clara y legible. En nuestro caso, la representación que tiene sentido comparar es la denominada jerárquica, en la que los nodos se organizan en niveles según su profundidad en el DAG. Para resolver este problema, la aproximación por excelencia es el método de Sugiyama que se describe en [35]. En él, divide el problema en los siguientes pasos:

1. Asignar cada nodo a un nivel, de forma que los nodos que están relacionados entre sí se encuentren en niveles cercanos.
2. Por cada arista que conecta dos nodos en niveles diferentes, crear una serie de nodos ficticios entre los niveles y partir la arista en varias que conecten los nodos originales con los nodos ficticios, de forma que cada arista solo conecte nodos de niveles adyacentes.
3. Reordenar los nodos dentro de cada nivel para minimizar el número de cruces entre las aristas.
4. Asignar coordenadas a cada nodo para que la representación sea clara y legible.

Con estos cuatro pasos, se pueden conseguir representaciones de DAGs bastante legibles, aunque el problema de minimizar el número de cruces entre las aristas es *NP-hard* y asignar coordenadas a

cada nodo es también *NP-hard*.

Representar DAGs es una tarea muy estudiada y de mucho interés, ya que este tipo de estructuras de datos se encuentran en multitud de ámbitos, como la informática, la biología, la sociología, etc. Cada año, se publican multitud de artículos sobre nuevas formas de representar DAGs, e incluso se organizan conferencias específicas sobre este tema, como la International Symposium on Graph Drawing and Network Visualization (GD) o la International Conference on Information Visualization (IV).

2.2.3. Representación de diagramas de Hasse

Si bien, tal y como se ha visto en la sección anterior, los diagramas de Hasse son un caso especial de DAGs, sus particularidades no son suficientes para aplicar métodos focalizados sobre ellos. Por ende, representar diagramas de Hasse es tan complejo como representar DAGs; aunque con la ventaja de que, al reducir el número de aristas, la representación es más legible.

2.2.4. Representación de grafos

De nuevo, al igual que los DAGs, existen múltiples representaciones para un mismo grafo. No obstante, a priori, pareciera que la tarea de representar un grafo es enormemente más compleja que la de representar un DAG, ya que en un grafo no existe una relación de orden entre los nodos.

Sin embargo, una de las formas concretas de representar un grafo es aplicando un paso adicional al método de Sugiyama, para transformarlo en un DAG. Tal y como se detalla en [26], los pasos serían:

1. Dar una dirección aleatoria a cada arista del grafo, de forma que se convierta en un grafo dirigido.
2. Cambiar la dirección de las aristas que forman ciclos, de forma que el grafo dirigido resultante sea acíclico.
3. Aplicar el método de Sugiyama para representar el DAG resultante.

El único paso complejo es el 2, pero hay muchos métodos heurísticos propuestos que resuelven el problema. Por tanto, representar un grafo es casi tan complejo como representar un DAG, lo que pone de manifiesto la gran complejidad de representar un DAG y, en consecuencia, la gran complejidad de representar un esquema de Matrix Harris.

2.2.5. Representación de esquemas de Matrix Harris

Por lo que hemos visto, representar un esquema de Matrix Harris es equivalente a representar un hipergrafo jerárquico con enrutamiento ortogonal. En concreto, el hecho de necesitar que las hiperaristas sean ortogonales, es decir, que estén formadas por segmentos horizontales y verticales, hace que

la tarea de representación sea relativamente más compleja que la de un DAG.

Aunque detallaremos más adelante cuáles son los pasos, los listaremos a continuación para que el lector tenga una visión general de la complejidad de esta tarea:

1. Asignar a cada nodo del hipergrafo un nivel, de forma que los nodos que están relacionados entre sí se encuentren en niveles cercanos.
2. Por cada hiperarista que conecta dos nodos en niveles diferentes, crear una serie de nodos ficticios entre los niveles y partir la hiperarista en varias que conecten los nodos originales con los nodos ficticios, de forma que cada hiperarista solo conecte nodos de niveles adyacentes.
3. Transformar convenientemente el hipergrafo resultante en un DAG, para poder aplicar un algoritmo de minimización de cruces entre aristas.
4. Transformar de otra forma distinta el hipergrafo en otro DAG, para poder aplicar un algoritmo de asignación de coordenadas a cada nodo.
5. Evitar solapamientos horizontales y verticales entre las hiperaristas.
6. Asignar puertos a cada nodo.
7. Asignar una altura a cada hiperarista y cada nodo.

Estos pasos aparecen listados y recogidos en [19] y es la manera en la que hemos conseguido representar esquemas de Matrix Harris. El problema reside en que cada uno de ellos es *NP-hard*, por lo que se han de usar aproximaciones que consigan soluciones subóptimas en tiempos razonables.

En conclusión, la tarea de representar un esquema de Matrix Harris es enormemente compleja y así lo demuestran la complejidad de los algoritmos necesarios para llevar a cabo esta tarea (NP-hard). Cuando uno ve diagramas como el de la Figura 1.1, no se imagina la complejidad que hay detrás de esa representación. De hecho, le da la sensación que es algo muy cercano a representar un árbol, pero como se ha visto a lo largo de esta sección, la complejidad de representar un esquema de Matrix Harris es muchísimo mayor que la de representar un árbol, y es incluso mayor que la de representar un DAG o un grafo. Por tanto, el problema a resolver en este proyecto es enormemente complejo, lo que pone de manifiesto la gran magnitud del mismo. La Figura 2.4 trata de evidenciar este punto.

2.3. Análisis comparativo de la competencia

En esta sección, hemos realizado un análisis comparativo de las distintas aplicaciones ya existentes para representar esquemas de Matrix Harris. Este análisis se ha llevado a cabo teniendo en cuenta los gustos del cliente y se encuentra en el Anexo A.1. Las conclusiones de este análisis son claras: no

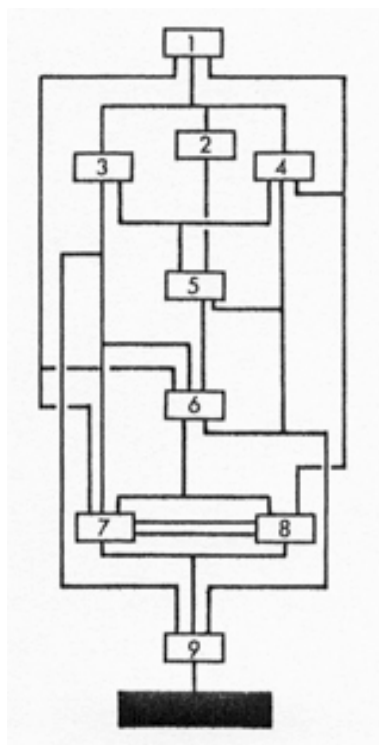


Figura 2.4: Ejemplo complejo de Matrix Harris.

existe ninguna aplicación actual que satisfaga las necesidades del cliente. No obstante, hemos tomado nota de algunas funcionalidades que podríamos aprovechar de estas aplicaciones.

2.4. Análisis comparativo de las tecnologías

El lenguaje de programación utilizado para el desarrollo de Matrix-Harris App es C++ y se integrará con la biblioteca gráfica Qt. En esta sección, se justifica esta elección tecnológica y se comparan otras alternativas que se consideraron durante el proceso de análisis tecnológico. Esta digresión aparece recogida en el Anexo A.2.

2.5. Entorno de desarrollo y de control de versiones

Siendo el lenguaje elegido C++, el editor que se ha utilizado es Visual Studio, que es un entorno ampliamente utilizado para el desarrollo en C++. Para la parte gráfica, se ha hecho uso de Qt Creator. Por otra parte, el control de versiones ha sido gestionado mediante Git. En [28], se puede acceder al repositorio principal de Matrix-Harris App.

ANÁLISIS DEL SISTEMA

En esta sección, se recoge el análisis de funcionalidad realizado en el sistema, mediante la educación de requisitos. Dichos requisitos han sido recopilados a través de entrevistas con el cliente, análisis de la competencia y revisión de la literatura académica relacionada con el problema que se pretende resolver.

Cabe destacar que, en este caso, no se hará ninguna subdivisión de requisitos por módulos o componentes del sistema. Si bien es cierto que la arquitectura lógica de la aplicación distingue claramente entre la lógica de dominio del hipergrafo, la lógica de aplicación (deshacer/rehacer) y la lógica de interacción, esta separación arquitectónica no se traslada a la especificación de requisitos, que se realiza a nivel del comportamiento observable del sistema completo. No obstante, esta distinción sí se tiene en cuenta a la hora del desarrollo e implementación del sistema.

La razón es que ambas capas actúan siempre de forma coordinada: cualquier operación de edición (crear un nodo, añadir una conexión, fusionar dos nodos) implica simultáneamente una modificación de la estructura del hipergrafo, un recálculo del *layout*, un guardado del estado anterior y una actualización de la representación visual, todo ello dentro de una única transacción. Formular requisitos separados para cada capa introduciría una fragmentación artificial que dificultaría la comprensión del comportamiento del sistema sin aportar información adicional.

Por este motivo, los requisitos funcionales se agrupan por ámbito funcional (gestión de proyectos, gestión de diagramas, gestión de nodos e hiperaristas, *layout* y visualización, diagrama conjunto y mecanismo de deshacer y rehacer) empleando el prefijo **RF** sin distinción de subsistema.

Dada la extensión de los requisitos funcionales y no funcionales, se ha incluido un catálogo completo de casos de uso y diagramas de secuencia en el Anexo B.3 para complementar esta sección. Como resumen, el requisito más importante en el que el cliente ha hecho hincapié es el de garantizar la legibilidad visual del esquema por encima de todo a la vez que se automatiza el proceso. Esta ha sido la máxima que se ha tenido en cuenta. Adicionalmente, también en dicho anexo se muestra el diagrama de casos de uso general del sistema, se describen todos los casos de uso de forma textual y se incluyen sus diagramas de secuencia asociados.

DISEÑO DEL SISTEMA

En este capítulo, se describen los aspectos relacionados con el diseño de la aplicación. Se detallan las decisiones de diseño tomadas y la arquitectura general del sistema.

4.1. Maquetas de la interfaz de usuario

Uno de los primeros pasos en el proceso de diseño fue la creación de maquetas de la interfaz de usuario. Esto era necesario no solo para entender los requisitos funcionales, sino también para visualizar cómo se organizarían las distintas funcionalidades en la aplicación y cómo interactuarían entre sí. El enfoque de las maquetas fue distinto al habitual: se optó por presentar una versión muy simplificada (solo creaba árboles) de mi idea de aplicación, para poder mostrarle al cliente de manera interactiva. Estas maquetas se encuentran en el Anexo C.1 y fueron fundamentales para validar con el cliente la dirección que estaba tomando el diseño de la aplicación.

4.2. Arquitectura del sistema

La arquitectura de la aplicación se articula en tres capas bien diferenciadas. La primera, implementada en el módulo `hypergraph_logic`, encapsula toda la lógica de dominio pura: la estructura del hipergrafo, sus invariantes (acicilicidad, propiedad de Hasse, gestión de aristas largas) y los algoritmos de *layout* que transforman esa estructura en coordenadas visuales. La segunda capa, formada por los editores y la clase `Project`, actúa como intermediaria entre el modelo de datos y la interfaz: gestiona el ciclo de vida de los diagramas, la persistencia en JSON y el mecanismo transaccional de deshacer y rehacer. La tercera capa es la interfaz gráfica, que consume los datos de *layout* producidos por las capas inferiores para renderizar el diagrama e interceptar las acciones del usuario.

Esta división se hace latente en el diagrama de clases (Figura 4.1): si se analiza de arriba a abajo, se puede observar cómo se construyen las capas de hipergrafos, después los editores y el proyecto, y por último la interfaz gráfica. También, cobra gran visibilidad en la propia implementación, como se discutirá en el capítulo correspondiente.

DESARROLLO

5.1. Entorno de Desarrollo y Herramientas

Como ya se argumentó en la Sección 2.4, el desarrollo de Matrix-Harris App se ha llevado a cabo utilizando el lenguaje de programación C++ y la biblioteca gráfica Qt. Para la gestión del proyecto, se ha utilizado CMake como sistema de construcción, lo que facilita la portabilidad. Además, la parte de independencia de las capas comentada en la Sección 4.2 se ha visto reforzada por la organización de las librerías de compilación en el propio CMake, con cada capa (dominio, aplicación e interfaz gráfica) organizada en su propia librería.

En otro orden de cosas, se han utilizado dos entornos de desarrollo distintos: Visual Studio para las capas de dominio y aplicación, y Qt Creator para la capa de interfaz gráfica. Este cambio de IDE fue el principal motivo por el que se usó CMake como sistema de construcción, pues permite una transición fluida entre ambos entornos. Finalmente, se ha empleado Git para el control de versiones, con un repositorio principal alojado en GitHub (véase `itematrixharrisapp`).

5.2. Ciclo de vida del desarrollo

El ciclo de vida adoptado para el desarrollo de esta aplicación encaja con un modelo más bien híbrido, que combina la estructura secuencial del desarrollo en cascada con la flexibilidad y mitigación de riesgos del desarrollo iterativo.

Esta elección se justifica por la naturaleza dual del proyecto: por un lado, la necesidad de obtener una definición precisa de los requisitos junto al cliente y, por otro, la alta complejidad técnica de la lógica de hipergrafos, que exigía una validación constante.

Fase de requisitos (Enfoque en cascada)

La fase inicial consistió en una serie de reuniones y comunicaciones con el cliente orientadas a la especificación de requisitos y la dinámica que debería presentar la aplicación. Este proceso, aunque

costoso en tiempo, resultó beneficioso a largo plazo al resolver las ambigüedades antes de iniciar el desarrollo, pues se evitaron cambios de requisitos en fases tardías.

Fase de desarrollo (Enfoque iterativo por capas)

Una vez estabilizados los requisitos, el desarrollo se organizó siguiendo la estrategia de desarrollo por capas descrita en la Sección 4.2. No se avanzaba de una capa a la siguiente hasta que la capa actual no estaba completamente implementada y validada mediante pruebas unitarias.

La capa de dominio se subdividió en dos fases secuenciales de implementación e iteración. La primera se dedicó a la lógica estructural del hipergrafo: la gestión de nodos, hiperaristas, capas y las invariantes del modelo. La segunda abordó el *pipeline* de *layout*: los algoritmos de minimización de cruces, asignación de coordenadas y gestión de puertos. En ambas fases se siguió una política de pruebas unitarias continuas: cada clase se sometía a su correspondiente batería de pruebas inmediatamente después de su implementación, antes de avanzar al siguiente componente. Esta filosofía se asemeja a una metodología *Test-Driven Development* (TDD), aunque no se siguió de forma estricta.

La capa de aplicación siguió un enfoque diferente: dado que los editores y la clase `Project` dependen directamente del dominio ya validado, fue posible implementar la capa completa en primer lugar y someterla a pruebas unitarias una vez finalizada.

Finalmente, la capa de interfaz gráfica se desarrolló e integró sobre las capas inferiores ya estables, y su validación se realizó mediante pruebas de integración que verificaron el comportamiento del sistema completo.

IMPLEMENTACIÓN: REPRESENTACIÓN

ORTOGONAL DE HIPERGRAFOS

JERÁRQUICOS

Este capítulo es el corazón de esta memoria, ya que constituye la parte más técnica y compleja de todo el proyecto. En él, voy a describir con detalle toda la algoritmia del dominio necesaria para que Matrix-Harris App pueda representar esquemas de Matrix Harris de forma clara y visualmente atractiva.

Resulta conveniente recalcar que, en toda la literatura que se precie sobre representación de grafos, los algoritmos se presentan de forma abstracta, en formato de pseudocódigo. Siguiendo esta línea, todos los algoritmos que se describan en esta sección se presentarán también de forma abstracta, en inglés y con formato de pseudocódigo. Para consultar la implementación concreta, se puede acudir al código fuente de la aplicación, referenciado en [28]. Como apunte, todos los algoritmos que se presentan en esta sección y sus apéndices son originales, esto es, fruto de mis propias reflexiones y adaptaciones de ideas de la literatura académica, pero no son traducciones de ningún algoritmo ya existente.

La solución tomada está basada en el paper *A Mixed-Integer Program for Drawing Orthogonal Hyperedges in a Hierarchical Hypergraph* [19]. En este artículo, se presenta un MIP (Mixed-Integer Program) para representar un hipergrafo jerárquico desde cero, es decir, solo con la información de sus vértices y sus hiperaristas. No obstante, al tratarse de un entorno de actualización dinámica, no es viable ejecutar un MIP cada vez que se actualice el grafo, ya que esto sería demasiado costoso computacionalmente.

En su lugar, también en [19] se presenta una heurística para realizar la misma tarea de forma mucho más eficiente, aunque con una calidad de representación algo menor. El enfoque de la aplicación ha sido tomar esta heurística como base, en el sentido de subdivisión del problema global, pero ir resolviendo cada uno de los pasos de forma independiente, en ocasiones adaptando algoritmos ya establecidos y en otras con soluciones originales. De una forma u otra, la heurística se compone de los siguientes pasos:

1. Ordenar los vértices de cada capa de forma que se minimicen los cruces entre las hiperaristas.
2. Asignar a cada vértice una coordenada horizontal, de forma que se respete la ordenación anterior y se minimice la distancia para con sus nodos padres e hijos.

3. Ordenar las hiperaristas (en función de la altura de su segmento horizontal) de forma que se minimicen los cruces entre ellas.
4. Asignar puertos a cada nodo, que son las abscisas desde las que partirán los segmentos verticales de las hiperaristas, de forma que se minimicen los cruces y no existan solapamientos entre ellos.
5. Asignar una coordenada vertical a cada hiperarista, respetando la ordenación anterior y tratando de minimizar la altura total del esquema.

No obstante, para poder aplicar esta heurística, es necesario tener el siguiente objeto:

Definición 6.1 (Hipergrafo propio de k capas). Sea $H = (V, \mathcal{E})$ un hipergrafo jerárquico con capas $L_0, L_1, \dots, L_{k-1}$. Se dice que H es un **hipergrafo propio de k capas** si toda hiperarista $e = (S, T) \in \mathcal{E}$ satisface que los vértices destino pertenecen exactamente a la capa inmediatamente inferior a la de los vértices origen:

$$\exists i \in \{0, \dots, k-2\} \text{ tal que } S \subseteq L_i \text{ y } T \subseteq L_{i+1}.$$

Por tanto, lo primero que se necesita en este proceso es disponer de una estructura de datos que represente un hipergrafo propio de k capas y que permita acoplar las políticas de actualización dinámica que se describen en la Sección B.1. De hecho, a partir de este instante, cada vez que se haga referencia a un hipergrafo, se estará hablando de un hipergrafo propio de k capas.

6.1. La estructura de datos del Hipergrafo

Asignación de capas

En circunstancias habituales, la asignación de capas a los vértices se realiza mediante un algoritmo heurístico que trata de minimizar el número de capas y la longitud total de las aristas. Sin embargo, en este contexto, la asignación de capas se realiza mediante la regla de la profundidad, es decir, cada vértice se asigna a la capa inmediatamente inferior a la máxima de sus padres. Esto se hace así por requerimientos del cliente y porque ayuda a reflejar lo que se va encontrando conforme se va excavando. De esta forma, se puede establecer el paralelismo entre la estratificación del suelo y el esquema Matrix Harris, tal y como queda ilustrado en la Figura 1.1.

Dividir hiperaristas largas

Definición 6.2 (Hiperarista larga). Sea $H = (V, \mathcal{E})$ un hipergrafo jerárquico en k capas (no necesariamente propio) y sea $\lambda : V \rightarrow \{0, \dots, k-1\}$ la función que asigna a cada vértice su capa. Se dice que una hiperarista $e = (S, T) \in \mathcal{E}$ es **larga** si:

$$\max_{u \in S, v \in T} (\lambda(u) - \lambda(v)) > 1.$$

De igual forma, diremos que una arista es **corta** si no es larga y, consecuentemente, se puede definir un hipergrafo propio de k capas como aquel que no tiene hiperaristas largas. En la práctica, según el usuario vaya introduciendo nodos y conexiones, es inevitable que se introduzcan hiperaristas largas, por lo que será necesario dividir las en una serie de hiperaristas cortas (que denotaremos con el nombre de **segmentos**), introduciendo nodos *dummy* en el proceso. Para ello, en *Hypergraph.cpp* se implementa el método `splitLongEdge`, cuyo pseudocódigo se muestra a continuación.

```

Input: Hipergrafo por capas  $H = (V, \mathcal{E}, \lambda)$ , donde  $\lambda : V \rightarrow \mathbb{N}$  asigna cada vértice a cada capa y una hiperarista  $e \in \mathcal{E}$  a ser subdividida.
Output:  $H$  modificado in situ:  $e$  es reemplazado por una cadena de hiperaristas cortas, enlazadas por nodos dummy.

1   $src[] \leftarrow$  sources of  $e$ , indexed by  $\lambda$ ,    $tgt[] \leftarrow$  targets of  $e$ , indexed by  $\lambda$ ;
2   $\ell_{\min}^s \leftarrow \min \lambda(src)$     $\ell_{\max}^s \leftarrow \max \lambda(src)$ ;
3   $\ell_{\min}^t \leftarrow \min \lambda(tgt)$     $\ell_{\max}^t \leftarrow \max \lambda(tgt)$ ;
4  if  $\ell_{\min}^s \geq \ell_{\max}^t - 1$  then
5  |   return;                               // La arista ya es corta - nada que hacer
6  end
7  removeFromHypergraph( $H, e$ );
8   $carry \leftarrow \emptyset$ ;
9  for  $L \leftarrow \ell_{\min}^s$  to  $\ell_{\max}^t - 1$  do
10 |    $segSrc \leftarrow []$ ;
11 |   if  $L \leq \ell_{\max}^s$  and  $src[L]$  exists then
12 | |    $segSrc \leftarrow segSrc \cup src[L]$ ;
13 |   end
14 |   if  $carry \neq \emptyset$  then
15 | |    $segSrc \leftarrow segSrc \cup [carry]$ ;
16 |   end
17 |    $segTgt \leftarrow []$ ;
18 |   if  $L + 1 \geq \ell_{\min}^t$  and  $tgt[L + 1]$  exists then
19 | |    $segTgt \leftarrow segTgt \cup tgt[L + 1]$ ;
20 |   end
21 |   if  $L + 1 < \ell_{\max}^t$  then
22 | |    $carry \leftarrow \text{createDummyNode}(L + 1)$ ;
23 | |    $V \leftarrow V \cup \{carry\}$ ;
24 | |    $\text{addNodeToLayer}(L + 1, carry)$ ;
25 | |    $segTgt \leftarrow segTgt \cup [carry]$ ;
26 |   else
27 | |    $carry \leftarrow \emptyset$ 
28 |   end
29 |   createHyperedge( $H, segSrc, segTgt$ );
30 end

```

Algoritmo 6.1: `splitLongEdge( $H, e$ )`: reemplaza la hiperarista larga e por una cadena de cortas.

De esta forma, podemos garantizar que, a lo largo de todo el proceso de actualización dinámica, el hipergrafo se mantenga como un hipergrafo propio de k capas, para luego poder aplicar el *pipeline* de heurísticas de representación.

Agrupación de conexiones

En un grafo normal, cada arista conecta exactamente dos vértices. Sin embargo, en un hipergrafo, cada hiperarista puede conectar a un conjunto de vértices origen con un conjunto de vértices destino. Múltiples formas de agrupar estas conexiones pueden resultar en representaciones visualmente distintas, pero con la misma información subyacente. La Figura 2.2 es un ejemplo de esto, donde en ambos casos A está por encima de B y C , pero en el primero esta conexión aparece por separado y, en el segundo, aparece agrupada.

Resulta complicado saber cuándo se ha de agrupar y cuándo no ya que, en ese mismo ejemplo, el usuario podría utilizar la conexión única entre A y C para añadir un origen nuevo y, por tanto, información nueva. Como no es posible anticipar las acciones del usuario, lo que se ha hecho es permitirle elegir la manera en la que se agrupan las conexiones.

Otra forma de plantearlo hubiera sido implementar una heurística que se encargara de esto de forma automática, pero eso tiene dos inconvenientes grandes. En primer lugar, el mapa mental que el usuario tiene de su esquema puede verse muy alterado por la nueva heurística y, en segundo lugar, no existen heurísticas que funcionen bien en todos los casos. Este problema, se describe como *Edge concentration* y está demostrado ser *NP-hard*, es decir, particularmente difícil de resolver de forma eficiente (véase [24]).

Existen heurísticas para resolver este problema en el caso de un grafo (o hipergrafo) bipartito (véase [11] o [29]), pero no para un grafo (o hipergrafo) que no cumpla esta propiedad. En la práctica, se busca esta propiedad de manera local, ya sea considerando solo conexiones entre capas adyacentes o entre capas no adyacentes. Ambas tienen sus desventajas y no existe motivo para pensar que los esquemas generados se ajusten a una u otra, por lo que se ha optado por dejar esta decisión en manos del usuario, para que pueda elegir la que mejor se adapte a sus necesidades.

El resto de detalles técnicos y algoritmos relativos a la estructura de datos del hipergrafo, como la gestión de nodos y conexiones, se describen el Apéndice D.1. A partir de este punto, se asume que se tiene una estructura de datos de hipergrafo propio de k capas, con la que se puede trabajar para aplicar el *pipeline* de heurísticas de representación automática.

6.2. Minimización de cruces entre hiperaristas

Este es el primer paso del *pipeline* de heurísticas de representación automática. El objetivo de este paso es ordenar los vértices de cada capa de forma que se minimicen los cruces entre las hiperaristas. En este caso, dado que tenemos actualización dinámica, no podemos estar constantemente minimizando cruces por cada interacción, ya que esto desvirtuaría el mapa mental que el usuario tiene de su esquema. En su lugar, se ha tratado de mitigar la aplicación de este paso, aplicándolo solo en

operaciones muy disruptivas, como la fusión de nodos o la recolocación masiva de nodos.

También, para reubicar un nodo dentro de la misma capa (a petición del usuario), se aplica este algoritmo para colocar los subgrafos que cuelgan de ese nodo de forma que se minimicen los cruces, pero manteniendo intacta la ordenación del resto de capas superiores. Adicionalmente, el usuario tiene la capacidad de solicitar una minimización global de cruces, que se aplicará a todo el esquema, para cuando desee mejorar la calidad de la representación.

Volviendo al problema de minimización de cruces, la aproximación tomada es convertir el hipergrafo en un DAG, pues se conocen múltiples métodos para minimizar cruces en DAGs. Para ello, una primera aproximación pudiera ser considerar cada hiperarista como un conjunto de aristas binarias, es decir, cada conexión entre un vértice origen y un vértice destino se consideraría como una arista independiente. No obstante, esta aproximación es errónea, como se expone en la Figura 6.1. La parte a) de la figura muestra las representaciones de un hipergrafo solo teniendo en cuenta las conexiones entre vértices, mientras que la parte b) muestra la representación real del hipergrafo. Se puede observar como erróneamente pudiera parecer que se reducen los cruces entre hiperaristas en la representación (a.b); cuando en realidad, aumentan a nivel de hipergrafo ((b.b)).

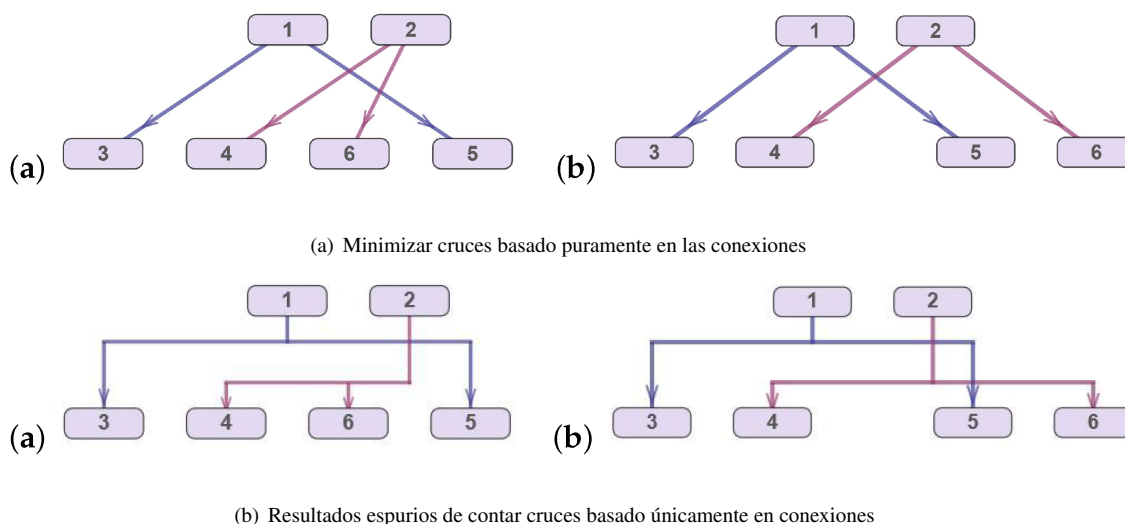


Figura 6.1: Contar cruces correctamente en un hipergrafo

Para solucionar esto, hay que pensar que se está haciendo mal al contar los cruces. Comprimir todos los segmentos horizontales a un solo punto es de gran utilidad para ver la solución adecuada. Lo que ocurre es que no se está teniendo en cuenta el nodo invisible que queda representado por el segmento horizontal. Por tanto, para contar los cruces correctamente, tendremos que introducir un nodo invisible para cada hiperarista, que quedará conectado a los vértices origen y destino de dicha hiperarista. Todos estos nodos intermedios los situaremos en una capa intermedia entre la de los vértices origen y la de los vértices destino, tal y como se muestra en la Figura 6.2.

Una vez que tenemos este DAG, podemos aplicar cualquier algoritmo de minimización de cruces

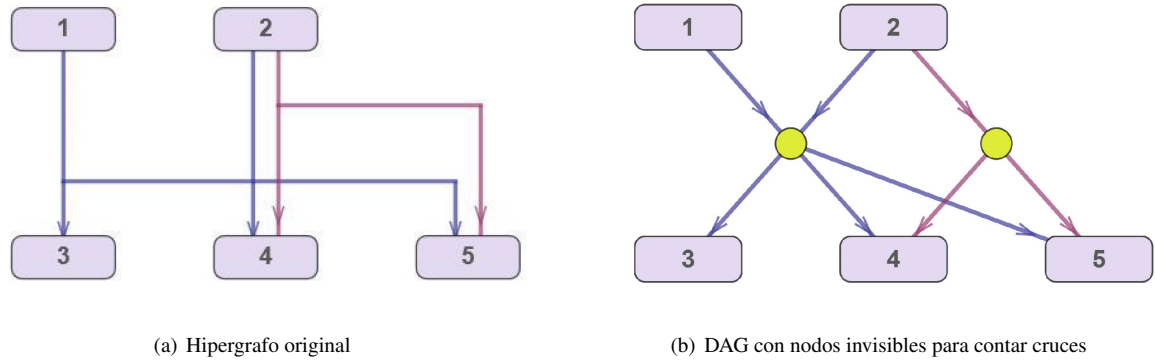


Figura 6.2: Transformar hipergrafo en un DAG para contar cruces

para DAGs, pues también minimizará los cruces a nivel de hipergrafo. El problema de minimización de cruces en DAGs es un problema *NP-hard*, incluso para solo dos capas. En un contexto multicapa, este problema se vuelve especialmente complejo, por lo que se han desarrollado múltiples heurísticas para abordarlo. En [24] y [26], se presenta un amplio repertorio de ellos. Para justificar esta elección, es necesario introducir el concepto de **segmentos**.

Así pues, como se mencionó anteriormente, al subdividir una hiperarista larga se procede introduciendo nodos *dummy* y creando una cadena de hiperaristas cortas, que denominamos segmentos. Diremos que un segmento es **interior** si es una conexión binaria entre dos nodos *dummy* (ver Figura 6.3).

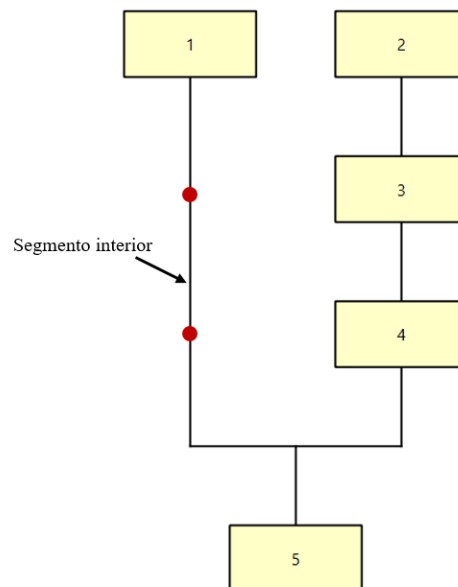


Figura 6.3: Segmentos interiores

De esta forma, se definen tres tipos de cruces entre hiperaristas: tipo 0, tipo 1 y tipo 2, que vienen a contar el número de segmentos interiores que participan en dicho cruce. Así, un cruce de tipo 2 se

produce entre dos segmentos interiores. Uno de las métricas de estética que más legibilidad aporta a la representación es que las hiperaristas largas (como la de la Figura 6.3) se representen lo más verticalmente posible, es decir, que los segmentos interiores consecutivos queden alineados verticalmente. Esto se consigue primero eliminando los cruces de tipo 2 y después aplicando un algoritmo de asignación de coordenadas horizontales que fuerce esta alineación. Para la primera parte, se necesita escoger un algoritmo de minimización de cruces adecuado.

A estos efectos, la mejor heurística para evitar cruces de tipo 2 es el algoritmo de *Global Sifting* presentado en [2]. De hecho, tal y como se compara en [26], en general este algoritmo es uno de los mejores para minimizar cruces en DAGs multicapa (solo superado por otro en algunas ocasiones). Además, esta heurística resulta especialmente útil ya que permite fijar ciertos nodos, lo que es fundamental cuando no se quiere aplicar en todo el esquema, sino solo en ciertas partes de él.

Por último, este algoritmo es especialmente importante en este escenario concreto, pues es muy sensible a la hora de encontrar la posición óptima para cada nodo del hipergrafo y, por tanto, especialmente útil para un entorno con *hubs*, que poseen un gran número de conexiones y cuyo correcto posicionamiento es fundamental para la legibilidad de la representación. En el Apéndice D.2 se explica con más detalle el algoritmo de *Global Sifting* y cómo se ha implementado en la aplicación.

6.3. Asignación de coordenadas horizontales

Una vez conocemos el orden en que se han de mostrar los nodos de cada capa de izquierda a derecha, ya sea por haber aplicado el paso anterior o por la ordenación actual del hipergrafo, el siguiente paso es asignar coordenadas horizontales (abscisas) a cada nodo. El objetivo de este paso es respetar la ordenación dada por el paso anterior y a su vez minimizar la distancia de un nodo para con sus nodos padres e hijos. De esta forma, se consigue cierta sensación de agrupamiento entre los nodos relacionados, lo que mejora la legibilidad de la representación. Además, se ha de garantizar un espacio mínimo entre los nodos para evitar solapamientos.

Como se expuso en la sección anterior, resulta importante representar las hiperaristas largas de la forma más vertical posible. Uno de los ingredientes (eliminar cruces de tipo 2) nos lo da el algoritmo de minimización; mientras que el otro es responsabilidad de este paso.

De nuevo, la manera de obrar será transformar convenientemente el hipergrafo en un DAG y aplicar algún método de asignación de abscisas para DAGs. En este caso, añadir nodos *hub* entre medias no es de utilidad. La justificación es sencilla: los *hubs* surgían de comprimir la barra horizontal de una hiperarista a un único punto, lo que permitía contar cruces correctamente porque un cruce es un evento puntual. La asignación de coordenadas horizontales, en cambio, no busca detectar eventos, sino distribuir nodos en el espacio; y la barra horizontal que el *hub* representaría tiene una longitud que depende de las coordenadas de todos los nodos que agrupa, por lo que no puede reducirse a una

única abscisa y, por tanto, no puede definirse un concepto de “estar cerca del *hub*”.

En este caso, la mejor forma de transformar el hipergrafo en un DAG es la intuitiva, es decir, considerar cada hiperarista como un conjunto de aristas binarias, lo que se muestra en la Figura 6.4. A partir de este DAG se pueden aplicar toda la maquinaria de asignación de coordenadas horizontales para DAGs. Se recomienda al lector tratar de resolver este problema por su cuenta, es decir, dado un DAG por capas y un orden de nodos dentro de cada capa, asignar coordenadas horizontales a cada nodo respetando dicho orden y tratando de minimizar la distancia para con sus nodos padres e hijos. Incluso para cuando se tenga únicamente dos capas, verá que este problema es no trivial y conseguir resultados visualmente agradables es especialmente complejo.

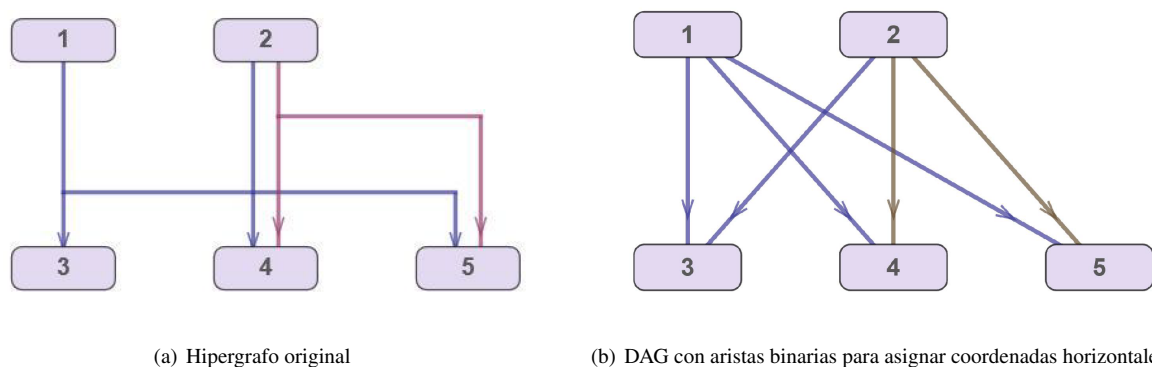


Figura 6.4: Transformar hipergrafo en un DAG para asignar coordenadas horizontales

Dado que este algoritmo se va a tener que aplicar cada vez que exista una actualización, es necesario que sea lo más eficiente posible. También, cabe recordar que es importante mantener las hiperaristas largas lo más verticales posible. Por todo ello, se ha optado por implementar el algoritmo de *Brandes and Köpf* presentado en [6], que es uno de los algoritmos más eficientes para asignar coordenadas horizontales a DAGs por capas, con una complejidad de $O(|V| + |E|)$. No obstante, [6] contenía una errata que fue corregida en [7], por lo que se ha implementado la versión corregida. De nuevo, en el Apéndice D.3 se explica con más detalle el algoritmo de *Brandes and Köpf* y cómo se ha implementado en la aplicación.

Si no se representaran los hipergrafos de manera ortogonal, realmente con los dos pasos anteriores el proceso habría concluido, pues el resto sería unir con líneas rectas. Sin embargo, al representar los hipergrafos de forma ortogonal, es necesario aplicar pasos adicionales. Una vez conocidas las coordenadas de cada nodo, el objetivo ahora consiste en organizar las hiperaristas de forma que no ocurran solapamientos entre ellas. La prioridad es que el grafo sea lo más legible posible.

A partir de aquí, el concepto de segmento vuelve a adquirir su definición geométrica y, cuando se hable de segmentos verticales u horizontales, se estará haciendo referencia a los segmentos (líneas) que forman la representación visual de las hiperaristas.

6.4. Ordenación de hiperaristas

Lo primero que se evitará son los solapamientos entre los segmentos horizontales de las hiperaristas. Es evidente que estos solapamientos solo pueden suceder entre hiperaristas de una misma capa, luego las ordenaciones son independientes y se realizan capa por capa. Por otro lado, pudiera parecer que el orden de altura (de arriba a abajo) en que se muestran las hiperaristas es irrelevante, pero lo cierto es que, en función de este orden, pueden darse más o menos cruces entre hiperaristas (Figura 6.5). ¿Cómo es esto posible? Muy sencillo, si una hiperarista e_1 tiene un segmento horizontal que está más arriba que el segmento horizontal de otra hiperarista e_2 , entonces los segmentos verticales que parten de los orígenes de e_2 y llegan hasta el segmento horizontal de e_2 pueden cruzarse en el camino con el segmento horizontal de e_1 . Similarmente, los segmentos verticales que parten de los destinos de e_1 y llegan hasta el segmento horizontal de e_1 pueden cruzarse con el segmento horizontal de e_2 . El caso donde e_1 está por debajo de e_2 es completamente análogo.

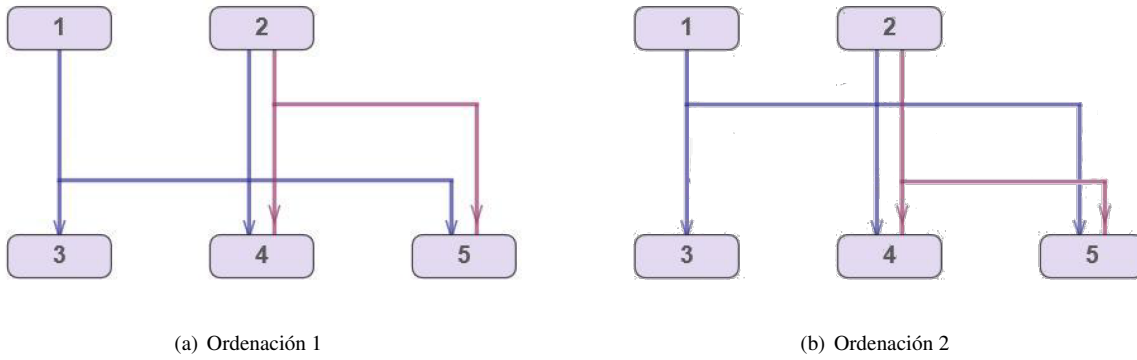


Figura 6.5: Ordenación de hiperarista afecta en los cruces

Para contabilizar estos cruces, se introduce el concepto de **span** de una hiperarista, que es el intervalo de coordenadas horizontales que abarca el segmento horizontal de dicha hiperarista. Dado que en este punto no se han resuelto los solapamientos entre segmentos verticales, se asume que cada uno de estos parte desde la abscisa del nodo origen o destino. Por tanto, dada una hiperarista e , $\text{span}(e) = [\min_{u \in S \cup T} x(u), \max_{u \in S \cup T} x(u)]$, donde $x(u)$ es la coordenada horizontal del nodo u .

Por tanto, dadas dos aristas e_1 y e_2 , el número de cruces entre ellas, que denotaremos por CT_{e_1, e_2} vendrá dado por:

$$CT_{e_1, e_2} = \begin{cases} |\{u \in S_2 : x(u) \in \text{span}(e_1)\}| + |\{u \in T_1 : x(u) \in \text{span}(e_2)\}| & \text{si } e_1 \text{ está por encima de } e_2 \\ |\{u \in S_1 : x(u) \in \text{span}(e_2)\}| + |\{u \in T_2 : x(u) \in \text{span}(e_1)\}| & \text{si } e_1 \text{ está por debajo de } e_2 \end{cases}$$

En consecuencia, el objetivo de este paso es minimizar la siguiente función: $\sum_{1 \leq i < j \leq |\mathcal{E}|} CT_{e_i, e_j}$.

Llevado esto a programación, dado que este paso mejora bastante la legibilidad de la representación, se ha optado por resolver el MIP asociado a este problema, que se presenta en [19]. Además,

se ha comprobado empíricamente que el MIP se resuelve en microsegundos para los casos de uso habituales y por debajo de un segundo para casos de gran tamaño, por lo que es perfectamente viable para un entorno de actualización dinámica. Adicionalmente, teniendo en cuenta que esta ordenación de hiperaristas es capa a capa, el número de capas afectadas tras una actualización es pequeño.

Para plantear el MIP, se introducen las siguientes variables binarias: HO_{e_i, e_j} que es igual a 1 si e_i está por encima de e_j y 0 en caso contrario. Además, como se conocen las coordenadas horizontales de cada nodo, se puede saber cuántos caen dentro del span de cada hiperarista. De esta forma, se introducen $acs(e_i, e_j) = |\{u \in S_j : x(u) \in \text{span}(e_i)\}|$ y $act(e_i, e_j) = |\{u \in T_j : x(u) \in \text{span}(e_i)\}|$, que son constantes. Así, el MIP, queda como sigue:

$$\sum_{1 \leq i < j \leq |\mathcal{E}|} CT_{e_i, e_j} \text{ es mínimo} \quad (6.1)$$

$$CT_{e_i, e_j} \geq acs(e_i, e_j) + act(e_j, e_i) - M \cdot HO_{e_i, e_j} \quad \forall 1 \leq i < j \leq |\mathcal{E}| \quad (6.2)$$

$$CT_{e_i, e_j} \geq acs(e_j, e_i) + act(e_i, e_j) - M \cdot (1 - HO_{e_i, e_j}) \quad \forall 1 \leq i < j \leq |\mathcal{E}| \quad (6.3)$$

$$0 \leq HO_{e_i, e_j} - HO_{e_i, e_k} + HO_{e_j, e_k} \leq 1 \quad \forall 1 \leq i < j < k \leq |\mathcal{E}| \quad (6.4)$$

$$CT_{e_i, e_j} \leq 0, \quad HO_{e_i, e_j} \in \{0, 1\} \quad \forall 1 \leq i < j \leq |\mathcal{E}| \quad (6.5)$$

donde M denota una constante suficientemente grande. La primera ecuación es la función objetivo, que busca minimizar el número total de cruces entre hiperaristas. La segunda y tercera ecuación linealizan la función descrita anteriormente para el número de cruces entre dos hiperaristas. Si $HO_{e_i, e_j} = 1$, entonces (6.2) se vuelve verdadera siempre ($CT_{e_i, e_j} \geq$ nº negativo) y (6.3) queda $CT_{e_i, e_j} \geq acs(e_i, e_j) + act(e_j, e_i)$. El caso $HO_{e_i, e_j} = 0$ es análogo. La cuarta ecuación garantiza la transitividad de la relación de orden entre las hiperaristas. Si e_i está por encima de e_j y e_j está por encima de e_k , entonces e_i debe estar por encima de e_k . Por último, la quinta ecuación describe el dominio de las variables.

6.5. Asignación de puertos

El siguiente paso es asignar puertos a cada nodo, que son las abscisas desde las que partirán los segmentos verticales de las hiperaristas. Así pues, cuando un nodo actúe como origen de una hiperarista, el segmento vertical partirá desde la abscisa asignada al puerto y desde la parte de abajo de la caja que representa al nodo, mientras que cuando un nodo actúe como destino de una hiperarista, el segmento vertical llegará hasta la abscisa asignada al puerto y hasta la parte de arriba de la caja que representa al nodo. Al primer caso lo denominaremos como puerto origen y al segundo caso como puerto destino, como se muestra en la Figura 6.6.

Por otro lado, hay que tener especial cuidado con los nodos *dummy* que se introducen para dividir las hiperaristas largas, pues estos no tienen una representación en caja, sino que se representan como

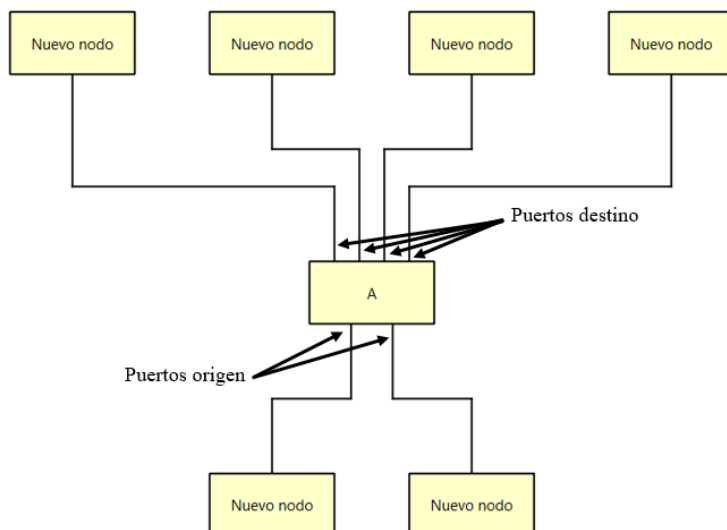


Figura 6.6: Puertos origen y puertos destino

la continuación vertical de una hiperarista larga. Por construcción, los nodos *dummy* tienen siempre exactamente un puerto origen y un puerto destino y es deseable que las abscisas otorgadas a estos puertos sean iguales, para no tener que introducir un codo horizontal uniéndolos, lo que haría que la hiperarista larga no tuviera una representación vertical.

La asignación de puertos es el paso que permite eliminar los solapamientos entre segmentos verticales de las hiperaristas que parten o llegan a un mismo nodo, hecho que refleja la Figura 6.7. Evidentemente, para que haya solapamiento de puertos en un mismo nodo, estos tienen que ser del mismo tipo, es decir, o ambos puertos son origen o ambos puertos son destino. De esta forma, asignaremos de manera independiente los puertos origen y los puertos destino de cada nodo.

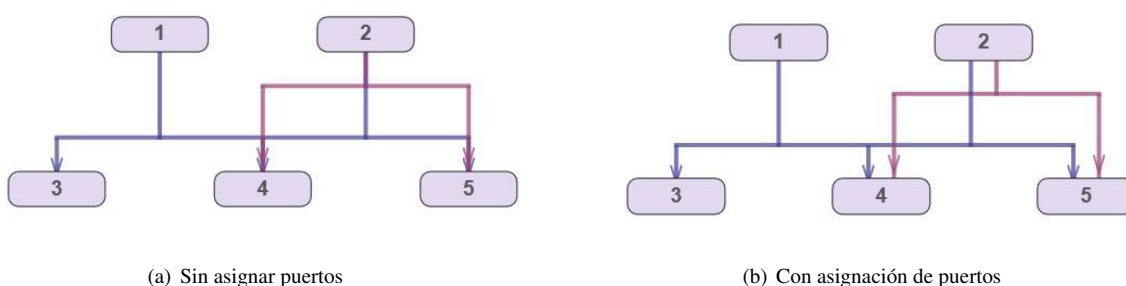


Figura 6.7: Solapamientos entre segmentos verticales

La idea será repartir el espacio horizontal de cada nodo entre el número de puertos, pero el orden en que estos se dispongan también es importante, pues puede generar más o menos cruces entre hiperaristas. Por ejemplo, en la Figura 6.7(b), si el puerto destino de la hiperarista roja en el nodo 5 se hubiera colocado a la izquierda del puerto destino de la hiperarista azul, entonces el segmento vertical de la hiperarista roja se cruzaría con el segmento horizontal de la hiperarista azul, lo que empeoraría

la legibilidad de la representación. Por tanto, el objetivo de este paso es asignar puertos a cada nodo de forma que se minimicen los cruces entre hiperaristas.

Para ello, será necesario distinguir entre tres tipos de puertos, que quedará determinado por la posición del nodo asociado dentro de la hiperarista:

- **Tipo 0:** el nodo es el nodo más a la izquierda de la hiperarista, entonces el puerto correspondiente deberá ubicarse lo más a la derecha posible en el extremo de la caja del nodo, para disminuir los cruces.
- **Tipo 1:** Si el nodo no es ni el más a la izquierda ni el más a la derecha, entonces se le asignará un puerto que se halle en una posición intermedia.
- **Tipo 2:** Si el nodo es el nodo más a la derecha de la hiperarista, entonces el puerto correspondiente deberá ubicarse lo más a la izquierda posible en el extremo de la caja del nodo, para disminuir los cruces.

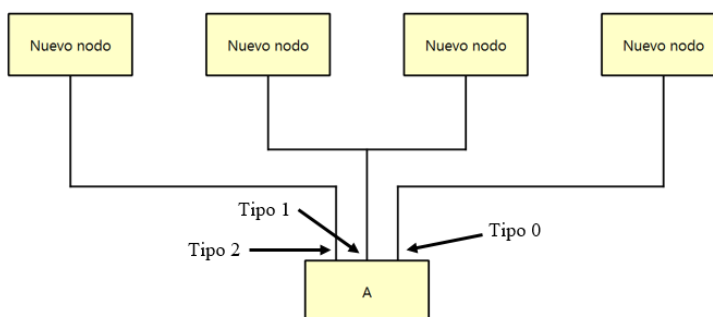


Figura 6.8: Tipos de puertos

El proceso de ordenación es el siguiente, los puertos de tipo 2 se sitúan a la izquierda del nodo, los puertos de tipo 0 se sitúan a la derecha y los puertos de tipo 1 se sitúan entre medias. Por otra parte, pueden surgir empates para puertos del mismo tipo. Para resolver estos empates, se opta por la opción que menos cruces genere.

- **Conflictos de tipo 0:** La manera de proceder es distinta en función de si se trata de un puerto origen o un puerto destino. En el caso de un puerto origen, el puerto más a la derecha será el que tenga la hiperarista con el segmento horizontal más elevado, mientras que en el caso de un puerto destino, el puerto más a la derecha será el que tenga la hiperarista con el segmento horizontal más bajo. De esta forma, se minimizan los cruces entre hiperaristas.
- **Conflictos de tipo 2:** El proceso es completamente análogo al caso de tipo 0, pero a la inversa. En el caso de un puerto origen, el puerto más a la izquierda será el que tenga la hiperarista con el segmento horizontal más bajo y viceversa en el caso de un puerto destino.

- **Conflictos de tipo 1:** En este caso, el orden de los puertos es completamente indiferente.

En la Figura 6.6 se puede apreciar el porqué de esta política. Si tomamos los puertos destino del nodo A como ejemplo, nótese que hay dos puertos de tipo 2 y dos de tipo 0. En el caso de los puertos de tipo 2, si el que tiene la horizontal más alta se hubiera colocado a la derecha, entonces se generaría un cruce; mientras que con la política aplicada, se evita. Lo mismo ocurre para los otros dos puertos de tipo 0.

Por último, cabe destacar que la asignación de puertos evita los solapamientos entre segmentos verticales de un mismo nodo, pero no entre segmentos verticales de nodos distintos. Como se ve en la Figura 6.9, el segmento vertical de la arista roja que emana del nodo 2, se solapa con el segmento vertical de la arista azul que llega al nodo 5. Para evitar esto, en [19] se propone un MIP para resolver este problema, desplazando ligeramente los nodos para evitar este tipo de situaciones.

No obstante, dada la rareza de este tipo de casos (por la elección del algoritmo de asignación de coordenadas) y el hecho de que, como norma general, se espera un número reducido de puertos por nodo, se ha optado por un procedimiento donde se detectan estos solapamientos y se resuelven de manera local, desplazando ligeramente los puertos implicados para evitar el solapamiento a cambio de reducir la distancia entre puertos. Sobre cómo se ha abordado algorítmicamente esta cuestión y algunos otros detalles de la asignación de puertos que no se han mencionado, se recomienda consultar el Apéndice D.4.

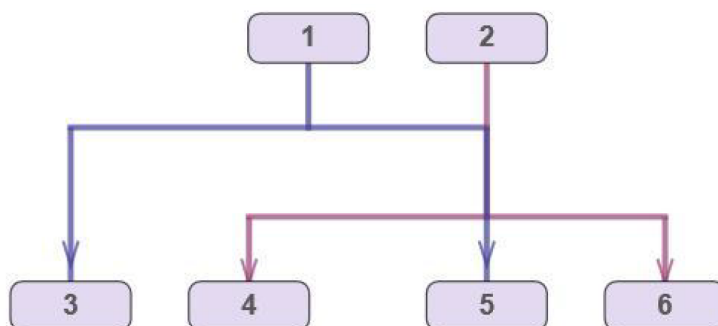


Figura 6.9: Solapamientos entre segmentos verticales de nodos distintos

6.6. Asignación de coordenadas verticales

Ahora, una vez se conocen las abscisas de cada segmento vertical de cada hiperarista, se conoce la extensión horizontal de cada hiperarista. Además, al tener también la ordenación de arriba a abajo de las hiperaristas, el último paso que queda es asignar coordenadas verticales a cada segmento horizontal de cada hiperarista, de forma que se minimice la altura del dibujo y se eviten solapamientos entre segmentos horizontales de una misma capa. A continuación, se muestra el algoritmo empleado:

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$ con las hiperaristas de cada capa ordenadas de arriba a abajo, y las coordenadas x de los puertos de cada nodo ya calculadas.

Output: Para cada capa ℓ , su coordenada y_ℓ (la altura a la que se dibuja su fila de nodos). Para cada hiperarista $e \in \mathcal{E}$ no trivial, su coordenada $\bar{y}(e)$ (la altura a la que se dibuja su segmento horizontal).

```

1   $y_0 \leftarrow 0$ ;
2  foreach capa  $\ell$  en orden creciente do
    //  $E_\ell$ : hiperaristas ordenadas que cruzan las capas  $(\ell-1, \ell)$ .
3    if  $\ell = 0$  or  $E_\ell = \emptyset$  then
4         $y_\ell \leftarrow y_{\ell-1} - \delta_{\text{nodo}} - \delta_{\text{gap}}$ ;
5        continue
6    end
7     $y_{\text{top}} \leftarrow y_{\ell-1} - \frac{\delta_{\text{nodo}}}{2} - \delta_{\text{gap}}$ ;
8     $\text{niveles} \leftarrow [(y_{\text{top}}, [])]$ ;
9    foreach  $e \in E_\ell$  do
10       if  $x_{\text{mín}}(e) = x_{\text{máx}}(e)$  then
11            $\bar{y}(e) \leftarrow y_{\ell-1} - \frac{\delta_{\text{nodo}}}{2}$ ;
12       else
13            $\text{mejor} \leftarrow \emptyset$ ;
14           for  $i \leftarrow |\text{niveles}| - 1$  to 0 do
15               if  $[x_{\text{mín}}(e), x_{\text{máx}}(e)]$  no solapa ninguna barra de  $\text{niveles}[i]$  then
16                    $\text{mejor} \leftarrow i$ ; // libre -- seguir subiendo
17               else
18                   break; // conflicto - bajar
19               end
20           end
21           if  $\text{mejor} = \emptyset$  then
22               // Sin nivel libre: abrir uno nuevo más profundo.
23                $y_{\text{nuevo}} \leftarrow \text{niveles.back().y} - \delta_{\text{sep}}$ ;
24                $\text{niveles.push}((y_{\text{nuevo}}, [(x_{\text{mín}}(e), x_{\text{máx}}(e))]))$ ;
25                $\bar{y}(e) \leftarrow y_{\text{nuevo}}$ ;
26           else
27                $\text{niveles}[\text{mejor}].\text{barras.push}((x_{\text{mín}}(e), x_{\text{máx}}(e)))$ ;
28                $\bar{y}(e) \leftarrow \text{niveles}[\text{mejor}].y$ ;
29           end
30       end
31   end
32   // -- Fijar  $y_\ell$  para la fila de nodos de la capa  $\ell$  --
33   if  $\text{niveles}[0].\text{barras} = []$  then
34        $y_\ell \leftarrow y_{\ell-1} - \delta_{\text{gap}} - \delta_{\text{nodo}}$ ;
35   else
36        $y_\ell \leftarrow \text{niveles.back().y} - \delta_{\text{gap}} - \frac{\delta_{\text{nodo}}}{2}$ ;
37   end

```

Algoritmo 6.2: Asignación de coordenadas verticales a cada capa y a cada hiperarista.

En este algoritmo, se intenta colocar cada segmento horizontal de cada hiperarista lo más alto posible, siempre que no cause conflictos con hiperarista que la ordenación haya señalado que deben estar por encima de ella. Nótese que, en este ejercicio, aunque el orden haya establecido que una hiperarista e_1 debe estar por debajo de otra e_2 , si entre ellas no existe ningún conflicto, entonces e_1 puede estar incluso por encima de e_2 sin que esto cause ningún cruce entre ellas.

Otra parte destacable del algoritmo es que emplea tres constantes distintas de separación: δ_{gap} que es la separación mínima entre un nodo y una hiperarista o entre dos nodos de diferentes capas, δ_{sep} que es la separación mínima entre segmentos horizontales de una misma capa y δ_{nodo} que es la altura del rectángulo que representa a cada nodo.

Por último, nótese que la extensión de cada hiperarista se ha denotado con $x_{\min}(e)$ y $x_{\max}(e)$, que son las coordenadas horizontales de puerto más a la izquierda y del más a la derecha de la hiperarista e , respectivamente. Para aquellas hiperaristas donde ambos valores coinciden, el segmento horizontal se reduce a un punto, por lo que la hiperarista no es más que una conexión vertical y, por ende, no se tiene que preocupar por los solapamientos con otras hiperaristas.

Con todo esto, ya se conocen todas las coordenadas necesarias para dibujar el hipergrafo, por lo que el proceso de representación automática ha concluido.

6.7. Lógica transaccional: hacer y deshacer

El último aspecto destacable de la implementación es la lógica transaccional que se ha implementado para poder hacer y deshacer cualquier operación. La elección del patrón Memento basado en copias profundas requiere una justificación, pues el enfoque alternativo de implementar operaciones inversas resultó inviable.

Entre otros, los responsables principales son los métodos `removeTransitiveConnections` y `applyRelocationAndPropagate`: el primero elimina conexiones redundantes por transitividad en función del estado global del grafo en ese instante, información que el estado posterior ya no contiene; el segundo desencadena una cascada de reubicaciones, disoluciones y reconstrucciones de segmentos cuyos pasos intermedios dependen unos de otros. Invertir cualquiera de los dos requeriría haber registrado cada paso intermedio con el suficiente detalle como para equivaler, en la práctica, a haber guardado casi el estado completo. Además, habría que acercar la lógica de deshacer y rehacer a la capa de dominio, lo que resultaría conceptualmente erróneo.

Ante esto, la copia profunda previa a cada mutación es la única solución que garantiza la corrección del deshacer en todos los casos posibles, eso sí, con un coste en memoria asumible para los tamaños de grafo habituales.

VALIDACIÓN Y RESULTADOS

7.1. Testing

Las pruebas unitarias constituyen una parte esencial del proceso de validación de esta aplicación. El motor de *layout* de hipergrafos que sustenta el sistema es un componente de alta complejidad algorítmica en el que un error sutil en cualquier función puede propagarse a lo largo de toda la cadena de representación y producir resultados incorrectos difíciles de depurar visualmente. A esto se suma el carácter dinámico del hipergrafo: cada inserción, eliminación o modificación de nodos y aristas desencadena una cascada de actualizaciones estructurales, como la reubicación de nodos, la disolución y reconstrucción de segmentos o la eliminación de conexiones transitivamente redundantes. Cada una de estas operaciones es un punto potencial de fallo que debe ser verificado de forma exhaustiva.

Las pruebas se han implementado utilizando el *framework* Google Test [21] y se organizan en dos módulos independientes que se corresponden con las capas de la arquitectura:

- Sobre la capa de dominio (`hypergraph_logic`) se han ejecutado un total de 431 pruebas unitarias.
- Sobre la capa de aplicación (`editors`) se han ejecutado un total de 117 pruebas unitarias.
- La capa de interfaz de usuario (`ui`) no se ha sometido a pruebas unitarias, sino que se ha comprobado su correcto funcionamiento mediante pruebas de integración y validación visual.

En total, la batería de pruebas comprende 548 pruebas unitarias, todas ellas superadas satisfactoriamente. El desglose por clase y módulo se puede encontrar en el Anexo E.1.

7.2. Pruebas de validación algorítmica y rendimiento

Dado que se han implementado algoritmos ya existentes desde cero, es preciso comprobar que su rendimiento coincide con el que se describe en las referencias originales. Para ello, además de realizar pruebas unitarias que comprobaban cada subalgoritmo de forma aislada, se han diseñado pruebas de

validación algorítmica que miden esta casuística. Asimismo, se han ejecutado pruebas de rendimiento para verificar que el coste de cada algoritmo de representación es *asumible*.

Global Sifting

En el caso de minimización de cruces, se ha realizado un estudio combinado. En este sentido, se ha tomado una batería de 100 DAGs de prueba sobre los que minimizar cruces y se han medido tanto el número de cruces resultante como el tiempo de ejecución. El banco de pruebas se ha extraído de la colección de la *ASP Competition* (véase [1]), una competición internacional centrada en la evaluación de algoritmos para problemas combinatorios complejos. Entre ellos, catalogado como “*Beyond NP*” (más difícil que NP), se encuentra una batería sobre el problema de minimización de cruces en un DAG de k -capas.

En la figura 7.1 se representa, para cada instancia del banco de pruebas, el número de cruces *antes* de la optimización (eje x) frente al número de cruces *después* de la optimización (eje y). Las instancias se han ordenado de menor a mayor número de cruces previos a la optimización. La recta de regresión lineal ajustada sobre los 100 puntos tiene pendiente $\hat{\beta} = 0,3296$, lo que indica que el algoritmo se aproxima a una reducción del 67 % en el número de cruces; dato que concuerda con el 65 % que se obtiene en la fuente original (véase [2]). Para más información sobre tiempos y rendimiento en cada instancia, consúltese el Anexo E.2.

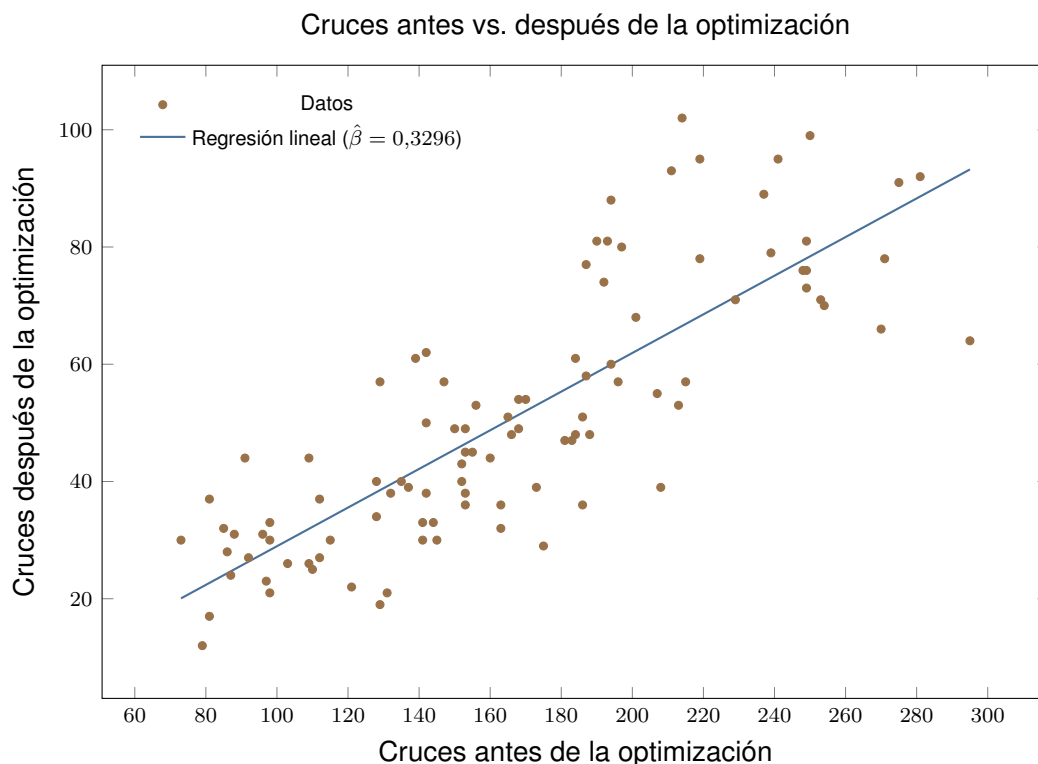


Figura 7.1: Resultados de la implementación de Global Sifting.

Brandes-Köpf

En este caso, se ha comprobado el correcto funcionamiento de la implementación de Brandes-Köpf de forma empírica, comparando el dibujo concreto de un grafo que los propios autores de la referencia original publican en su artículo como principal resultado (véase [6]). Esta comparativa se muestra en la Figura 7.2. Salvando el tamaño de las cajas, su nombre y las distancias inter-nodo e inter-capa, el resultado es prácticamente idéntico, lo que confirma la validez de la implementación.

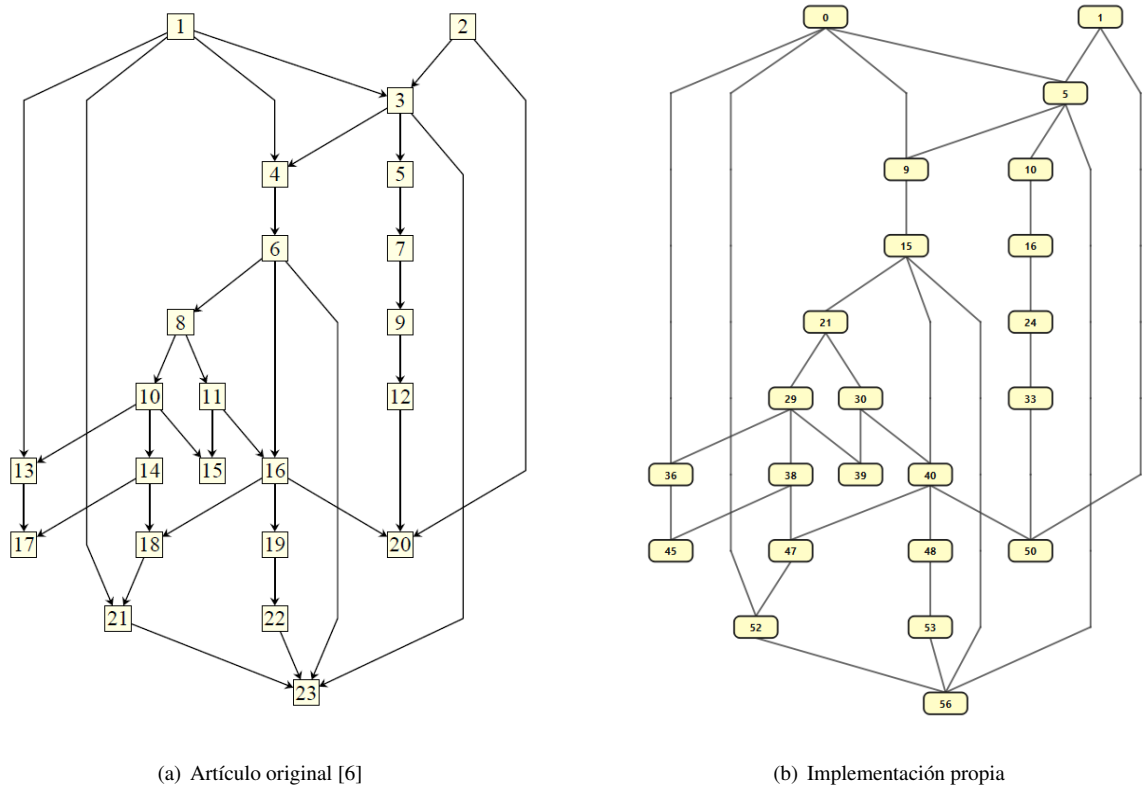


Figura 7.2: Resultados de la implementación de Brandes-Köpf.

Ordenación de hiperaristas

En la sección 6.4 se afirma que los tiempos de ejecución del MIP que resuelve el problema de ordenación de hiperaristas son *asumibles* para la aplicación. Para comprobar esta afirmación, se ha diseñado una prueba de rendimiento que mide el tiempo de ejecución del MIP en función del número de hiperaristas a ordenar. Además, también se mencionó la importancia de conseguir una buena ordenación para mejorar la legibilidad de la representación, por lo que también se evalúa el comportamiento.

En esta línea, se ha comparado el rendimiento del MIP frente al cálculo con fuerza bruta de todas las permutaciones posibles. Como la ordenación de hiperaristas es independiente para cada capa, basta evaluar el rendimiento para una sola capa. Por ello, se han generado 64 hipergrafos aleatorios

bicapa bajo la nomenclatura (n, e) donde n denota el número de nodos en las capas superior e inferior y e es el número de hiperaristas entre ambas capas. Es sobre este conjunto de DAGs sobre el que se han realizado todas las mediciones.

La Figura 7.3 muestra el número de cruces resultante para cada instancia, tanto para el MIP implementado como para la fuerza bruta. Se observa que, aunque el MIP no siempre consigue la mejor solución, sí que se aproxima a ella en la mayoría de los casos y, lo que es más importante, lo hace con tiempos de ejecución muy inferiores a los de fuerza bruta. Para más información sobre tiempos y rendimiento en cada instancia, consúltese el Anexo E.2.

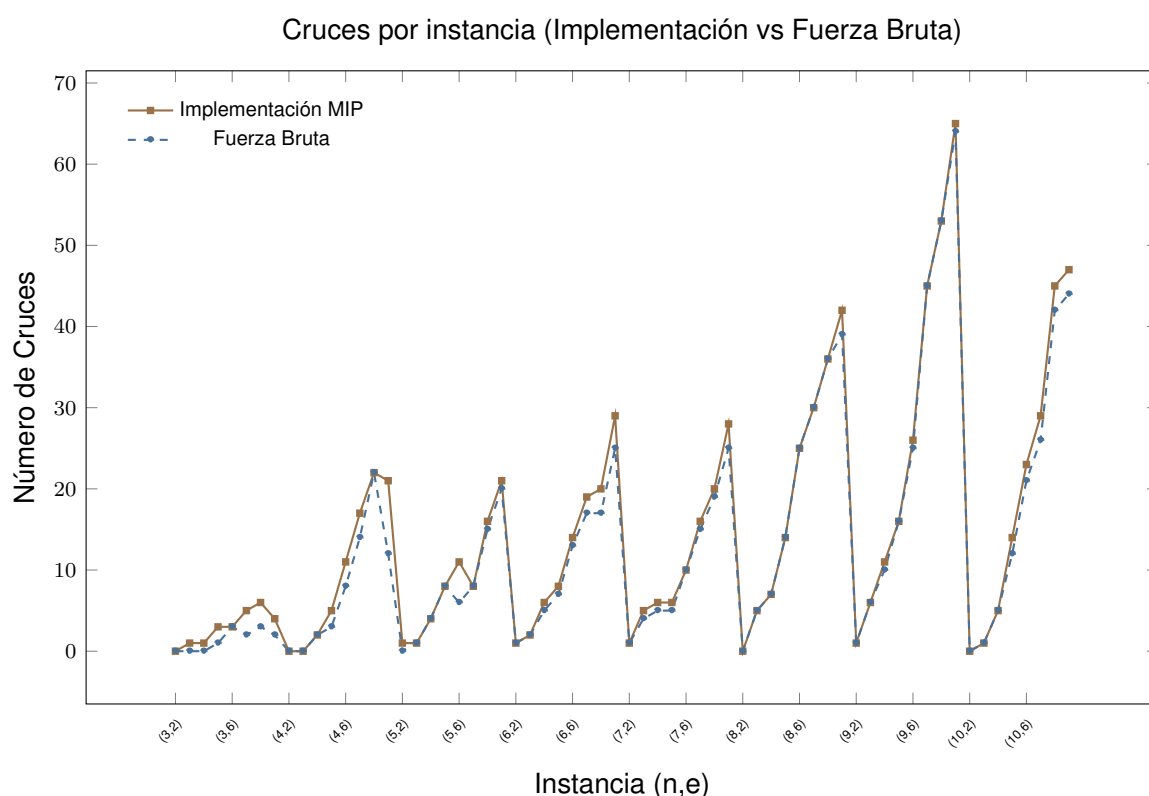


Figura 7.3: Implementación de ordenación de hiperaristas.

7.3. Encuesta de usabilidad

Para evaluar si la aplicación cumple con los requisitos funcionales y de experiencia de usuario establecidos, se ha diseñado y distribuido una encuesta de usabilidad entre un grupo reducido de usuarios. Esto es debido a que, al ser una aplicación muy especializada, resulta complicado encontrar expertos en el área que quieran dedicar tiempo a transmitir sus impresiones.

Para evaluar la usabilidad, se ha utilizado el cuestionario *System Usability Scale* (SUS), desarrollado por Brooke en [8]. De esta forma, se ha implementado la versión validada al castellano, atendiendo

a las propiedades psicométricas analizadas por Castilla et al. en [10], donde también presentan una plantilla de encuesta a utilizar cuando se desea aplicar un formulario de usabilidad breve, como es el caso.

La encuesta se ha estructurado en tres bloques diferenciados:

1. **Ejecución de tareas:** Para garantizar que los usuarios evaluaran la aplicación basándose en una experiencia de uso real, se les solicitó que realizaran una serie de tareas representativas del uso típico de la herramienta. Estas tareas fueron diseñadas para cubrir las funcionalidades clave de la aplicación.
2. **Cuestionario SUS:** Tras completar las tareas, los usuarios respondieron al cuestionario SUS, que consta de diecisiete ítems con afirmaciones positivas y negativas sobre la usabilidad del sistema. Cada ítem se evalúa en una escala Likert de 1 a 5, lo que permite obtener una puntuación global de usabilidad.
3. **Preguntas abiertas:** Para complementar la información cuantitativa obtenida a través del SUS, se incluyeron dos preguntas abiertas relativas a sugerencias de mejora y aspectos destacables de la experiencia de usuario. Estas preguntas permiten obtener visiones cualitativas que pueden ser muy valiosos para futuras mejoras.

Una vez desarrollada la encuesta, esta se sometió a una revisión por parte de un grupo de expertos para garantizar la claridad de las preguntas y la adecuación a la aplicación. Posteriormente, se creó un formulario digital en *Google Forms* y se distribuyó el enlace a un grupo de usuarios potenciales. La participación fue voluntaria y se garantizó la anonimidad de las respuestas.

Tras la recogida de datos, se procedió a su análisis. En conjunto, los resultados de la encuesta de usabilidad apuntan a que la aplicación es percibida como una herramienta usable y funcional. La puntuación SUS media obtenida en los doce ítems de usabilidad general es de **85,0 puntos** sobre 100, muy por encima del umbral de 68 que marca la usabilidad aceptable [8]. Los cinco ítems adicionales, orientados a evaluar la funcionalidad específica de la aplicación, obtuvieron una puntuación media de **96,0 puntos**, destacando especialmente la claridad visual del esquema (97,5), requisito en el que el cliente puso mayor énfasis.

En cuanto a las respuestas cualitativas, el aspecto más elogiado de forma recurrente fue la reordenación automática del diagrama,. Las sugerencias de mejora se concentran principalmente en dos líneas: la personalización visual de la interfaz y, de mayor relevancia para la adopción de la herramienta, la adaptación de la terminología al vocabulario estratigráfico arqueológico. Esta última línea constituye la principal área de mejora identificada de cara a futuras versiones.

CONCLUSIONES

El objetivo principal de este Trabajo de Fin de Grado es el desarrollo de una aplicación de escritorio para representar esquemas Matrix Harris, lo que en términos informáticos se traduce en la representación dinámica de hipergrafos jerárquicos de forma ortogonal. Tras completar el análisis, diseño, codificación y pruebas se pueden extraer las siguientes conclusiones.

La magnitud del problema de representación es considerable, como se ha demostrado en el análisis del estado del arte y reflejado en la complejidad de los algoritmos implementados. Sin embargo, se ha conseguido abordar este desafío adaptando ideas de literatura académica relevante y adoptando algoritmos clásicos de representación de grafos al problema en cuestión.

Tras un proceso de pruebas de validación exhaustivo, se puede aseverar que uno de los mayores requerimientos del cliente ha sido cumplido: las representaciones son claras, legibles y estéticamente agradables. Adicionalmente, se han incorporado el resto de funcionalidades solicitadas, lo que sitúa a la aplicación como una herramienta de gran utilidad para la creación de esquemas Matrix Harris.

La elección de separar la aplicación en tres capas independientes (dominio, aplicación e interfaz de usuario) ha sido un acierto, ya que ha permitido organizar el código de forma modular y facilitar el proceso de pruebas unitarias.

Por último, en cuanto a líneas de trabajo futuro, se podrían considerar las siguientes mejoras y ampliaciones:

- Implementar un sistema de lógica transaccional que no esté basado en copias profundas, lo que podría mejorar el rendimiento en esquemas de gran tamaño.
- Valorar otras estrategias de representación de hipergrafos jerárquicos, que podrían ofrecer mejores visualizaciones en determinados casos.
- Investigar mejoras para la ordenación de bloques inicial en el algoritmo de minimización de cruces.
- Explorar la posibilidad de desarrollar una versión web de la aplicación.

BIBLIOGRAFÍA

- [1] ASP Competition 2011, “Crossing minimization — final problem description,” 2011. <https://www.mat.unical.it/aspcomp2011> [Último acceso: mayo de 2026].
- [2] C. Bachmaier, F. J. Brandenburg, W. Brunner, and F. Hübner, “A global k -level crossing reduction algorithm,” in *Proceedings of the 17th International Symposium on Graph Drawing (GD 2009)*, vol. 5849 of *Lecture Notes in Computer Science*, pp. 70–81, Springer, 2009.
- [3] W. Barth, M. Jünger, and P. Mutzel, “Simple and efficient bilayer cross counting,” in *Proceedings of the 10th International Symposium on Graph Drawing (GD 2002)*, vol. 2528 of *Lecture Notes in Computer Science*, pp. 130–141, Springer, 2002.
- [4] M. Bastian, S. Heymann, and M. Jacomy, “Gephi: An open source software for exploring and manipulating networks,” in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, pp. 361–362, AAAI Press, 2009. <https://gephi.org/publications/gephi-bastian-feb09.pdf> [Último acceso: 10 de diciembre de 2025].
- [5] Anónimo, “Software para crear matrix de harris,” 2008. <https://arqueologiaypatrimonio.blogspot.com> [Último acceso: 3 de diciembre de 2025].
- [6] U. Brandes and B. Köpf, “Fast and simple horizontal coordinate assignment,” in *Proceedings of the 9th International Symposium on Graph Drawing (GD 2001)*, vol. 2265 of *Lecture Notes in Computer Science*, pp. 31–44, Springer, 2002.
- [7] U. Brandes, J. Walter, and J. Zink, “Erratum: Fast and simple horizontal coordinate assignment,” in *Graph Drawing and Network Visualization (GD 2020)*, vol. 12590 of *Lecture Notes in Computer Science*, pp. 433–435, Springer, 2021.
- [8] J. Brooke, “Sus: a “quick and dirty” usability scale,” in *Usability evaluation in industry* (P. W. Jordan, B. Thomas, I. L. McClelland, and B. Weerdmeester, eds.), pp. 189–194, London: Taylor & Francis, 1996. Originalmente publicado en 1986, es la referencia base del método SUS.
- [9] J. M. Briffa, “Free software for a harris matrix,” 2020. <https://jmbriffa.net> [Último acceso: 7 de diciembre de 2025].
- [10] D. Castilla, I. Jaen, C. Suso-Ribera, G. García-Soriano, I. Zaragoza, J. Bretón-López, A. Mira, A. Díaz-García, and A. García-Palacios, “Psychometric properties of the spanish full and short forms of the system usability scale (sus): Detecting the effect of negatively worded items,” *International Journal of Human–Computer Interaction*, vol. 40, no. 15, pp. 4145–4151, 2023.
- [11] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, “Efficient exact algorithms for maximum balanced biclique search in bipartite graphs,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, ACM, 2022.
- [12] M. Chimani, C. Gutwenger, M. Jünger, G. W. Klau, K. Klein, and P. Mutzel, “The open graph drawing framework (OGDF),” in *Handbook of Graph Drawing and Visualization* (R. Tamassia, ed.), ch. 17,

- pp. 543–569, CRC Press, 2014. <https://github.com/ogdf/ogdf>.
- [13] ISO/IEC, “ISO/IEC 14882:2020 — programming languages — C++,” standard, International Organization for Standardization, 2020. <https://www.iso.org/standard/79358.html>.
- [14] draw.io Ltd and draw.io AG, “diagrams.net,” 2020. <https://www.diagrams.net/> [Último acceso: 10 de diciembre de 2025].
- [15] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Algorithms for drawing graphs: an annotated bibliography,” June 1994.
- [16] M. Eiglsperger, M. Siebenhaller, and M. Kaufmann, “An efficient implementation of Sugiyama’s algorithm for layered graph drawing,” *Journal of Graph Algorithms and Applications*, vol. 8, no. 2, pp. 180–221, 2004.
- [17] OpenJS Foundation, “Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS,” 2024. <https://www.electronjs.org>.
- [18] J. Ellson, E. R. Gansner, E. Koutsofios, S. C. North, and G. Woodhull, “Graphviz — open source graph drawing tools,” in *Graph Drawing. GD 2001*, vol. 2265 of *Lecture Notes in Computer Science*, pp. 483–484, Springer, 2002.
- [19] G. Fridman, Y. Vasiliev, V. Puhkalo, and V. Ryzhov, “A mixed-integer program for drawing orthogonal hyperedges in a hierarchical hypergraph,” *Mathematics*, vol. 9, no. 2, pp. 1–16, 2021.
- [20] E. R. Gansner, E. Koutsofios, S. C. North, and K.-P. Vo, “A technique for drawing directed graphs,” *IEEE Transactions on Software Engineering*, vol. 19, no. 3, pp. 214–230, 1993.
- [21] Google LLC, “GoogleTest: Google testing and mocking framework,” 2024. <https://github.com/google/googletest>.
- [22] steveroush and erg, “Regarding graphviz’s “orthogonal” edge routing,” 2023. <https://forum.graphviz.org>.
- [23] Gurobi Optimization, LLC, “Gurobi Optimizer reference manual,” 2024. <https://www.gurobi.com>.
- [24] P. Healy and N. S. Nikolov, “Hierarchical drawing algorithms,” in *Handbook of Graph Drawing and Visualization* (R. Tamassia, ed.), ch. 13, pp. 409–453, CRC Press, 2013.
- [25] ArchPro, “Harris matrix composer,” 2024. <https://hmc.archpro.science/> [Último acceso: 10 de diciembre de 2025].
- [26] A. A. K. Ismaeel, *Dynamic Hierarchical Graph Drawing*. PhD thesis, Karlsruher Institut für Technologie (KIT), Karlsruhe, Germany, 2012.
- [27] OpenJFX contributors, “JavaFX — open source client application platform,” 2024. <https://openjfx.io>.
- [28] J. San Andrés, “Matrix-Harris App: Repositorio principal,” 2026. <https://github.com/javiersanandres/Matrix-Harris-app>.
- [29] F. J. Newbery, “Edge concentration: A method for clustering directed graphs,” in *Proceedings of the Workshop on Visual Languages*, IEEE, 1989.
- [30] The Qt Company, “Qt framework, version 6,” 2024. <https://www.qt.io>.
- [31] Reddit — r/Archaeology, “Anyone know a good program for creating a harris matrix?,” 2012.

- <https://www.reddit.com/r/Archaeology> [Último acceso: 5 de diciembre de 2025].
- [32] G. Sander, "Layout of directed hypergraphs with orthogonal hyperedges," in *Proceedings of the 11th International Symposium on Graph Drawing (GD 2003)*, vol. 2912 of *Lecture Notes in Computer Science*, Springer, 2004.
- [33] J. G. Siek, L.-Q. Lee, and A. Lumsdaine, *The Boost Graph Library: User Guide and Reference Manual*. Addison-Wesley, 2002.
- [34] I. Herzog, "Stratify: Check of stratigraphic data and harris diagram," 2004. <http://www.stratify.org/> [Último acceso: 10 de diciembre de 2025].
- [35] K. Sugiyama, S. Tagawa, and M. Toda, "Methods for visual understanding of hierarchical system structures," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.
- [36] C. Traxler and W. Neubauer, "The Harris Matrix Composer — a new tool to manage archaeological stratigraphy," in *Digital Heritage: Proceedings of the 14th International Conference on Virtual Systems and Multimedia*, pp. 13–20, 2008. <https://www.cg.tuwien.ac.at/research/publications/2008/TRA08/> [Último acceso: 10 de diciembre de 2025].
- [37] Y. M. Vasiliev and G. M. Fridman, "Cooperation scheme visualization: Hyperedge routing method for hierarchical multilayer hypergraph," *Sovremennye Ekonomicheskie Problemy i Resheniia*, vol. 3, pp. 18–33, 2017. In Russian.
- [38] wxWidgets contributors, "wxWidgets: Cross-platform GUI library, version 3.2," 2024. <https://www.wxwidgets.org>.

APÉNDICES

ESTADO DEL ARTE

En este apéndice, se completan las informaciones del análisis comparativo de la competencia y el análisis tecnológico del texto principal, que se han tenido en cuenta a la hora de diseñar y desarrollar Matrix-Harris App. Se divide en dos secciones por propia naturaleza: la primera se centra en el análisis comparativo de las aplicaciones existentes para representar esquemas de Matrix Harris, mientras que la segunda se enfoca en el análisis de las tecnologías disponibles para el desarrollo de la aplicación.

A.1. Análisis comparativo de la competencia

Este análisis comparativo se ha realizado con el objetivo de identificar las fortalezas y debilidades de las aplicaciones existentes en el mercado que ofrecen funcionalidades similares a las que se implementan en este proyecto. Muchas de las aplicaciones analizadas han sido directamente señaladas por el cliente, con su respectiva justificación de qué aspectos no le gustan. Además, también se han tenido en cuenta otras aplicaciones sugeridas en distintos foros o blogs [5], [9] y [31].

Por otra parte, muchas de las mejoras que alumbren las aplicaciones analizadas serán incorporadas a la versión final de Matrix-Harris App. Otras, sin embargo, pueden verse como futuros proyectos de extensión derivados de este trabajo, que podrían ser abordados por otros estudiantes en el futuro. Por lo pronto y ya anticipando el análisis de las aplicaciones, no parece haber un software dedicado para lo que el cliente necesita, por lo que el proyecto de desarrollo de Matrix-Harris App se justifica plenamente.

diagrams.net (draw.io)

diagrams.net [14] es una herramienta de diagramación de propósito general, de código abierto y uso gratuito, disponible tanto como aplicación web como aplicación de escritorio. Ofrece soporte para una gran variedad de tipos de diagrama (flujo, UML, ER, etc.) y permite construir grafos dirigidos mediante una interfaz de arrastrar y soltar.

Tabla A.1: Análisis de diagrams.net (draw.io)

Aspectos favorables	Aspectos desfavorables	Mejoras para Matrix-Harris App
– Exportación a múltiples formatos (PNG, SVG, PDF, XML). – Disponible como aplicación web sin necesidad de instalación, accesible desde cualquier dispositivo con navegador. – Integración directa con Google Drive y OneDrive, permitiendo guardar y compartir diagramas en la nube de forma inmediata. – Soporte de colaboración en tiempo real entre múltiples usuarios sobre el mismo diagrama.	– El posicionamiento de nodos y aristas es completamente manual. – No incorpora el concepto de hiperarista, solo aristas binarias. – No dispone de funcionalidades específicas para la Matrix Harris: no valida la acicilicidad ni aplica la propiedad de Hasse. – No ofrece ninguna vista conjunta que permita comparar o combinar varios diagramas en paralelo.	– Incorporar exportación del diagrama a formatos de imagen (PNG, SVG) y PDF. – Incluir plantillas de proyectos predefinidas con estructuras estratigráficas típicas.

Stratify

Stratify [34] es una herramienta de escritorio desarrollada específicamente para la gestión y representación de esquemas Matrix Harris en contextos arqueológicos. Fue diseñada para sistemas Windows de generaciones anteriores y no ha recibido actualizaciones significativas desde hace varios años. La última fue del 2008 y su desarrollo se ha abandonado, haciéndola incompatible con Windows 11. No ha sido posible probarla directamente, pero he recopilado información sobre sus funcionalidades en la propia web.

Tabla A.2: Análisis de Stratify

Aspectos favorables	Aspectos desfavorables	Mejoras para Matrix-Harris App
– Permite agrupar unidades estratigráficas y establecer conexiones entre ellas. Además, establece una especie de base de datos por cada unidad. – Permite exportar a PDF y a formatos de imagen (BMP, JPG). – Tiene comprobaciones de consistencia para detectar ciclos o relaciones contradictorias en el esquema.	– Incompatible con Windows 11 y versiones recientes del sistema operativo; no puede ejecutarse en equipos modernos. – Software abandonado y sin mantenimiento activo: no recibe actualizaciones ni correcciones de errores. – Interfaz visual anticuada que no cumple los estándares de usabilidad actuales. – No soporta hiperaristas: las relaciones se representan como aristas binarias. – No dispone de vista conjunta ni de ningún mecanismo para comparar varios esquemas en paralelo.	– Asociar a cada nodo del hipergrafo una ficha de metadatos editable (descripción de la unidad, tipo de contexto, arqueólogo responsable, fecha de registro), de modo que el diagrama actúe también como base de datos estratigráfica básica.

Harris Matrix Composer (HMC)

Harris Matrix Composer [25] es una herramienta de escritorio desarrollada orientada a arqueólogos profesionales. Su acceso está sujeto a una licencia comercial de coste elevado; aunque presenta una demostración gratuita.

Tabla A.3: Análisis de Harris Matrix Composer (HMC)

Aspectos favorables	Aspectos desfavorables	Mejoras para Matrix-Harris App
– Permite la gestión de fases y periodos cronológicos con agrupación visual automática de las unidades contemporáneas y asignación de dataciones. – Incluye herramientas de análisis de la secuencia (cálculo del camino crítico, identificación de la secuencia mínima) que ayudan al arqueólogo a interpretar la historia deposicional. – Genera automáticamente una lista numerada de todas las unidades del proyecto con su estado de registro y sus relaciones, exportable como informe.	– Requiere una licencia comercial de coste elevado, inaccesible para la mayoría de proyectos académicos y equipos con recursos limitados. – No produce hipergrafos: el modelo de datos es un DAG estándar con aristas binarias, impidiendo representar relaciones de muchos a muchos. – No está muy automatizado: los nodos han de ser depositados de manera manual. – No incorpora funcionalidad de fusión de unidades estratigráficas.	– Añadir herramientas de análisis de la secuencia estratigráfica, como el cálculo del camino más largo (secuencia mínima), la identificación de nodos clave o el resaltado automático de las cadenas de relaciones entre dos unidades seleccionadas. – Ampliar el modelo de relaciones para incluir, además de anterioridad y posterioridad, relaciones de contemporaneidad entre unidades, representadas con un trazo manual.

Gephi

Gephi [4] es una plataforma de código abierto para la visualización y el análisis de redes y grafos complejos, ampliamente utilizada en ciencias sociales, bioinformática y análisis de redes de datos. Aunque no está orientada específicamente a la Matrix Harris, sus capacidades avanzadas de visualización y análisis la convierten en una referencia útil para identificar funcionalidades aprovechables en Matrix-Harris App.

Tabla A.4: Análisis de Gephi

Aspectos favorables	Aspectos desfavorables	Mejoras para Matrix-Harris App
– Cómodo de usar y con muchas opciones para personalizar la apariencia del grafo, incluyendo diferentes tipos de layout. – Permite aplicar filtros dinámicos sobre el grafo para centrar la visualización en subconjuntos de nodos de interés.	– No está diseñada para el dominio arqueológico y carece de las restricciones propias de un esquema Harris. – No soporta hiperaristas: el modelo de datos es un grafo estándar con aristas binarias. – El enrutamiento ortogonal de aristas no está disponible. – Interfaz orientada a usuarios técnicos con conocimientos de análisis de redes; la curva de aprendizaje es elevada.	– Añadir un sistema de filtrado dinámico que permita resaltar ciertos nodos por encima de otros para que el usuario pueda decidir qué partes del grafo quiere visualizar en cada momento. – Permitir que el usuario asigne colores diferenciados a los nodos según su tipo.

A.2. Análisis comparativo de las Tecnologías

En esta sección, se justificarán las decisiones tecnológicas adoptadas durante el desarrollo de la aplicación. En primer lugar, se discutirá la librería gráfica y el lenguaje de programación elegidos. En segundo lugar, se analizarán las librerías de dibujo de grafos y los solvers de programación lineal entera mixta que podrían haber servido de base algorítmica. Es importante recalcar que estas decisiones están fuertemente ligadas a cómo resolvemos el problema de representación, siguiendo el modelo que se expone en [19].

A.2.1. Librería gráfica y lenguaje de programación

El proceso de selección tecnológica comenzó por la librería gráfica y, condicionado a esa elección, se determinó el lenguaje de programación. Los criterios de búsqueda fueron: soporte nativo para trazado de líneas con formas complejas, funcionalidades integradas de zoom, *pan* y menús contextuales sobre elementos individuales, y portabilidad entre plataformas.

Librerías gráficas candidatas

wxWidgets.

wxWidgets [38] es un *framework* muy utilizado a la hora de desarrollar aplicaciones de escritorio. Sin embargo, wxWidgets no tiene un modelo de escena orientado a objetos, lo que significa que toda la lógica de interacción tendría que estar configurada manualmente.

Electron / aplicación web (HTML5 Canvas o SVG).

Una alternativa radicalmente diferente habría sido construir la aplicación como una aplicación web empaquetada con Electron [17] o similar, utilizando HTML5 Canvas o SVG para el renderizado. No obstante, el cliente abogaba más por una aplicación de escritorio tradicional, y esta opción introduce una complejidad en la distribución.

JavaFX.

JavaFX [27] fue otro de los candidatos considerados. Es un *framework* gráfico moderno para Java y eso es lo que lo limita principalmente: el lenguaje Java. Tendría que obligar a mi cliente a instalar una máquina virtual de Java, lo cual ya es un poco tedioso per se.

Elección: Qt

Qt [30] fue seleccionado como *framework* gráfico por ser la opción que cubre de forma más completa y natural los dos requisitos gráficos fundamentales del proyecto. Permite dibujar los *paths* de una manera cómoda, que cada objeto de la escena tenga su propia lógica de interacción y tiene incorporada de forma nativa funcionalidades como zoom y pan. Además, Qt es una biblioteca moderna y con una portabilidad excelente, lo que facilita el desarrollo de la aplicación. Además, su distribución es también bastante sencilla, lo que situó a Qt como la opción más adecuada para este proyecto.

Elección del lenguaje: C++20

Una vez elegido Qt como *framework* gráfico, la elección del lenguaje quedó en gran medida determinada. Qt está escrito en C++ y su API más natural, completa y documentada es la de C++. Bien es cierto que Qt también ofrece enlaces para otros lenguajes como Python.

No obstante, más allá de Qt, en cierta etapa del *pipeline* se requería el uso de un *solver* de programación lineal entera mixta (MIP), para el que se seleccionó Gurobi, como se justifica más adelante. En ello, Gurobi está escrito en C y la integración que menos latencia y overhead introduce es la de C/C++.

En otro orden de cosas, a lo largo del devenir del proyecto, se hace necesario definir varios tipos de datos personalizados y estructuras de datos, lo que es más propio de un lenguaje de bajo nivel como

C++ que de un lenguaje interpretado como Python.

Finalmente, el rendimiento es un factor a tener en cuenta, aunque no es crítico. C++ es un lenguaje de alto rendimiento y permite optimizar el código de manera más efectiva que otros lenguajes de más alto nivel, lo que puede ser beneficioso para garantizar una experiencia de usuario fluida, especialmente en diagramas complejos. No obstante, es posible que el usuario no note diferencias de rendimiento significativas entre una implementación en C++ y una en Python, pero puede llegar a ser un factor a considerar en escenarios con diagramas muy grandes o con muchos elementos interactivos.

A.2.2. Librerías de dibujo de grafos y solvers MIP

Graphviz

Graphviz [18] es la herramienta de dibujo automático de grafos de referencia en entornos académicos e industriales. Su motor `dot` implementa el marco de Sugiyama para grafos dirigidos jerárquicos y produce resultados de gran calidad. No obstante, según destaca uno de los desarrolladores de Graphviz en el foro oficial del proyecto [22], el modo de enrutamiento ortogonal de Graphviz no funciona cuando se especifican puertos de conexión en los nodos, lo que es un requisito fundamental para el modelo de representación de Matrix-Harris App.

OGDF (Open Graph Drawing Framework)

OGDF [12] es una biblioteca de C++ desarrollada y mantenida por varias universidades europeas (Osnabrück, Passau, Konstanz, TU Viena, TU Ilmenau) que implementa una amplia colección de algoritmos de dibujo de grafos, incluyendo el marco de Sugiyama completo. A priori, OGDF podría haber sido utilizado para delegar la implementación de los dos algoritmos más complejos del *pipeline*: Global Sifting y Brandes-Köpf. Sin embargo, ambas implementaciones presentan problemas que las hacen inutilizables en el contexto de este proyecto. Además, fue necesario implementar personalizaciones específicas de ambos algoritmos que no están expuestas en las interfaces públicas de OGDF, lo que habría resultado considerablemente más tedioso.

Boost.Graph

La biblioteca Boost.Graph [33] es la referencia en C++ para algoritmos de teoría de grafos de propósito general. Sin embargo, no implementa ningún algoritmo del marco de Sugiyama. Podría haber sido útil para el modelo de datos subyacente del hipergrafo, pero al ser este personalizado y específico, adaptar las estructuras de datos de Boost.Graph habría introducido una complejidad adicional innecesaria.

Solvers MIP: elección de Gurobi

En cierta etapa del *pipeline* de *layout* se necesita resolver un MIP (Mixed Integer Program) tal y como se describe en el artículo de referencia [19]. Este paso es importante porque influye enormemente en la legibilidad del diagrama, por lo que se requería un *solver* eficiente con buena integración en C++. La opción más adecuada es Gurobi [23] y, de hecho, es el que Fridman et al. emplean en su implementación de referencia. Por ese motivo, se decidió Gurobi aun siendo este un software comercial, aunque con licencia académica gratuita. En futuras versiones de la aplicación, se podría plantear sustituir Gurobi por un *solver* de código abierto; pero de momento Gurobi es la opción más adecuada para garantizar un buen rendimiento y resultados de calidad.

ANÁLISIS DEL SISTEMA

B.1. Catálogo de Requisitos funcionales

A continuación se detallan los requisitos funcionales de la aplicación Matrix-Harris:

- **RF-1.** El sistema permitirá al usuario crear un nuevo proyecto vacío, el cual contendrá un diagrama inicial vacío y un diagrama conjunto (*joint*) también vacío.
- **RF-2.** El sistema permitirá al usuario guardar el estado completo del proyecto (incluyendo todos los diagramas y el diagrama conjunto) en un fichero, eligiendo la ruta de destino mediante un diálogo del sistema operativo.
 - **RF-2.1.** Una vez que el proyecto haya sido guardado por primera vez, el sistema permitirá al usuario guardar los cambios sobre el fichero existente sin necesidad de volver a seleccionar la ruta.
- **RF-3.** El sistema permitirá al usuario abrir un proyecto previamente guardado seleccionando el fichero correspondiente, restaurando el estado de todos los diagramas tal y como se encontraban en el momento del guardado.
- **RF-4.** El sistema informará al usuario de la existencia de cambios no guardados antes de cerrar la aplicación o abrir un proyecto diferente, solicitando confirmación para continuar.
- **RF-5.** El sistema permitirá al usuario añadir nuevos diagramas al proyecto, cada uno con un nombre único generado automáticamente que el usuario podrá modificar.
- **RF-6.** El sistema permitirá al usuario eliminar un diagrama del proyecto, excepto el diagrama conjunto, que no puede ser eliminado y siempre permanece presente.
- **RF-7.** El sistema permitirá al usuario renombrar cualquier diagrama existente del proyecto, incluyendo el diagrama conjunto.
- **RF-8.** El sistema permitirá al usuario navegar entre los diagramas del proyecto y el diagrama conjunto mediante una barra de pestañas, mostrando en el área central el diagrama activo en cada momento.

- **RF-9.** El sistema permitirá al usuario crear un nodo raíz nuevo en el diagrama activo y asignándole una etiqueta introducida por el usuario.
 - **RF-9.1.** En función de dónde el usuario haga clic para crear dicho nodo, el sistema lo ubicará en la posición que más se ajuste a la ubicación del clic dentro de la capa raíz del diagrama.
- **RF-10.** El sistema permitirá al usuario crear un nodo nuevo como hijo de un nodo existente.
 - **RF-10.1.** El sistema ubicará automáticamente el nodo nuevo en la capa inmediatamente inferior a la del nodo padre, en la posición que menos conflictos de cruces genere con respecto a las conexiones ya existentes y con un sesgo hacia la derecha.
 - **RF-10.2.** El sistema permitirá al usuario añadir una etiqueta al nuevo nodo introducido.
- **RF-11.** El sistema permitirá al usuario crear un nodo nuevo como padre de un nodo existente.
 - **RF-11.1.** El sistema ubicará automáticamente el nodo nuevo en la capa raíz, en la posición que menos conflictos de cruces genere con respecto a las conexiones ya existentes y con un sesgo hacia la derecha.
 - **RF-11.2.** El sistema permitirá al usuario añadir una etiqueta al nuevo nodo introducido.
- **RF-12.** El sistema permitirá al usuario insertar un nodo nuevo en el interior de una hiperarista existente, dividiéndola en dos.
 - **RF-12.1.** El sistema ubicará automáticamente el nodo nuevo según la regla de la profundidad, en la posición que menos conflictos de cruces genere con respecto a las conexiones ya existentes y con un sesgo hacia la derecha.
 - **RF-12.2.** El sistema permitirá al usuario añadir una etiqueta al nuevo nodo introducido.
- **RF-13.** El sistema permitirá al usuario crear un nuevo nodo origen (*source*) para una hiperarista existente, siguiendo los mismos criterios de ubicación y etiquetado que en el RF-11.
- **RF-14.** El sistema permitirá al usuario crear un nuevo nodo destino (*target*) para una hiperarista existente, colocándolo en la capa adecuada según la profundidad de los nodos origen de dicha hiperarista y siguiendo los mismos criterios de ubicación y etiquetado que en el RF-12.
- **RF-15.** El sistema permitirá al usuario eliminar un nodo del diagrama. El sistema reconectará automáticamente los nodos padre e hijo del nodo eliminado para preservar la estructura jerárquica del diagrama y la información anterior a la eliminación.
- **RF-16.** El sistema permitirá al usuario renombrar un nodo existente directamente sobre el elemento visual del diagrama.
- **RF-17.** El sistema permitirá al usuario reordenar la posición horizontal de un nodo dentro de su capa arrastrándolo con el ratón, actualizando el diagrama en consecuencia.

- **RF-18.** El sistema permitirá al usuario fusionar dos nodos en uno solo, transfiriendo al nodo resultante todas las relaciones de ambos nodos y asignando al nodo fusionado una nueva etiqueta indicada por el usuario. El sistema rechazará la fusión si esta generara un ciclo en el grafo.
- **RF-19.** El sistema permitirá al usuario añadir una conexión entre dos nodos existentes, creando una nueva hiperarista (binaria).
 - **RF-19.1.** El sistema rechazará la operación si generaría un ciclo o una conexión redundante por transitividad.
 - **RF-19.2.** Si la conexión es exitosa, el sistema actualizará el hipergrafo eliminando conexiones ahora redundantes por transitividad.
- **RF-20.** El sistema permitirá al usuario añadir un nodo existente como nodo origen adicional de una hiperarista existente.
 - **RF-20.1.** Se satisfarán los mismos requisitos que en el RF-19 para la validez de la operación y la actualización del hipergrafo.
 - **RF-20.2.** Si ya existe una conexión entre el nodo origen añadido y alguno de los nodos destino de la hiperarista, el sistema tomará dicha operación como una solicitud de reagrupación de la hiperarista, añadiendo el nodo origen y eliminando las conexiones redundantes.
- **RF-21.** El sistema permitirá al usuario añadir un nodo existente como nodo destino adicional de una hiperarista existente.
 - **RF-21.1.** Se satisfarán los mismos requisitos que en el RF-19 para la validez de la operación y la actualización del hipergrafo.
 - **RF-21.2.** Si ya existe una conexión entre el nodo destino añadido y alguno de los nodos origen de la hiperarista, el sistema tomará dicha operación como una solicitud de reagrupación de la hiperarista, añadiendo el nodo destino y eliminando las conexiones redundantes.
- **RF-22.** El sistema permitirá al usuario eliminar la conexión dirigida entre dos nodos, eliminando o modificando la hiperarista afectada según corresponda.
- **RF-23.** El sistema permitirá al usuario eliminar un nodo origen de una hiperarista existente. Si el nodo origen eliminado es el único nodo origen de dicha hiperarista, el sistema eliminará la hiperarista completa.
- **RF-24.** El sistema permitirá al usuario eliminar un nodo destino de una hiperarista existente. Si el nodo destino eliminado es el único nodo destino de dicha hiperarista, el sistema eliminará la hiperarista completa.
- **RF-25.** El sistema permitirá al usuario eliminar una hiperarista completa del diagrama.
- **RF-26.** El sistema calculará y mostrará automáticamente el *layout* del diagrama tras cada operación de edición.

- **RF-26.1.** El sistema mostrará un *layout* conforme a lo establecido en la Definición 2.10.
- **RF-26.2.** El sistema mostrará los nodos de una capa como rectángulos de tamaño fijo.
- **RF-26.3.** El sistema hará un pequeño arco cuando un segmento horizontal de una hiperarista se cruce con un segmento vertical de otra. Además, si la sucesión de cruces fuera demasiado densa, el sistema mostrará un único arco esquivando toda la sucesión de cruces.
- **RF-27.** El sistema permitirá al usuario ejecutar manualmente una pasada de minimización de cruces sobre el diagrama activo mediante un botón dedicado en la interfaz.
- **RF-28.** El sistema permitirá al usuario ampliar y reducir la escala del diagrama visible mediante los atajos de teclado correspondientes o las opciones del menú.
- **RF-29.** El sistema permitirá al usuario desplazar el área de visualización del diagrama (*pan*) manteniendo pulsado el botón izquierdo del ratón sobre el fondo del área de edición.
- **RF-30.** El sistema dispondrá de un diagrama conjunto (*joint view*) en el que el usuario podrá incorporar diagramas individuales del proyecto, mostrándolos de forma conjunta y alineada por capas.
- **RF-31.** El sistema permitirá al usuario incorporar un diagrama al diagrama conjunto, colocándolo a la izquierda o a la derecha del contenido ya presente en función de la posición del clic sobre el fondo del diagrama conjunto.
- **RF-32.** En el diagrama conjunto, el sistema permitirá al usuario añadir conexiones entre nodos de distintos diagramas incorporados, así como eliminar nodos, eliminar conexiones y fusionar nodos, aplicando las mismas restricciones de validez que en los diagramas individuales, pero no permitirá crear nuevos nodos ni eliminar diagramas incorporados.
- **RF-33.** El sistema permitirá al usuario deshacer la última operación de edición realizada sobre el diagrama activo, restaurando el estado anterior del diagrama.
- **RF-34.** El sistema permitirá al usuario rehacer la última operación deshecha sobre el diagrama activo, siempre que no se haya realizado ninguna operación de edición nueva tras el deshacer.
- **RF-35.** El sistema mantendrá historiales de deshacer y rehacer independientes para cada diagrama y para el diagrama conjunto.

B.2. Catálogo de Requisitos no funcionales

En esta sección se detallan todos los requisitos no funcionales de la aplicación Matrix-Harris. De entre todos, los más relevantes (y en los que el cliente hizo más énfasis) tienen que ver con el rendimiento y con la legibilidad visual del esquema Matrix Harris.

B.2.1. Requisitos de Rendimiento

- **RNF-RE-1.** El sistema garantizará que la actualización del *layout* del diagrama tras cualquier operación de edición (creación, eliminación, conexión o fusión de nodos) se complete en un tiempo inferior a dos segundos para grafos de hasta 100 nodos.
- **RNF-RE-2.** El sistema garantizará que la ejecución manual de la minimización de cruces sobre el diagrama activo se complete en un tiempo inferior a cinco segundos para grafos de hasta 100 nodos.

B.2.2. Requisitos de Fiabilidad

- **RNF-FI-1.** El sistema no confirmará ninguna operación de edición que haya fallado: en caso de error, el estado del diagrama permanecerá idéntico al que tenía antes de intentar la operación, y la pila de deshacer no se verá modificada.
- **RNF-FI-2.** El sistema rechazará toda operación que pudiera introducir un ciclo en el hipergrafo, informando al usuario del motivo del rechazo mediante un mensaje de error.
- **RNF-FI-3.** El sistema rechazará toda operación que generara una conexión redundante por transitividad, manteniendo en todo momento la propiedad del diagrama de Hasse.

B.2.3. Requisitos de Usabilidad

- **RNF-US-1.** El sistema proporcionará mensajes de error descriptivos al usuario cuando una operación no pueda llevarse a cabo, indicando la razón concreta del fallo (ciclo detectado, operación no válida, etc.).
- **RNF-US-2.** El sistema resaltará visualmente los nodos que constituyan destinos válidos para el segundo clic en operaciones de dos pasos (añadir conexión, fusionar nodos, añadir origen, añadir destino, etc.), facilitando la selección al usuario.
- **RNF-US-3.** El sistema permitirá cancelar cualquier operación de dos pasos en curso haciendo clic sobre el fondo del área de edición, devolviendo el diagrama al estado de reposo sin modificación alguna.
- **RNF-US-4.** El sistema garantizará en todo momento la legibilidad visual del diagrama mediante la minimización automática del número de cruces entre hiperaristas, el enrutamiento estrictamente ortogonal de todos los segmentos, la inserción de desvíos semicirculares en los puntos de cruce entre una barra horizontal y un segmento vertical, y el respeto de separaciones mínimas entre nodos y entre puertos de conexión adyacentes.

- **RNF-US-5.** La evaluación de la usabilidad estará basada en tres criterios diferenciados: eficacia (capacidad del sistema para permitir a los usuarios completar las tareas propuestas de forma precisa y completa), eficiencia (tiempo medio requerido para completar las tareas con eficacia) y satisfacción (grado de aceptación y comodidad percibido por los usuarios durante la interacción con el sistema). Para verificar el cumplimiento de estos criterios se llevarán a cabo pruebas de usabilidad con usuarios reales empleando un cuestionario de satisfacción diseñado para la aplicación.

B.2.4. Requisitos de Portabilidad y Compatibilidad

- **RNF-PO-1.** El sistema será portable a Windows, Linux y macOS, garantizando que la experiencia de usuario sea consistente en todas las plataformas.

B.2.5. Requisitos de Mantenibilidad

- **RNF-MA-1.** El sistema estará organizado en dos módulos claramente separados: la biblioteca de lógica de dominio (`hypergraph_logic`) y la capa de aplicación y presentación (`app`), de modo que los cambios en la interfaz de usuario no afecten a la lógica del grafo y viceversa.
- **RNF-MA-2.** El sistema contará con una suite de pruebas unitarias y de integración para cubrir toda la funcionalidad crítica de la aplicación.

B.2.6. Requisitos de Persistencia

- **RNF-PE-1.** El sistema almacenará el estado completo de un proyecto en un único fichero, de modo que la información pueda ser cargada y restaurada íntegramente en sesiones posteriores.

B.2.7. Requisitos de Documentación

- **RNF-DO-1.** El código fuente del sistema contará con comentarios de cabecera en todas las clases y métodos públicos, de manera que pueda ser comprendido por otros desarrolladores sin necesidad de consultar la implementación interna.
- **RNF-DO-2.** El sistema contará con documentación técnica que describa la arquitectura general, las dependencias del proyecto y el proceso de compilación e instalación.
- **RNF-DO-3.** El sistema contará con una guía de usuario a modo de tutorial que explique tanto la instalación como el uso de las funcionalidades principales de la aplicación.

B.3. Diagrama de casos de uso

A continuación, se expone el diagrama de casos de uso global de la aplicación Matrix Harris. Se recomienda leer este documento en la versión digital para poder hacer zoom sobre el diagrama y apreciar los detalles de cada caso de uso.

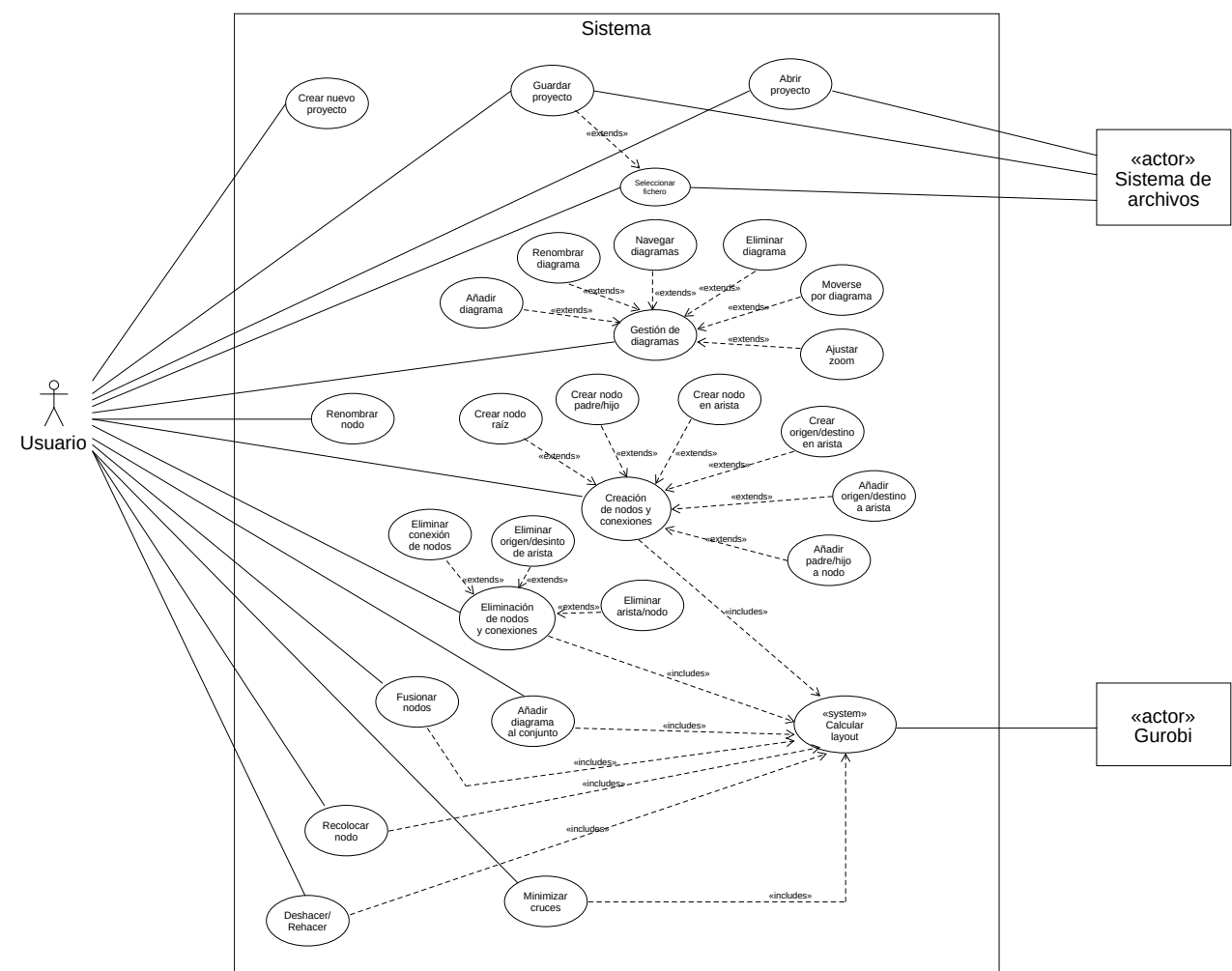


Figura B.1: Diagrama global de casos de uso de la aplicación Matrix Harris.

B.4. Catálogo de casos de uso

B.4.1. CU-1: Crear nuevo proyecto

Información general	
ID y Nombre	CU-1: Crear nuevo proyecto
Sigue en la siguiente página...	

Tabla B.1 – Continuación de la página anterior

Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario crear un nuevo proyecto vacío, con un diagrama inicial y un diagrama conjunto vacíos.
Estado del sistema	
Precondiciones	Ninguna
Postcondiciones	1. Se crea un nuevo proyecto vacío con un diagrama inicial. 2. El proyecto pasa a ser el proyecto activo en la aplicación.
Secuencia de eventos	
Flujo Principal	1. El usuario selecciona la opción “Nuevo proyecto” en el menú de la aplicación o abre la aplicación. 2. El sistema comprueba si existe un proyecto activo con cambios sin guardar. 3. El sistema crea un nuevo proyecto vacío con un diagrama inicial y un diagrama conjunto vacíos y lo muestra al usuario.
Excepciones	2a. Cambios sin guardar: Si existe un proyecto activo con cambios sin guardar, el sistema muestra un mensaje de advertencia solicitando confirmación antes de descartar los cambios. Si el usuario cancela, el sistema vuelve al estado anterior sin crear el nuevo proyecto.

Tabla B.1: Especificación del caso de uso CU-1: Crear nuevo proyecto

B.4.2. CU-2: Guardar proyecto

Información general	
ID y Nombre	CU-2: Guardar proyecto
Actor principal	Usuario
Actores secundarios	Sistema de archivos
Descripción	Permite al usuario guardar el estado completo del proyecto activo en un fichero JSON en el sistema de archivos.
Estado del sistema	
Precondiciones	1. Debe existir un proyecto activo en la aplicación.
Sigue en la siguiente página...	

Tabla B.2 – Continuación de la página anterior

Postcondiciones	1. El estado completo del proyecto queda serializado en un fichero JSON en la ruta seleccionada. 2. El indicador de cambios sin guardar se desactiva.
Secuencia de eventos	
Flujo Principal	1. El usuario selecciona la opción “Guardar” en el menú de la aplicación. 2. El sistema comprueba si el proyecto ya tiene una ruta de fichero asociada. 3. El sistema serializa el proyecto completo y lo escribe en el fichero existente.
Subflujo A	Guardar por primera vez <i>Condición de activación: el proyecto no tiene ruta de fichero asociada</i> 1. El sistema muestra un diálogo del sistema operativo para que el usuario seleccione la ruta y el nombre del fichero. 2. El usuario introduce el nombre del fichero y confirma la ruta. 3. El sistema serializa el proyecto y lo escribe en la ruta seleccionada.
Excepciones	A.2a. El usuario cancela el diálogo de guardado: el sistema no guarda ningún fichero y vuelve al estado anterior.

Tabla B.2: Especificación del caso de uso CU-2: Guardar proyecto

B.4.3. CU-3: Abrir proyecto

Información general	
ID y Nombre	CU-3: Abrir proyecto
Actor principal	Usuario
Actores secundarios	Sistema de archivos
Descripción	Permite al usuario abrir un proyecto previamente guardado a partir de un fichero JSON, restaurando el estado completo de todos los diagramas.
Estado del sistema	
Precondiciones	Ninguna
Sigue en la siguiente página...	

Tabla B.3 – Continuación de la página anterior

Postcondiciones	1. El proyecto queda cargado y pasa a ser el proyecto activo en la aplicación. 2. El estado de todos los diagramas y el diagrama conjunto queda restaurado.
Secuencia de eventos	
Flujo Principal	1. El usuario selecciona la opción “Abrir proyecto” en el menú de la aplicación. 2. El sistema comprueba si existe un proyecto activo con cambios sin guardar. 3. El sistema muestra un diálogo del sistema operativo para que el usuario seleccione el fichero JSON del proyecto. 4. El usuario selecciona el fichero y confirma. 5. El sistema deserializa el fichero y carga el proyecto. 6. El sistema muestra el proyecto cargado al usuario, esto es, el primer diagrama no conjunto.
Excepciones	2a. Cambios sin guardar: Si existe un proyecto activo con cambios sin guardar, el sistema muestra un mensaje de advertencia solicitando confirmación. Si el usuario cancela, el sistema vuelve al estado anterior. 3a. El usuario cancela el diálogo: el sistema no carga ningún fichero y vuelve al estado anterior. 5a. Fichero inválido o corrupto: si el fichero seleccionado no es un proyecto válido, el sistema muestra un mensaje de error y vuelve al estado anterior.

Tabla B.3: Especificación del caso de uso CU-3: Abrir proyecto

B.4.4. CU-4: Añadir diagrama

Información general	
ID y Nombre	CU-4: Añadir diagrama
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario añadir un nuevo diagrama vacío al proyecto activo.
Sigue en la siguiente página...	

Tabla B.4 – Continuación de la página anterior

Estado del sistema	
Precondiciones	1. Debe existir un proyecto activo en la aplicación.
Postcondiciones	1. El nuevo diagrama queda añadido al proyecto con un nombre único generado automáticamente. 2. El nuevo diagrama pasa a ser el diagrama activo. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario pulsa el botón de añadir diagrama en la barra de pestañas o selecciona la ruta “Archivo/Añadir diagrama” en el menú de la aplicación. 2. El sistema crea un nuevo diagrama vacío con un nombre único generado automáticamente. 3. El sistema añade una nueva pestaña al final de la barra de pestañas y activa el nuevo diagrama.
Excepciones	Ninguna

Tabla B.4: Especificación del caso de uso CU-4: Añadir diagrama

B.4.5. CU-5: Eliminar diagrama

Información general	
ID y Nombre	CU-5: Eliminar diagrama
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario eliminar un diagrama del proyecto activo. La operación no puede deshacerse.
Estado del sistema	
Precondiciones	1. Debe existir un proyecto activo con al menos un diagrama.
Postcondiciones	1. El diagrama queda eliminado del proyecto de forma permanente. 2. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.5 – Continuación de la página anterior

Flujo Principal	1. El usuario pulsa el botón de papelera sobre la pestaña del diagrama que desea eliminar. 2. El sistema elimina el diagrama del proyecto y actualiza la barra de pestañas. 3. El sistema activa el diagrama adyacente al eliminado o el diagrama conjunto si no quedan más diagramas.
Excepciones	Ninguna

Tabla B.5: Especificación del caso de uso CU-5: Eliminar diagrama

B.4.6. CU-6: Renombrar diagrama

Información general	
ID y Nombre	CU-6: Renombrar diagrama
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario cambiar el nombre de un diagrama existente haciendo doble clic sobre su pestaña.
Estado del sistema	
Precondiciones	1. Debe existir un proyecto activo con al menos un diagrama.
Postcondiciones	1. El diagrama queda renombrado con el nuevo nombre introducido. 2. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario hace doble clic sobre la pestaña del diagrama que desea renombrar. 2. El sistema habilita la edición en línea del nombre en la pestaña. 3. El usuario introduce el nuevo nombre y confirma pulsando Intro o haciendo clic fuera del campo. 4. El sistema actualiza el nombre del diagrama.
Excepciones	3a. El usuario pulsa Escape: el sistema cancela la edición y restaura el nombre anterior. 3b. El nuevo nombre está vacío: el sistema descarta el cambio y restaura el nombre anterior.
Sigue en la siguiente página...	

Tabla B.6 – Continuación de la página anterior

Tabla B.6: Especificación del caso de uso CU-6: Renombrar diagrama**B.4.7. CU-7: Navegar entre diagramas**

Información general	
ID y Nombre	CU-7: Navegar entre diagramas
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario cambiar el diagrama activo seleccionando una pestaña de la barra de pestañas, incluyendo el diagrama conjunto.
Estado del sistema	
Precondiciones	1. Debe existir un proyecto activo con al menos un diagrama.
Postcondiciones	1. El diagrama seleccionado pasa a mostrarse en el área central de edición y su miniatura desaparece de la barra de pestañas, quedando únicamente el nombre del diagrama activo.
Secuencia de eventos	
Flujo Principal	1. El usuario hace clic sobre la pestaña del diagrama al que desea navegar. 2. El sistema activa el diagrama seleccionado y actualiza el área central de edición con su contenido.
Excepciones	Ninguna

Tabla B.7: Especificación del caso de uso CU-7: Navegar entre diagramas**B.4.8. CU-8: Crear nodo raíz**

Información general	
ID y Nombre	CU-8: Crear nodo raíz
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario crear un nuevo nodo sin padre en la capa raíz del diagrama activo haciendo clic sobre el fondo del área de edición.
Estado del sistema	
Sigue en la siguiente página...	

Tabla B.8 – Continuación de la página anterior

Precondiciones	1. El diagrama activo debe ser un diagrama individual (no el diagrama conjunto).
Postcondiciones	1. El nuevo nodo queda añadido al diagrama en la capa raíz (capa 0). 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario hace clic sobre el fondo del área de edición del diagrama activo. 2. El sistema muestra un diálogo de edición en línea sobre el rectángulo del nodo solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo en la capa 0, posicionándolo en función de la abscisa donde se realizó el clic, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.8: Especificación del caso de uso CU-8: Crear nodo raíz

B.4.9. CU-9: Crear nodo hijo

Información general	
ID y Nombre	CU-9: Crear nodo hijo
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario crear un nuevo nodo como hijo directo de un nodo existente, colocándolo en la capa inmediatamente inferior.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Sigue en la siguiente página...	

Tabla B.9 – Continuación de la página anterior

Postcondiciones	1. El nuevo nodo queda añadido al diagrama en la capa del nodo padre más uno. 2. Se crea automáticamente una hiperarista entre el nodo padre y el nuevo nodo. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo existente y selecciona “Crear nodo debajo”. 2. El sistema muestra un diálogo de edición en línea solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo hijo en la capa correspondiente, insertándolo en la posición donde menor conflicto genere (con un sesgo hacia la derecha), crea la hiperarista de conexión, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.9: Especificación del caso de uso CU-9: Crear nodo hijo

B.4.10. CU-10: Crear nodo padre

Información general	
ID y Nombre	CU-10: Crear nodo padre
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario crear un nuevo nodo como padre de un nodo existente, colocándolo en la capa raíz del diagrama.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Sigue en la siguiente página...	

Tabla B.10 – Continuación de la página anterior

Postcondiciones	1. El nuevo nodo queda añadido en la capa 0 del diagrama. 2. Se crea automáticamente una hiperarista entre el nuevo nodo y el nodo seleccionado. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo existente y selecciona “Crear nodo arriba”. 2. El sistema muestra un diálogo de edición en línea solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo padre en la capa 0 insertándolo en la posición que menos conflictos genera, crea la hiperarista de conexión, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.10: Especificación del caso de uso CU-10: Crear nodo padre

B.4.11. CU-11: Crear nodo dentro de una hiperarista

Información general	
ID y Nombre	CU-11: Crear nodo dentro de una hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario insertar un nuevo nodo en el interior de una hiperarista existente, dividiéndola en dos hiperaristas.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Sigue en la siguiente página...	

Tabla B.11 – Continuación de la página anterior

Postcondiciones	1. El nuevo nodo queda insertado entre los nodos origen y destino de la hiperarista original. 2. La hiperarista original queda dividida en dos: una de los orígenes al nuevo nodo y otra del nuevo nodo a los destinos originales. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Crear nodo en arista”. 2. El sistema muestra un diálogo de edición en línea solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo, divide la hiperarista, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.11: Especificación del caso de uso CU-11: Crear nodo dentro de una hiperarista

B.4.12. CU-12: Crear origen de una hiperarista

Información general	
ID y Nombre	CU-12: Crear un nodo que se inserte como origen de una hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario insertar un nuevo nodo como origen de una hiperarista, convirtiéndose automáticamente en padre de los destinos de dicha hiperarista
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Sigue en la siguiente página...	

Tabla B.12 – Continuación de la página anterior

Postcondiciones	1. El nuevo nodo queda insertado en la capa raíz del diagrama. 2. La hiperarista original queda modificada con un nuevo origen, que es el nodo creado. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Crear origen”. 2. El sistema muestra un diálogo de edición en línea solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo, modifica la hiperarista, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.12: Especificación del caso de uso CU-12: Crear origen de una hiperarista

B.4.13. CU-13: Crear destino de una hiperarista

Información general	
ID y Nombre	CU-13: Crear un nodo que se inserte como destino de una hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario insertar un nuevo nodo como destino de una hiperarista, convirtiéndose automáticamente en hijo de los orígenes de dicha hiperarista
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Sigue en la siguiente página...	

Tabla B.13 – Continuación de la página anterior

Postcondiciones	1. El nuevo nodo queda insertado en la capa inmediatamente inferior a la capa de los orígenes de la hiperarista, esto es, el máximo de las profundidades de los orígenes más uno. 2. La hiperarista original queda modificada con un nuevo destino, que es el nodo creado. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Crear destino”. 2. El sistema muestra un diálogo de edición en línea solicitando la etiqueta del nuevo nodo. 3. El usuario introduce la etiqueta y confirma. 4. El sistema crea el nodo, modifica la hiperarista, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación.

Tabla B.13: Especificación del caso de uso CU-13: Crear destino de una hiperarista

B.4.14. CU-14: Añadir un nuevo padre a nodo

Información general	
ID y Nombre	CU-14: Añadir un nuevo padre a nodo
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario establecer una conexión dirigida entre dos nodos existentes, creando una nueva hiperarista binaria (un solo origen y un solo destino). En concreto, el destino será el nodo sobre el que se aplica la operación y el origen es el nodo seleccionado como nuevo padre.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos dos nodos.
Sigue en la siguiente página...	

Tabla B.14 – Continuación de la página anterior

Postcondiciones	1. La conexión dirigida entre los dos nodos queda establecida. 2. El nodo sobre el que se aplica la operación actualiza su capa a la máxima entre su capa actual y la capa del nuevo padre más uno. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo y selecciona “Añadir conexión arriba”. 2. El sistema resalta los nodos válidos como destino de la conexión. 3. El usuario hace clic sobre el nodo destino. 4. El sistema crea la conexión, recalcula la capa del nodo sobre el que se ejecuta la operación, elimina las relaciones redundantes por transitividad, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación. 4a. La conexión introduciría un ciclo en el grafo: el sistema muestra un mensaje de error y descarta la operación. 4b. La conexión ya existe en el diagrama, es decir, el nuevo padre ya es un padre directo del nodo destino: el sistema muestra un mensaje de error y descarta la operación. 4c. La conexión es redundante en el diagrama por transitividad, es decir, el nuevo padre tiene como hijo a algún ancestro del nodo destino: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.14: Especificación del caso de uso CU-14: Añadir un nuevo padre a nodo

B.4.15. CU-15: Añadir un nuevo hijo a nodo

Información general	
ID y Nombre	CU-15: Añadir un nuevo hijo a nodo
Actor principal	Usuario
Actores secundarios	Gurobi
Sigue en la siguiente página...	

Tabla B.15 – Continuación de la página anterior

Descripción	Permite al usuario establecer una conexión dirigida entre dos nodos existentes, creando una nueva hiperarista binaria (un solo origen y un solo destino). En concreto, el origen será el nodo sobre el que se aplica la operación y el destino es el nodo seleccionado como nuevo hijo.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos dos nodos.
Postcondiciones	1. La conexión dirigida entre los dos nodos queda establecida. 2. El nodo sobre seleccionado como hijo actualiza su capa a la máxima entre su capa actual y la capa del nuevo padre más uno. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo y selecciona “Añadir conexión debajo”. 2. El sistema resalta los nodos válidos como destino de la conexión. 3. El usuario hace clic sobre el nodo destino. 4. El sistema crea la conexión, recalcula la capa del nodo seleccionado, elimina las relaciones redundantes por transitividad, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación. 4a. La conexión introduciría un ciclo en el grafo: el sistema muestra un mensaje de error y descarta la operación. 4b. La conexión ya existe en el diagrama, es decir, el nuevo hijo ya es un hijo directo del nodo sobre el que se ejecuta la operación: el sistema muestra un mensaje de error y descarta la operación. 4c. La conexión es redundante en el diagrama por transitividad, es decir, el nodo seleccionado tiene como padre a algún descendiente del nodo sobre el que se ejecuta la operación: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.15: Especificación del caso de uso CU-15: Añadir un nuevo hijo a nodo

B.4.16. CU-16: Añadir nodo origen a hiperarista

Información general	
ID y Nombre	CU-16: Añadir nodo origen a hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario añadir un nodo existente como nodo origen adicional de una hiperarista existente.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista y un nodo adicional susceptible de ser origen.
Postcondiciones	1. El nodo seleccionado queda registrado como origen adicional de la hiperarista. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Añadir origen”. 2. El sistema resalta los nodos válidos para ser añadidos como origen. 3. El usuario hace clic sobre el nodo que desea añadir como origen. 4. El sistema registra el nuevo origen, elimina posibles redundancias por transitividad, recalcula el <i>layout</i> y actualiza la vista. Si ya existía una conexión entre el nuevo origen y alguno de los destinos de la hiperarista, el sistema reagrupará automáticamente dichas conexiones para evitar redundancias.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación. 4a. La operación introduciría un ciclo: el sistema muestra un mensaje de error y descarta la operación. 4b. La conexión es redundante en el diagrama por transitividad, es decir, el nodo seleccionado tiene como hijo a algún ancestro de los destinos de la hiperarista: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.16: Especificación del caso de uso CU-16: Añadir nodo origen a hiperarista

B.4.17. CU-17: Añadir nodo destino a hiperarista

Información general	
ID y Nombre	CU-17: Añadir nodo destino a hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario añadir un nodo existente como nodo destino adicional de una hiperarista existente.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista y un nodo adicional susceptible de ser destino.
Postcondiciones	1. El nodo seleccionado queda registrado como destino adicional de la hiperarista. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Añadir destino”. 2. El sistema resalta los nodos válidos para ser añadidos como destino. 3. El usuario hace clic sobre el nodo que desea añadir como destino. 4. El sistema registra el nuevo destino, elimina conexiones redundantes por transitividad y recalcula el <i>layout</i> y actualiza la vista. Si ya existía una conexión entre alguno de los orígenes de la hiperarista y el nuevo destino, el sistema reagrupará automáticamente dichas conexiones para evitar redundancias.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación. 4a. La operación introduciría un ciclo: el sistema muestra un mensaje de error y descarta la operación. 4b. La conexión es redundante en el diagrama por transitividad, es decir, ya existe una conexión de alguno de los orígenes de la hiperarista a un ancestro del nuevo destino: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.17: Especificación del caso de uso CU-17: Añadir nodo destino a hiperarista

B.4.18. CU-18: Eliminar nodo

Información general	
ID y Nombre	CU-18: Eliminar nodo
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario eliminar un nodo del diagrama activo. El sistema reconecta automáticamente los padres e hijos del nodo eliminado.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Postcondiciones	1. El nodo queda eliminado del diagrama. 2. Los padres e hijos del nodo eliminado quedan reconectados y reubicados a la capa correspondiente según la regla de la profundidad. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo y selecciona “Eliminar nodo”. 2. El sistema elimina el nodo, reconecta padres e hijos, reubica los nodos en capas, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	Ninguna

Tabla B.18: Especificación del caso de uso CU-18: Eliminar nodo**B.4.19. CU-19: Eliminar conexión entre nodos**

Información general	
ID y Nombre	CU-19: Eliminar conexión entre nodos
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario eliminar la conexión dirigida entre dos nodos, modificando o eliminando la hiperarista afectada según corresponda.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Sigue en la siguiente página...	

Tabla B.19 – Continuación de la página anterior

Postcondiciones	1. La conexión entre los nodos seleccionados queda eliminada. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario abre el menú contextual sobre un nodo y selecciona “Eliminar conexión”. 2. El sistema resalta los nodos válidos (sus padres o hijos) para eliminar la conexión. 3. El usuario hace clic sobre el nodo cuya conexión desea eliminar. 4. El sistema elimina la conexión (ya sea eliminando la hiperarista o modificando pertinentemente la hiperarista implicada), reubica los nodos afectados a su profundidad adecuada, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación.

Tabla B.19: Especificación del caso de uso CU-19: Eliminar conexión entre nodos

B.4.20. CU-20: Eliminar origen de una hiperarista

Información general	
ID y Nombre	CU-20: Eliminar origen de una hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario eliminar un origen de una hiperarista, modificando o eliminando la hiperarista afectada según el caso.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Postcondiciones	1. El nodo seleccionado deja de ser origen de la hiperarista. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.20 – Continuación de la página anterior

Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Eliminar origen”. 2. El sistema resalta los nodos válidos (las fuentes de dicha arista) para eliminar la conexión. 3. El usuario hace clic sobre el nodo fuente que desea eliminar. 4. El sistema elimina dicha fuente (ya sea eliminando la hiperarista o modificando pertinentemente la hiperarista implicada), reubica los nodos afectados a su profundidad adecuada, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación.

Tabla B.20: Especificación del caso de uso CU-20: Eliminar origen de una hiperarista

B.4.21. CU-21: Eliminar destino de una hiperarista

Información general	
ID y Nombre	CU-21: Eliminar destino de una hiperarista
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario eliminar un destino de una hiperarista, modificando o eliminando la hiperarista afectada según el caso.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Postcondiciones	1. El nodo seleccionado deja de ser destino de la hiperarista. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.21 – Continuación de la página anterior

Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Eliminar destino”. 2. El sistema resalta los nodos válidos (los sumideros de dicha arista) para eliminar la conexión. 3. El usuario hace clic sobre el nodo destino que desea eliminar. 4. El sistema elimina dicho sumidero (ya sea eliminando la hiperarista o modificando pertinentemente la hiperarista implicada), reubica los nodos afectados a su profundidad adecuada, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación.

Tabla B.21: Especificación del caso de uso CU-21: Eliminar destino de una hiperarista

B.4.22. CU-22: Eliminar hiperarista completa

Información general	
ID y Nombre	CU-22: Eliminar hiperarista completa
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario eliminar una hiperarista completa del diagrama, desconectando todos sus nodos origen y destino.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos una hiperarista.
Postcondiciones	1. La hiperarista queda completamente eliminada del diagrama. 2. El diagrama reubica los nodos afectados a su profundidad adecuada según la regla de la profundidad. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.22 – Continuación de la página anterior

Flujo Principal	1. El usuario abre el menú contextual sobre una hiperarista y selecciona “Eliminar arista”. 2. El sistema elimina la hiperarista completa, reubica los nodos afectados de acuerdo a la regla de profundidad, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	Ninguna

Tabla B.22: Especificación del caso de uso CU-22: Eliminar hiperarista completa

B.4.23. CU-23: Fusionar nodos

Información general	
ID y Nombre	CU-23: Fusionar nodos
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario fusionar dos nodos en uno solo, transfiriendo todas las relaciones de ambos al nodo resultante con una nueva etiqueta.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos dos nodos.
Postcondiciones	1. Los dos nodos quedan fusionados en uno con la nueva etiqueta indicada. 2. El nodo resultante hereda todas las relaciones de ambos nodos originales. 3. El diagrama actualiza su <i>layout</i> automáticamente. 4. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.23 – Continuación de la página anterior

Flujo Principal	1. El usuario abre el menú contextual sobre un nodo y selecciona “Fusionar”. 2. El sistema resalta los nodos válidos como destino de la fusión. 3. El usuario hace clic sobre el segundo nodo a fusionar. 4. El sistema muestra un diálogo solicitando la nueva etiqueta del nodo resultante. 5. El usuario introduce la etiqueta y confirma. 6. El sistema fusiona los nodos, transfiere las relaciones, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario hace clic sobre el fondo o pulsa Escape: el sistema cancela la operación. 5a. El usuario cancela o deja la etiqueta vacía: el sistema descarta la operación. 6a. La fusión introduciría un ciclo en el grafo: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.23: Especificación del caso de uso CU-23: Fusionar nodos

B.4.24. CU-24: Renombrar nodo

Información general	
ID y Nombre	CU-24: Renombrar nodo
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario cambiar la etiqueta de un nodo existente mediante edición en línea directamente sobre el elemento visual.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Postcondiciones	1. La etiqueta del nodo queda actualizada con el nuevo nombre. 2. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.24 – Continuación de la página anterior

Flujo Principal	1. El usuario hace doble clic sobre un nodo en el área de edición. 2. El sistema habilita la edición en línea de la etiqueta del nodo. 3. El usuario introduce el nuevo nombre y confirma pulsando Intro o haciendo clic fuera del campo. 4. El sistema actualiza la etiqueta del nodo y refresca la vista.
Excepciones	3a. El usuario pulsa Escape: el sistema cancela la edición y restaura la etiqueta anterior. 3b. La nueva etiqueta está vacía: el sistema descarta el cambio y restaura la etiqueta anterior.

Tabla B.24: Especificación del caso de uso CU-24: Renombrar nodo

B.4.25. CU-25: Reordenar nodo en su capa

Información general	
ID y Nombre	CU-25: Reordenar nodo en su capa
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario cambiar la posición horizontal de un nodo dentro de su capa arrastrándolo con el ratón.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Postcondiciones	1. El nodo queda reposicionado dentro de su capa según la nueva posición indicada y todo el subgrafo descendiente se mueve con él, posicionándose automáticamente de la manera menos conflictiva posible. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.25 – Continuación de la página anterior

Flujo Principal	1. El usuario pulsa y arrastra un nodo horizontalmente dentro del área de edición. 2. Al soltar el nodo, el sistema interpreta la nueva posición horizontal como señal de reordenamiento dentro de la capa. 3. El sistema reordena el nodo en la capa, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	Ninguna

Tabla B.25: Especificación del caso de uso CU-25: Reordenar nodo en su capa

B.4.26. CU-26: Minimizar cruces manualmente

Información general	
ID y Nombre	CU-26: Minimizar cruces manualmente
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario ejecutar manualmente un algoritmo de minimización de cruces sobre el diagrama activo mediante el botón dedicado en la interfaz.
Estado del sistema	
Precondiciones	1. El diagrama activo debe contener al menos un nodo.
Postcondiciones	1. El orden horizontal de los nodos en cada capa queda optimizado para minimizar el número de cruces entre hiperaristas. 2. El diagrama actualiza su <i>layout</i> automáticamente. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Flujo Principal	1. El usuario pulsa el botón “Minimizar cruces” en la interfaz. 2. El sistema ejecuta el algoritmo de minimización de cruces (<i>Global Sifting</i>) sobre el diagrama activo. 3. El sistema recalcula el <i>layout</i> completo y actualiza la vista.
Excepciones	Ninguna

Tabla B.26: Especificación del caso de uso CU-26: Minimizar cruces manualmente

B.4.27. CU-27: Ajustar zoom del diagrama

Información general	
ID y Nombre	CU-27: Ajustar zoom del diagrama
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario ampliar o reducir la escala del diagrama visible mediante atajos de teclado o las opciones del menú.
Estado del sistema	
Precondiciones	1. Debe existir un diagrama activo visible en el área de edición o uno no activo en la barra de pestañas.
Postcondiciones	1. La escala de visualización del diagrama queda actualizada según la acción del usuario.
Secuencia de eventos	
Flujo Principal	1. El usuario utiliza el atajo de teclado de acercar o alejar zoom, o selecciona la opción correspondiente en el menú. 2. El sistema actualiza la escala del área de visualización del diagrama.
Excepciones	Ninguna

Tabla B.27: Especificación del caso de uso CU-27: Ajustar zoom del diagrama**B.4.28. CU-28: Arrastrarse a lo largo del diagrama**

Información general	
ID y Nombre	CU-28: Arrastrarse a lo largo del diagrama
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario navegar por la vista de un diagrama para posicionarse en una perspectiva concreta.
Estado del sistema	
Precondiciones	1. Debe existir un diagrama activo visible en el área de edición o uno no activo en la barra de pestañas.
Postcondiciones	1. La porción visible del diagrama queda desplazada en la dirección indicada por el usuario.
Sigue en la siguiente página...	

Tabla B.28 – Continuación de la página anterior

Secuencia de eventos	
Flujo Principal	1. El usuario hace clic izquierdo sobre el fondo del área de edición y arrastra el ratón en la dirección deseada. 2. El sistema actualiza la porción visible del diagrama desplazándola en la dirección indicada por el usuario.
Excepciones	Ninguna

Tabla B.28: Especificación del caso de uso CU-28: Arrastrarse a lo largo del diagrama

B.4.29. CU-29: Incorporar diagrama al diagrama conjunto

Información general	
ID y Nombre	CU-29: Incorporar diagrama al diagrama conjunto
Actor principal	Usuario
Actores secundarios	Gurobi
Descripción	Permite al usuario añadir un diagrama individual al diagrama conjunto, mostrándolo de forma alineada por capas junto al contenido ya presente.
Estado del sistema	
Precondiciones	1. El diagrama activo debe ser el diagrama conjunto. 2. El proyecto debe contener al menos un diagrama individual no incorporado previamente al diagrama conjunto.
Postcondiciones	1. El diagrama seleccionado queda incorporado al diagrama conjunto. 2. El diagrama conjunto actualiza su <i>layout</i> mostrando el nuevo diagrama alineado por capas. 3. El proyecto queda marcado con cambios sin guardar.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.29 – Continuación de la página anterior

Flujo Principal	1. El usuario hace clic derecho sobre el fondo del área de edición del diagrama conjunto. 2. El sistema determina si el clic fue en la mitad izquierda o derecha del área y muestra un selector de diagramas disponibles. 3. El usuario selecciona el diagrama a incorporar. 4. El sistema incorpora una copia del diagrama al lado seleccionado del diagrama conjunto, recalcula el <i>layout</i> y actualiza la vista.
Excepciones	3a. El usuario cancela el selector: el sistema no incorpora ningún diagrama. 4a. El diagrama ya ha sido incorporado previamente: el sistema muestra un mensaje de error y descarta la operación.

Tabla B.29: Especificación del caso de uso CU-29: Incorporar diagrama al diagrama conjunto

B.4.30. CU-30: Calcular layout

Información general	
ID y Nombre	CU-30: Calcular layout
Actor principal	Ninguno (iniciado por el sistema)
Actores secundarios	Gurobi
Descripción	Caso de uso interno del sistema que calcula las coordenadas visuales de todos los elementos del diagrama activo. Es invocado automáticamente tras cualquier operación de edición que modifique la estructura del hipergrafo, así como al ejecutar la minimización de cruces manual. Ejecuta un <i>pipeline</i> de cinco etapas que determina el orden horizontal de los nodos, sus coordenadas cartesianas, el orden vertical de las barras de las hiperaristas, los puertos de conexión de cada nodo y las coordenadas verticales de cada capa.
Estado del sistema	
Sigue en la siguiente página...	

Tabla B.30 – Continuación de la página anterior

Precondiciones	1. El diagrama activo debe contener al menos un nodo. 2. El caso de uso es invocado por el sistema como consecuencia de alguno de los siguientes casos de uso: CU-B.8, CU-B.9, CU-B.10, CU-B.11, CU-B.12, CU-B.13, CU-B.14, CU-B.15, CU-B.16, CU-B.17, CU-B.18, CU-B.19, CU-B.20, CU-B.21, CU-B.22, CU-B.23, CU-B.25, CU-B.26 o CU-B.29.
Postcondiciones	1. Cada nodo del diagrama tiene asignadas una coordenada x y una coordenada y que determinan su posición visual. 2. Cada hiperarista tiene asignada la coordenada y de su barra horizontal y la coordenada x de los puertos de conexión en los nodos origen y destino. 3. Cada capa del diagrama tiene asignada la coordenada y de su fila de nodos. 4. La vista del diagrama queda actualizada con el nuevo layout.
Secuencia de eventos	
Sigue en la siguiente página...	

Tabla B.30 – Continuación de la página anterior

Flujo Principal	1. Minimización de cruces (<i>Global Sifting</i>): el sistema reordena los nodos en cada capa para minimizar el número de cruces entre hiperaristas mediante el algoritmo <i>Global Sifting</i>. 2. Asignación de coordenadas x (<i>Brandes-Köpf</i>): el sistema asigna una coordenada horizontal a cada nodo mediante el algoritmo <i>Brandes-Köpf</i>, que compacta horizontalmente el diagrama respetando el orden establecido en la etapa anterior. 3. Ordenación de segmentos horizontales de hiperaristas: el sistema invoca a Gurobi para resolver un programa de enteros mixtos (MIP) que determina el orden vertical óptimo de las barras horizontales de las hiperaristas en cada par de capas adyacentes, minimizando el número de cruces entre segmentos verticales y horizontales. 4. Asignación de puertos y resolución de solapamientos: el sistema construye y distribuye los puertos de conexión en cada nodo para cada par de capas adyacentes, y resuelve los solapamientos entre segmentos verticales de hiperaristas distintas mediante el <i>PortAssigner</i>. 5. Asignación de coordenadas y: el sistema asigna una coordenada vertical a cada fila de nodos y a cada barra horizontal de hiperarista en función de los puertos calculados en la etapa anterior.
Excepciones	1a. El diagrama activo no sufre ningún algoritmo de minimización de cruces: el sistema utiliza el orden de nodos de la iteración anterior para asignar las abscisas de la etapa 2. 3a. Gurobi no está disponible o no tiene licencia activa: el sistema omite la optimización MIP de la etapa 3 y asigna un orden de barras por defecto basado en la posición horizontal de los nodos origen de cada hiperarista. El resto del <i>pipeline</i> continúa con normalidad.

Tabla B.30: Especificación del caso de uso CU-30: Calcular layout

B.4.31. CU-31: Deshacer operación

Información general	
ID y Nombre	CU-31: Deshacer operación
Sigue en la siguiente página...	

Tabla B.31 – Continuación de la página anterior

Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario revertir la última operación de edición realizada sobre el diagrama activo, restaurando el estado anterior del diagrama.
Estado del sistema	
Precondiciones	1. La pila de deshacer del diagrama activo debe contener al menos una operación.
Postcondiciones	1. El diagrama activo queda restaurado al estado previo a la última operación de edición. 2. La operación deshecha pasa a estar disponible en la pila de rehacer.
Secuencia de eventos	
Flujo Principal	1. El usuario selecciona la opción “Deshacer” en el menú o utiliza el atajo de teclado Ctrl+Z. 2. El sistema restaura el estado anterior del diagrama activo desde la pila de deshacer y actualiza la vista.
Excepciones	1a. La pila de deshacer está vacía: el sistema no realiza ninguna acción.

Tabla B.31: Especificación del caso de uso CU-31: Deshacer operación

B.4.32. CU-32: Rehacer operación

Información general	
ID y Nombre	CU-32: Rehacer operación
Actor principal	Usuario
Actores secundarios	Ninguno
Descripción	Permite al usuario re-aplicar la última operación deshecha sobre el diagrama activo.
Estado del sistema	
Precondiciones	1. La pila de rehacer del diagrama activo debe contener al menos una operación. 2. No debe haberse realizado ninguna operación de edición nueva tras el último deshacer.
Sigue en la siguiente página...	

Tabla B.32 – Continuación de la página anterior

Postcondiciones	1. El diagrama activo queda restaurado al estado posterior a la operación re-aplicada. 2. La operación rehecha pasa a estar disponible de nuevo en la pila de deshacer.
Secuencia de eventos	
Flujo Principal	1. El usuario selecciona la opción “Rehacer” en el menú o utiliza el atajo de teclado Ctrl+Y. 2. El sistema re-aplica la operación deshecha sobre el diagrama activo y actualiza la vista.
Excepciones	1a. La pila de rehacer está vacía: el sistema no realiza ninguna acción.

Tabla B.32: Especificación del caso de uso CU-32: Rehacer operación

B.5. Catálogo de diagramas de secuencia

B.5.1. SD-01: Crear nuevo proyecto

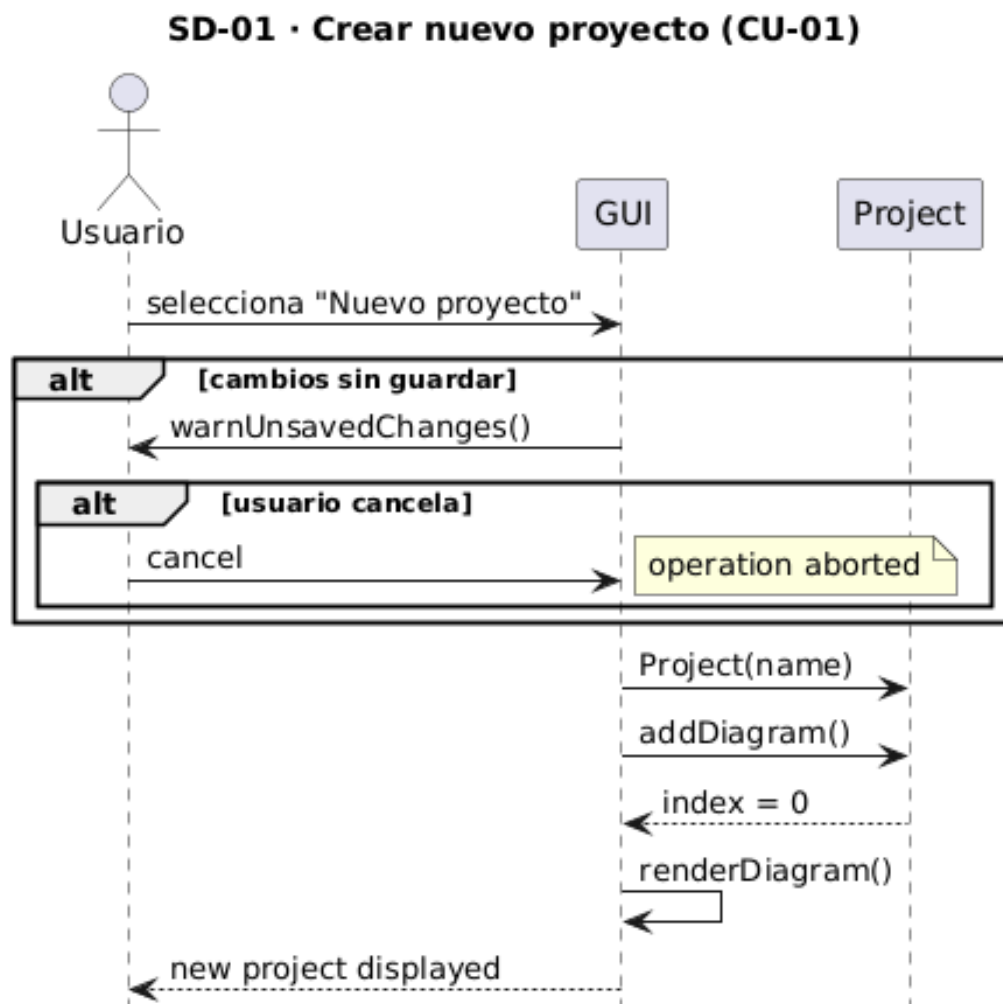


Figura B.2: Diagrama de secuencia del B.1: Crear nuevo proyecto.

B.5.2. SD-02: Guardar proyecto

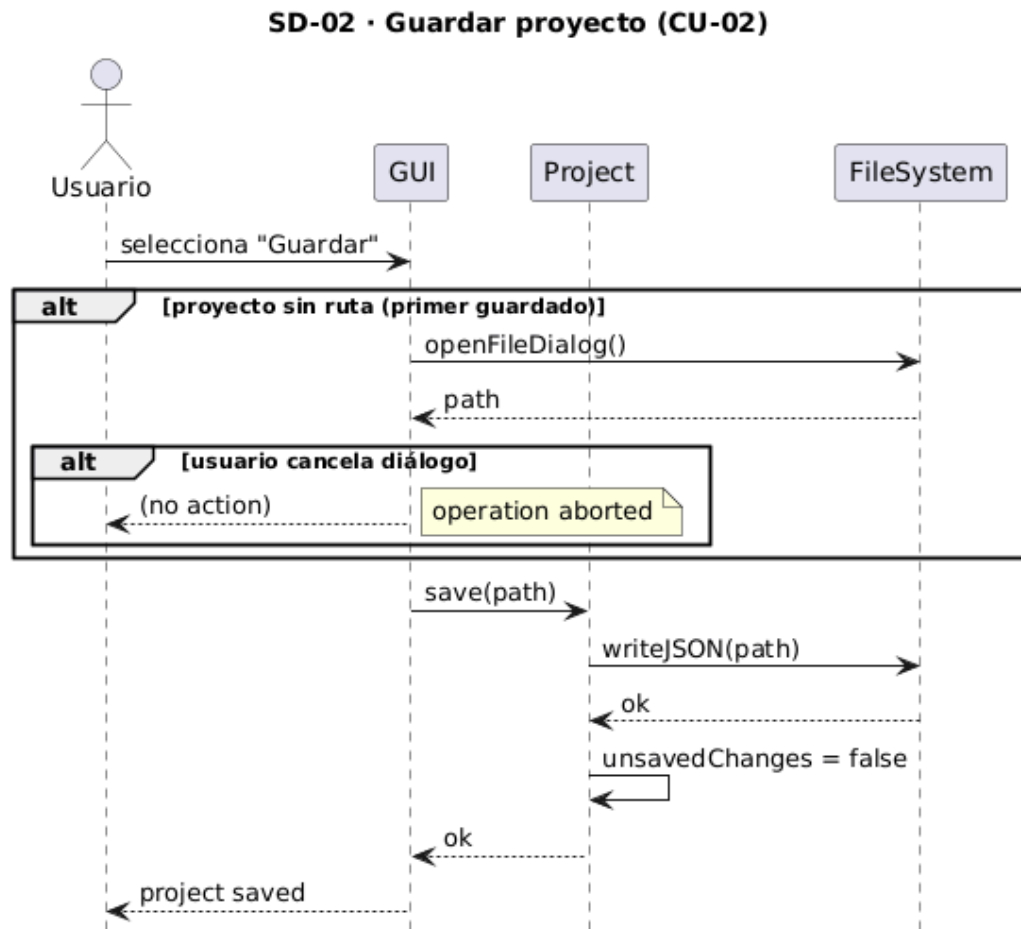


Figura B.3: Diagrama de secuencia del B.2: Guardar proyecto.

B.5.3. SD-03: Abrir proyecto

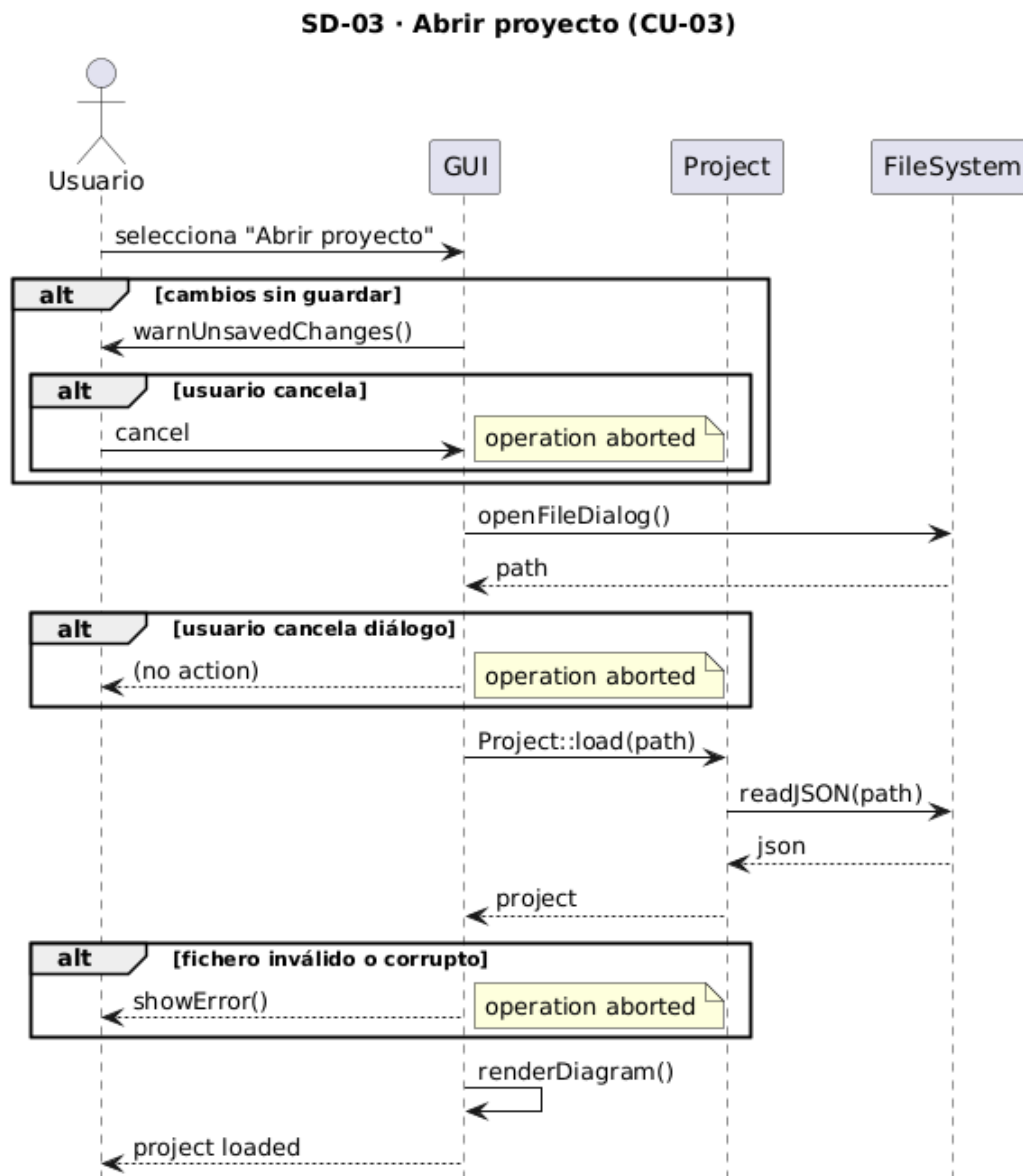


Figura B.4: Diagrama de secuencia del B.3: Abrir proyecto.

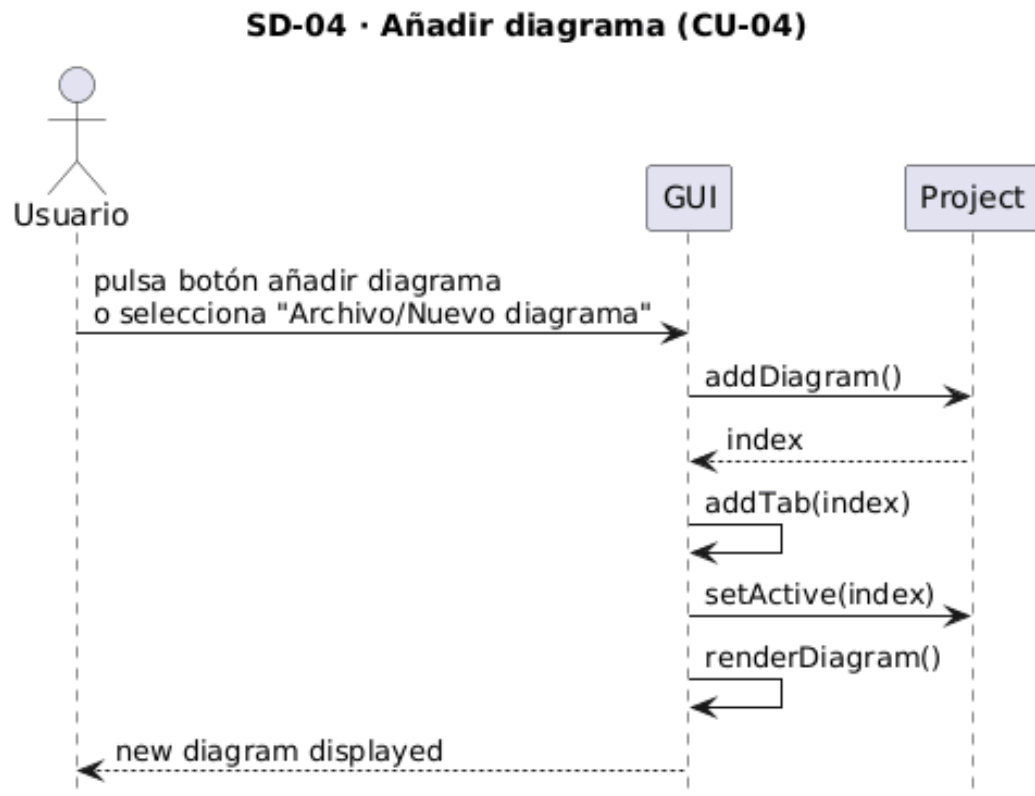
B.5.4. SD-04: Añadir diagrama

Figura B.5: Diagrama de secuencia del B.4: Añadir diagrama.

B.5.5. SD-05: Eliminar diagrama

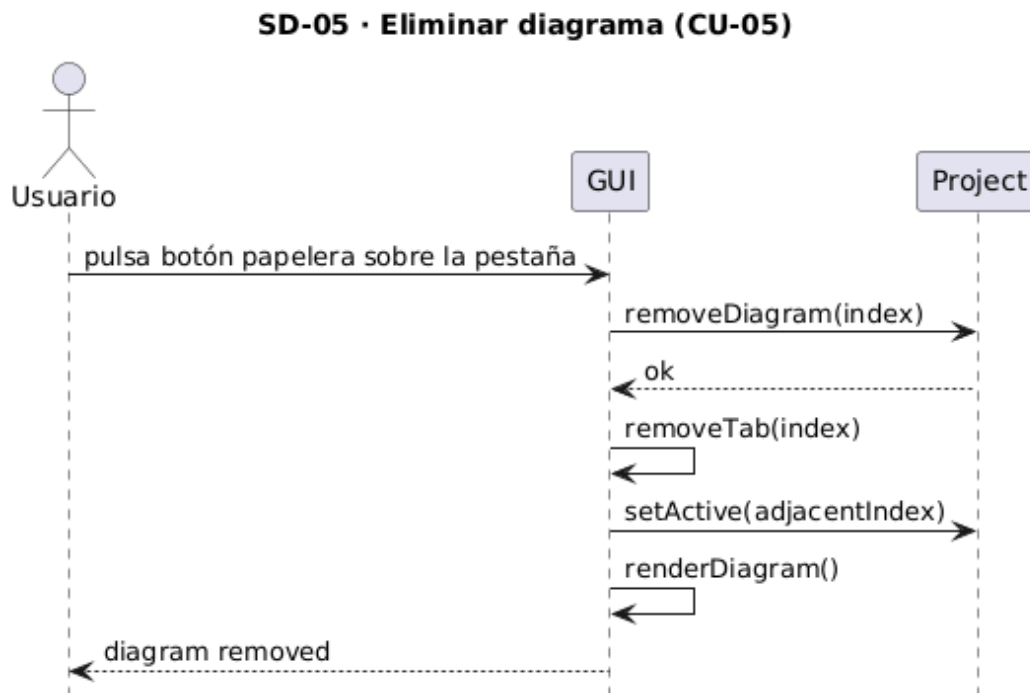
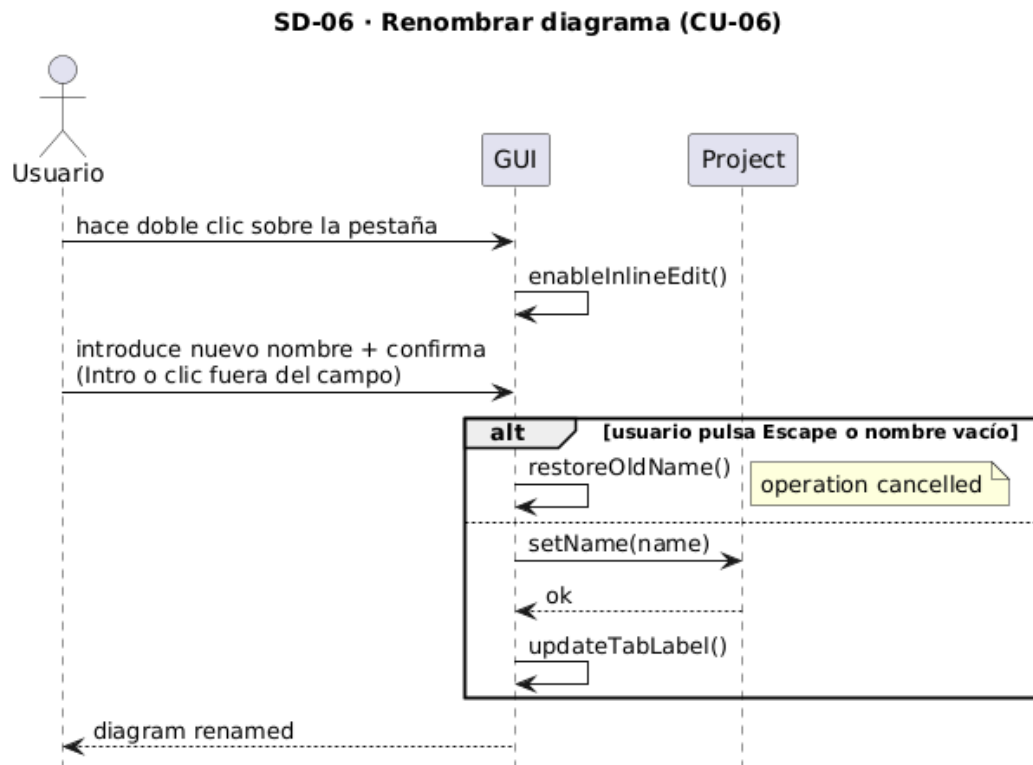


Figura B.6: Diagrama de secuencia del B.5: Eliminar diagrama.

B.5.6. SD-06: Renombrar diagrama**Figura B.7:** Diagrama de secuencia del B.6: Renombrar diagrama.

B.5.7. SD-07: Navegar entre diagramas

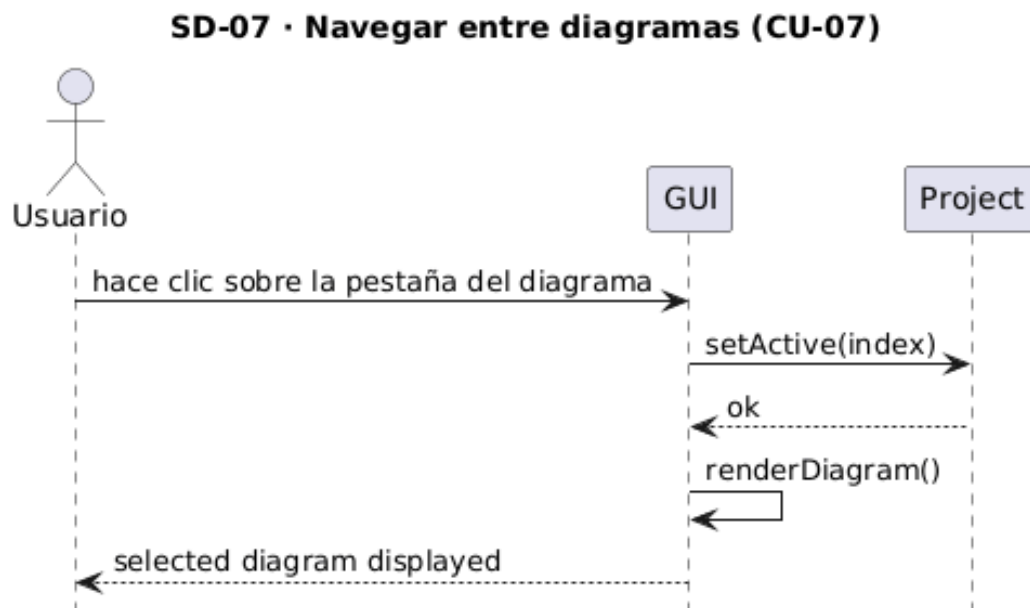


Figura B.8: Diagrama de secuencia del B.7: Navegar entre diagramas.

B.5.8. SD-08: Crear nodo raíz

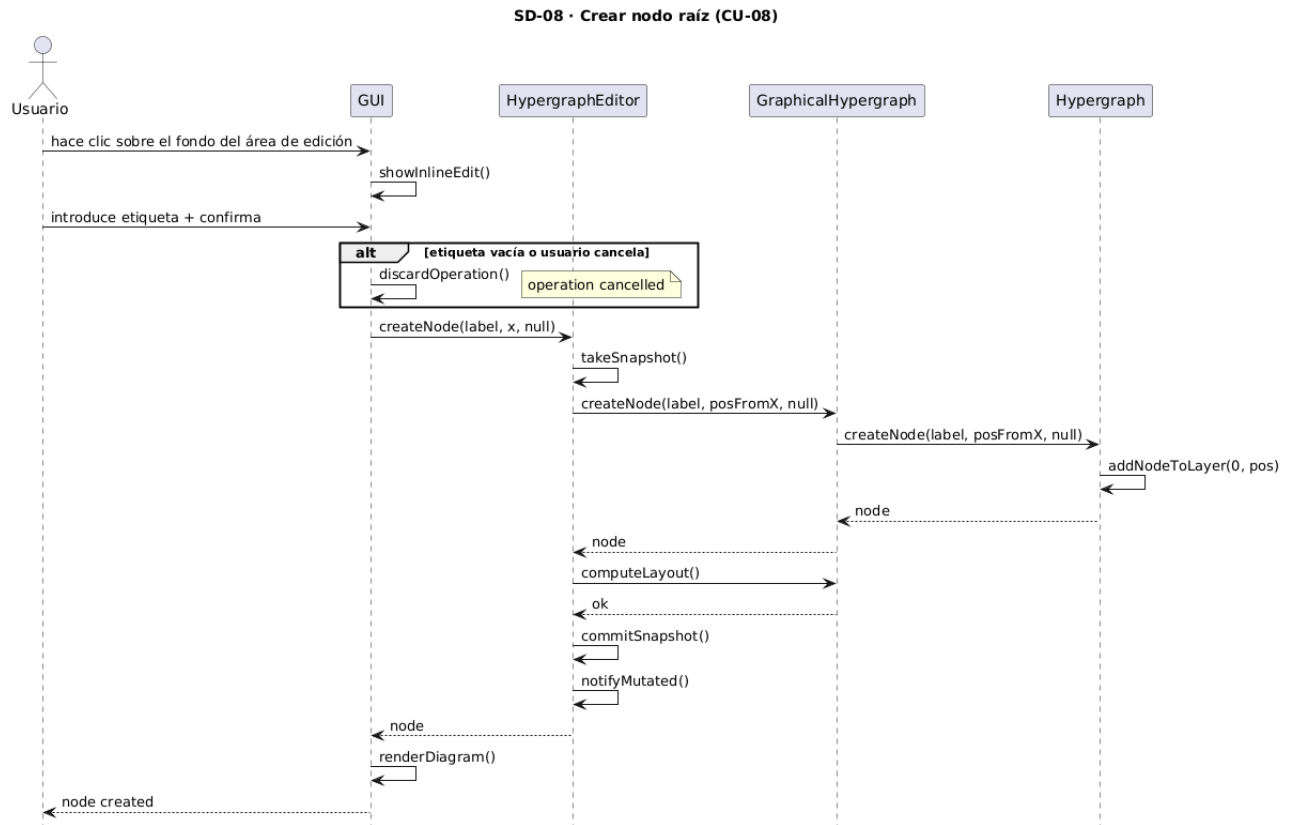


Figura B.9: Diagrama de secuencia del B.8: Crear nodo raíz.

B.5.9. SD-09: Crear nodo hijo

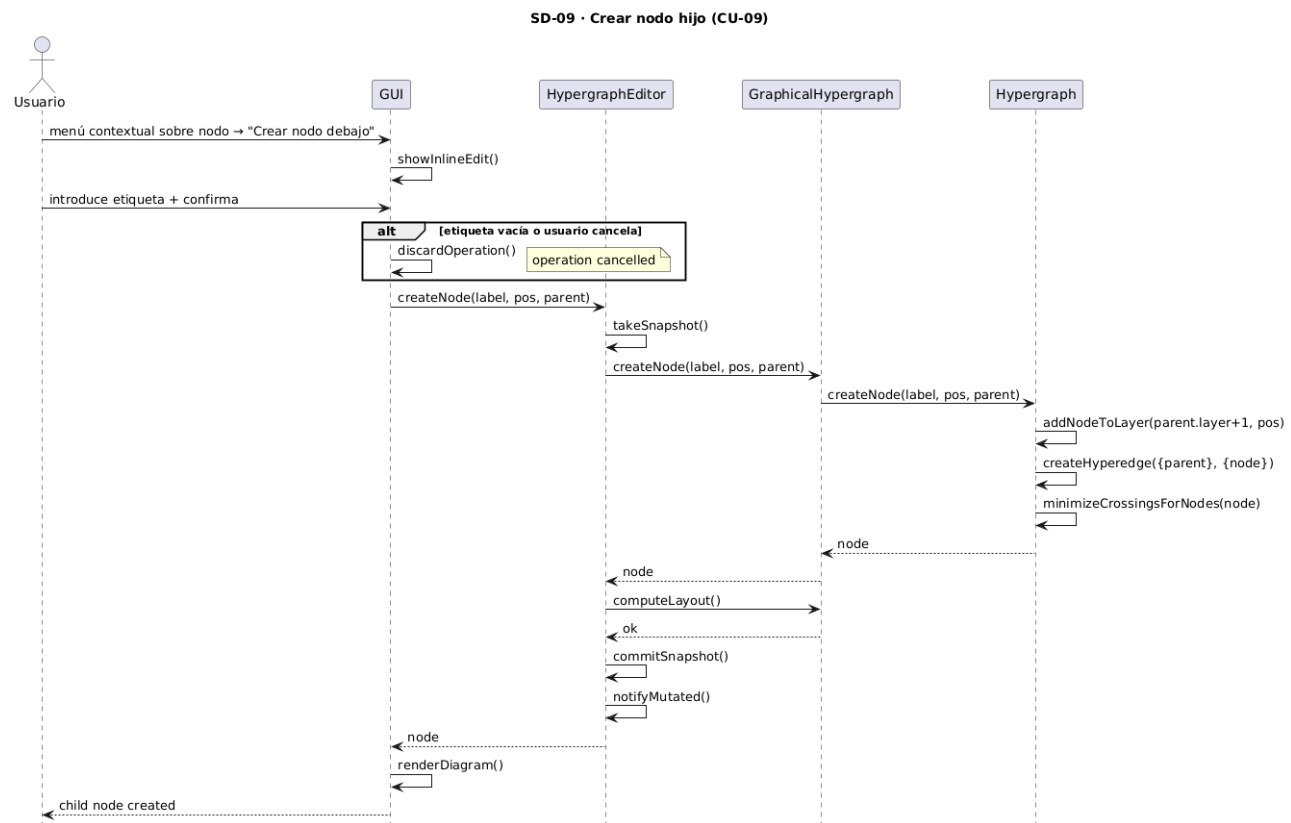


Figura B.10: Diagrama de secuencia del B.9: Crear nodo hijo.

B.5.10. SD-10: Crear nodo padre

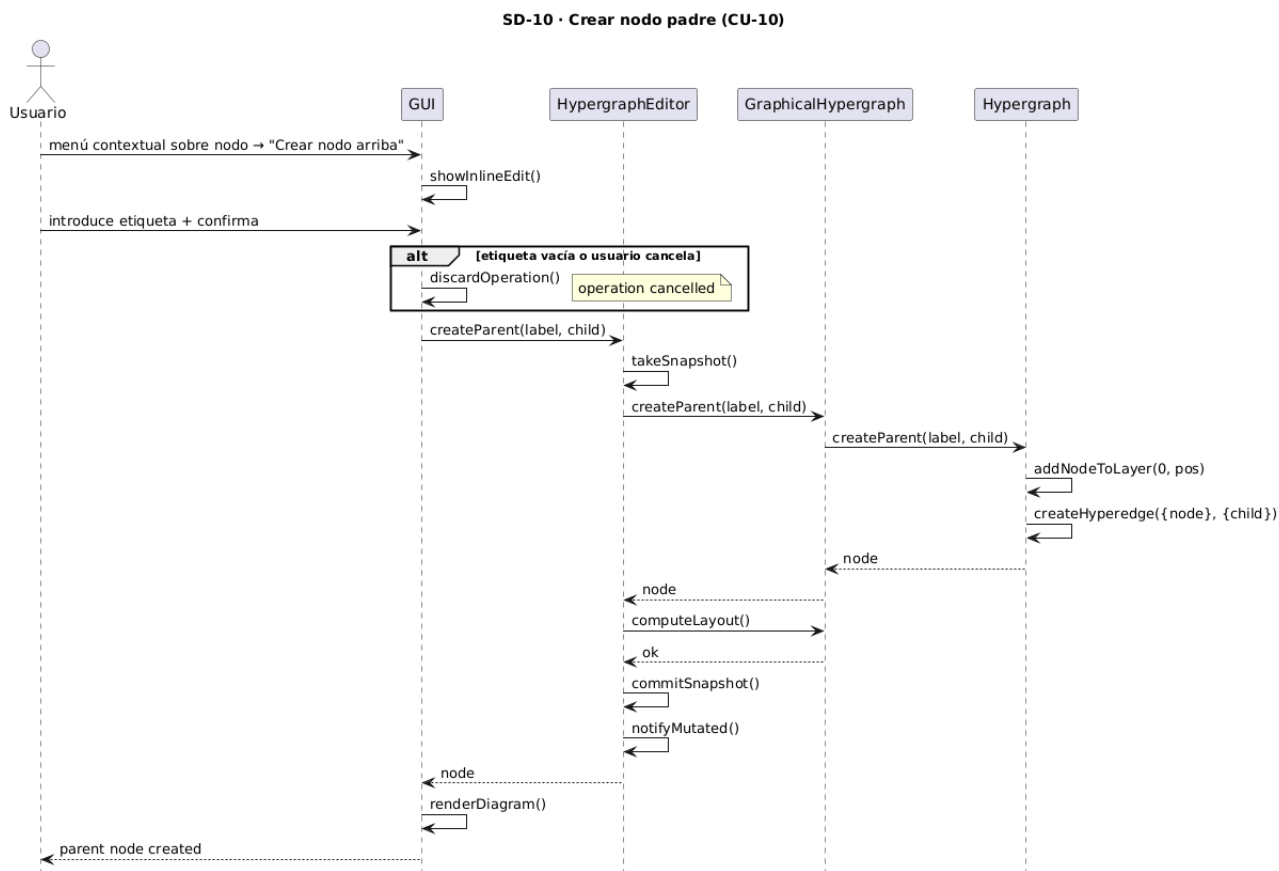


Figura B.11: Diagrama de secuencia del B.10: Crear nodo padre.

B.5.11. SD-11: Crear nodo dentro de una hiperarista

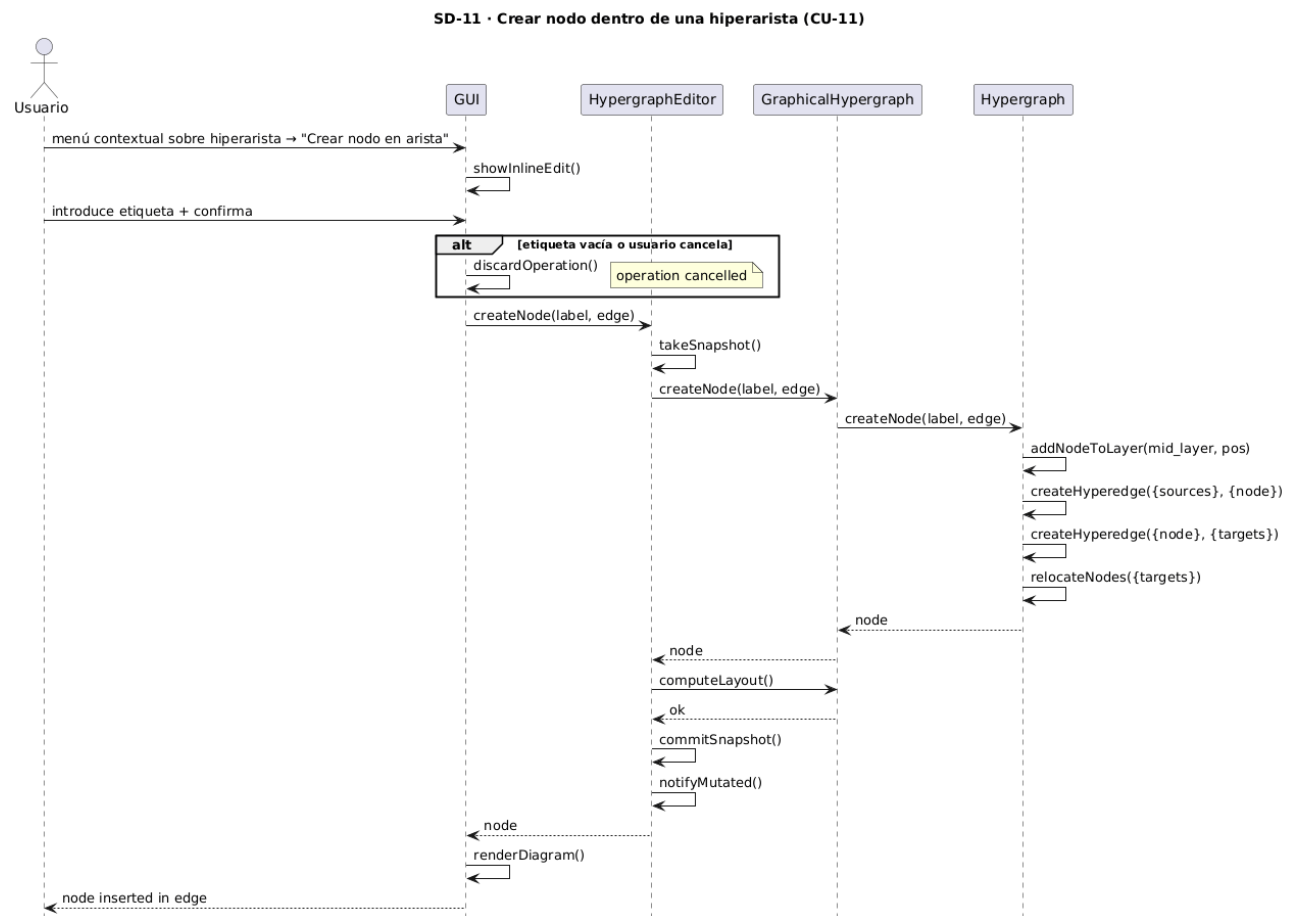


Figura B.12: Diagrama de secuencia del B.11: Crear nodo dentro de una hiperarista.

B.5.12. SD-12: Crear origen de una hiperarista

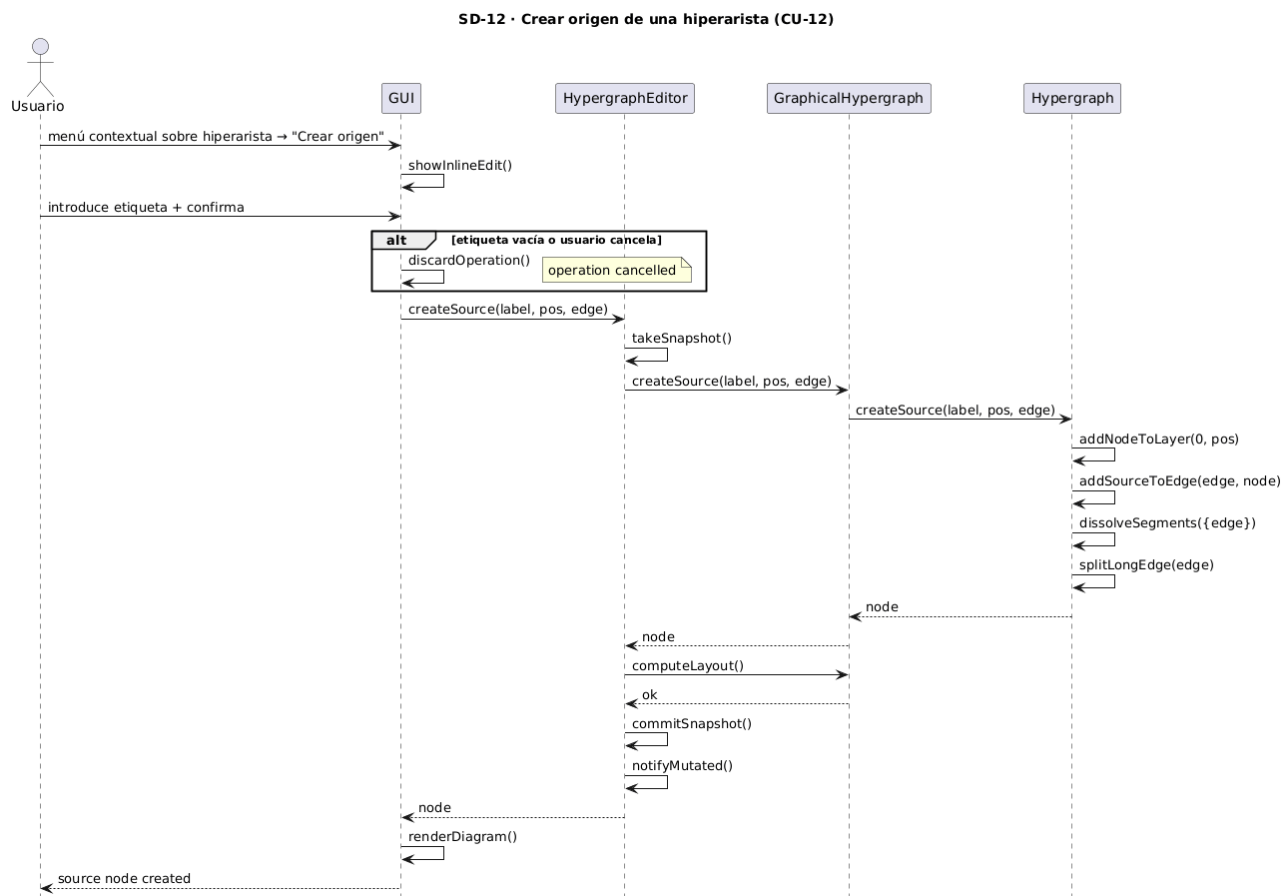


Figura B.13: Diagrama de secuencia del B.12: Crear origen de una hiperarista.

B.5.13. SD-13: Crear destino de una hiperarista

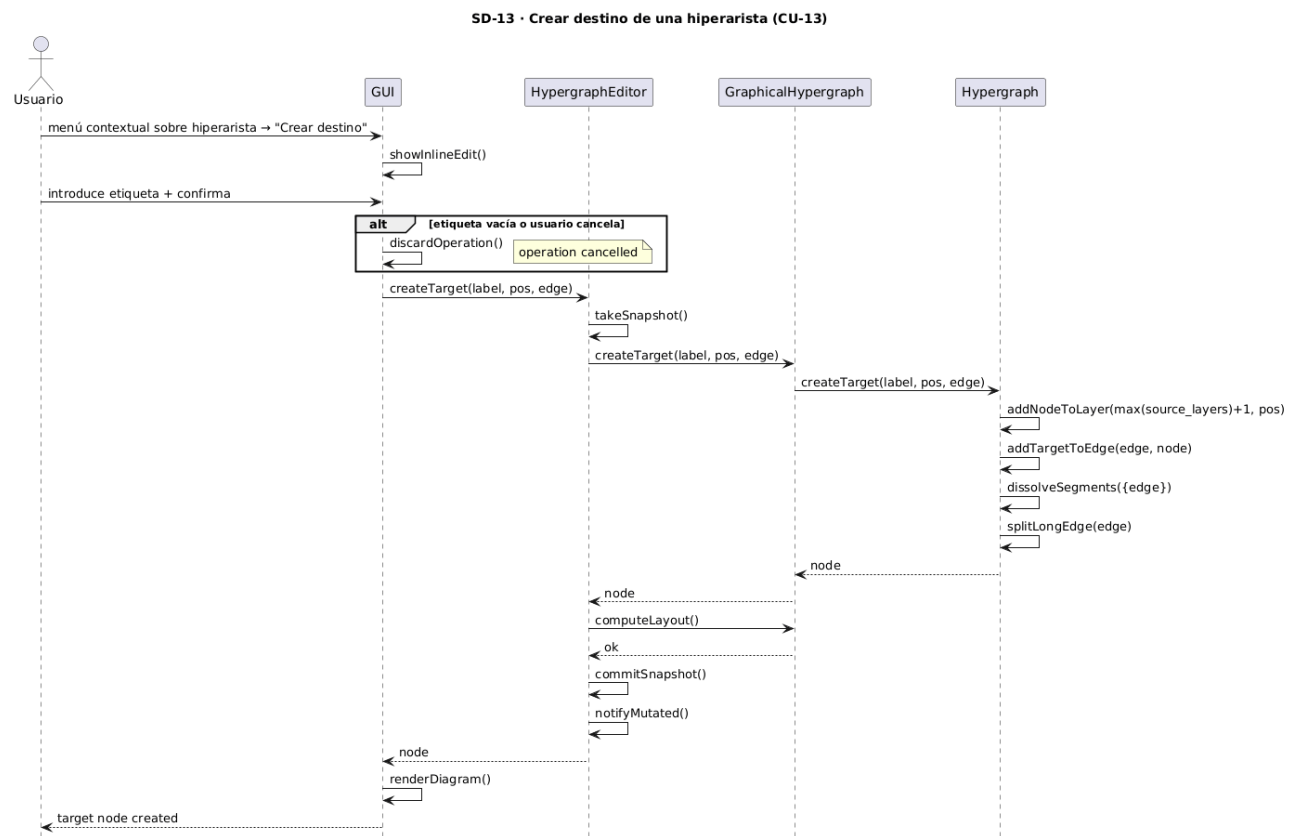


Figura B.14: Diagrama de secuencia del B.13: Crear destino de una hiperarista.

B.5.14. SD-14: Añadir un nuevo padre a nodo

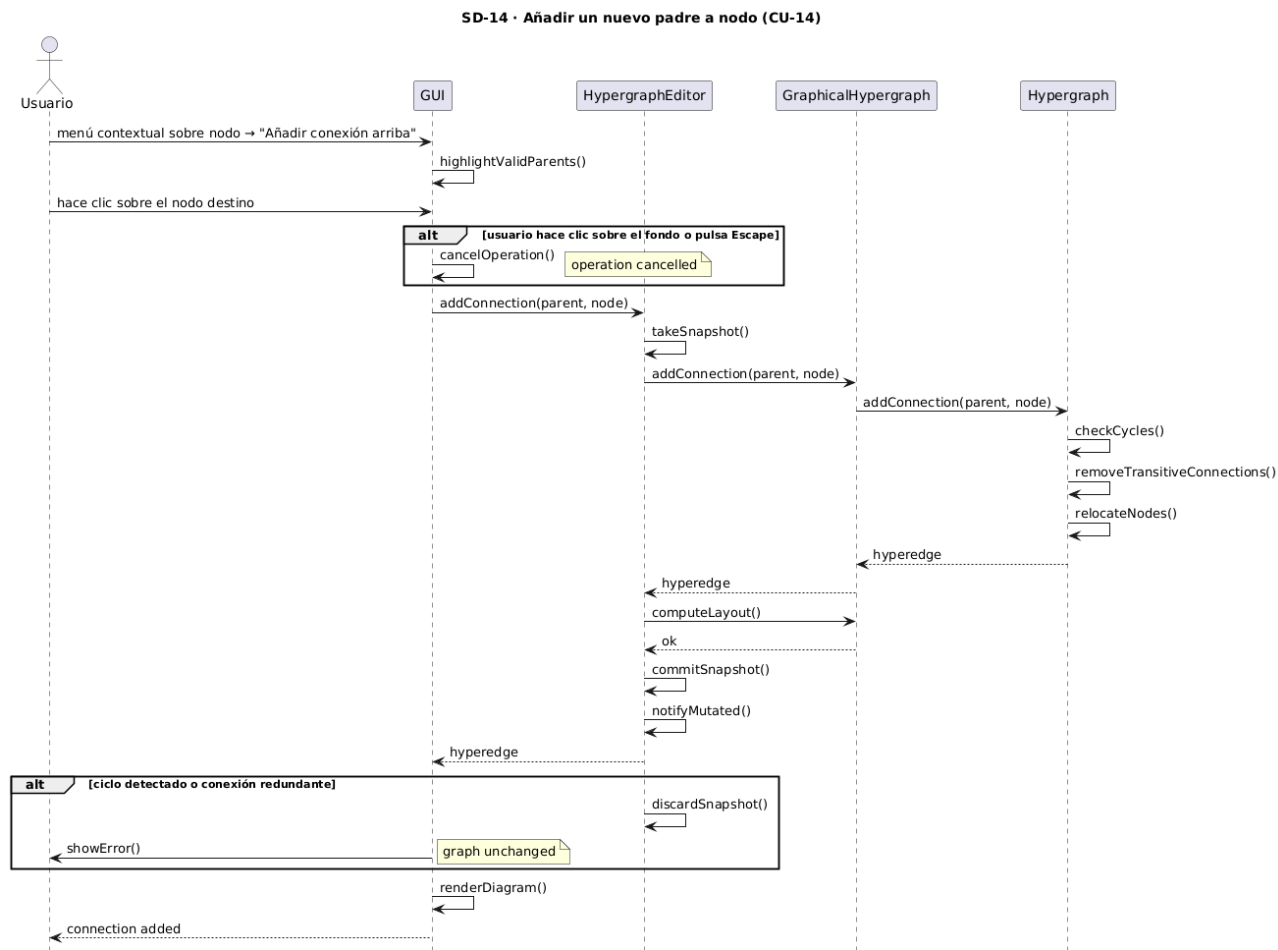


Figura B.15: Diagrama de secuencia del B.14: Añadir un nuevo padre a nodo.

B.5.15. SD-15: Añadir un nuevo hijo a nodo

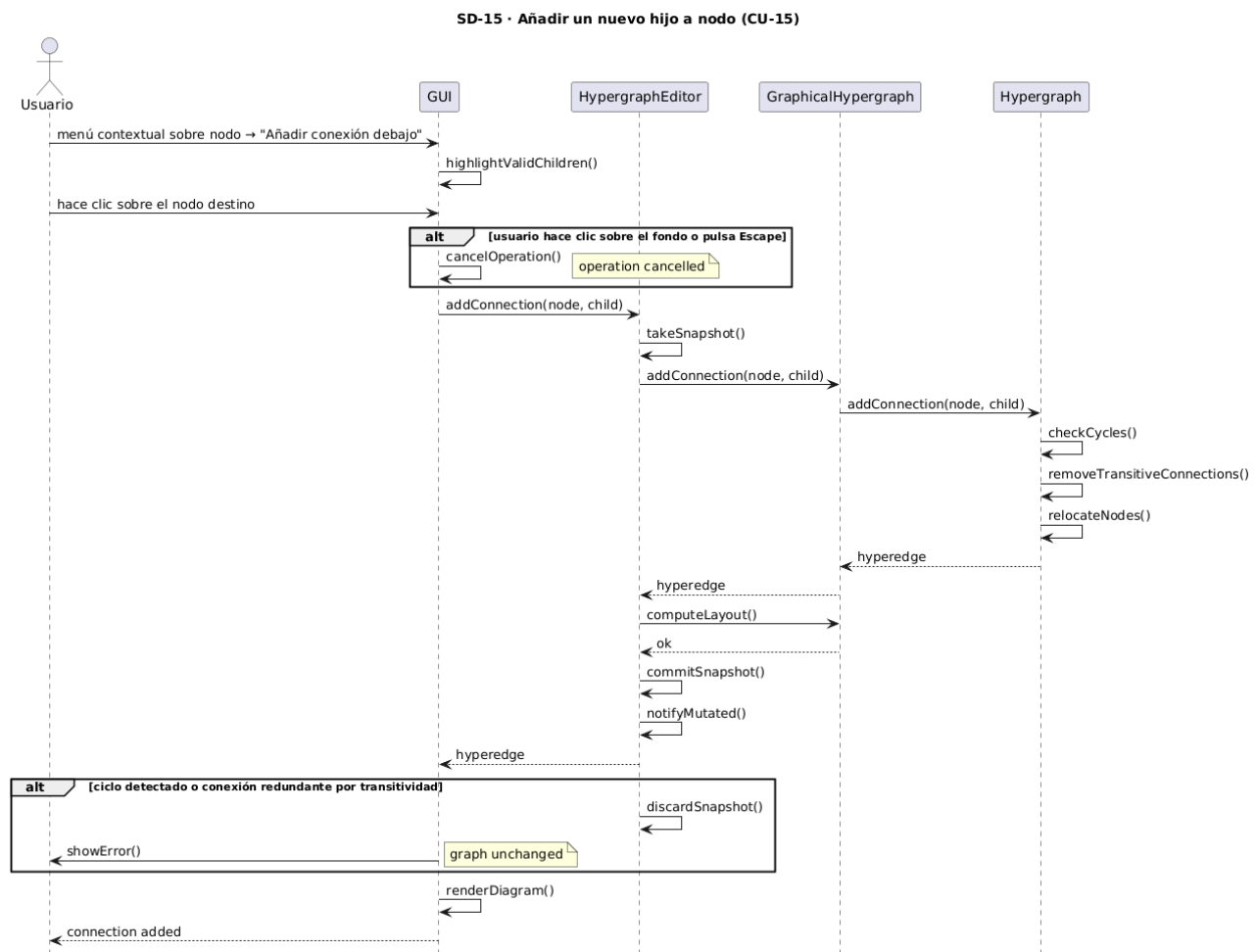


Figura B.16: Diagrama de secuencia del B.15: Añadir un nuevo hijo a nodo.

B.5.16. SD-16: Añadir nodo origen a hiperarista

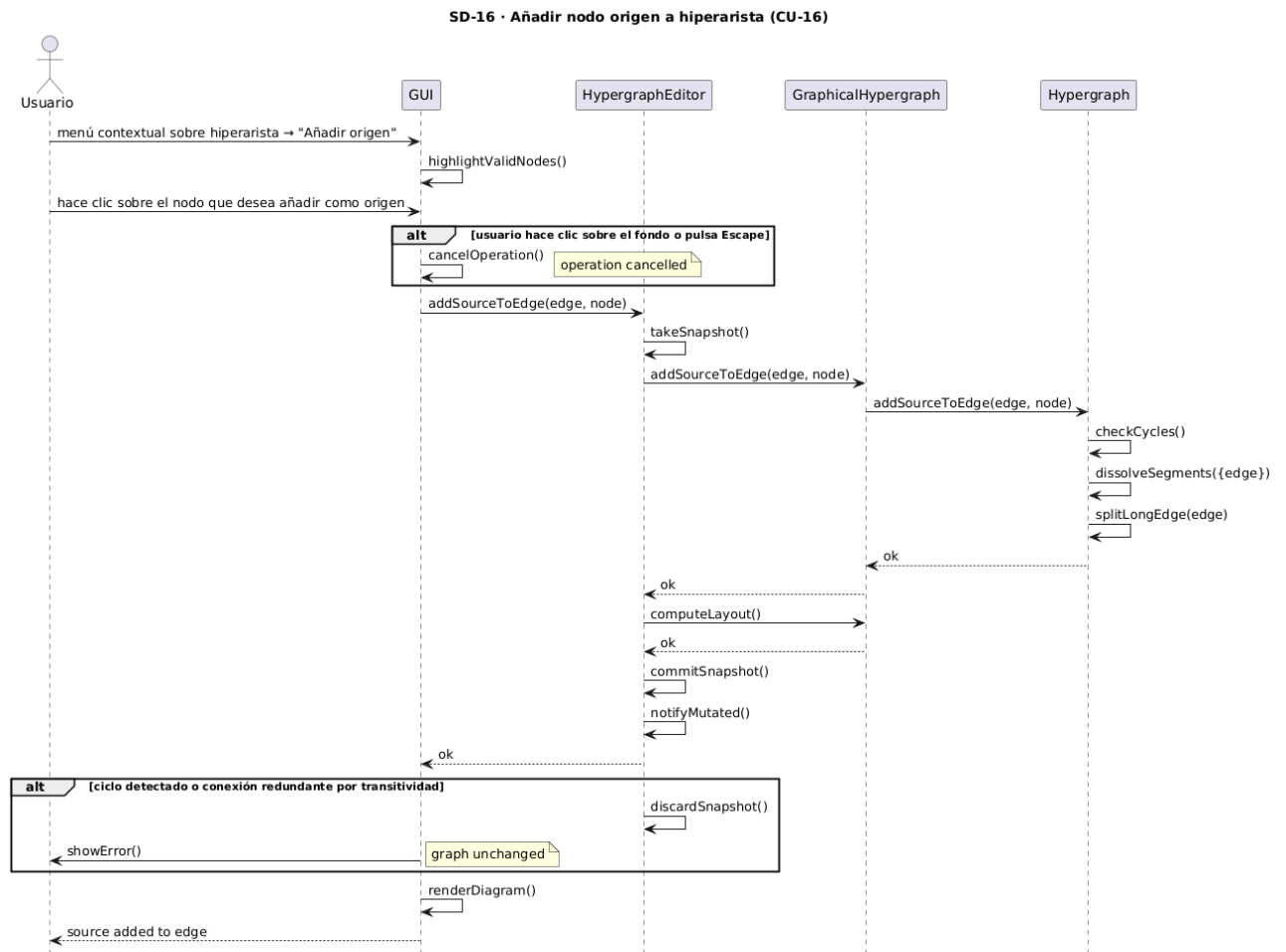


Figura B.17: Diagrama de secuencia del B.16: Añadir nodo origen a hiperarista.

B.5.17. SD-17: Añadir nodo destino a hiperarista

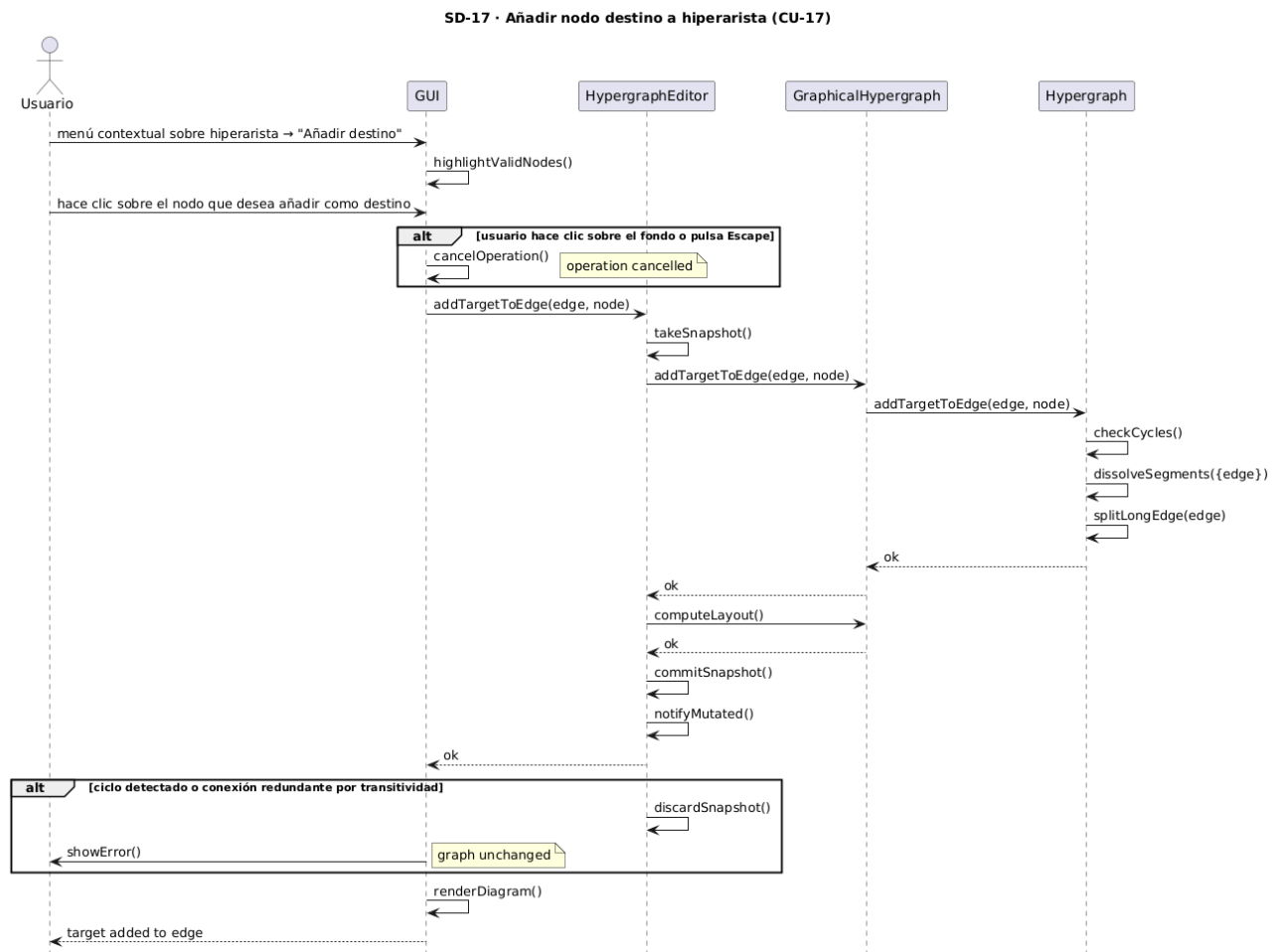


Figura B.18: Diagrama de secuencia del B.17: Añadir nodo destino a hiperarista.

B.5.18. SD-18: Eliminar nodo

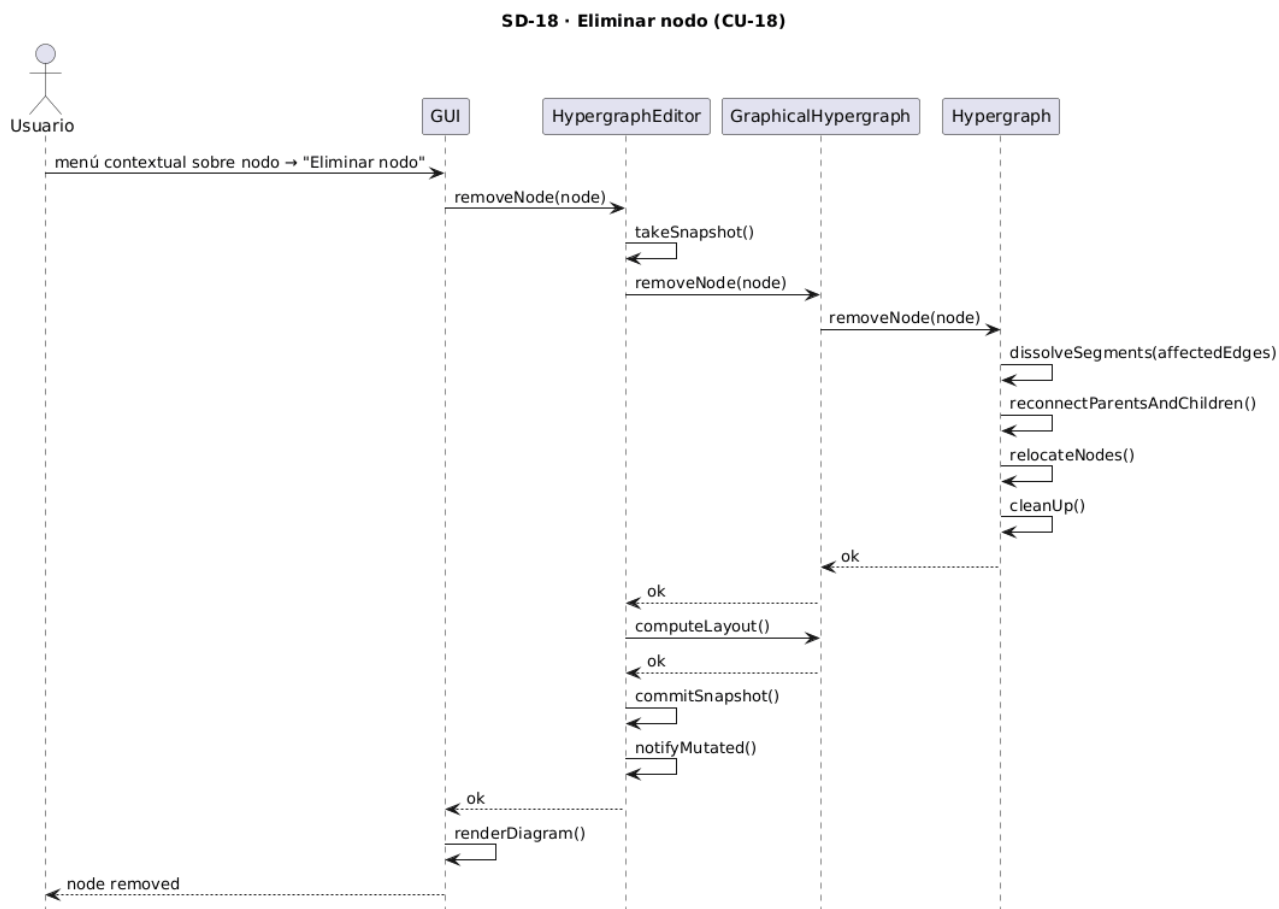


Figura B.19: Diagrama de secuencia del B.18: Eliminar nodo.

B.5.19. SD-19: Eliminar conexión entre nodos

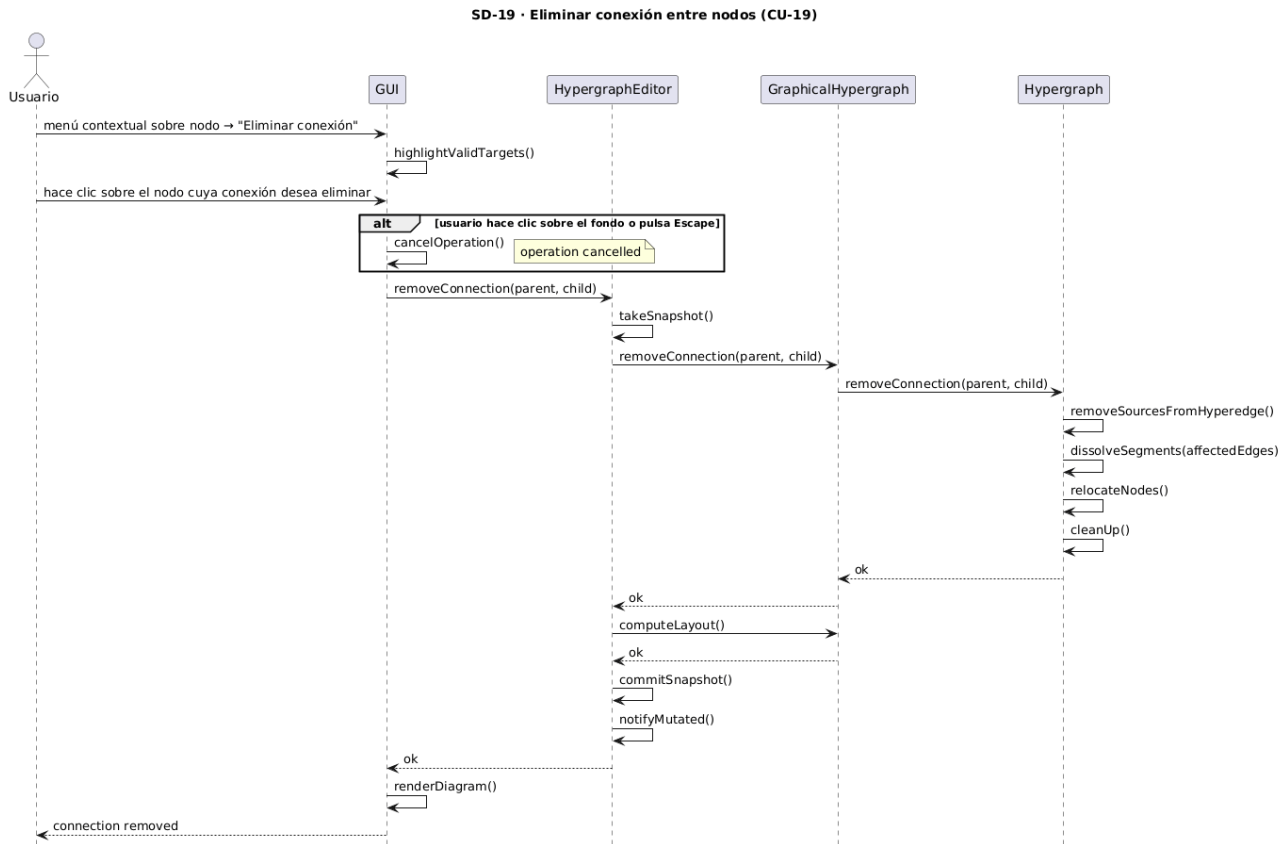


Figura B.20: Diagrama de secuencia del B.19: Eliminar conexión entre nodos.

B.5.20. SD-20: Eliminar origen de una hiperarista

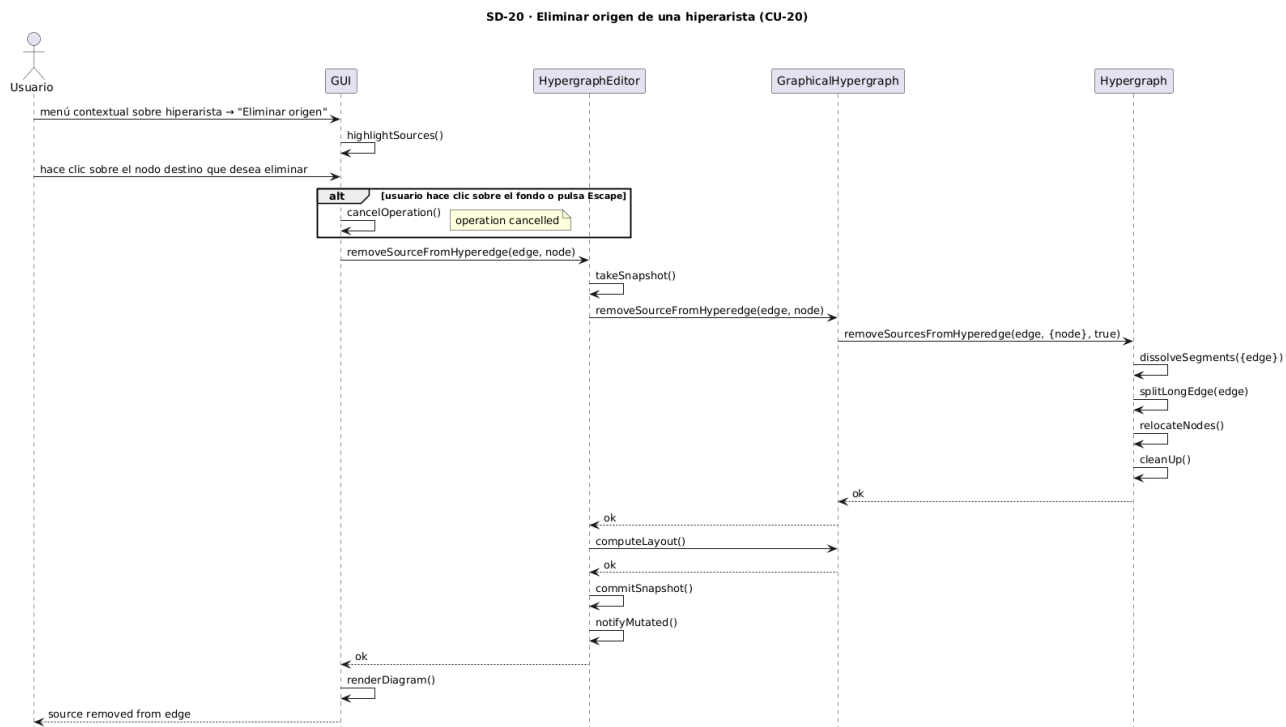


Figura B.21: Diagrama de secuencia del B.20: Eliminar origen de una hiperarista.

B.5.21. SD-21: Eliminar destino de una hiperarista

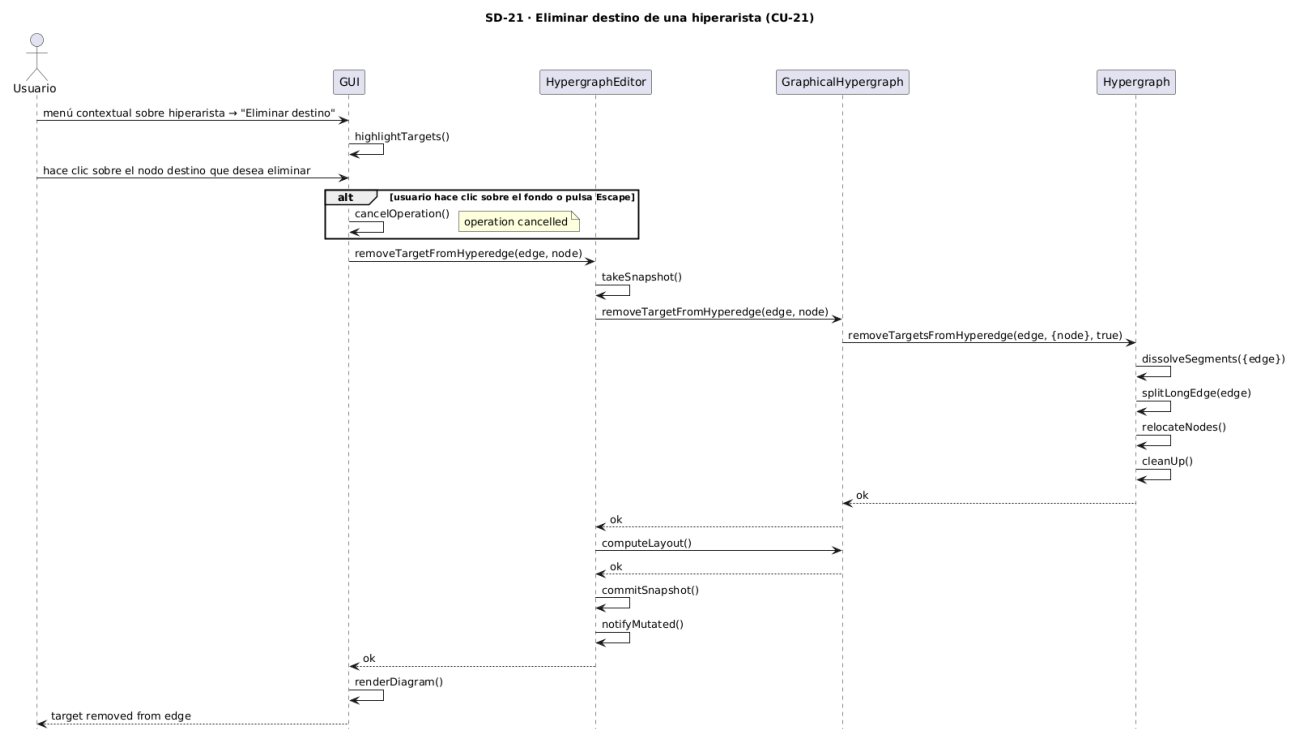


Figura B.22: Diagrama de secuencia del B.21: Eliminar destino de una hiperarista.

B.5.22. SD-22: Eliminar hiperarista completa

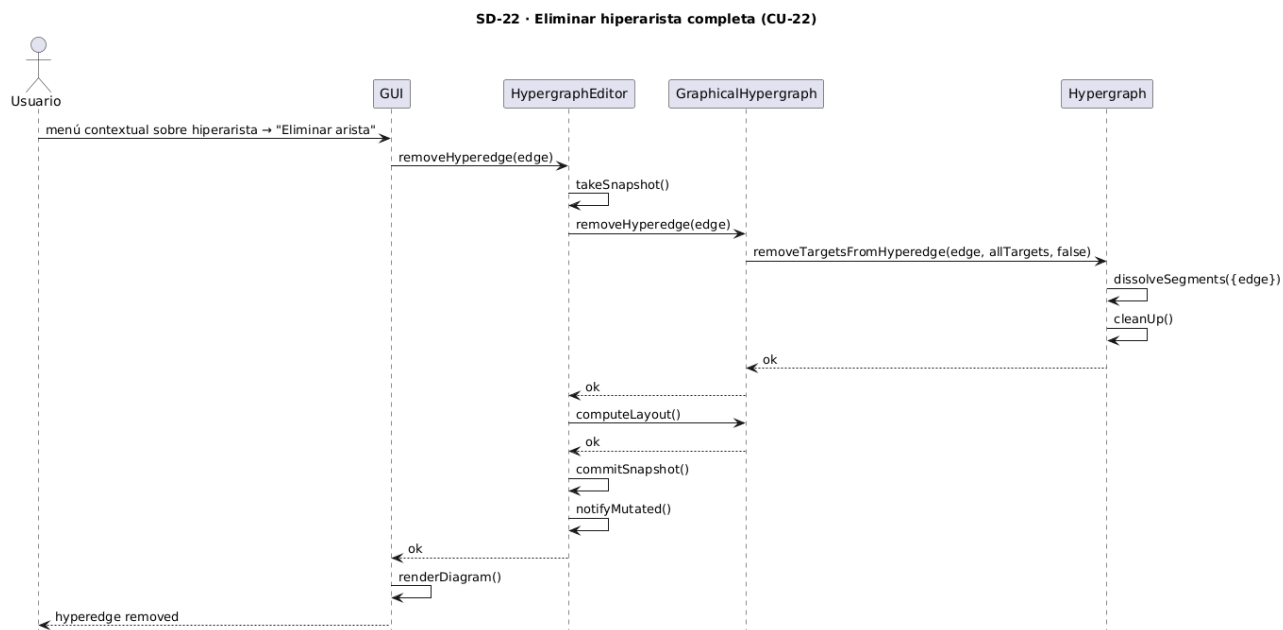


Figura B.23: Diagrama de secuencia del B.22: Eliminar hiperarista completa.

B.5.23. SD-23: Fusionar nodos

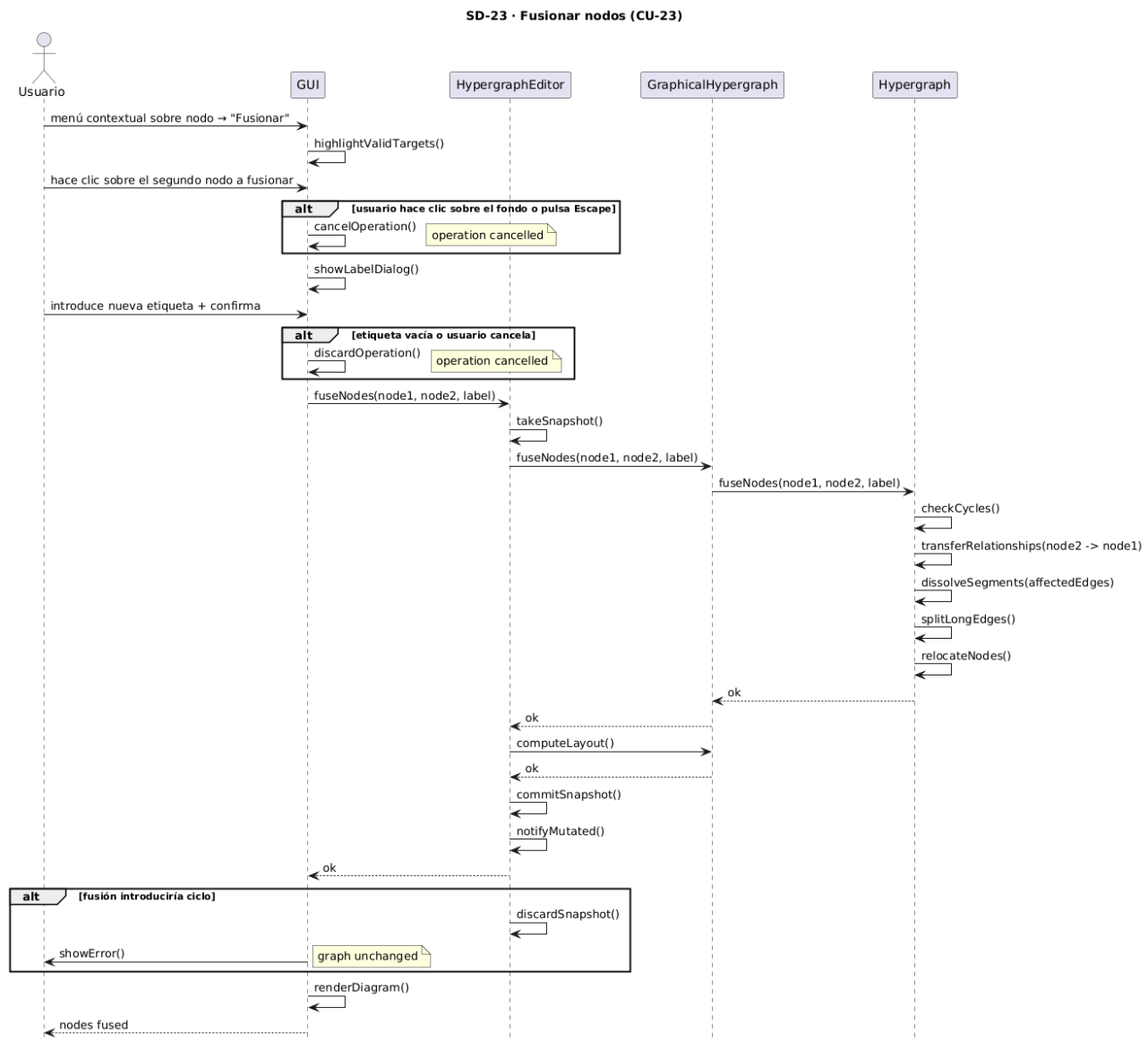


Figura B.24: Diagrama de secuencia del B.23: Fusionar nodos.

B.5.24. SD-24: Renombrar nodo

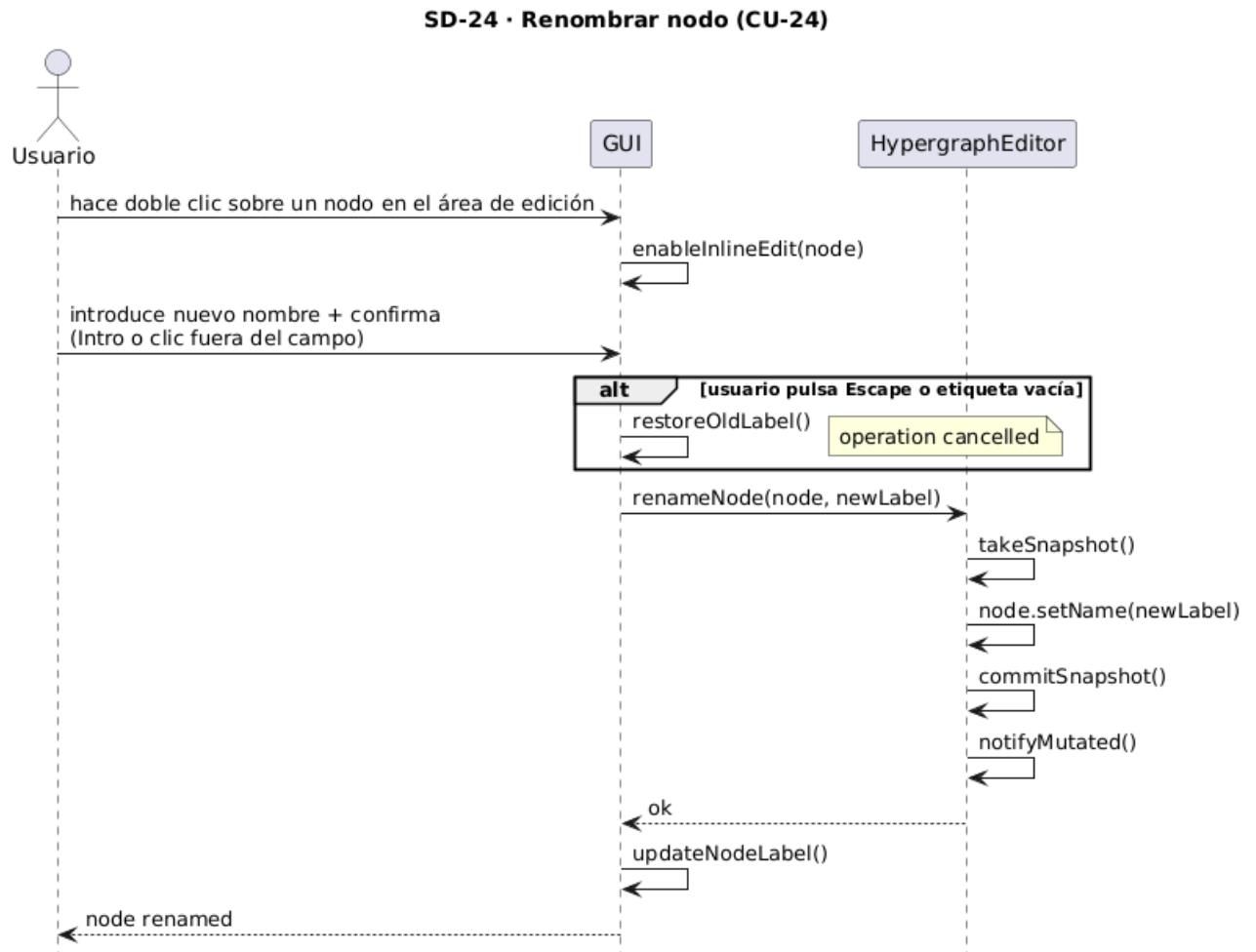


Figura B.25: Diagrama de secuencia del B.24: Renombrar nodo.

B.5.25. SD-25: Reordenar nodo en su capa

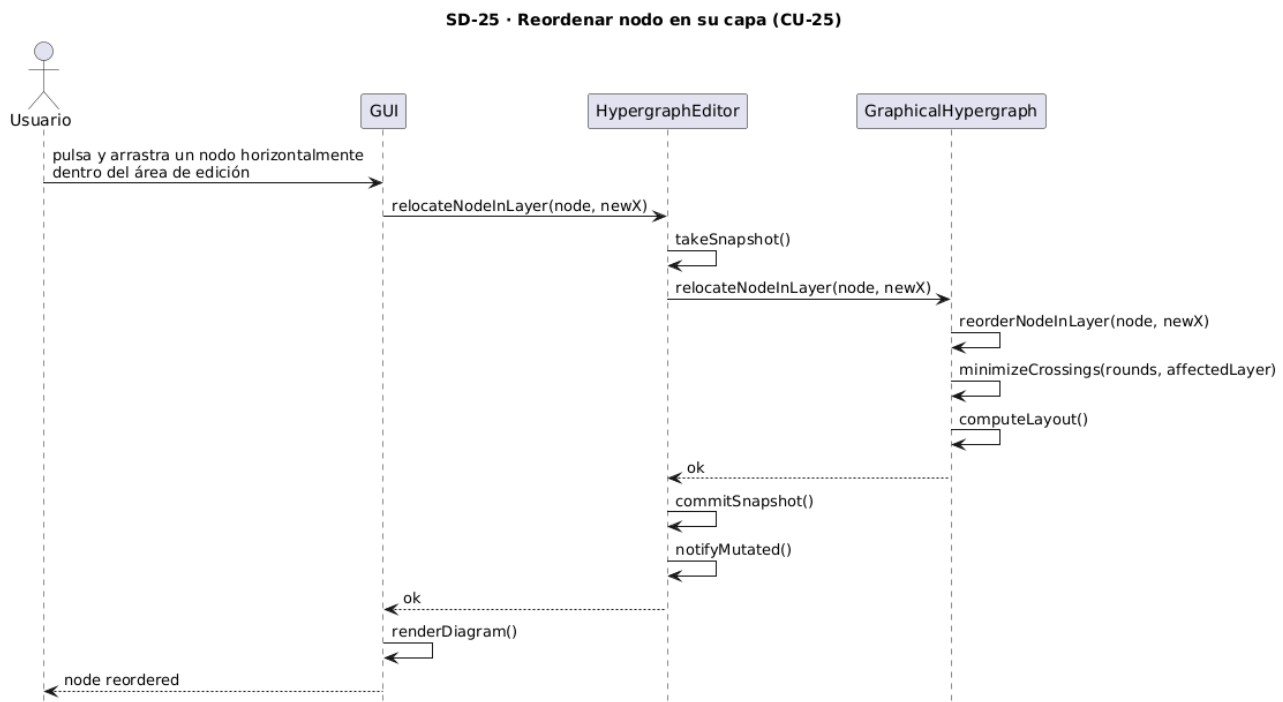


Figura B.26: Diagrama de secuencia del B.25: Reordenar nodo en su capa.

B.5.26. SD-26: Minimizar cruces manualmente

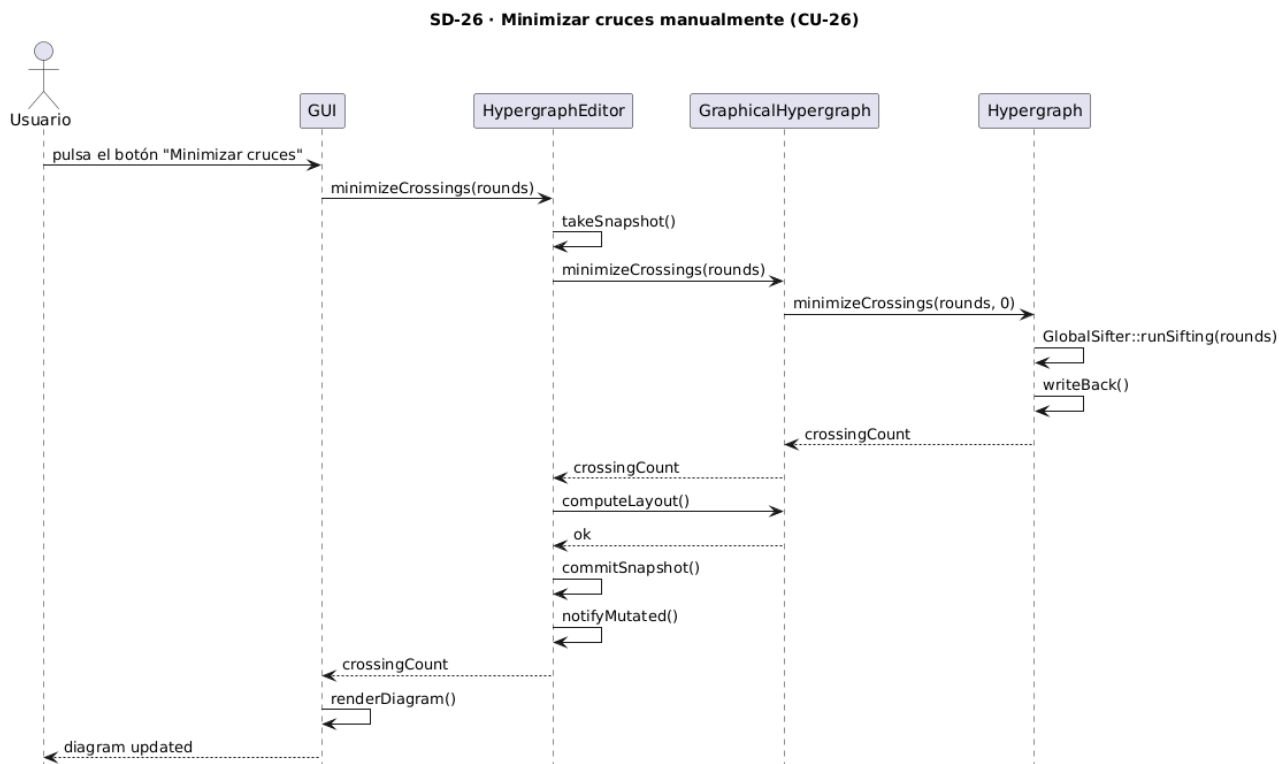


Figura B.27: Diagrama de secuencia del B.26: Minimizar cruces manualmente.

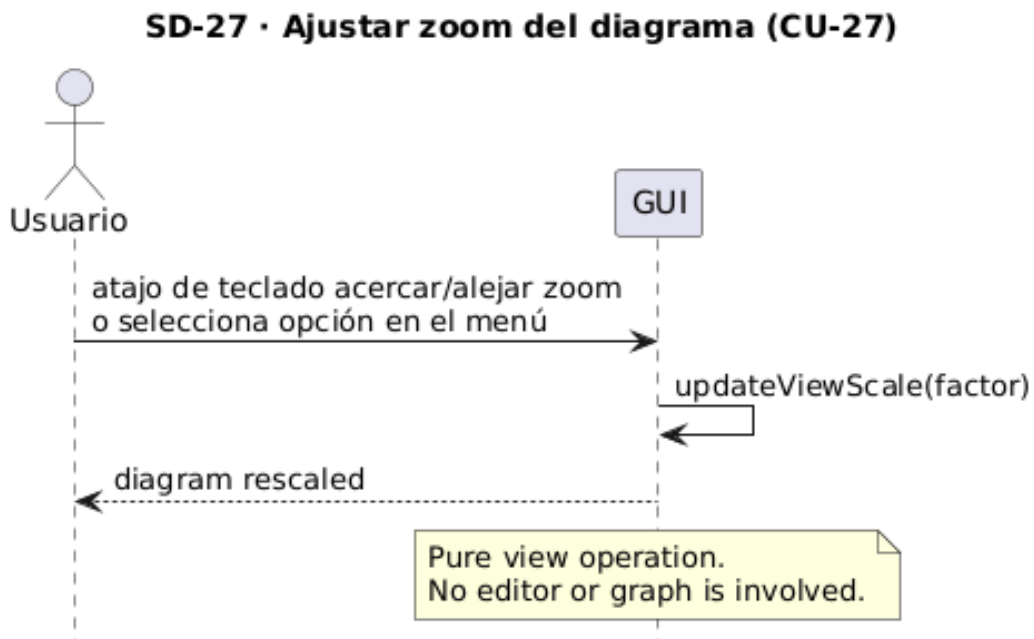
B.5.27. SD-27: Ajustar zoom del diagrama

Figura B.28: Diagrama de secuencia del B.27: Ajustar zoom del diagrama.

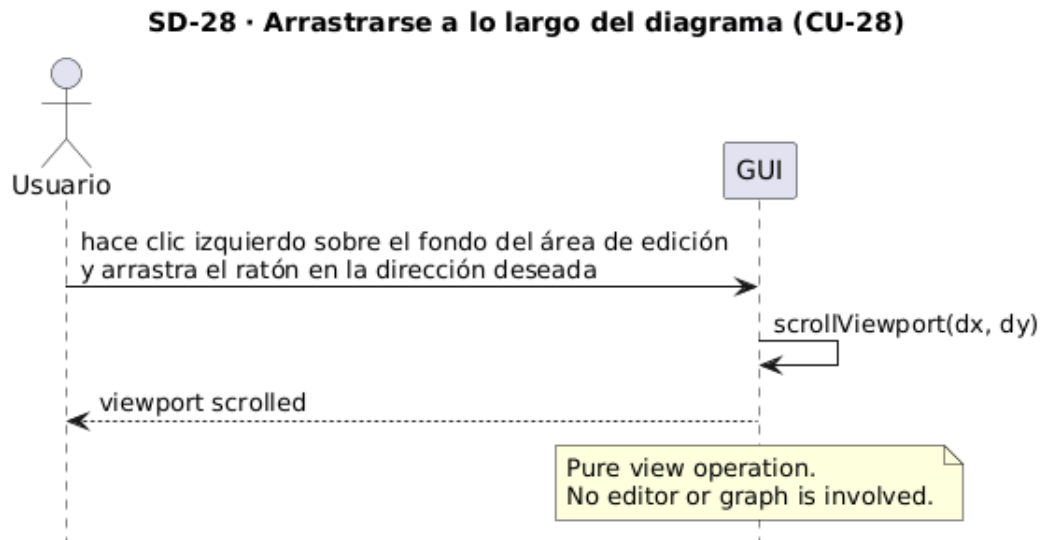
B.5.28. SD-28: Arrastrarse a lo largo del diagrama

Figura B.29: Diagrama de secuencia del B.28: Arrastrarse a lo largo del diagrama.

B.5.29. SD-29: Incorporar diagrama al diagrama conjunto

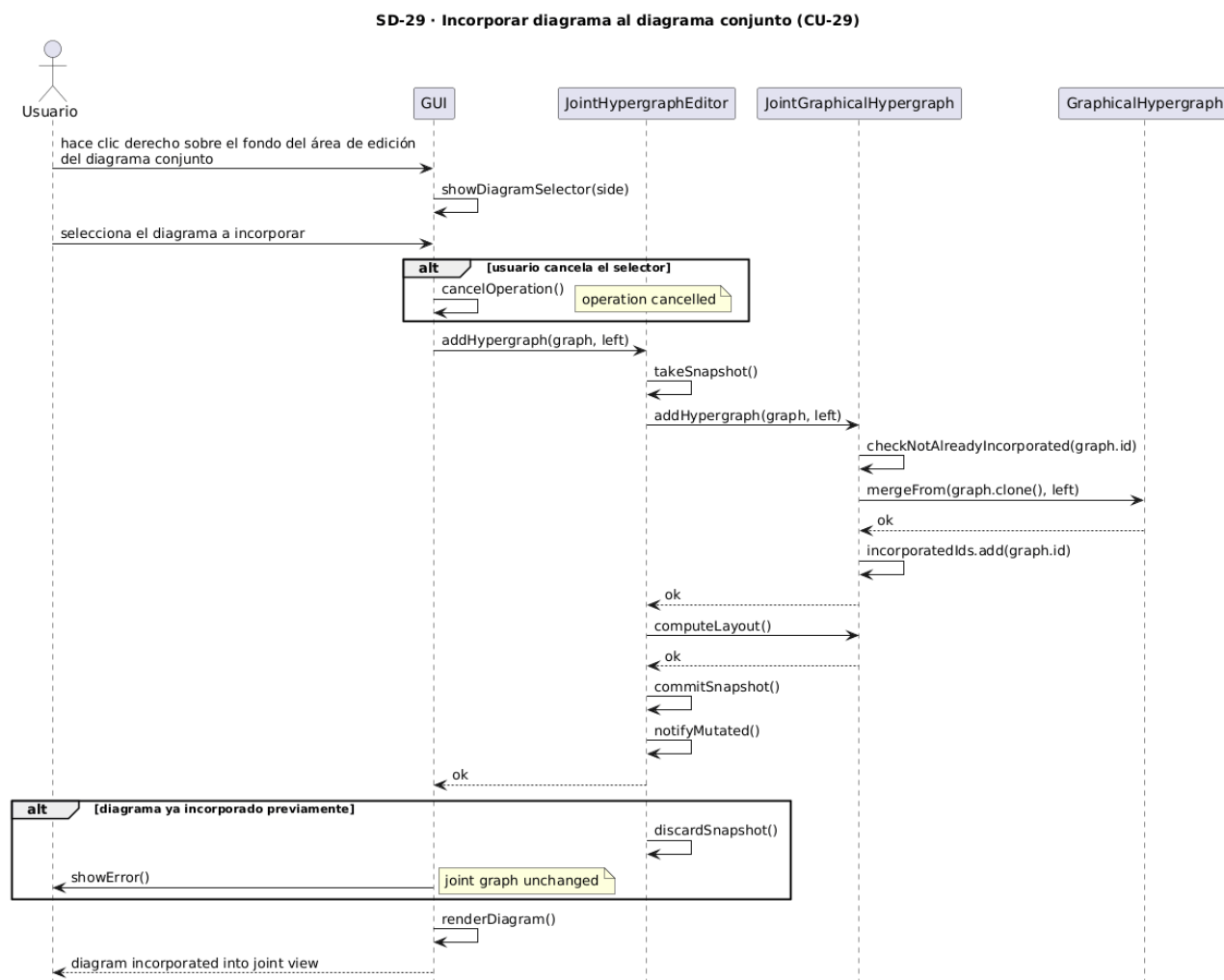


Figura B.30: Diagrama de secuencia del B.29: Incorporar diagrama al diagrama conjunto.

B.5.30. SD-30: Calcular layout

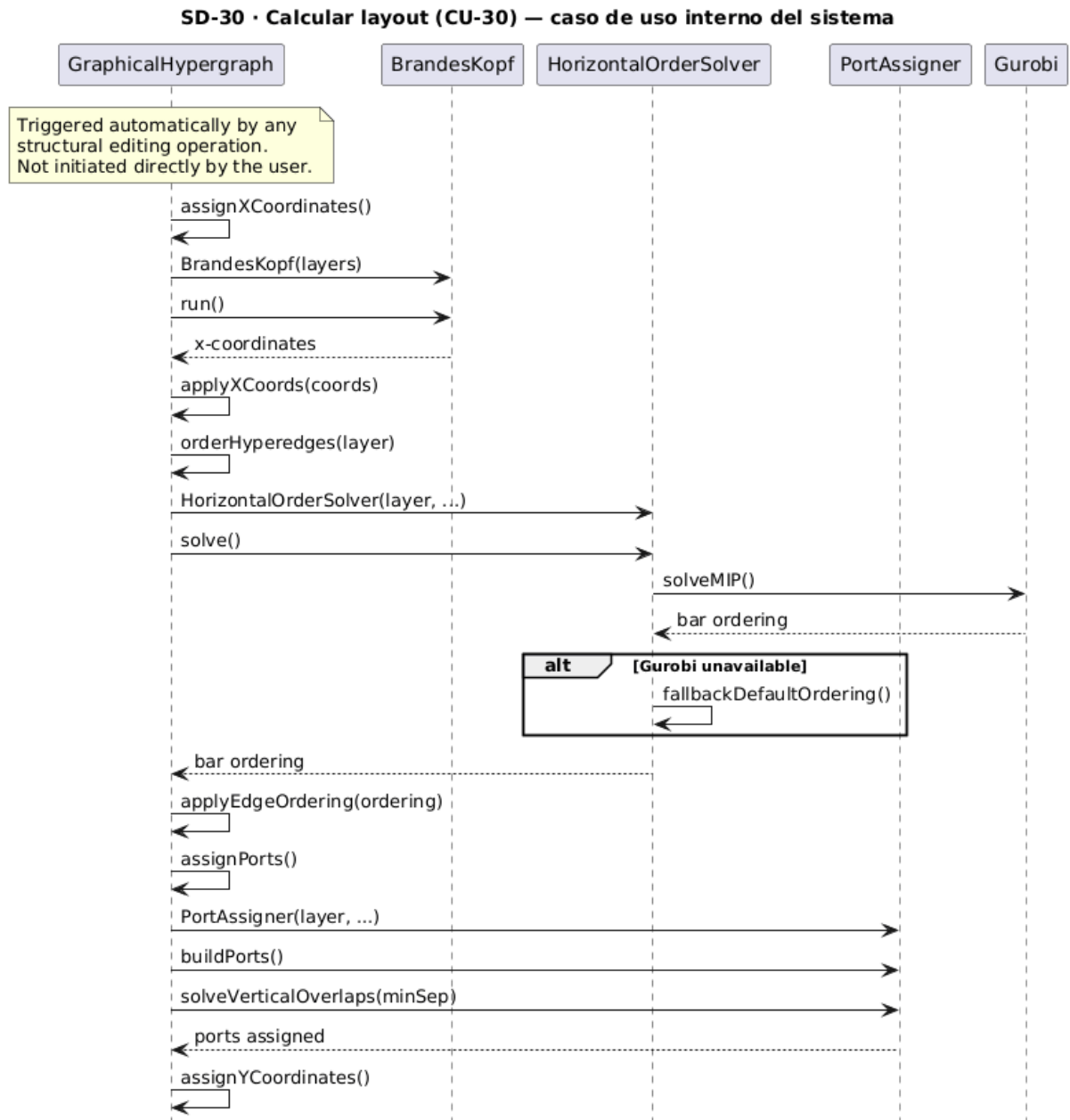


Figura B.31: Diagrama de secuencia del B.30: Calcular layout.

B.5.31. SD-31: Deshacer operación

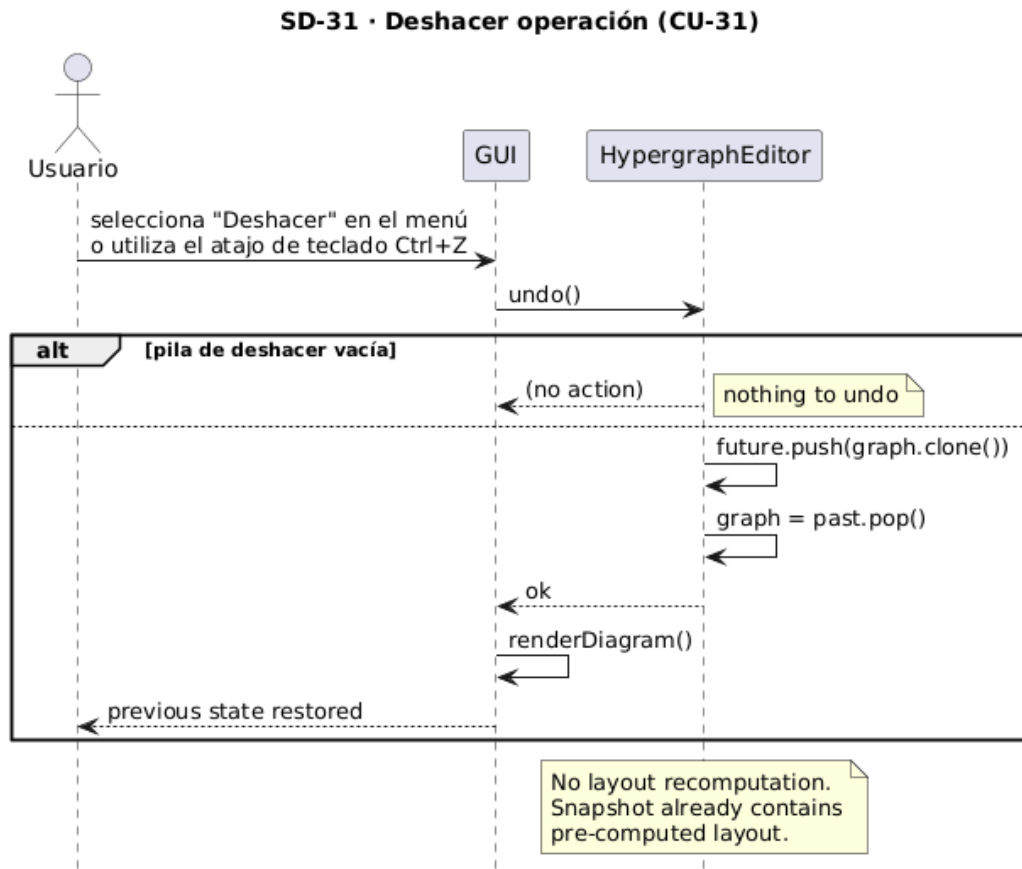
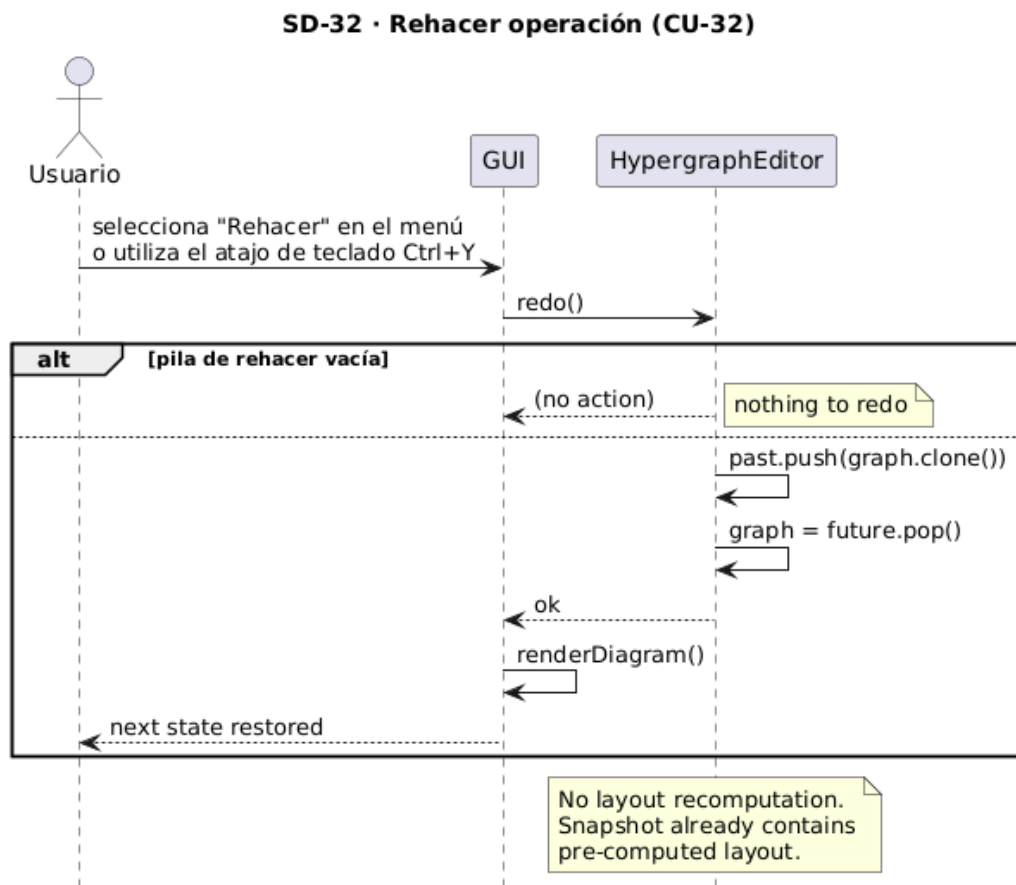


Figura B.32: Diagrama de secuencia del B.31: Deshacer operación.

B.5.32. SD-32: Rehacer operación**Figura B.33:** Diagrama de secuencia del B.32: Rehacer operación.

DISEÑO DEL SISTEMA

C.1. Catálogo de Maquetas de la interfaz de usuario

Como ya se ha mencionado en el texto principal, las maquetas que se presentaron tuvieron un matiz novedoso, ya que se optó por hacer una versión muy simplificada de la aplicación, centrada únicamente en la creación de árboles. Esto permitió mostrar al cliente una versión interactiva de la idea de aplicación desde las primeras etapas del diseño, lo que facilitó la validación de la dirección que se estaba tomando.

Este movimiento fue especialmente beneficioso dado que el cliente no tiene conocimientos de informática, por lo que una maqueta interactiva era mucho más efectiva para comunicar la visión de la aplicación que un conjunto de diagramas estáticos. A continuación, en las figuras siguientes aparecen los primeros conceptos de la aplicación, incluso se incorporan algunas capturas de pantalla de la reunión que se mantuvo.

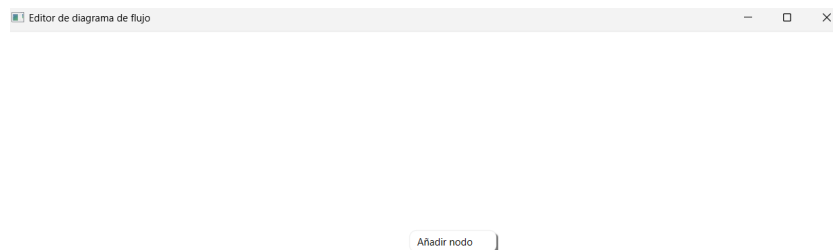


Figura C.1: Maqueta de creación de nuevos nodos.

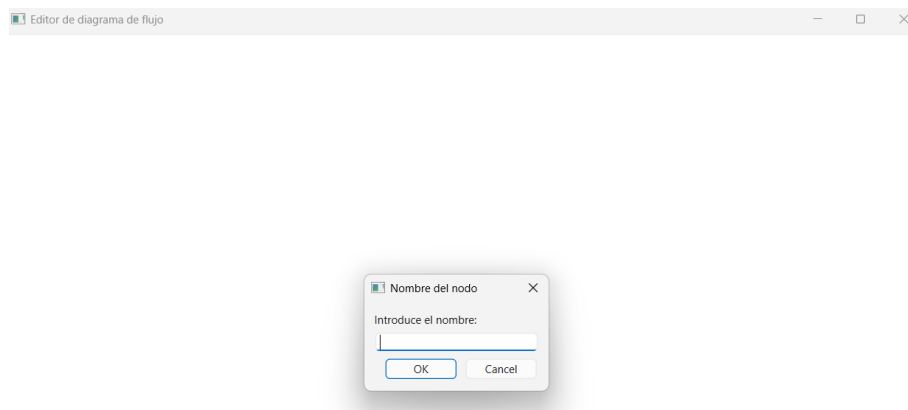


Figura C.2: Maqueta de asignación de nombre a nodos nuevos.



Figura C.3: Maqueta de menú de despliegue de un nodo.

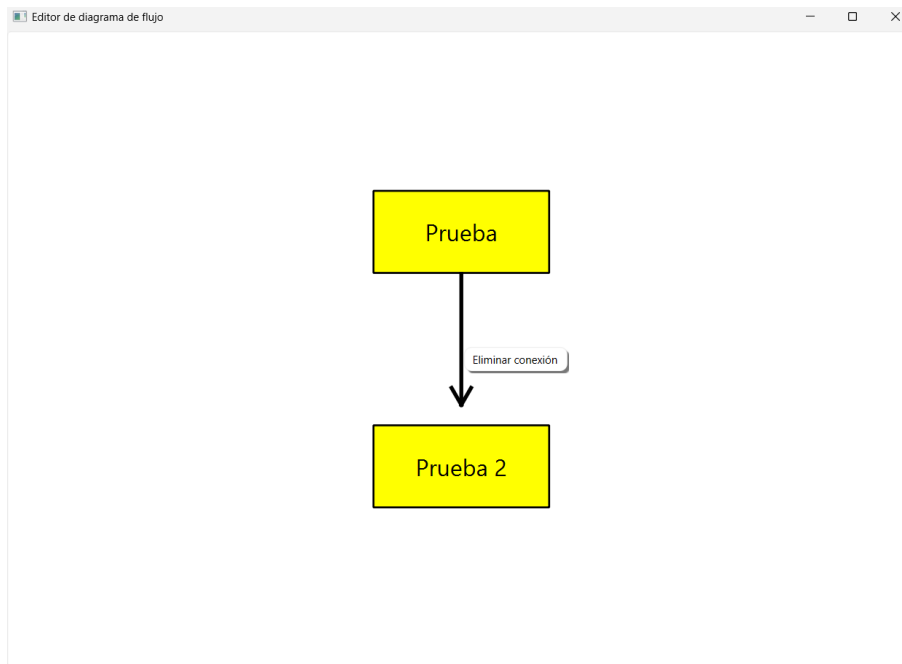
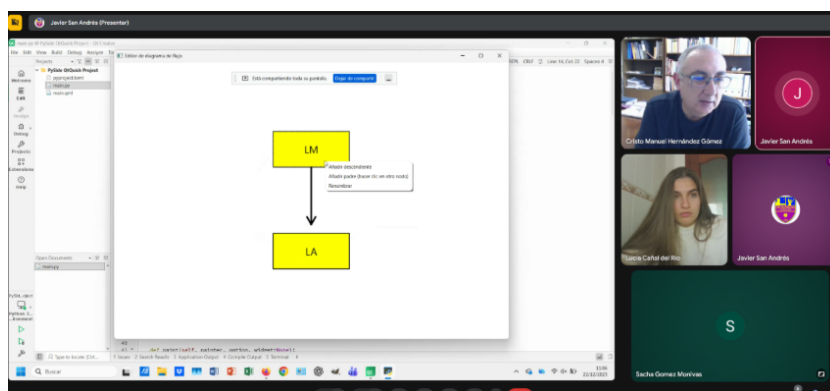
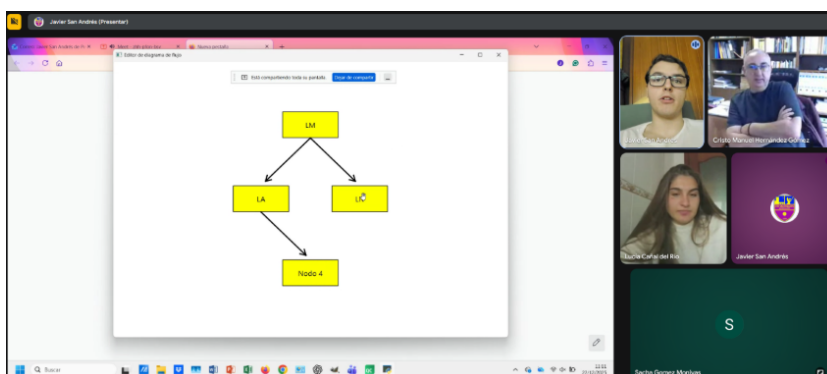


Figura C.4: Maqueta de menú de despliegue de una arista.



(a)



(b)

Figura C.5: Capturas de pantalla de la reunión de validación con el cliente.

C.2. Arquitectura y patrones de diseño

C.2.1. Diseño de la aplicación

Visión arquitectónica general

Como ya se ha comentado en el texto principal (Sección 4.2), la arquitectura de la aplicación Matrix Harris se basa en una estricta separación de responsabilidades entre tres capas: la capa de dominio, que modela el hipergrafo y los algoritmos de cálculo; la capa de aplicación, que gestiona los editores y el proyecto; y la capa de interfaz gráfica, que se encarga de renderizar la escena y gestionar la interacción del usuario.

Esta arquitectura tiene una consecuencia directa en la modularidad y mantenibilidad del sistema: el módulo `hypergraph_logic` puede ser reutilizado por cualquier otro interfaz gráfico sin que ello afecte a su implementación. De hecho, se podría emplear incluso con regla y bolígrafo para representar los diagramas una vez `hypergraph_logic` ha calculado las coordenadas de los nodos y las hiperaristas. Por tanto, este componente es ampliamente reutilizable para cualquier otra aplicación que requiera modelar y visualizar hipergrafos jerárquicos dirigidos, lo que lo convierte en un activo valioso más allá del alcance de esta aplicación concreta.

Capa de dominio: `hypergraph_logic`

La capa de dominio modela el hipergrafo jerárquico y los algoritmos necesarios para calcular su representación visual. Su jerarquía de clases sigue un patrón de herencia progresiva: `Hypergraph` contiene la topología y las operaciones estructurales; `GraphicalHypergraph` extiende `Hypergraph` añadiendo el *pipeline* de *layout* y los datos de coordenadas; y `JointGraphicalHypergraph` extiende `GraphicalHypergraph` incorporando la funcionalidad de diagrama conjunto.

Además, la relación entre `Hypergraph` y sus elementos (`Node`, `Hyperedge` y `LayerData`) es de composición, por lo que el grafo se encarga de gestionar el ciclo de vida de todos sus elementos.

Para calcular el *layout*, se necesita ejecutar un *pipeline* de algoritmos secuenciales que transforman la estructura del grafo en coordenadas visuales. Esta implementación está basada en [19], aunque con relativas modificaciones. De una forma u otra, para resolver cada uno de estos algoritmos se ha optado por delegar su implementación a clases auxiliares independientes, lo que facilita la posibilidad de modificar o sustituir cualquier etapa del *pipeline* sin afectar al resto.

Por último, la clase `JointGraphicalHypergraph` introduce la funcionalidad de diagrama conjunto e implementa una aproximación de lo que sería el patrón Singleton. A diferencia de un Singleton clásico, que impediría la posibilidad de clonar el grafo para mantener versiones anteriores (utilizado por las funciones de deshacer y rehacer), se ha optado por una aproximación que permite convivir a

En la Figura C.6 siguiente se muestra el diagrama de clases solo teniendo en cuenta la capa de dominio, para poder exhibir su estructura con mayor claridad.

Si la mutación tiene éxito, la instantánea previa se empuja a la pila de deshacer; si falla, se descarta sin modificar ninguna pila.

Por último, `Project` actúa como fachada (*Facade*) de la capa de aplicación: agrega todos los editores y el editor conjunto, exponiendo una interfaz unificada para la gestión de diagramas, la persistencia y el mecanismo de deshacer y rehacer.

En la Figura C.7 se muestra el diagrama de clases de esta capa, con una visión simplificada de la capa de `hypergraph_logic`. Aquí, se pueden observar los patrones que comentábamos.

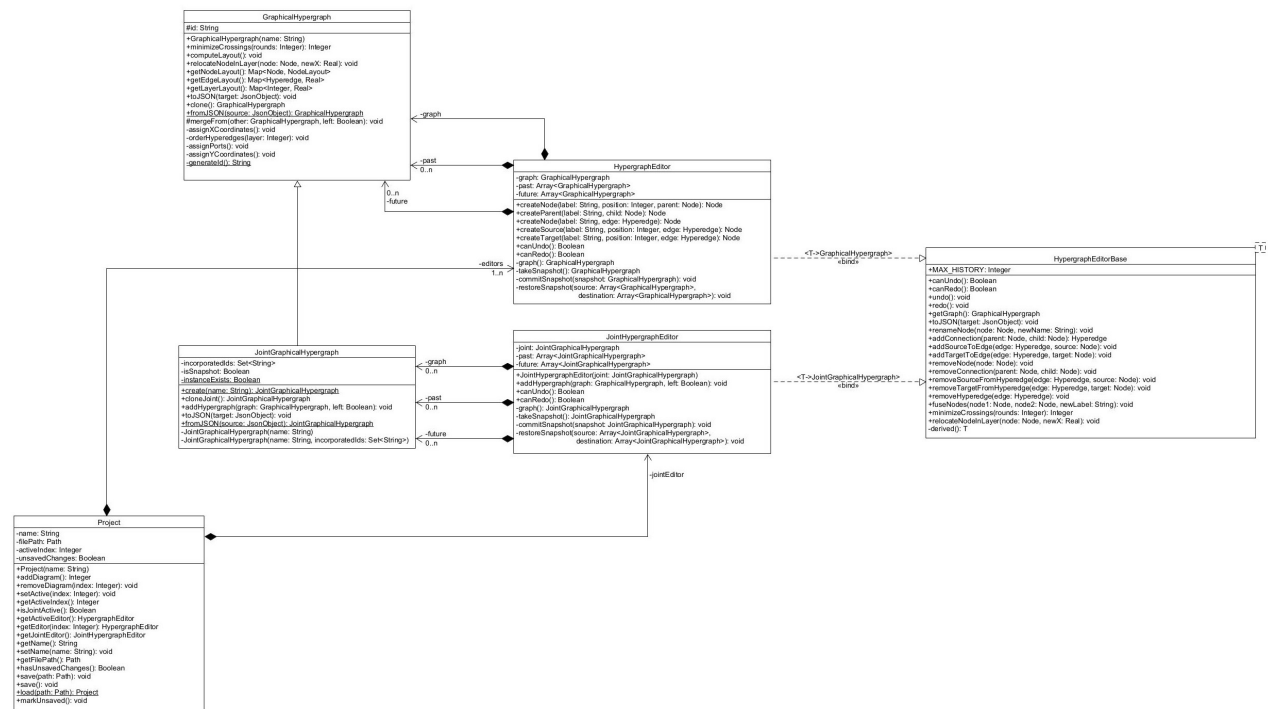


Figura C.7: Diagrama de clases de la capa de aplicación (editores y proyecto).

Capa de interfaz: `ui`

En esta capa, se gestiona toda la interacción con el usuario y el renderizado de la escena. La clase `GUI` actúa como controlador principal, gestionando la ventana principal, los menús y la barra de herramientas. La `GUI` interactúa con el proyecto para realizar sus operaciones, siendo completamente independiente de la lógica de dominio. Adicionalmente, contiene un método que sirve para dibujar el hipergrafo correspondiente, por lo que hasta cierto punto tiene constancia de la estructura del grafo o, mejor dicho, de sus coordenadas de *layout*, aunque no de su topología.

En el diagrama de clases, esta capa se representa con una clase genérica `Application` para no sobrecargar el diagrama con detalles específicos del framework gráfico utilizado. En la implementación, sin embargo, esta clase sí adquiere la nomenclatura previamente utilizada.

REPRESENTACIÓN ORTOGONAL DE HIPERGRAFOS JERÁRQUICOS

D.1. Nodos, Hiperaristas y el Hipergrafo

D.1.1. Nodo

La clase `Node` representa un vértice del hipergrafo. Su estado interno se reduce a cinco atributos:

Código D.1: Atributos privados de la clase `Node`.

```
1  bool is_dummy_;
2  std::string name_;
3  int layer_;
4  std::vector<WeakNodePtr> parents_;
5  std::vector<WeakNodePtr> children_;
```

En la Sección 6.1 hemos hablado de la necesidad de tener un hipergrafo propio de k capas, lo que requiere crear nodos `dummy` para rellenar las capas intermedias cuando se subdivide una arista. En este sentido, el campo `is_dummy_` distingue los nodos reales de los ficticios. Los nodos `dummy` carecen de nombre y son invisibles para el usuario final. El campo `layer_` lo mantiene exclusivamente el hipergrafo: el nodo no decide en qué capa reside, sino que recibe ese valor a través de `setLayer()`, un método privado al que solo tiene acceso la propia clase `Hypergraph`.

Los atributos `parents_` y `children_` almacenan punteros débiles (`weak_ptr`) a los vecinos directos del nodo en la jerarquía. El uso de punteros débiles en ambas listas rompe cualquier ciclo de propiedad; la propiedad real de todos los nodos recae en el hipergrafo, a través de su vector `all_nodes_`, como se verá más adelante. La idea es que, cuando el usuario solicita una conexión, no tenemos manera de comprobar si eso generaría un ciclo y, por ende, no podemos usar `shared_ptr` para mantener la propiedad de los nodos (acabaríamos con pérdidas de memoria).

Un aspecto fundamental del diseño es que los métodos de mutación (`addParent`, `addChild`, `removeParent`, `removeChild`) actúan *únicamente sobre la lista propia del nodo receptor*. Por ejemplo, llamar a `u.addChild(v)` registra `v` en la lista `children_` de `u`, pero no inscribe `u` en

la lista `parents_` de `v`.

Esta implementación es intencionada, pues nos interesa que los nodos reales no conozcan los nodos ficticios con los que están conectados a través de subdivisiones de aristas largas. En este sentido, los nodos *dummy* tienen un ciclo de vida relativamente corto y se crean o destruyen según lo marquen las operaciones de inserción, eliminación o fusión. Los nodos reales, en cambio, tienen un ciclo de vida más largo y solo se eliminan cuando el usuario los borra explícitamente.

La idea es poder recomponer la estructura original del hipergrafo a partir de los nodos reales, sin que los nodos *dummy* interfieran en ese proceso. Esto permite agilizar la comprobación de ciclos, la eliminación de conexiones redundantes por transitividad y, en general, las operaciones de adición o eliminación que exigen recolocaciones masivas de nodos entre capas. Con este diseño, podemos obtener el subgrafo afectado por una operación de inserción o eliminación de manera eficiente. Además, la capa asignada a cada nodo solo puede quedar determinada por la profundidad de sus padres reales, no por la de los nodos ficticios, lo que arroja otro argumento para mantener este diseño.

D.1.2. Hiperarista

La clase `Hyperedge` representa una hiperarista dirigida $e = (S, T)$. Sus atributos privados son:

Código D.2: Atributos privados de la clase `Hyperedge`.

```
1  bool is_segment_;
2  WeakHyperedgePtr origin_;
3  int layer_;
4  std::vector<WeakNodePtr> sources_;
5  std::vector<WeakNodePtr> targets_;
```

El campo `is_segment_` distingue las hiperaristas originales (las conexiones reales), suministradas por el usuario, de los segmentos creados al subdividir una arista larga. Cuando una hiperarista es un segmento, `origin_` contiene un puntero débil a la arista original a la que pertenece, lo que permite reconstruir la cadena completa de segmentos. Esta información no es usada por la clase *Hypergraph*, que tiene una forma conveniente de almacenar esta información, pero sí por la función de renderizado de un hipergrafo, que necesita conocer el camino completo de una hiperarista para dibujarla correctamente. En las aristas originales, `origin_` está vacío. Aunque no es estrictamente necesario, se utiliza un `weak_ptr` para evitar que un segmento prolongue artificialmente la vida de su origen: la propiedad de todas las hiperaristas, tanto originales como segmentos, corresponde al hipergrafo.

El campo `layer_` indica la capa de la que parte la hiperarista, es decir, la capa de sus nodos fuente. El principal cometido que tiene este campo es facilitar la detección de hiperaristas largas, que tendrán este campo a -1 . No obstante, las aristas cortas o segmentos tendrán este campo al valor de

la capa de sus nodos fuente que, por definición, es la misma para todos ellos.

Las listas `sources_` y `targets_` almacenan punteros débiles a los nodos fuente y destino respectivamente. Al igual que ocurre con `Node`, los métodos de mutación (`addSource`, `addTarget`, `removeSource`, `removeTarget`) actúan *únicamente sobre la lista propia de la hiperarista*. Añadir un nodo a `sources_` no establece ningún vínculo padre-hijo sobre ese nodo; el hipergrafo se encarga de mantener sincronizadas las listas de adyacencia de los nodos afectados después de cada operación estructural.

En el hipergrafo, almacenaremos tanto las hiperaristas originales como los segmentos resultantes de subdividirlos. Esto es necesario para cuando se actualice la estructura del hipergrafo y, por ejemplo, los nodos tengan que cambiar su capa. En ese caso, necesitamos conocer las hiperaristas originales (que son las que realmente representan las conexiones entre los nodos) para poder actualizar correctamente los segmentos que las subdividen.

Por último, pudiera parecer repetitivo almacenar la información de conexiones tanto en nodos como hiperaristas, pero esta redundancia es intencionada para facilitar la eficiencia en la navegación a través del grafo. Hay métodos como la detección de ciclos o reubicaciones masivas de nodos que requieren tener esta información de manera inmediata, sin necesidad de recorrer las hiperaristas para conocer los vecinos de un nodo o viceversa. Por otra parte, mantener las hiperaristas es necesario para poder representar las conexiones entre los nodos de manera explícita, lo que no sería posible si solo tuviéramos las listas de adyacencia de los nodos.

D.1.3. Hipergrafo

La clase `Hypergraph` es el núcleo del sistema: posee todos los nodos y hiperaristas, mantiene el mapa de capas y es la única entidad con autoridad para modificar el estado interno de `Node` y `Hyperedge`. Sus atributos protegidos son:

Código D.3: Atributos protegidos de la clase `Hypergraph`.

```
1  std::string name_;
2  std::map<int, LayerData> layers_;
3  std::vector<NodePtr> all_nodes_;
4  std::unordered_map<HyperedgePtr, std::vector<HyperedgePtr>> all_hyperedges_;
```

El atributo `layers_` es un `std::map` ordenado por número de capa. Cada entrada almacena un `LayerData` con dos listas: los nodos presentes en esa capa (`nodes`) y las hiperaristas que parten de ella (`outgoing_edges`). El orden del mapa garantiza que cualquier iteración sobre capas se realiza de la más superficial a la más profunda, propiedad que explotan los algoritmos de minimización de cruces y de renderizado. Además, mantener las capas es fundamental para aplicar todo el *pipeline* de

layout, ya que las capas recogen la representación del hipergrafo como hipergrafo propio de k capas.

El atributo `all_nodes_` contiene todos los nodos del grafo en orden de inserción, tanto nodos reales como nodos *dummy*. Este vector es la única fuente de `shared_ptr` vivos para los nodos; el resto del código usa `weak_ptr` o punteros en bruto para referirse a ellos. De esta forma, el hipergrafo mantiene la propiedad exclusiva de todos los nodos, lo que simplifica la gestión de memoria y evita pérdidas.

El atributo `all_hyperedges_` es una tabla hash cuya clave es la hiperarista original y cuyo valor es el vector de segmentos en que dicha arista ha sido descompuesta. Una arista corta, que no ha necesitado descomposición, tiene un vector de segmentos vacío. Esta estructura centraliza en un único lugar el ciclo de vida completo de cada arista y de todos sus segmentos: crear, dividir, disolver y eliminar se hacen siempre a través de este mapa. Esto hace que sea mucho más eficiente actualizar la estructura del hipergrafo cuando se añaden o eliminan conexiones, pues solo es necesario consultar esta tabla para poder actualizar todos los segmentos afectados por una operación en concreto.

La clase `Hypergraph` presenta relativa complejidad computacional debido a que, tras cada operación, actualiza solo la parte del grafo que se ve afectada, manteniendo la estructura de hipergrafo propio de k capas. Evidentemente, existe una manera más sencilla de realizar la misma lógica: subdividir el hipergrafo entero en cada operación. No obstante, esta solución es ineficiente, por lo que se ha optado por una implementación más compleja pero que solo actualice la parte necesaria. Para que el lector pueda entender la dificultad, piense qué ocurre cuando un nodo (con todo un subgrafo debajo) cambia de capa. En ese caso, no solo hay que actualizar la capa de ese nodo, sino también posiblemente la de sus descendientes, lo que genera un efecto dominó y que haya que actualizar las hiperaristas, porque habrá algunas que ahora sean largas, otras que dejen de serlo y otras que mantengan su estado pero sea necesario añadir o eliminar segmentos.

A continuación, listaremos con pseudocódigo o con descripciones formales los métodos que utiliza la clase `Hypergraph` para gestionar toda esta lógica.

addNodeToLayer

Código D.4: Cabecera de `addNodeToLayer`.

```
1 void addNodeToLayer(int layer, int position, const NodePtr& node);
```

Inserta un nodo en la capa indicada, en la posición `position` dentro del vector `nodes` de esa capa. Si la capa no existe, se crea. Si la posición está fuera de rango, el nodo se añade al final. La operación actualiza el campo `layer_` del nodo a través de `setLayer()`. En la práctica, los nodos se insertan al final y es el hipergrafo con métodos de minimización el que se encarga de reposicionarlos posteriormente para conseguir un mejor *layout*.

removeNodeFromLayer

Código D.5: Cabeceras de las dos sobrecargas de `removeNodeFromLayer`.

```
3 void removeNodeFromLayer(int layer, const NodePtr& node);
4 void removeNodeFromLayer(int layer, const std::unordered_set<Node*>& nodes);
```

Elimina uno o varios nodos del vector `nodes` de la capa indicada. Existe una variante individual y una variante por lotes, más eficiente cuando hay que retirar muchos nodos de golpe (por ejemplo, al tener que eliminar muchos nodos *dummy* al mismo tiempo).

getAllAncestors y getAllDescendants

Código D.6: Cabeceras de `getAllAncestors` y `getAllDescendants`.

```
6 std::unordered_set<Node*> getAllAncestors(const std::vector<NodePtr>& nodes);
7 std::unordered_set<Node*> getAllDescendants(const std::vector<NodePtr>& nodes);
```

Devuelven, respectivamente, el conjunto de todos los ancestros estrictos y el de todos los descendientes estrictos de un conjunto de nodos dado, recorriendo los enlaces padre-hijo mediante DFS con conjunto de visitados para evitar repeticiones. Estos métodos se utilizan para poder mantener la propiedad de Hasse y eliminar conexiones redundantes por transitividad cuando se efectúa una nueva conexión en el hipergrafo.

createHyperedge

Código D.7: Cabeceras de las dos sobrecargas de `createHyperedge`.

```
9 HyperedgePtr createHyperedge(const std::vector<NodePtr>& sources,
10                             const std::vector<NodePtr>& targets, int layer);
11 HyperedgePtr createHyperedge(const WeakHyperedgePtr& origin,
12                             const std::vector<NodePtr>& sources,
13                             const std::vector<NodePtr>& targets, int layer);
```

La primera sobrecarga crea una hiperarista original: la registra en `all_hyperedges_` con lista de segmentos vacía, la añade a la capa indicada y establece los enlaces padre-hijo entre todos los pares (fuente, destino). La segunda sobrecarga crea un segmento: lo añade al vector de segmentos de su arista origen y aplica una política de enlazado más fina para que los nodos reales no tengan conocimiento de la infraestructura *dummy*.

addHyperedgeToLayer

Código D.8: Cabecera de `addHyperedgeToLayer`.

```

15 void addHyperedgeToLayer(int layer, const HyperedgePtr& edge);

```

Registra una hiperarista existente en `outgoing_edges` de la capa indicada y actualiza su campo `layer_`. Es idempotente: si la arista ya está en esa capa, no hace nada.

removeHyperedgeFromLayer

Código D.9: Cabeceras de las dos sobrecargas de `removeHyperedgeFromLayer`.

```

17 void removeHyperedgeFromLayer(int layer, const HyperedgePtr& edge);
18 void removeHyperedgeFromLayer(int layer, const std::unordered_set<Hyperedge*>& edges);

```

Elimina una o varias hiperaristas de `outgoing_edges` de la capa indicada y resetea su campo `layer_` a `-1`. Al igual que su análogo para nodos, existe una variante por lotes para limpiezas masivas, como la que ocurre al disolver segmentos.

edgeIsShort

Código D.10: Cabecera de `edgeIsShort`.

```

20 int edgeIsShort(const HyperedgePtr& edge);

```

Comprueba si todos los pares (fuente, destino) de una hiperarista están en capas adyacentes, es decir, si $\lambda(t) = \lambda(s) + 1$ para todo $s \in S, t \in T$. Devuelve la capa de las fuentes si la arista es corta, o `-1` si es larga, vacía o nula. Es el predicado central que determina si una arista necesita descomponerse en segmentos o añadirse a una capa concreta.

splitLongEdge

El pseudocódigo de este método se presentó ya en la sección 6.1. No obstante, la implementación introduce dos diferencias respecto al pseudocódigo que conviene señalar.

En primer lugar, el pseudocódigo omite el paso previo de disolución: si la arista ya había sido subdividida en una llamada anterior (es decir, su lista de segmentos no está vacía), la implementación invoca `dissolveSegments` antes de volver a dividirla. Esto garantiza que la lista de segmentos

refleja siempre la posición actual de los nodos y no una descomposición obsoleta. En segundo lugar, en el pseudocódigo se refleja como si la hiperarista original fuera eliminada, cuando en realidad se mantiene intacta y es la infraestructura de segmentos la que se actualiza.

dissolveSegments

Este método se utiliza cada vez que una hiperarista larga necesita ser re-subdividida, al comprobarse que ya es corta o al eliminarse completamente. Se encarga de eliminar todos los segmentos y nodos *dummy* asociados a un conjunto de hiperaristas originales. Utilizando las sobrecargas por lotes de los métodos de eliminación de nodos y aristas, se consigue que esta operación sea eficiente incluso cuando hay que realizar limpiezas profundas

```

Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$  y un conjunto  $E' \subseteq \mathcal{E}$  de hiperaristas originales cuyos segmentos se desean
disolver.
Output:  $H$  modificado: los segmentos de toda arista en  $E'$  son eliminados de  $\mathcal{E}$  y sus nodos dummy son
eliminados de  $V$ ; las aristas originales permanecen intactas.

1   $deadDummies \leftarrow \{\}$ ;
2   $deadSegments \leftarrow \{\}$ ;
3  foreach  $e \in E'$  do
4      foreach segment  $\sigma$  of  $e$  do
5          foreach  $v \in sources(\sigma) \cup targets(\sigma)$  do
6              if  $v$  es dummy then
7                   $deadDummies \leftarrow deadDummies \cup \{v\}$ ;
8              end
9          end
10          $deadSegments \leftarrow deadSegments \cup \{\sigma\}$ ;
11     end
12      $segments(e) \leftarrow []$ ;                                // vaciar la lista de segmentos
13 end

// Eliminar segmentos y dummies de las estructuras del
hipergrafo
14 foreach  $l \in \lambda(deadSegments)$  do
15      $removeHyperedgeFromLayer(l, \sigma)$ ;
16 end
17 foreach  $l \in \lambda(deadDummies)$  do
18      $removeNodeFromLayer(l, d)$ ;
19 end
20  $V \leftarrow V \setminus deadDummies$ ;

```

Algoritmo D.1: Disolución de segmentos

relocateNodes y applyRelocationAndPropagate

Estos dos métodos trabajan siempre en tándem: `relocateNodes` determina qué nodos necesitan moverse y delega la ejecución en `applyRelocationAndPropagate`, que realiza los movimientos

y propaga estas modificaciones hacia abajo. Se invocan conjuntamente desde todas las operaciones de adición y eliminación de conexiones cuando el grafo de capas debe reequilibrarse.

En primer lugar, $\text{relocateNodes}(H, N')$: identifica los nodos de N' cuya capa ya no coincide con su profundidad correcta y lanza la propagación.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$ y un conjunto de nodos $N' \subseteq V$ a evaluar.
Output: `true` si se reubicó al menos un nodo (en cuyo caso H queda modificado por $\text{applyRelocationAndPropagate}$); `false` si ningún nodo necesitaba moverse.

```

1  relocs  $\leftarrow []$ ;
2  foreach  $v \in N'$  do
3       $\ell^* \leftarrow \begin{cases} 0 & \text{si } \text{parents}(v) = \emptyset; \\ \max_{p \in \text{parents}(v)} \lambda(p) + 1 & \text{en otro caso} \end{cases}$ ;
4      if  $\ell^* \neq \lambda(v)$  then
5           $\text{relocs} \leftarrow \text{relocs} \cup [(v, \ell^*)]$ ;
6      end
7  end
8  if  $\text{relocs} = []$  then
9      return false; // nada que reubicar
10 else
11      $\text{applyRelocationAndPropagate}(H, \text{relocs})$ ;
12     return true;
13 end
```

Algoritmo D.2: Determinación de reubicaciones

En segundo lugar, $\text{applyRelocationAndPropagate}$ es una función mucho más compleja que merece una explicación previa más allá del propio pseudocódigo. El problema es el siguiente, una vez sabemos que varios nodos han de moverse al mismo tiempo, ¿cómo actualizar correctamente las capas (profundidad) de los nodos afectados? Necesitamos recolectar todos los nodos que potencialmente necesiten actualizar su profundidad y dirimir si así lo necesitan. El problema reside en que, para cuando se comprueba si un nodo necesita actualizar su profundidad, es obligatorio que sus padres ya estén en la capa correcta.

Mi primer pensamiento para realizar este código fue un BFS, pero es fácil encontrar contraejemplos donde esto no funciona. En la Figura D.1, supongamos que tanto A como B necesitan moverse de capa; en concreto, supongamos que A debe ir a la capa 1 y B a la capa 2. Si se procesa primero A , se mueve a la capa 1 y se insertaría D en la cola. Después, al procesar B se mueve a la capa 2 y se insertaría C a la cola. Finalmente, al procesar D se quedaría en la capa 2, ya que su padre (C) aún no ha sido procesado y no ha cambiado de capa, lo que es incorrecto porque D debería ir a la capa 4.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$ y una lista $relocs = [(v_1, \ell_1^*), \dots]$ de pares (nodo, nueva capa).

Output: H modificado: todos los nodos en $relocs$ y sus descendientes afectados ocupan la capa correcta según la regla $\lambda(v) = \max_{p \in \text{parents}(v)} \lambda(p) + 1$, y todas las aristas afectadas han sido reconstruidas.

```
// -- Fase 1: mover los nodos --
1  foreach (v,  $\ell^*$ )  $\in relocs$  do
2      removeNodeFromLayer( $\lambda(v)$ , v);
3      addNodeToLayer( $\ell^*$ , v);
4  end
5  incoming  $\leftarrow \{e \in \mathcal{E} \mid \text{targets}(e) \cap \{v : (v, \_) \in relocs\} \neq \emptyset\}$ ;
6  dissolveSegments(H, incoming);
7  foreach e  $\in incoming$  do
8      if edgeIsShort(e)  $\geq 0$  then
9          addHyperedgeToLayer(edgeIsShort(e), e);
10     else
11         splitLongEdge(H, e);
12     end
13 end

// -- Fase 2: propagar a los descendientes --
14 affected  $\leftarrow \text{getAllDescendants}(\{v : (v, \_) \in relocs\}) \setminus \{v : (v, \_) \in relocs\}$ ;
15 foreach capa  $\ell$  en orden creciente tal que  $\exists w \in affected$  con  $\lambda(w) = \ell$  do
16     foreach w  $\in affected$  con  $\lambda(w) = \ell$  do
17          $\ell_w^* \leftarrow \max_{p \in \text{parents}(w)} \lambda(p) + 1$ ;
18         if  $\ell_w^* \neq \ell$  then
19             removeNodeFromLayer( $\ell$ , w);
20             addNodeToLayer( $\ell_w^*$ , w);
21         end
22     end
23 end

// -- Fase 3: reconstruir aristas --
24 moved  $\leftarrow \{w \in affected \mid \lambda(w) \text{ cambió en la Fase 2}\}$ ;
25 foreach e  $\in \mathcal{E}$  tal que  $(\text{sources}(e) \cup \text{targets}(e)) \cap moved \neq \emptyset$  do
26     if edgeIsShort(e)  $\geq 0$  then
27         dissolveSegments(H, {e});
28         addHyperedgeToLayer(edgeIsShort(e), e);
29     else
30         splitLongEdge(H, e);
31     end
32 end
```

Algoritmo D.3: Aplicación de reubicaciones y propagación

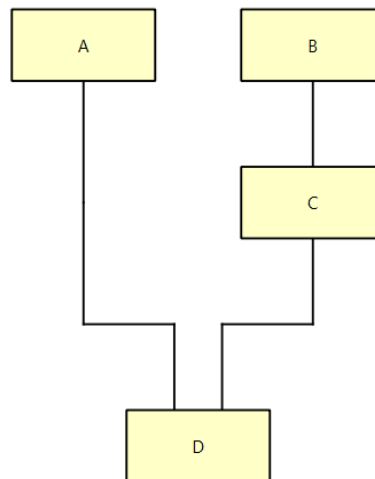


Figura D.1: Ejemplo de hipergrafo donde BFS falla.

parentIsInAncestors

Este método se usa como guardia antes de añadir cualquier conexión nueva al grafo, para detectar si la conexión ya existe de forma transitiva. En concreto, comprueba si un nodo p es ancestro de algún nodo que se liste en C mediante DFS con memoria para no expandir en nodos ya visitados. El método `childIsInDescendants` es completamente análogo, recorriendo el grafo hacia abajo en lugar de hacia arriba, y se emplea de la misma forma para los destinos.

checkCycles

Este método se invoca de forma preventiva, antes de comprometer cualquier cambio estructural que añada nuevos enlaces padre-hijo. Este método se ejecuta de manera más eficiente debido a la decisión de diseño de que los nodos mantengan listas de adyacencia. Además, el hecho de que los nodos reales no conozcan a los ficticios, también reduce el número de llamadas en la pila.

El algoritmo implementado es una modificación de DFS con conjunto de visitados y conjunto de nodos en el camino actual. Veamos por qué un DFS normal no es suficiente. Consideremos la Figura D.2:

- Empecemos DFS desde el nodo 1. Seguirá el camino $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ y marcará todos estos nodos como visitados.
- Como no hay más nodos a los que ir desde el nodo 6, el DFS retrocede a 5, luego a 4 y finalmente a 3.
- Desde 3, irá al nodo 7, que no ha sido visitado y, finalmente, desde 7 irá al nodo 5, que ya ha sido visitado.

```

Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$ , un conjunto de nodos  $C \subseteq V$  y un nodo  $p \in V$ .
Output: true si  $p$  es ancestro estricto de algún nodo de  $C$ ; false en caso contrario.

1   $visited \leftarrow \{\}$ ;
2  foreach  $c \in C$  tal que  $\lambda(c) > \lambda(p)$  do
3      foreach  $q \in \text{parents}(c)$  tal que  $\lambda(q) \geq \lambda(p)$  y  $q \notin visited$  do
4          if  $\lambda(q) = \lambda(p)$  then
5              if  $q = p$  then
6                  return true
7              end
8              continue;                                // condición de poda
9          end
10          $visited \leftarrow visited \cup \{q\}$ ;
11         if  $\text{parentIsInAncestors}(H, \{q\}, p)$  then
12             return true
13         end
14     end
15 end
16 return false;

```

Algoritmo D.4: Detección de ancestro

Por lo tanto, erróneamente, el DFS concluiría que hay un ciclo, porque hay un nodo que se ha visitado dos veces, cuando en realidad no lo hay. El problema está en que el nuevo camino para llegar a 5 no incluye al propio 5, aunque este hubiera sido visitado previamente por otro camino. Por eso, es necesario llevar la cuenta del camino actual, para detectar si el nodo visitado previamente pertenece a ese camino (lo que sí indicaría un ciclo) o no (lo que no indicaría un ciclo).

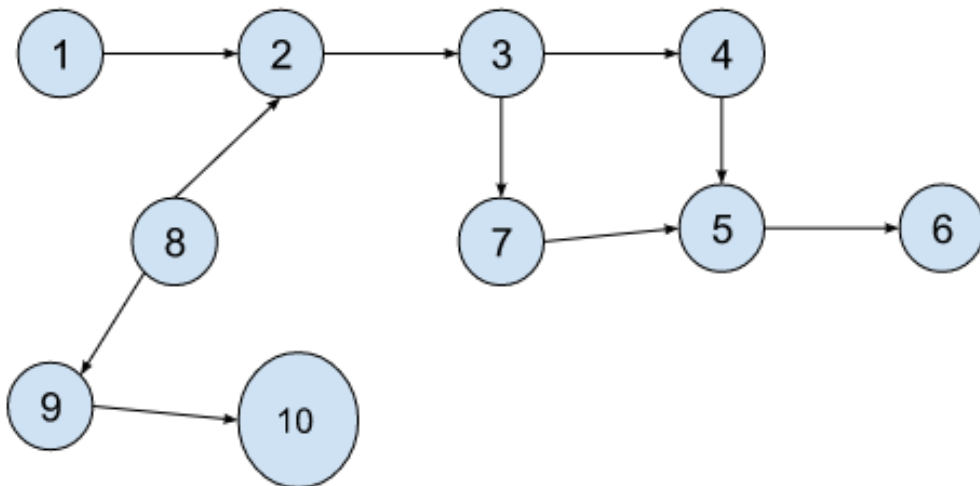


Figura D.2: DFS normal no basta para detectar ciclos.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$ y un nodo $v \in V$ desde el que iniciar la búsqueda.
Output: `true` si existe un ciclo alcanzable desde v ; `false` en caso contrario.

```

1  visited  $\leftarrow \{\}$ ;
2  path  $\leftarrow \{\}$ ;                                // nodos en el camino DFS actual

// Función auxiliar interna (checkCyclesUtil)
3  Function checkCyclesUtil( u ) :
4      visited  $\leftarrow$  visited  $\cup \{u\}$ ;
5      path  $\leftarrow$  path  $\cup \{u\}$ ;
6      foreach  $w \in \text{children}(u)$  do
7          if  $w \notin \text{visited}$  then
8              if checkCyclesUtil( w ) then
9                  return true
10             end
11         else
12             if  $w \in \text{path}$  then
13                 return true;                    // arista hacia atrás -- ciclo
14             end
15         end
16     end
17     path  $\leftarrow$  path  $\setminus \{u\}$ ;
18     return false;
19 return checkCyclesUtil( v );

```

Algoritmo D.5: Detección de ciclos

removeTransitiveConnections

Este método se invoca desde `addConnection`, `addSourceToEdge` y `addTargetToEdge` inmediatamente después de confirmar que la nueva conexión no crea ciclos. Su propósito es mantener el invariante de diagrama de Hasse: si la nueva arista ($P \rightarrow C$) hace que alguna arista preexistente ($P' \rightarrow C'$) sea redundante por transitividad (porque $P' \preceq P$ y $C \preceq C'$), esa arista preexistente debe eliminarse o modificarse para eliminar los destinos redundantes. Cabe destacar que tanto P como C pueden ser conjuntos de nodos.

Para ejemplificarlo, tómese la Figura D.1 y supongamos que queremos añadir la conexión $A \rightarrow C$. En ese caso, la conexión $A \rightarrow D$ se vuelve redundante por transitividad, porque $A \preceq C$ y $C \preceq D$. Por lo tanto, esta conexión debe eliminarse o recortarse para eliminar el destino redundante D . En este caso, como D es el único destino de esa arista, la conexión $A \rightarrow D$ se elimina completamente.

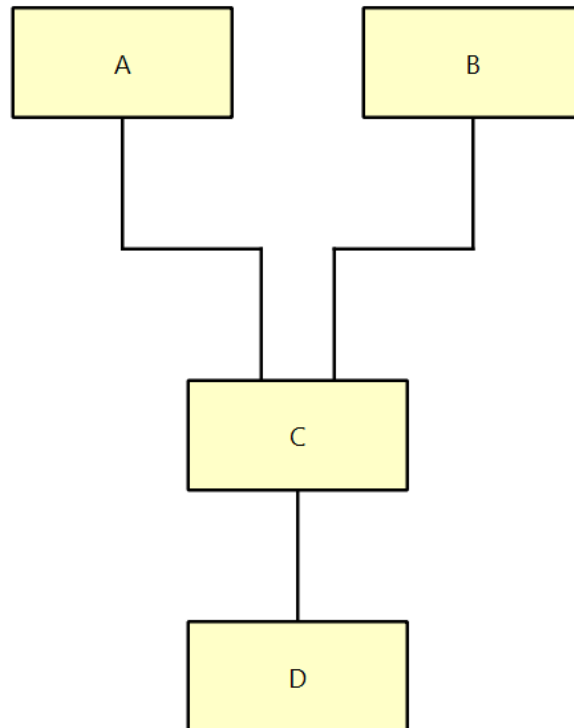


Figura D.3: Ejemplo de eliminación de conexión transitiva.

Sin embargo, si hubiera otro destino E en esa arista (que nada tiene que ver con B), entonces la conexión se recortaría a $A \rightarrow E$, eliminando solo el destino redundante D .

Con estos métodos privados, el hipergrafo ya está listo para soportar todas las operaciones públicas de adición, eliminación y fusión, manteniendo siempre por dentro la estructura de hipergrafo propio de k capas y el invariante de diagrama de Hasse. No obstante, quedarían dos métodos relativos a la

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, un conjunto de nodos fuente $P \subseteq V$ y un conjunto de nodos destino $C \subseteq V$ de la nueva conexión que se está añadiendo.

Output: H modificado: todas las conexiones redundantes por transitividad son eliminadas o recortadas. Se devuelve la capa más superficial ℓ^* en la que se haya creado un nuevo nodo *dummy* como consecuencia de un recorte, o $+\infty$ si no se creó ninguno.

```

1   $anc \leftarrow P \cup \text{getAllAncestors}(P);$ 
2   $desc \leftarrow C \cup \text{getAllDescendants}(C);$ 
3   $\ell^* \leftarrow +\infty;$ 
4  foreach  $e \in \mathcal{E}$  do
5       $ancSrc \leftarrow \text{sources}(e) \cap anc;$ 
6      if  $ancSrc = \emptyset$  then
7          continue
8      end
9       $redTgt \leftarrow \text{targets}(e) \cap desc;$ 
10      $survTgt \leftarrow \text{targets}(e) \setminus desc;$ 
11     if  $redTgt = \emptyset$  then
12         continue;                                // ningún destino es redundante
13     end
14      $\text{removeSourcesFromHyperedge}(e, ancSrc);$ 
15     if  $survTgt \neq \emptyset$  then
16          $e' \leftarrow \text{createHyperedge}(ancSrc, survTgt);$ 
17         if  $\text{edgeIsShort}(e') \geq 0$  then
18              $\text{addHyperedgeToLayer}(\text{edgeIsShort}(e'), e');$ 
19         else
20              $\text{splitLongEdge}(H, e');$ 
21             foreach  $a \in ancSrc$  do
22                  $\ell^* \leftarrow \min(\ell^*, \lambda(a) + 1);$ 
23             end
24         end
25     end
26 end
27 return  $\ell^*;$ 

```

Algoritmo D.6: Eliminación de conexiones transitivas

minimización de cruces, que ahora se asumirán como cajas negras, pero que se explicarán en detalle en la sección D.2. Aun así, expliquemos brevemente qué hacen estos métodos:

- `minimizeCrossingsForNodes( $N$ ,  $\ell_{min}$ ,  $\ell_{max}$ )`: dado un conjunto de nodos N (cada uno en diferentes capas) que se han añadido o movido a capas entre ℓ_{min} y ℓ_{max} , este método intenta encontrar la posición óptima para cada nodo en su capa respectiva, minimizando los cruces con otras hiperaristas ya existentes.
- `minimizeCrossings(rounds, startLayer)`: este método se encarga de realizar una minimización global de cruces a partir de una capa dada, con un número máximo de rondas. A mayor número de rondas, mejor será la minimización, aunque también más se desvirtuará el mapa mental del usuario. Por eso, utilizaremos valores bajos para operaciones no tan agresivas y valores más altos para operaciones que impliquen cambios estructurales importantes.

`createNode(label, position, parent)`

Crea un nodo real con etiqueta `label` y lo inserta en el hipergrafo. Si se proporciona un nodo padre, el nuevo nodo se sitúa en la capa inmediatamente inferior y se crea una hiperarista que los une. En caso contrario, el nodo se ubica en la capa 0. Si no se especifica posición concreta (`position = -1`), se invoca `minimizeCrossingsForNodes` para colocar el nodo en la posición menos disruptiva de su capa.

En la práctica, la posición viene determinada por dónde se efectúe el clic si el nodo se ubica en la capa 0 o por lo que diga el algoritmo de minimización de no ser así. Al final, sería un poco tedioso pedirle al cliente que indique la posición de inserción cada vez que se añada un nodo, así que lo más común es dejar que el algoritmo de minimización se encargue de colocar el nodo en la mejor posición posible. Eso sí, este algoritmo tiene un sesgo hacia la derecha, ya que es la posición más usual de inserción.

`createParent`

Crea un nodo real en la capa 0 y lo conecta como padre del nodo `child`. Si el hijo necesita reubicarse (porque su nueva profundidad mínima ha cambiado), se lanza una minimización global de cruces. En caso contrario, si la arista resultante es larga, se subdivide; si es corta, basta con situar al nuevo padre en la posición menos conflictiva de la capa 0.

`createNode(label, edge)`

Esta segunda sobrecarga de `createNode` intercala un nuevo nodo real *sobre* una hiperarista existente e : elimina las fuentes de e , sitúa el nuevo nodo en la capa inmediatamente inferior a ellas y

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, una etiqueta `label`, una posición de inserción `position` y un nodo padre opcional `parent`.

Output: El nuevo nodo v , insertado en H con sus enlaces correctamente establecidos.

```

1   $v \leftarrow \text{newNode}(\text{label});$ 
2   $V \leftarrow V \cup \{v\};$ 
3  if  $\text{parent} = \emptyset$  then
4     $\text{addNodeToLayer}(0, \text{position}, v);$ 
5  else
6     $\text{createHyperedge}(\{\text{parent}\}, \{v\}, \lambda(\text{parent}));$ 
7     $\text{addNodeToLayer}(\lambda(\text{parent}) + 1, \text{position}, v);$ 
8  end
9  if  $\text{position} = -1$  then
10    $\text{minimizeCrossingsForNodes}(\{v\}, \lambda(v), \lambda(v));$ 
11 end
12 return  $v$ ;

```

Algoritmo D.7: Crear un nodo real con padre opcional

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, una etiqueta `label` y un nodo hijo $\text{child} \in V$.

Output: El nuevo nodo padre v , insertado en H con la hiperarista $(v \rightarrow \text{child})$ correctamente establecida.

```

1   $v \leftarrow \text{newNode}(\text{label});$ 
2   $V \leftarrow V \cup \{v\};$ 
3   $\text{addNodeToLayer}(0, -1, v);$ 
4   $e \leftarrow \text{createHyperedge}(\{v\}, \{\text{child}\}, -1);$ 
5  if  $\text{relocateNodes}(\{\text{child}\})$  then
6    // La reubicación es global - minimizar en todo el grafo
7     $\text{minimizeCrossings}(10, 0);$ 
8  else
9    // No hay reubicaciones -- solo colocar los nuevos nodos
10   if  $\text{edgeIsShort}(e) \geq 0$  then
11      $\text{addHyperedgeToLayer}(\text{edgeIsShort}(e), e);$ 
12      $\text{minimizeCrossingsForNodes}(\{v\}, 0, 0);$ 
13   else
14      $\text{splitLongEdge}(H, e);$ 
15      $D \leftarrow \{d \in \text{dummies}(e)\};$  // dummies creados al subdividir
16      $\text{minimizeCrossingsForNodes}(\{v\} \cup D, 0, \lambda(\text{child}) - 1);$ 
17   end
18 end
19 return  $v$ ;

```

Algoritmo D.8: Crear un nodo que se inserte como padre de un nodo existente

recrea dos nuevas hiperaristas, una de las fuentes al nuevo nodo y otra del nuevo nodo a los destinos originales. Es análoga en estructura a `createParent`, pero actúa sobre una arista completa en lugar de sobre un nodo hijo.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, una etiqueta `label` y una hiperarista $e \in \mathcal{E}$.

Output: El nuevo nodo v intercalado entre las fuentes y los destinos de e ; e es reemplazada por dos nuevas hiperaristas.

```

1  src ← sources(e)  tgt ← targets(e)  ℓe ← λ(e);
2  v ← newNode(label);
3  V ← V ∪ {v};
4  ℓv ← máxs ∈ src λ(s) + 1;
5  removeSourcesFromHyperedge(e, src, relocation = false);
6  addNodeToLayer(ℓv, -1, v);
7  if ℓe ≥ 0 then
    // Arista original corta: las dos nuevas también
8      createHyperedge(src, {v}, ℓe);
9      e2 ← createHyperedge({v}, tgt, ℓe + 1);
10     relocateNodes(targets(e2));
11 else
12     e1 ← createHyperedge(src, {v}, -1);
13     if edgeIsShort(e1) ≥ 0 then
14         addHyperedgeToLayer(edgeIsShort(e1), e1);
15     else
16         splitLongEdge(H, e1);
17     end
18     e2 ← createHyperedge({v}, tgt, -1);
19     if ¬relocateNodes(targets(e2)) then
20         if edgeIsShort(e2) ≥ 0 then
21             addHyperedgeToLayer(edgeIsShort(e2), e2);
22         else
23             splitLongEdge(H, e2);
24         end
25     end
26 end
27 ℓmín ← míns ∈ src λ(s) + 1;
28 minimizeCrossings(3, ℓmín);
29 return v;
    
```

Algoritmo D.9: Crear un nodo intercalado sobre una arista existente

createSource y createTarget

Estas dos funciones son análogas entre sí: crean un nuevo nodo real y lo añaden como fuente o destino de una hiperarista existente e , respectivamente. `createSource` sitúa siempre el nuevo nodo en la capa 0, ya que toda fuente nueva se asume raíz; `createTarget` lo sitúa en la capa inmediatamente inferior a la más profunda de las fuentes de e . En ambos casos, si la arista resultante es larga, se re-subdivide y se minimizan los cruces de los nuevos nodos dummy; si es corta, basta con mini-

mizar la posición del nuevo nodo en su capa. Aunque ambas cuentan con un parámetro de posición, en la práctica se llaman siempre con `position = -1`, dejando que el algoritmo de minimización se encargue de colocar el nuevo nodo en la posición menos disruptiva de su capa.

```

Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$ , una etiqueta label, una posición y una hiperarista  $e \in \mathcal{E}$ .
Output: El nuevo nodo fuente  $v$ , insertado en la capa 0 y enlazado con todos los destinos de  $e$ .

1   $v \leftarrow \text{newNode}(\text{label});$ 
2   $V \leftarrow V \cup \{v\};$ 
3   $\text{addNodeToLayer}(0, \text{position}, v);$ 
4   $e.\text{addSource}(v);$ 
5  foreach  $t \in \text{targets}(e)$  do
6     $t.\text{addParent}(v)$   $v.\text{addChild}(t);$ 
7  end
8  if  $\text{edgeIsShort}(e) < 0$  then
9    // La arista es ahora larga -- re-subdividir
10   if  $\lambda(e) \geq 0$  then
11      $\text{removeHyperedgeFromLayer}(\lambda(e), e);$ 
12   end
13    $\text{splitLongEdge}(H, e);$ 
14    $D \leftarrow \{d \in \text{dummies}(e)\};$ 
15    $\ell_{\text{máx}} \leftarrow \max_{d \in D} \lambda(d);$ 
16    $\text{minimizeCrossingsForNodes}(\{v\} \cup D, 0, \ell_{\text{máx}});$ 
17 else
18   // La arista sigue siendo corta -- solo colocar el nuevo
19   nodo
20    $\text{minimizeCrossingsForNodes}(\{v\}, 0, 0);$ 
21 end
22 return  $v;$ 

```

Algoritmo D.10: Crear un nodo fuente o destino para una arista existente

addConnection

Añade una conexión directa entre un nodo padre y un nodo hijo ya existentes. Antes de comprometer el cambio comprueba que la conexión no es redundante (el padre ya es ancestro del hijo), que no crea un ciclo y elimina las conexiones previas que la nueva haría transitivamente redundantes. Después, dependiendo de si las capas de ambos nodos ya son adyacentes, si el hijo está más abajo o si el hijo necesita reubicarse, elige entre tres estrategias de actualización estructural y encuentra la mejor posición para los nuevos nodos *dummy* que se puedan crear. En el peor caso, el hijo debe aumentar su capa y, con él, todos sus descendientes, lo que implica una reubicación global.

Para verificar ciclos, no es necesario recorrer el grafo completo, ya que un nuevo ciclo solo podría introducirse a través de la nueva conexión padre-hijo. Por lo tanto, basta con realizar un DFS desde el nodo hijo o el nodo padre indistintamente. En nuestro caso, se ha optado por iniciarlo desde el nodo hijo.

```

1  Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$  y dos nodos  $\text{parent}, \text{child} \in V$ .
2  Output: La nueva hiperarista  $e$  que representa la conexión, con  $H$  actualizado y libre de ciclos y redundancias.
3
4  if  $\text{parentIsInAncestors}(\{\text{child}\}, \text{parent})$  then
5      | lanzar «conexión ya existente»;
6  end
7
8   $\text{child.addParent}(\text{parent})$   $\text{parent.addChild}(\text{child})$ ;
9  if  $\text{checkCycles}(\text{child})$  then
10     |  $\text{child.removeParent}(\text{parent})$   $\text{parent.removeChild}(\text{child})$ ;
11     | lanzar «ciclo detectado»;
12 end
13
14  $\ell^* \leftarrow \text{removeTransitiveConnections}(\{\text{parent}\}, \{\text{child}\})$ ;
15  $\ell_p \leftarrow \lambda(\text{parent})$   $\ell_c \leftarrow \lambda(\text{child})$ ;
16
17 if  $\ell_p = \ell_c - 1$  then
18     | // Caso simple: capas adyacentes
19     |  $e \leftarrow \text{createHyperedge}(\{\text{parent}\}, \{\text{child}\}, \ell_p)$ ;
20 else if  $\ell_p < \ell_c$  then
21     | // El hijo ya está más abajo -- subdividir la arista
22     |  $e \leftarrow \text{createHyperedge}(\{\text{parent}\}, \{\text{child}\}, -1)$ ;
23     |  $\text{splitLongEdge}(H, e)$ ;
24     | if  $\ell^* < +\infty$  then
25         |  $\text{minimizeCrossings}(3, \ell^*)$ ;
26     | else
27         |  $D \leftarrow \{d \in \text{dummies}(e)\}$ ;
28         |  $\text{minimizeCrossingsForNodes}(D, \ell_p + 1, \ell_c - 1)$ ;
29     | end
30 else
31     | // Peor caso: el hijo debe aumentar su capa
32     |  $e \leftarrow \text{createHyperedge}(\{\text{parent}\}, \{\text{child}\}, \ell_p)$ ;
33     |  $\text{applyRelocationAndPropagate}(\{(\text{child}, \ell_p + 1)\})$ ;
34     |  $\ell^* \leftarrow \min(\ell^*, \min_{p \in \text{parents}(\text{child})} \lambda(p) + 1)$ ;
35     |  $\text{minimizeCrossings}(10, \ell^*)$ ;
36 end
37 return  $e$ ;

```

Algoritmo D.11: Añadir una conexión binaria entre dos nodos existentes

addSourceToEdge y addTargetToEdge

Estas dos operaciones son simétricas: añaden un nodo ya existente como fuente o como destino de una hiperarista e ya existente. Ambas siguen el mismo patrón de cuatro fases: (1) comprobar si la conexión ya existe o es redundante, permitiendo reagrupaciones cuando procede; (2) verificar ausencia de ciclos añadiendo el enlace temporalmente; (3) eliminar conexiones transitivamente redundantes con `removeTransitiveConnections`; y (4) reubicar o re-subdividir según sea necesario. Solo se describe `addSourceToEdge`; el algoritmo de `addTargetToEdge` es completamente análogo intercambiando fuentes y destinos, y usando `childIsInDescendants` en lugar de `parentIsInAncestors`.

La parte de reagrupación es la más interesante. En la Figura 2.2 se muestra lo que ocurriría si partiendo del grafo de la izquierda, se añadiera el nodo C como destino de la arista $A \rightarrow B$. En ese caso, aunque C ya era un hijo de A a través de la arista $A \rightarrow C$, lo que interpretamos es que se quiere establecer otra reagrupación diferente de las conexiones para otorgar mayor claridad, resultando en la figura de la derecha; donde la conexión binaria $A \rightarrow C$ desaparece.

Las reagrupaciones se permiten siempre que efectuarla elimine conexiones existentes y no sea necesario introducir nuevas.

- En el caso de añadir un nuevo origen (v) a una arista e y supongamos que e tiene como destino un nodo t que ya es hijo de ese nuevo origen a través de otra arista e' . Entonces, se permite la reagrupación si el conjunto de orígenes de e' es un subconjunto del conjunto de orígenes de $e \cup \{v\}$. Pero, como ya existe la arista e con destino t y dado que se preserva la integridad del grafo (no hay conexiones redundantes), entonces el conjunto de orígenes de e no puede tener intersección con el conjunto de orígenes de e' , por lo que la condición se reduce a comprobar que v es el único origen de e' . El mismo criterio se aplica para el resto de destinos de e y solo se permite la reagrupación si se cumple para todos ellos.
- En el caso de añadir un nuevo destino (v) a una arista e y supongamos que e tiene como origen un nodo p que ya es padre de ese nuevo destino a través de otra arista e' . Entonces, se permite la reagrupación si el conjunto de orígenes de e' es un subconjunto del conjunto de orígenes de e . De alguna manera, permitimos reagrupación si la nueva arista e puede absorber a e' . El mismo criterio se aplica para el resto de orígenes de e y solo se permite la reagrupación si se cumple para todos ellos.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, una hiperarista $e \in \mathcal{E}$ y un nodo $source \in V$.

Output: H modificado con $source$ como nueva fuente de e , sin ciclos ni redundancias. (continúa en el algoritmo siguiente)

```
// Fase 1 -- Detectar reagrupaciones permitidas
1  affTgt ← ∅  affEdges ← [];
2  foreach e' ∈ E tal que source ∈ sources(e') do
3      foreach t ∈ targets(e) tal que t ∈ targets(e') do
4          affTgt ← affTgt ∪ {t}  affEdges ← affEdges || [e'];
5          if |sources(e')| ≠ 1 then
6              lanzar «conexión ya existente»;
7          end
8      end
9  end
10 if affTgt = ∅ and parentIsInAncestors(targets(e), source) then
11     lanzar «conexión ya existente»;
12 end

// Fase 2 -- Verificar ciclos temporalmente
13 foreach t ∈ targets(e) \ affTgt do
14     source.addChild(t)  t.addParent(source);
15 end
16 if checkCycles(source) then
17     foreach t ∈ targets(e) \ affTgt do
18         source.removeChild(t)  t.removeParent(source);
19     end
20     lanzar «ciclo detectado»;
21 end

// Fase 3 -- Reagrupar destinos afectados si procede
22 if affTgt ≠ ∅ then
23     foreach e' ∈ affEdges do
24         removeTargetsFromHyperedge(e', affTgt, relocation = false);
25     end
26     foreach t ∈ affTgt do
27         source.addChild(t)  t.addParent(source);
28     end
29 end
30 ℓ* ← removeTransitiveConnections({source}, targets(e));
31 e.addSource(source);
```

Algoritmo D.12: Añadir una fuente o destino a una arista existente (Parte I).

Input: (continuación) Variables *affTgt* y *affEdges* calculadas en la Parte I.

// Fase 5 -- Reubicar o re-subdividir según corresponda

```

1  relocs  $\leftarrow [(t, \lambda(\text{source}) + 1) \mid t \in \text{targets}(e), \lambda(\text{source}) + 1 > \lambda(t)];$ 
2  if relocs = [] then
3      if edgeIsShort(e)  $\geq 0$  then
4          addHyperedgeToLayer(edgeIsShort(e), e);
5      else
6          splitLongEdge(H, e);
7          if  $\ell^* < +\infty$  then
8              minimizeCrossings(3,  $\ell^*$ );
9          else
10              $D \leftarrow \{d \in \text{dummies}(e)\};$ 
11             minimizeCrossingsForNodes(D,  $\min_d \lambda(d)$ ,  $\max_d \lambda(d)$ );
12         end
13     end
14     if  $\ell^* < +\infty$  then
15         minimizeCrossings(3,  $\ell^*$ )
16     end
17 else
18     applyRelocationAndPropagate(relocs);
19      $\ell^* \leftarrow \min(\ell^*, \min_{s \in \text{sources}(e)} \lambda(s) + 1);$ 
20     minimizeCrossings(10,  $\ell^*$ );
21 end

```

Algoritmo D.13: Añadir una fuente o destino a una arista existente (Parte II).

removeNode

Elimina un nodo existente del hipergrafo. El comportamiento depende de si el nodo tiene padres, hijos o ambos. Si no tiene hijos, simplemente se retira de todas las aristas en que es destino. Si tiene hijos pero no padres, se elimina de todas las aristas en que es fuente y se reubican los hijos. Si tiene ambos padres e hijos, se crea una nueva hiperarista directa de los padres a los hijos para no perder la información de conectividad, y entonces se elimina.

```

Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$  y un nodo  $v \in V$ .
Output:  $H$  modificado:  $v$  eliminado y la conectividad de su entorno preservada.

1   $P \leftarrow \text{parents}(v)$    $C \leftarrow \text{children}(v)$ ;
2  if  $C = \emptyset$  then
3      // Sin hijos
4      foreach  $e \in \mathcal{E}$  tal que  $v \in \text{targets}(e)$  do
5          |  $\text{removeTargetsFromHyperedge}(e, \{v\}, \text{relocation} = \text{false})$ ;
6      end
7  else if  $P = \emptyset$  then
8      // Sin padres pero con hijos
9       $\text{toRelocate} \leftarrow []$ ;
10     foreach  $e \in \mathcal{E}$  tal que  $v \in \text{sources}(e)$  do
11         |  $\text{toRelocate} \leftarrow \text{toRelocate} \parallel \text{targets}(e)$ ;
12         |  $\text{removeSourcesFromHyperedge}(e, \{v\}, \text{relocation} = \text{false})$ ;
13     end
14      $\text{relocateNodes}(\text{toRelocate})$ ;
15 else
16     foreach  $e \in \mathcal{E}$  tal que  $v \in \text{sources}(e)$  do
17         |  $\text{removeSourcesFromHyperedge}(e, \{v\}, \text{relocation} = \text{false})$ ;
18     end
19     foreach  $e \in \mathcal{E}$  tal que  $v \in \text{targets}(e)$  do
20         |  $\text{removeTargetsFromHyperedge}(e, \{v\}, \text{relocation} = \text{false})$ ;
21     end
22      $e' \leftarrow \text{createHyperedge}(P, C, -1)$ ;
23     if  $\neg \text{relocateNodes}(\text{targets}(e'))$  then
24         | if  $\text{edgeIsShort}(e') \geq 0$  then
25             |  $\text{addHyperedgeToLayer}(\text{edgeIsShort}(e'), e')$ ;
26         | else
27             |  $\text{splitLongEdge}(H, e')$ ;
28         | end
29     else
30         end
31 end
32  $V \leftarrow V \setminus \{v\}$ ;
33  $\text{removeNodeFromLayer}(\lambda(v), v)$ ;
34  $\text{cleanUp}(H)$ ;
    
```

Algoritmo D.14: Eliminar un nodo existente

removeConnection

Elimina la conexión directa entre un nodo padre y un nodo hijo ya existentes. Localiza la hiperarista que los conecta, retira el hijo de sus destinos y, si la arista queda sin fuentes, simplemente reubica al hijo. En caso contrario, crea una nueva arista con las fuentes restantes, gestiona la posible reubicación del hijo y subdivide si la nueva arista resulta larga. Como solo puede existir una hiperarista que conecte a un padre con un hijo, el algoritmo se detiene tan pronto como encuentra la primera.

```

1  Input: Hipergrafo  $H = (V, \mathcal{E}, \lambda)$  y dos nodos  $parent, child \in V$  cuya conexión existe.
2  Output:  $H$  modificado: la conexión  $parent \rightarrow child$  eliminada y la estructura actualizada.
3  foreach  $e \in \mathcal{E}$  tal que  $parent \in sources(e)$  and  $child \in targets(e)$  do
4       $remSrc \leftarrow sources(e) \setminus \{parent\};$ 
5       $removeTargetsFromHyperedge(e, \{child\}, relocation = false);$ 
6      if  $remSrc = \emptyset$  then
7          if  $relocateNodes(\{child\})$  then
8               $\ell^* \leftarrow \min_{p \in parents(child)} \lambda(p) + 1;$ 
9               $minimizeCrossings(10, \ell^*);$ 
10             end
11         else
12              $e' \leftarrow createHyperedge(remSrc, \{child\}, -1);$ 
13             if  $relocateNodes(\{child\})$  then
14                 if  $children(child) = \emptyset$  then
15                      $D \leftarrow \{d \in dummies(e')\};$ 
16                      $minimizeCrossingsForNodes(D, \min_d \lambda(d), \max_d \lambda(d));$ 
17                 else
18                      $\ell^* \leftarrow \min_{p \in parents(child)} \lambda(p) + 1;$ 
19                      $minimizeCrossings(10, \ell^*);$ 
20                 end
21             else
22                 if  $edgeIsShort(e') \geq 0$  then
23                      $addHyperedgeToLayer(edgeIsShort(e'), e');$ 
24                 else
25                      $splitLongEdge(H, e');$ 
26                      $D \leftarrow \{d \in dummies(e')\};$ 
27                      $minimizeCrossingsForNodes(D, \min_d \lambda(d), \max_d \lambda(d));$ 
28                 end
29             end
30         end
31     end
32     return;
33 end

```

Algoritmo D.15: Eliminar una conexión entre dos nodos existentes**removeSourcesFromHyperedge y removeTargetsFromHyperedge**

Estas dos operaciones son simétricas: eliminan un conjunto de fuentes o de destinos de una hiperarista original e . En ambos casos, si la arista queda vacía (sin fuentes o sin destinos), se disuelve completamente. Si la arista tenía segmentos y tras la eliminación resulta ser corta, se disuelven los

segmentos. Si sigue siendo larga, se propaga la eliminación a través de la cadena de segmentos en la dirección correspondiente: de arriba hacia abajo al eliminar fuentes (pues un segmento sin fuentes vivas mata a su *dummy* destino), y de abajo hacia arriba al eliminar destinos (pues un segmento sin destinos vivos mata a su *dummy* fuente). Solo se describe `removeSourcesFromHyperedge`; `removeTargetsFromHyperedge` es completamente análogo con el recorrido invertido.

fuseNodes

Fusiona dos nodos existentes v_1 y v_2 en uno solo. Todos los padres y los hijos de v_2 que no sean ya padres o hijos de v_1 se traspasan a v_1 ; los que sean comunes simplemente se desvinculan de v_2 . Antes de comprometer el cambio, la fusión se aplica temporalmente y se comprueba que no se crea ningún ciclo. Una vez confirmada, se reemplazan todas las apariciones de v_2 en las hiperaristas, se eliminan las hiperaristas que hayan quedado duplicadas y se reubica v_1 si su profundidad correcta ha cambiado. La idea es utilizar uno de los nodos como nodo resultante de la fusión y renombrarlo al final, dando la sensación que se han fusionado; cuando en realidad el término más correcto sería hablar de absorción.

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$, una hiperarista original $e \in \mathcal{E}$ y un conjunto de fuentes a eliminar $R \subseteq \text{sources}(e)$.
Output: H modificado: las fuentes en R retiradas de e y de sus segmentos; los dummies y segmentos que queden huérfanos son eliminados; los destinos se reubican si el parámetro *relocation* es verdadero.

```

1  foreach  $s \in R$  do
2       $e.\text{removeSource}(s)$ ;
3      foreach  $t \in \text{targets}(e)$  do
4           $s.\text{removeChild}(t)$    $t.\text{removeParent}(s)$ ;
5      end
6  end
7  if  $\text{sources}(e) = \emptyset$  then
8       $\text{dissolveSegments}(H, \{e\})$ ;
9       $\mathcal{E} \leftarrow \mathcal{E} \setminus \{e\}$ ;
10     if relocation then
11          $\text{relocateNodes}(\text{targets}(e))$ 
12     end
13     return;
14 end
15 if  $\text{segments}(e) = []$  then
16     return;                                     // era corta -- nada más que hacer
17 end
18 if  $\text{edgeIsShort}(e) \geq 0$  then
19     // Ahora es corta -- disolver segmentos
20      $\text{dissolveSegments}(H, \{e\})$ ;
21      $\text{addHyperedgeToLayer}(\text{edgeIsShort}(e), e)$ ;
22     return;
23 else
24      $\text{deadDummies} \leftarrow \{\}$    $\text{deadSegs} \leftarrow \{\}$ ;
25     foreach segmento  $\sigma$  de  $e$  en orden creciente de capa do
26         foreach  $s \in R \cup \text{deadDummies}$  do
27              $\sigma.\text{removeSource}(s)$ ;
28         end
29         if  $\text{sources}(\sigma) = \emptyset$  then
30              $\text{deadSegs} \leftarrow \text{deadSegs} \cup \{\sigma\}$ ;
31             foreach  $t \in \text{targets}(\sigma)$  tal que  $t$  es dummy do
32                  $\text{deadDummies} \leftarrow \text{deadDummies} \cup \{t\}$ ;
33             end
34         end
35         foreach  $\sigma \in \text{deadSegs}$  do
36              $\text{removeHyperedgeFromLayer}(\lambda(\sigma), \sigma)$ ;
37         end
38         foreach  $d \in \text{deadDummies}$  do
39              $\text{removeNodeFromLayer}(\lambda(d), d)$ ;
40              $V \leftarrow V \setminus \{d\}$ ;
41         end
42         if relocation and  $\text{relocateNodes}(\text{targets}(e))$  then
43              $\ell^* \leftarrow \min_{p \in \text{parents}(\text{targets}(e))} \lambda(p) + 1$ ;
44              $\text{minimizeCrossings}(10, \ell^*)$ ;
45         end
46 end

```

Algoritmo D.16: Eliminar fuentes o destinos de una arista existente

Input: Hipergrafo $H = (V, \mathcal{E}, \lambda)$ y dos nodos distintos $v_1, v_2 \in V$.

Output: H modificado: v_2 eliminado y sus conexiones absorbidas por v_1 , que adopta el nombre `name`. Sin ciclos ni hiperaristas duplicadas.

```

1   $P_1 \leftarrow \text{parents}(v_1)$    $C_1 \leftarrow \text{children}(v_1)$ ;
2   $P_2 \leftarrow \text{parents}(v_2)$    $C_2 \leftarrow \text{children}(v_2)$ ;
3  foreach  $p \in P_2$  do
4      if  $p \notin P_1$  then
5           $p.\text{replaceChild}(v_2, v_1)$    $v_1.\text{addParent}(p)$ ;
6      else
7           $p.\text{removeChild}(v_2)$ ;
8      end
9  end
10 foreach  $c \in C_2$  do
11     if  $c \notin C_1$  then
12          $c.\text{replaceParent}(v_2, v_1)$    $v_1.\text{addChild}(c)$ ;
13     else
14          $c.\text{removeParent}(v_2)$ ;
15     end
16 end
17 if  $\text{checkCycles}(v_1)$  then
18     // Rollback
19     lanzar «ciclo detectado»;
20 end
21  $v_1.\text{setName}(\text{name})$ ;
22 foreach  $e \in \mathcal{E}$  do
23      $\text{update}(e, v_2, v_1)$    $\text{update}(e, v_2, v_1)$ ;
24 end
25 // Eliminar hiperaristas duplicadas
26  $\text{seen} \leftarrow \{\}$    $\text{toDel} \leftarrow \{\}$ ;
27 foreach  $e \in \text{modified}$  do
28      $k \leftarrow (\text{sources}(e), \text{targets}(e))$ ;
29     if  $k \notin \text{seen}$  then
30          $\text{seen} \leftarrow \text{seen} \cup \{k\}$ 
31     else
32          $\text{removeHyperedge}(H, e)$ 
33     end
34 end
35  $V \leftarrow V \setminus \{v_2\}$ ;
36  $\text{removeNodeFromLayer}(\lambda(v_2), v_2)$ ;
37 if  $\text{relocateNodes}(\{v_1\})$  then
38      $\ell^* \leftarrow \min_{p \in \text{parents}(v_1)} \lambda(p) + 1$ ;
39      $\text{minimizeCrossings}(10, \ell^*)$ ;
40 end

```

Algoritmo D.17: Fusionar dos nodos existentes

D.2. Minimización de cruces

El objetivo de esta sección es complementar las explicaciones realizadas en la Sección 6.2 sobre el algoritmo de minimización de cruces entre hiperaristas. La implementación concreta se encuentra en el archivo *minimizeCrossings.cpp* de [28].

A diferencia de la sección anterior, los pseudocódigos que se encontrarán aquí serán más bien reducidos, pues no se repetirá el pseudocódigo del artículo original *A Global k -Level Crossing Reduction Algorithm* [2]. Es de suma importancia que el lector consulte primero dicho artículo para conocer los detalles concretos de operación del algoritmo, pues aquí se explicará superficialmente su lógica y las particularidades de su implementación en la aplicación.

Planteamiento del problema

Se recuerda que el hipergrafo propio se había transformado en un DAG con k capas, tal y como se observaba en la Figura 6.2. El pseudocódigo de esa transformación se omite por ser sencillo y estar disponible en [19]. Lo único relevante de la transformación es que se añaden *hubs* entre cada par de capas consecutivas, duplicando así el número de capas del grafo auxiliar.

El problema de minimización de cruces sobre este DAG consiste en encontrar una ordenación de los vértices de cada capa que minimice el número de cruces entre aristas, sujeta a la restricción de que no pueden existir conflictos de tipo 2. La primera idea que se viene a la cabeza es probar todas las combinaciones posibles de ordenaciones y descartar las que no cumplen la restricción. Sin embargo, el número de combinaciones es $O((n!)^k)$, lo que resulta inasumible incluso para DAGs pequeños. Por ello, se recurre a la heurística de *Global Sifting* propuesta por Bachmaier et al. en [2].

El concepto de bloque

Para garantizar la ausencia de conflictos de tipo 2 y que las aristas largas queden lo más verticales posible, hay conjuntos de nodos que deben moverse como una unidad indivisible. A estas unidades se las denomina **bloques**. En lugar de ordenar nodos individuales, *Global Sifting* ordena bloques, lo que reduce el espacio de búsqueda y garantiza la restricción estructural de forma implícita.

Cada nodo real constituye un bloque individual. Para las hiperaristas largas, se identifican sus segmentos interiores y se agrupan aquellos que son consecutivos, es decir, los que representan la misma vertical de la misma hiperarista. Cada agrupación maximal de segmentos consecutivos forma un bloque, que comprende desde el primer nodo *dummy* de la cadena hasta el último, incluyendo tanto nodos *dummy* como *hubs*. Los nodos *hub* o *dummy* que no pertenezcan a ninguna cadena de segmentos interiores forman bloques individuales.

La Figura D.4 ilustra la formación de bloques a partir de un hipergrafo propio. En este caso, los

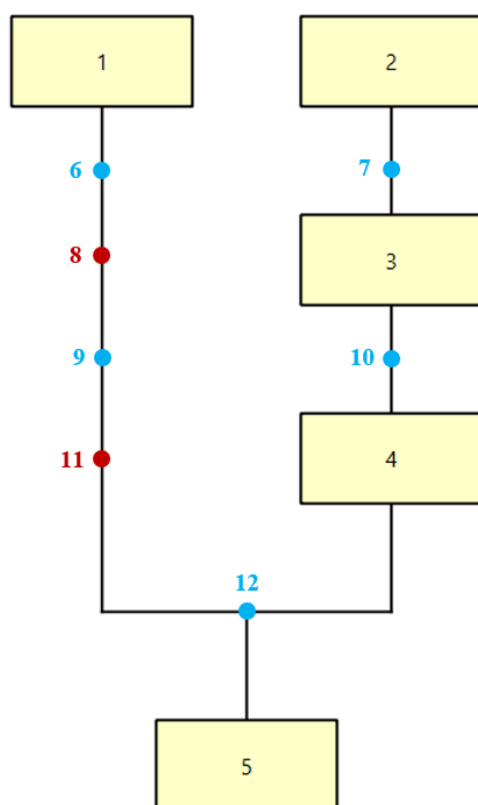


Figura D.4: Bloques en *Global Sifting*

bloques resultantes son:

- Bloques 1–7: nodos reales 1 al 7, uno por bloque.
- Bloque 8: nodos 8, 9 y 11 (cadena de segmentos interiores).
- Bloque 9: nodo 10.
- Bloque 10: nodo 12.

El algoritmo de *Global Sifting*

Una vez definidos los bloques, la idea fundamental de *Global Sifting* es la siguiente: si $\mathcal{B}$ es la lista ordenada de bloques, los nodos se depositan en las capas de izquierda a derecha siguiendo el orden dado por $\mathcal{B}$. Por ejemplo, una posible ordenación que refleje la disposición de la Figura D.4 es $[B_1, B_2, B_8, B_3, B_4, B_5, B_6, B_7, B_9, B_{10}]$. Nótese que la ordenación no es única: cualquier permutación de bloques cuyas capas no tengan intersección produce la misma ordenación de nodos dentro de cada capa. Por ejemplo, permutar B_4 y B_{10} en la lista anterior mantiene el mismo orden de nodos en cada capa.

El algoritmo busca la ordenación de $\mathcal{B}$ que minimice el número de cruces mediante la siguiente

estrategia: se elige un bloque A y se prueba a desplazarlo a todas las posiciones posibles dentro de B , registrando en cuál se obtienen menos cruces. Esta operación se denomina *Sifting-Step* en [2] y se repite para cada bloque. El proceso completo se itera un número determinado de rondas; en el artículo original se proponen 10.

Para mayor eficiencia, en lugar de recalcular el número total de cruces en cada posición, se calcula la *variación* en el número de cruces al permutar dos bloques contiguos, lo que se denomina *Sifting-Swap*. Este cálculo es un problema matemático sencillo que se invita al lector a resolver por sí mismo.

```

Input: Grafo propio por capas  $G' = (V', E', \lambda')$ , lista ordenada de bloques  $B$  y un bloque  $A \in B$  a desplazar.
Output:  $B$  reordenado de forma que  $A$  ocupa la posición que minimiza el número de cruces global.

1  Mover  $A$  al frente de  $B$ ;
2   $\delta \leftarrow 0$    $\delta^* \leftarrow 0$    $p^* \leftarrow 0$ ;           //  variación acumulada, mínima y posición
    óptima
3  for  $p \leftarrow 1$  to  $|B| - 1$  do
4       $\delta \leftarrow \delta + \text{siftingSwap}(A, B[p]);$  //  acumular la variación al intercambiar
         $A$  con  $B[p]$ 
5      if  $\delta < \delta^*$  then
6           $\delta^* \leftarrow \delta$    $p^* \leftarrow p$ ;
7      end
8  end
9  Insertar  $A$  en la posición  $p^*$  de  $B$ ;

```

Algoritmo D.18: Sifting Step

Como se aprecia en el pseudocódigo, el bloque A se desplaza desde el frente (variación inicial de 0) hacia la derecha, acumulando la variación de cruces en cada intercambio. La posición óptima es aquella en la que la variación acumulada es mínima.

Ordenación inicial de bloques

Los propios autores señalan que una ordenación inicial adecuada puede suponer grandes mejoras en el resultado final, efecto que se ha podido corroborar en las pruebas de integración realizadas. Se sugiere utilizar el orden lexicográfico, considerando primero la capa y después la posición original del bloque dentro de ella.

La implementación desarrollada sigue esta indicación, pero introduce una política adicional motivada por el comportamiento del sistema ante actualizaciones dinámicas. Cuando se tiene que insertar un nodo a una capa, el nodo implicado se añade al *final*, independientemente de su posición ideal. Esto provoca que fuentes y sus destinos queden inicialmente muy alejados entre sí, generando un gran número de cruces artificiales antes incluso de que el *Sifting* comience a actuar.

Es conveniente destacar que este tipo de situaciones son frecuentes en un entorno con actualización dinámica, pues suceden cada vez que se añade un nuevo nodo, se subdivide una arista, se crea una hiperarista larga o un nodo y su descendencia tienen que ser reubicados hacia otras capas. La Figura D.5 muestra un ejemplo: al insertar el nodo *E*, el hipergrafo lo inserta al final de su capa, lejos de su fuente *A*, lo que introduce cruces innecesarios.

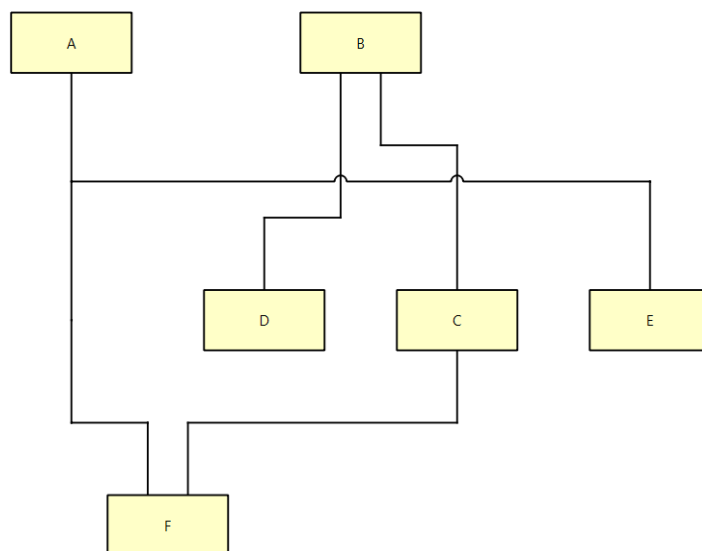


Figura D.5: Inicialización pobre tras actualización dinámica

Para mitigar este efecto, el método `buildBlockOrder` construye el orden inicial procesando la capa ancla ℓ_0 con su orden original y ordenando cada capa sucesiva en función de la posición de las fuentes en la capa inmediatamente superior, acercando así fuentes y destinos desde el principio. Esta política tiene además una consecuencia importante para la usabilidad: cuando el usuario reordena manualmente dos nodos en una misma capa, el algoritmo respeta esa decisión como punto de partida en lugar de devolver los nodos a su posición anterior.

Los dos métodos auxiliares que sustentan esta ordenación inicial son `sortLayer`, que ordena los nodos de cada capa en función de sus fuentes, y `sortHubs`, que ordena los *hubs* de cada gap de forma coherente con las capas adyacentes. Sus pseudocódigos se muestran a continuación.

Input: Mapa $upperOrder = \{v \mapsto pos(v)\}$ con las posiciones de los nodos de la capa superior, lista de nodos L de la capa inferior y lista de hiperaristas E que conectan ambas capas.

Output: L reordenada y mapa $lowerOrder = \{v \mapsto pos(v)\}$ con las posiciones resultantes.

// Para cada nodo destino, calcular la posición más a la izquierda de sus fuentes en la capa superior

```

1  leftOf[v]  $\leftarrow +\infty \quad \forall v \in L$ ;
2  foreach  $e \in E$  do
3       $p_{\min} \leftarrow \min_{s \in sources(e)} upperOrder[s]$ ;
4      foreach  $t \in targets(e)$  do
5           $leftOf[t] \leftarrow \min(leftOf[t], p_{\min})$ ;
6      end
7  end

// Ordenar  $L$  de forma estable por  $leftOf$  creciente
8   $L \leftarrow \text{stableSort}(L, u \prec v \iff leftOf[u] < leftOf[v])$ ;
9   $lowerOrder \leftarrow \{L[i] \mapsto i \mid 0 \leq i < |L|\}$ ;
10 return  $lowerOrder$ ;

```

Algoritmo D.19: Ordenación de la capa inferior.

Input: Mapa $upperOrder$ con las posiciones de los nodos de la capa $\ell-1$, mapa $lowerOrder$ con las posiciones de los nodos de la capa ℓ y conjunto de hubs H del gap $(\ell-1, \ell)$.

Output: H reordenado.

// Orden lexicográfico: primero por fuente más a la izquierda; en caso de empate, por destino más a la izquierda

```

1  sort( $H, h_a \prec h_b \iff$ ;
2       $\min_{p \in parents(h_a)} upperOrder[p] < \min_{p \in parents(h_b)} upperOrder[p]$ ;
3       $\vee (\min_{p \in parents(h_a)} upperOrder[p] \text{ iguales} \wedge \min_{c \in children(h_a)} lowerOrder[c] < \min_{c \in children(h_b)} lowerOrder[c])$ );

```

Algoritmo D.20: Ordenación de los hubs del gap.

Con estos dos métodos, se construye el orden inicial de bloques para el algoritmo de minimización:

```

Input: Grafo auxiliar  $G' = (V', E', \lambda')$ , conjunto de bloques  $B$  y capa ancla  $\ell_0$ .
Output: Lista ordenada  $B$  de bloques en la que el orden refleja la disposición topológica del grafo de arriba a abajo,
con fuentes antes que sus destinos.

1   $upperOrder \leftarrow \{v \mapsto pos(v) \mid v \in nodes(\ell_0)\};$  // orden original de la capa ancla
2  foreach  $v \in nodes(\ell_0)$  en orden do
3  |    $B \leftarrow B \parallel [block(v)];$ 
4  end

// -- Barrido capa a capa por debajo de la ancla --
5  for  $\ell \leftarrow \ell_0 + 1$  to  $\ell_{\text{máx}}$  do
6  |    $lowerOrder \leftarrow sortLayer(upperOrder, nodes(\ell));$ 
7  |   // Insertar primero los hubs del gap  $(\ell-1, \ell)$ 
8  |    $sortHubs(upperOrder, lowerOrder, \ell);$ 
9  |   foreach hub  $h$  en el gap  $(\ell-1, \ell)$  en el orden calculado do
10 | |   if  $block(h) \notin B$  then
11 | | |    $B \leftarrow B \parallel [block(h)];$ 
12 | |   end
13 |   end
14 |   // Insertar después los nodos reales de la capa  $\ell$ 
15 |   foreach  $v \in nodes(\ell)$  en el orden de  $lowerOrder$  do
16 | |   if  $block(v) \notin B$  then
17 | | |    $B \leftarrow B \parallel [block(v)];$ 
18 | |   end
19 |   end
20 |    $upperOrder \leftarrow lowerOrder;$ 
21 end

```

Algoritmo D.21: Construcción del orden inicial de bloques

Minimización local para nodos recién insertados

Se ha implementado una política de minimización adicional para los casos en los que solo es necesario reubicar un conjunto reducido de bloques mientras el resto permanece inmóvil. Es la función `minimizeCrossingsForNodes`, empleada por el hipergrafo en la sección anterior. Se aplica, por ejemplo, cuando al añadir una conexión larga se insertan nuevos nodos *dummy* en las capas intermedias y hay que encontrar la posición menos conflictiva para ellos.

La Figura D.6 ilustra este escenario: al añadir una conexión entre los nodos 1 y 5 se crean dos nodos *dummy* que deben ubicarse de la forma óptima.

Una primera aproximación consistiría en ejecutar el *Sifting-Step* para cada nuevo bloque de forma independiente. Sin embargo, los bloques a mover pertenecen a capas diferentes, lo que hace que sus posiciones relativas entre sí no afecten al número de cruces. El *Sifting-Step* individual solo buscaría

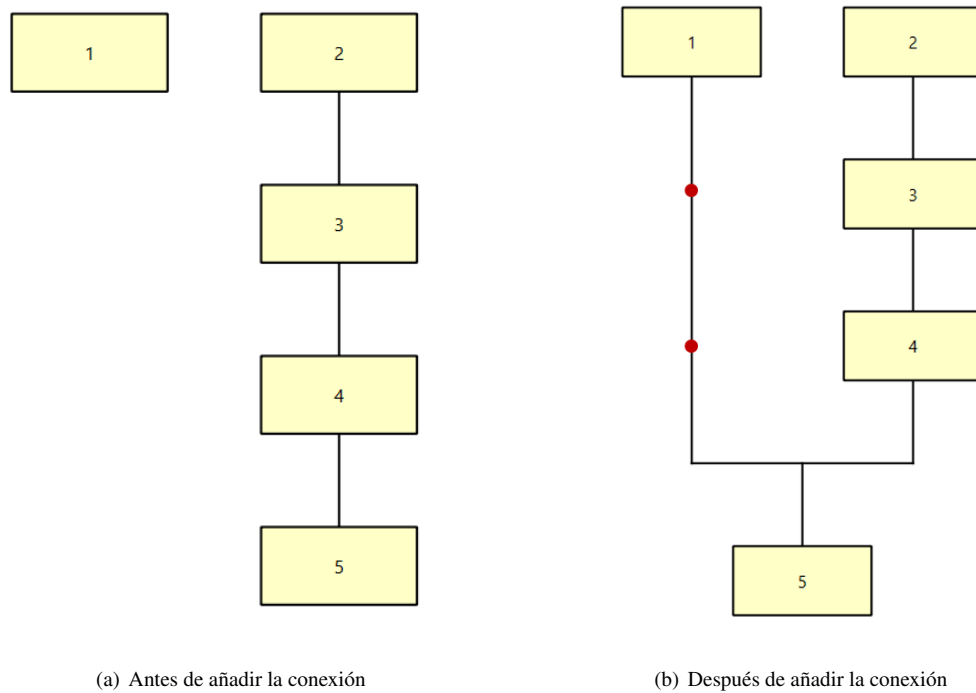


Figura D.6: Encontrar la posición óptima para nuevos nodos *dummy*

la mejor posición relativa de dichos bloques entre sí, devolviendo siempre la posición original como óptima, lo que no es correcto.

Lo que se necesita es encontrar la mejor posición para todos los bloques móviles simultáneamente. Como el orden relativo entre ellos no afecta al número de cruces y el orden de los bloques fijos debe mantenerse, el número de casos a explorar se reduce a $C(n_B, m)$, donde n_B es el número total de bloques y m el número de bloques móviles. Haciendo uso de *Sifting-Swap*, la variación de cruces de cada combinación se calcula de forma incremental, sin necesidad de recalcular el total, lo que hace viable esta aproximación. El pseudocódigo de la búsqueda recursiva se muestra a continuación.

Naturalmente, la búsqueda exhaustiva garantiza la posición óptima, pero su coste crece como $O(C(n_B, m))$. Dado que dicho número es fácilmente computable antes de iniciar la búsqueda, se establece un umbral de $\Theta = 10^6$ combinaciones. Cuando se supera, los bloques móviles se tratan como un único bloque compuesto y se desplazan conjuntamente mediante un *Sifting-Step* de coste $O(n_B \cdot m)$. El desplazamiento se realiza de forma suave: en cada paso, los m bloques móviles se intercambian uno a uno con el bloque fijo que entra por la derecha del grupo, manteniendo en todo momento su orden relativo interno. Este algoritmo final se ubica en el pseudocódigo D.23.

Input: Lista de bloques B , índice $movableStart$ del primer bloque móvil aún sin colocar, bloque final A_{last} , conjunto $movableSet$ de identificadores de bloques móviles y variación acumulada de cruces χ .

Output: Actualiza $best_\chi$ y $best_B$ cuando se encuentra una ordenación con menos cruces que la mejor conocida.

```

1   $pos \leftarrow$  primer índice  $\geq movableStart$  tal que  $B[pos] \in movableSet$ ;
2   $A \leftarrow B[pos]$ ;           // bloque móvil a desplazar en este nivel
3  while true do
4      if  $A = A_{last}$  then
5          // Caso base: todos los bloques móviles colocados
6          if  $\chi < best_\chi$  then
7               $best_\chi \leftarrow \chi$    $best_B \leftarrow B$ ;
8          end
9      else
10         // Caso recursivo: fijar posición actual de  $A$  y colocar
11         el resto
12          $snapshot \leftarrow (S.blocks, S.\pi, B)$ ;
13          $siftNodesSearch(B, movableStart + 1, A_{last}, movableSet, \chi)$ ;
14          $(S.blocks, S.\pi, B) \leftarrow snapshot$ ;
15     end
16     if  $pos = fixedCount$  or  $B[pos - 1] \in movableSet$  then
17         break;
18     end
19      $\chi \leftarrow \chi + siftingSwap(B[pos - 1], A)$ ;
20      $swap(B[pos - 1], B[pos])$ ;
21      $pos \leftarrow pos - 1$ ;
22 end

```

Algoritmo D.22: Búsqueda de posición óptima para bloques móviles.

Input: Lista de bloques B , conjunto de nodos $nodes$ recién insertados y umbral $\Theta = 10^6$ de combinaciones.

Output: B reordenado de forma que los bloques móviles ocupan la posición que minimiza el número de cruces global.

```

// -- Fase 1: identificar bloques móviles --
// Son móviles los bloques de los nodos dados, más cualquier
// hub cuyos fuentes o destinos sean todos también móviles.
1   $M \leftarrow \{\text{block}(v) \mid v \in nodes\} \cup \{h \in hubs \mid \text{parents}(h) \subseteq M \text{ or } \text{children}(h) \subseteq M\};$ 
2  if  $M = \emptyset$  then
3  |   return
4  end

// -- Fase 2: sesgo a la derecha --
5   $B \leftarrow \text{stablePartition}(B, b \mapsto b \notin M);$ 
6   $\text{sortAdjacencies}();$ 

// -- Fase 3: elegir estrategia según  $C(n_B, |M|)$  --
7   $n_B \leftarrow |B|$    $m \leftarrow |M|$    $comb \leftarrow C(n_B - \text{fixedCount}, m);$ 
8  if  $comb \leq \Theta$  then
|   // Búsqueda exhaustiva con backtracking
9    $\chi^* \leftarrow 0$    $B^* \leftarrow B;$ 
10   $\text{siftNodesSearch}(n_B - m, n_B, B[n_B - 1], M, 0, \chi^*, B^*);$ 
11   $B \leftarrow B^*;$ 
12 else
|   // Fallback: desplazamiento suave del bloque compuesto
13   $\chi \leftarrow 0$    $\chi^* \leftarrow 0$    $B^* \leftarrow B;$ 
14  for  $k \leftarrow n_B - m$  to  $\text{fixedCount} + 1$  do
15  |   for  $j \leftarrow k$  to  $k + m - 1$  do
16  |   |    $\chi \leftarrow \chi + \text{siftingSwap}(B[j - 1], B[j]);$ 
17  |   |    $\text{swap}(B[j - 1], B[j]);$ 
18  |   end
19  |   if  $\chi < \chi^*$  then
20  |   |    $\chi^* \leftarrow \chi$    $B^* \leftarrow B;$ 
21  |   end
22  end
23   $B \leftarrow B^*;$ 
24 end
25  $\text{sortAdjacencies}();$ 

```

Algoritmo D.23: Posicionamiento de bloques móviles.

D.3. Asignación de coordenadas horizontales

A diferencia del algoritmo de minimización de cruces, que sí ha obtenido bastante grado de personalización, el algoritmo de asignación de coordenadas horizontales se ha implementado siguiendo la descripción dada por Brandes y Köpf en [6] sin apenas modificaciones, más allá de las correcciones de [7]. En este caso, la implementación concreta se puede encontrar en el fichero *horizontalCoordina-tes.cpp* en [28].

Dado el escaso grado de personalización, en esta sección plantearemos el problema de asignación de coordenadas horizontales y describiremos la idea detrás del algoritmo de Brandes y Köpf. No obstante, es muy recomendable leer las fuentes originales para entender en profundidad el algoritmo, ya que la descripción que se ofrece aquí es muy superficial.

El problema es el siguiente: dado un DAG en k capas y una ordenación de los nodos dentro de cada capa, asignar coordenadas horizontales a cada nodo de forma que se minimice la distancia de un nodo para con sus padres e hijos y se respete una determinada distancia mínima entre nodos. Además, se añade la restricción de representar las aristas largas lo más vertical posible, alineando segmentos interiores consecutivos de la misma arista.

Para resolver el problema, la idea central que propone el algoritmo de Brandes y Köpf es que, en un grafo por capas con un orden de nodos ya fijado, la coordenada horizontal de cada nodo debería ser, idealmente, la mediana de las coordenadas de sus vecinos en la capa adyacente. Sin embargo, satisfacer todas las medianas simultáneamente es en general imposible, por lo que el algoritmo se decanta por sesgos: alinear con respecto a la mediana izquierda o derecha y con respecto a los padres o a los hijos. De esta forma, se generan cuatro soluciones diferentes, cada una con un sesgo diferente, y se combinan posteriormente para obtener una solución equilibrada.

Alineación en bloques

El núcleo del método es la noción de bloque: un conjunto de nodos, uno por capa, que el algoritmo decide alinear verticalmente, es decir, asignarles la misma coordenada horizontal. Nótese cómo los bloques permiten de manera natural alinear segmentos interiores consecutivos de una misma arista larga, cumpliendo con la restricción anteriormente impuesta. Adicionalmente, para asegurar esta alineación, cualquier arista que se cruce con un segmento interior y produzca, por ende, un conflicto de tipo 1, es ignorada y no se tiene en cuenta a la hora de construir los bloques.

La construcción de los bloques se realiza de la siguiente manera: se recorre el grafo y se selecciona, para cada nodo, con qué vecino de la capa superior se va a alinear. En caso de que dos nodos quieran alinearse con el mismo vecino, la preferencia se tendrá en función del sesgo horizontal elegido (izquierda o derecha). El vecino con el que se alinee dependerá también del sesgo vertical (elegir padres o hijos). Como, dentro de cada bloque, todos los nodos recibirán exactamente la misma

coordenada x , por lo que el problema se reduce a asignar coordenadas a los bloques respetando las restricciones de separación mínima entre nodos de la misma capa.

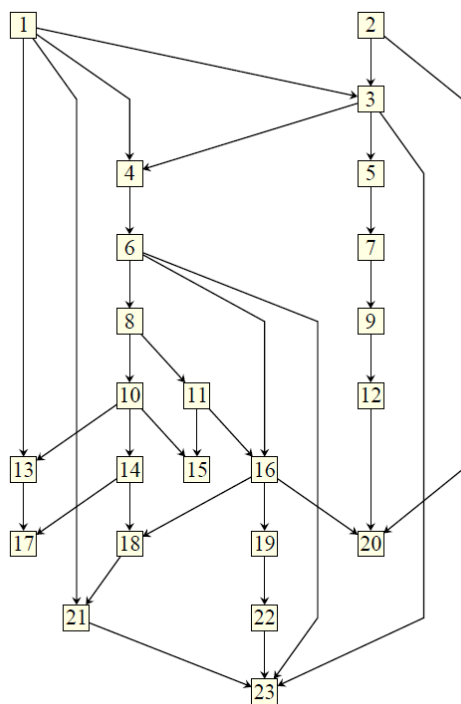
Los cuatro sesgos

El algoritmo se ejecuta cuatro veces, una por cada combinación de dos decisiones independientes: la dirección de recorrido vertical o sesgo vertical (de arriba a abajo o de abajo a arriba) y el sesgo horizontal (preferencia por la izquierda o por la derecha). Esto da lugar a cuatro variantes: sesgo superior-izquierdo, superior-derecho, inferior-izquierdo e inferior-derecho.

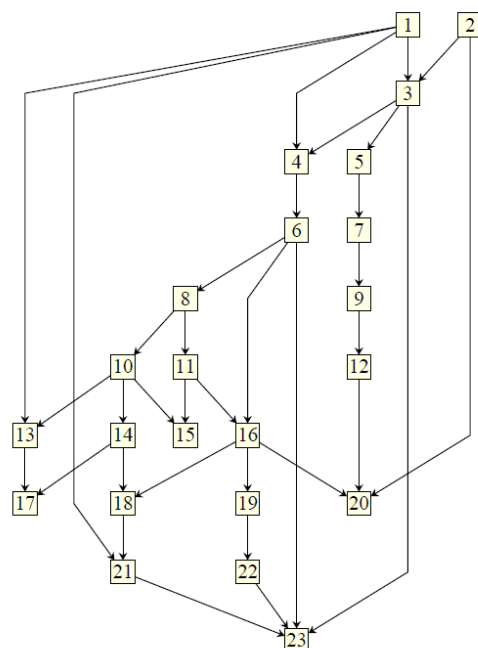
El motivo de ejecutar las cuatro variantes es que cada una introduce un sesgo diferente en la disposición resultante: las variantes de izquierda tienden a pegar los nodos hacia la izquierda del espacio disponible y las de derecha hacia la derecha, mientras que las variantes de arriba y abajo difieren en qué capa tiene más influencia sobre las demás. Ninguna variante es universalmente mejor que las otras, pero cada una captura bien una parte de la estructura del grafo. Esto aparece reflejado en la Figura D.7.

Combinación de las cuatro soluciones

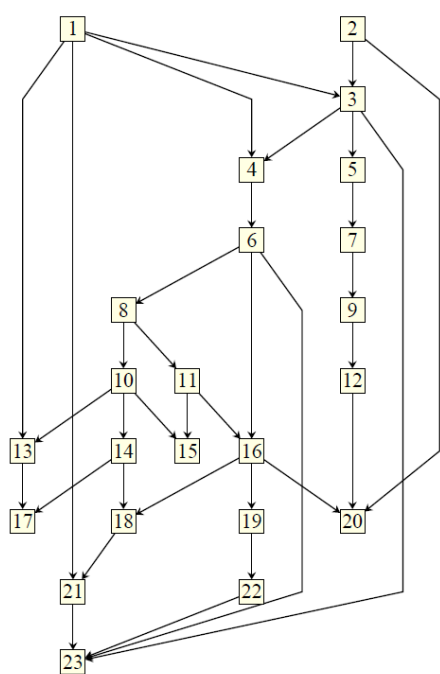
Una vez obtenidas las cuatro asignaciones de coordenadas, se combinan en una solución única. Para ello, se normalizan las cuatro soluciones de forma que ocupen el mismo rango horizontal, y para cada nodo se toma la mediana de sus cuatro coordenadas. Este promedio por mediana elimina los extremos introducidos por los sesgos individuales y produce una disposición equilibrada que, en la práctica, minimiza la longitud total de las aristas y mantiene los nodos bien centrados respecto a sus vecinos.



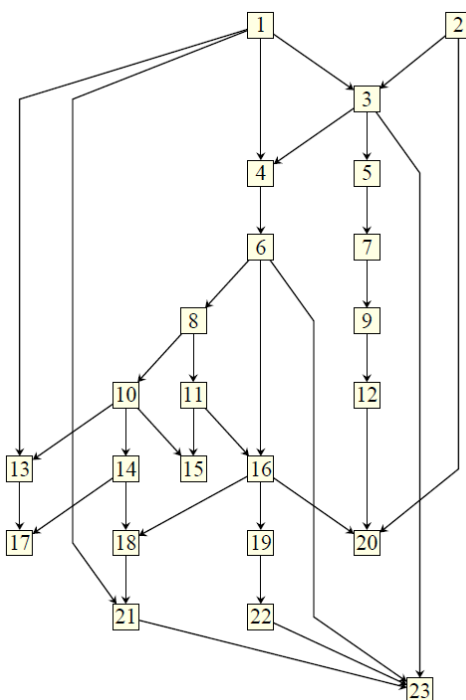
(a) Sesgo superior-izquierdo



(b) Sesgo superior-derecho



(c) Sesgo inferior-izquierdo



(d) Sesgo inferior-derecho

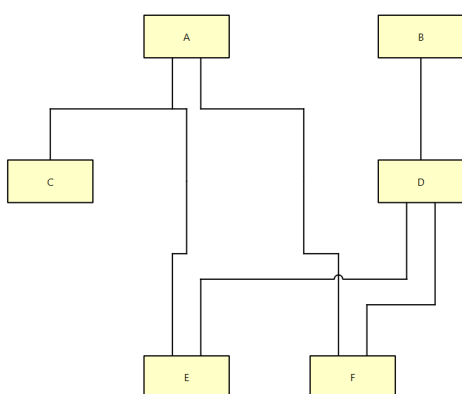
Figura D.7: Representación del mismo grafo para los cuatro sesgos

D.4. Asignación de puertos

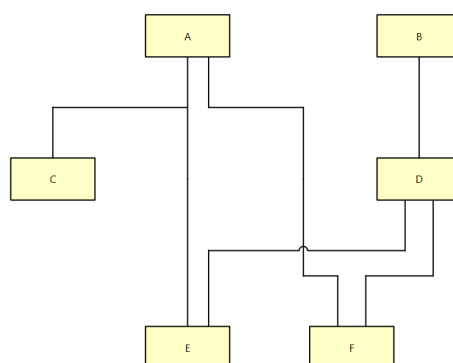
Esta sección complementa la explicación de la asignación de puertos de la Sección ???. Se describen dos aspectos que no se trataron en el texto principal: la reducción de codos horizontales innecesarios y el mecanismo de detección y resolución de conflictos entre segmentos verticales de nodos distintos. La implementación concreta de la asignación de puertos se puede encontrar en el fichero *portAssignment.cpp* en [28].

Reducción de codos horizontales

Tras asignar los puertos de forma simétrica dentro de cada nodo, pueden aparecer **codos horizontales** innecesarios en la representación. Un codo horizontal ocurre cuando el puerto fuente de una hiperarista en el nodo superior y su puerto destino en el nodo inferior no comparten la misma abscisa, pero sí están muy cerca; lo que puede traducirse en una elongación horizontal pequeña e innecesaria que no aporta ninguna información al diagrama y lo hace visualmente más ruidoso. Es esa elongación horizontal la que se ha denominado codo horizontal, aunque la notación es propia de la aplicación y no es un término estándar en el ámbito de los diagramas por capas.



(a) Ejemplo de codo horizontal innecesario



(b) Eliminación de codo horizontal.

Figura D.8: Resolución de codos horizontales

Estos codos pueden aparecer en tres situaciones concretas: cuando una hiperarista tiene exactamente un nodo fuente y un nodo destino; cuando tiene exactamente dos nodos en posición más a la izquierda y cuando tiene exactamente dos nodos en posición más a la derecha. En los tres casos, los dos puertos implicados son candidatos a ser alineados, pues representan el mismo extremo de la hiperarista y, por construcción de la política de ordenación de puertos, no deberían generar cruces si comparten abscisa.

La alineación solo se ejecuta si es posible efectuarla sin invadir el espacio de los puertos vecinos. Para ello, se calcula el span de movimiento de cada puerto, definido como el intervalo horizontal en el

que puede desplazarse sin colisionar con su vecino inmediato (o con el borde del nodo si es el puerto más exterior). En el caso de los nodos *dummy*, dado que mover sus puertos generaría uno de estos codos en sí mismo, se considera que su span de movimiento es un punto fijo, que es la abcisa del propio puerto.

Sea como fuere, si la intersección de los dos spans no es vacía, ambos puertos se desplazan al punto medio de dicha intersección; en caso contrario, no se realiza ningún movimiento. Nótese que, en caso de que uno de los nodos sea *dummy*, el punto medio será el propio punto fijo del nodo *dummy*, por lo que el puerto del nodo real se desplazará hasta él y el otro quedará anclado, lo que garantiza la alineación sin generar codos. Una vez fijados los puertos alineados, los puertos libres restantes del nodo se redistribuyen de forma simétrica en el espacio que queda entre los puertos anclados y los bordes del nodo. Esto queda reflejado en el Algoritmo D.24.

D.4.1. Detección y resolución de conflictos de solapamiento

Como se mencionó en el texto principal, la asignación de puertos evita los solapamientos entre segmentos verticales que parten o llegan al *mismo* nodo, pero no necesariamente entre segmentos de *nodos distintos*. Un solapamiento de este tipo ocurre cuando el puerto fuente (asociado a una hiperarista e_1) de un nodo de la capa superior y el puerto destino (asociado a una hiperarista e_2) de un nodo de la capa inferior están muy próximos en abcisa y, si e_1 está por debajo de e_2 , entonces los segmentos verticales que emanan de esos puertos se solapan.

La detección y resolución de estos conflictos se realiza en dos fases.

Detección de conflictos

La primera fase consiste en marcar posibles conflictos entre nodos de la capa superior y nodos de la capa inferior. Estos posibles conflictos se dan cuando los puertos de un nodo están muy cerca de los puertos de otro nodo, lo que podría resultar en un posible solapamiento de segmentos verticales. Es importante recalcar que, al final, la última palabra la tiene la ordenación respectiva de la hiperaristas asociadas y que en esta fase solo se marcan como conflictivos aquellos pares de nodos que cumplen la condición necesaria (no suficiente). Este algoritmo de detección se refleja en D.25.

Resolución de conflictos

Para cada par potencialmente conflictivo detectado, se identifican los rangos exactos de puertos que se solapan mediante un segundo barrido sobre las listas de puertos de ambos nodos. En este segundo barrido se identifican conflictos reales que han de ser solventados. La idea para resolverlos será desplazar ligeramente la abcisa de los puertos asociados para que queden a una distancia mayor a la que tienen actualmente. Evidentemente, este proceso es una mitigación del problema y no una

Input: Para cada gap $(\ell, \ell + 1)$, las hiperaristas salientes de la capa ℓ con sus puertos fuente y destino ya asignados simétricamente.

Output: Puertos fuente y destino alineados donde sea posible; los puertos libres redistribuidos simétricamente en el espacio restante.

```

1  ancladosSrc  $\leftarrow \{\}$   ancladosTgt  $\leftarrow \{\}$ ;          // puertos que han sido alineados
2  foreach hiperarista e en el gap do
    // Identificar el par (puerto fuente, puerto destino)
    candidato a alineación
3  if  $|sources(e)| = 1$  and  $|targets(e)| = 1$  then
4  |    $(s, t) \leftarrow (source(e), target(e));$ 
5  else if  $|leftmost(e)| = 2$ , uno fuente y otro destino then
6  |    $(s, t) \leftarrow$  el fuente y el destino más a la izquierda;
7  else if  $|rightmost(e)| = 2$ , uno fuente y otro destino then
8  |    $(s, t) \leftarrow$  el fuente y el destino más a la derecha;
9  else
10 |   continue
11 end

    // Calcular el span de movimiento de cada puerto
12  $span_s \leftarrow$  [vecino izquierdo de  $p_s$ , vecino derecho de  $p_s$ ];
13  $span_t \leftarrow$  [vecino izquierdo de  $p_t$ , vecino derecho de  $p_t$ ];
14  $inter \leftarrow span_s \cap span_t$ ;
15 if  $inter \neq \emptyset$  then
16 |    $x_{new} \leftarrow midpoint(inter);$ 
17 |    $p_s.x \leftarrow x_{new}$    $p_t.x \leftarrow x_{new}$ ;
18 |    $ancladosSrc[s] \leftarrow ancladosSrc[s] \cup \{p_s\};$ 
19 |    $ancladosTgt[t] \leftarrow ancladosTgt[t] \cup \{p_t\};$ 
20 end
21 end

    // Redistribuir simétricamente los puertos no anclados
22 foreach nodo v con puertos anclados en ancladosSrc do
23 |    $redistributePorts(sourcePorts(v), ancladosSrc[v]);$ 
24 end
25 foreach nodo v con puertos anclados en ancladosTgt do
26 |    $redistributePorts(targetPorts(v), ancladosTgt[v]);$ 
27 end

```

Algoritmo D.24: Reducción de codos horizontales.

Input: Nodos de la capa superior ordenados por x creciente, nodos de la capa inferior ordenados por x creciente y separación mínima entre puertos $\delta_{\min}$.

Output: Lista de pares (u, v) con u en la capa superior y v en la capa inferior que constituyen conflictos potenciales.

```

1  conflicts ← []  i ← 0  j ← 0;
2  while i < |upper| and j < |lower| do
3      u ← upper[i]  v ← lower[j];
4      if x(u) < x(v) then
5          // u está a la izquierda de v: comprobar si el puerto
6             más derecho de u y el más izquierdo de v están
7             demasiado cerca
8          if pright(u) − pleft(v) < δmín then
9              conflicts ← conflicts || [(u, v)];
10             end
11             i ← i + 1;
12         else if x(u) > x(v) then
13             // u está a la derecha de v: comprobar el puerto más
14                 izquierdo de u y el más derecho de v
15             if pleft(u) − pright(v) < δmín then
16                 conflicts ← conflicts || [(u, v)];
17             end
18             j ← j + 1;
19         else
20             // Misma abscisa: conflicto seguro si ambos tienen
21                 puertos
22             if sourcePorts(u) ≠ ∅ and targetPorts(v) ≠ ∅ then
23                 conflicts ← conflicts || [(u, v)];
24             end
25             // Avanzar el nodo más estrecho (o ambos si son
26                 iguales)
27             if width(u) < width(v) then
28                 i ← i + 1
29             else
30                 if width(u) > width(v) then
31                     j ← j + 1
32                 else
33                     i ← i + 1  j ← j + 1
34                 end
35             end
36         end
37     end
38 end
39 return conflicts;
```

Algoritmo D.25: Detección de posibles conflictos entre segmentos verticales

solución, pero para el tipo de diagramas que se manejan en la aplicación, es más que suficiente para eliminar la mayoría de los solapamientos sin introducir demasiada distorsión en la disposición.

Una vez identificados los rangos conflictivos, la resolución depende de si los nodos implicados son reales o *dummy*. Los nodos *dummy* tienen siempre exactamente un puerto, que no puede desplazarse libremente sin introducir codos en la cadena de segmentos que representan. Por ello, se trata su puerto como un punto fijo y se mueven los puertos del nodo real de forma que queden a una distancia suficiente usando un desplazamiento baricéntrico: el puerto se mueve un tercio de la distancia disponible hacia el vecino más próximo (o hasta el borde del nodo si no hay vecino), de forma proporcional al espacio disponible.

Si diera la casualidad de que ambos nodos fueran *dummy*, lo cual es sumamente improbable, entonces no quedaría otra que desplazar uno de los puertos el mínimo necesario para alcanzar la separación mínima. Dada la configuración específica de constantes de separación entre nodos y anchura de los mismos, este desplazamiento puede realizarse sin generar ningún otro conflicto. En este caso, se ha optado por desplazar el puerto del nodo superior, porque el puerto del nodo inferior podría ser el inicio de una cadena de segmentos interiores que han de alinearse verticalmente, lo que haría que el desplazamiento de este nodo inferior generara un codo horizontal en la hiperarista larga que representa.

Cuando ninguno de los dos nodos es *dummy*, la idea es fusionar los puertos en el rango conflictivo de ambos nodos, esto es, hacer que el nodo inferior cuente también con los puertos del nodo superior que están en conflicto, y viceversa. El orden de esta lista fusionada se establece por la posición horizontal de los puertos y, en caso de empate, ateniendo a la ordenación que menor número de cruces genere. Una vez ordenada esta lista, los puertos se redistribuyen de forma equidistante en el intervalo $[x_{\text{mín}}, x_{\text{máx}}]$, calculado también mediante la regla del tercio respecto a los puertos vecinos.

Input: Nodo superior u con sus puertos fuente ordenados por x , nodo inferior v con sus puertos destino ordenados por x y separación mínima $\delta_{\min}$.

Output: Puertos de u y v reubicados de forma que no haya solapamientos entre sus segmentos verticales.

// Identificar los rangos de puertos conflictivos

```

1  [ $i_{\min}, i_{\max}$ ]  $\leftarrow \emptyset$   [ $j_{\min}, j_{\max}$ ]  $\leftarrow \emptyset$ ;
2   $i \leftarrow 0$    $j \leftarrow 0$ ;
3  while  $i < |\text{sourcePorts}(u)|$  and  $j < |\text{targetPorts}(v)|$  do
4      if  $|p_u[i].x - p_v[j].x| < \delta_{\min}$  and  $\text{order}(p_u[i].\text{edge}) > \text{order}(p_v[j].\text{edge})$  then
5          [ $i_{\min}, i_{\max}$ ]  $\leftarrow [\min(i_{\min}, i), \max(i_{\max}, i)]$ ;
6          [ $j_{\min}, j_{\max}$ ]  $\leftarrow [\min(j_{\min}, j), \max(j_{\max}, j)]$ ;
7      end
8      if  $p_u[i].x < p_v[j].x$  then
9           $i \leftarrow i + 1$ 
10     else
11         if  $p_u[i].x > p_v[j].x$  then
12              $j \leftarrow j + 1$ 
13         else
14              $i \leftarrow i + 1$    $j \leftarrow j + 1$ 
15         end
16     end
17 end
18 if [ $i_{\min}, i_{\max}$ ] =  $\emptyset$  then
19     return ; // no hay conflicto real
20 end
21 if  $u$  es dummy and  $v$  es dummy then
22     // Ambos tienen exactamente un puerto: desplazar el de  $u$ 
23     la cantidad justa para alcanzar  $\delta_{\min}$ 
24     ajustarDummy( $u, v, \delta_{\min}$ );
25 else if  $u$  es dummy then
26     // Puerto de  $u$  fijo: desplazar el puerto conflictivo de  $v$ 
27     un tercio de la distancia disponible hacia su vecino
28     shiftWithFixedPort( $p_u[0], v, p_v[j_{\min}], \dots, p_v[j_{\max}], \delta_{\min}$ );
29 else if  $v$  es dummy then
30     // Simétrico: puerto de  $v$  fijo, desplazar los de  $u$ 
31     shiftWithFixedPort( $p_v[0], u, p_u[i_{\min}], \dots, p_u[i_{\max}], \delta_{\min}$ );
32 else
33      $merged \leftarrow \text{merge}(p_u[i_{\min} \dots i_{\max}], p_v[j_{\min} \dots j_{\max}])$ ;
34      $x_{\min} \leftarrow \text{leftBound}(u, v, merged, \delta_{\min})$ ;
35      $x_{\max} \leftarrow \text{rightBound}(u, v, merged, \delta_{\min})$ ;
36     redistribute( $merged, x_{\min}, x_{\max}$ );
37 end
    
```

Algoritmo D.26: Resolución de conflictos entre puertos

Por último, La regla del tercio ha sido utilizada para mantener una relación de aspecto 2:1 entre el espacio disponible y el espacio ocupado por los puertos, lo que reduce el contraste visual de tener puertos muy juntos o muy separados. Además, esta relación de aspecto es la que mejor se adapta en el caso más frecuente de conflicto, que es únicamente involucrando a dos puertos.

Si llamamos p_1 al puerto que después de la resolución quede más a la izquierda y p_2 al que quede más a la derecha, entonces la distancia entre p_1 y p_2 será un tercio de la distancia entre el vecino izquierdo de p_1 y p_1 más otro tercio de la distancia entre p_2 y su vecino derecho. Si ambas distancias son iguales, después de la actualización, la distancia del vecino de p_1 a p_1 , la distancia entre p_1 y p_2 y la distancia entre p_2 y su vecino derecho serán iguales, lo que da lugar a una distribución equidistante de los puertos en el espacio disponible. Esto se refleja en la siguiente figura:

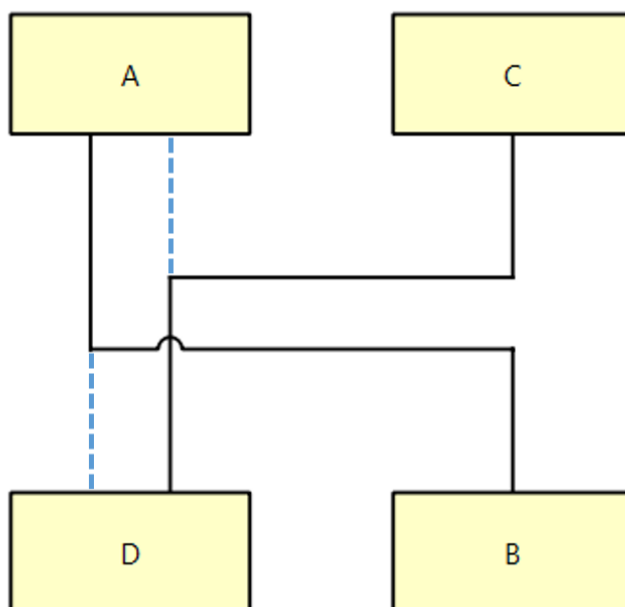


Figura D.9: Elección de proporción baricéntrica

RESULTADOS Y VALIDACIÓN

E.1. Desglose de pruebas unitarias

Como ya se introdujo en la sección 7.1, en esta aplicación, el proceso de validación mediante pruebas unitarias ha sido exhaustivo y se ha organizado en dos módulos independientes que se corresponden con las capas de la arquitectura: la capa de dominio (`hypergraph_logic`) y la capa de aplicación (`editors`). La capa de interfaz de usuario (`ui`) no se ha sometido a pruebas unitarias, sino que se ha comprobado su correcto funcionamiento mediante pruebas de integración y validación visual. En esta sección, se ofrece el desglose detallado de las pruebas unitarias ejecutadas en cada módulo.

La Tabla E.1 recoge el detalle por clase para la capa de dominio y la Tabla E.2 para la capa de aplicación.

Fichero de test	Clase / módulo probado	Nº tests
<code>test_node.cpp</code>	Node	30
<code>test_hyperedge.cpp</code>	Hyperedge	35
<code>test_hypergraph.cpp</code>	Hypergraph	99
<code>test_minimize_crossings.cpp</code>	<i>Global Sifting</i> (unitario)	118
<code>test_horizontal_coordinates.cpp</code>	Brandes-Köpf	44
<code>test_horizontal_order.cpp</code>	Ordenación de hiperaristas	33
<code>test_port_assignment.cpp</code>	Asignación de puertos	35
<code>test_graphicalhypergraph.cpp</code>	GraphicalHypergraph	33
<code>test_jointgraphicalhypergraph.cpp</code>	JointGraphicalHypergraph	4
Total		431

Tabla E.1: Pruebas unitarias de la capa de dominio (`hypergraph_logic`).

Fichero de test	Clase / módulo probado	Nº tests
test_editors.cpp	HypergraphEditor / JointHypergraphEditor	76
test_project.cpp	Project	41
Total		117

Tabla E.2: Pruebas unitarias de la capa de aplicación (editors).

E.2. Pruebas de validación algorítmica y rendimiento

Global Sifting

En la sección 7.2, se presentaron resultados reducidos del análisis de la implementación de minimización de cruces. En esta sección, se ofrece un desglose más detallado de los resultados obtenidos, incluyendo tiempos de ejecución y evolución del número de cruces en cada instancia.

Los datos mostrados en la gráfica 7.1 se han obtenido a partir de la tabla E.3, que se muestra a continuación:

Tabla E.3: Análisis detallado de la optimización de cruces (Todas las instancias)

Instancia	Cruces antes	Cruces después	Reducción (%)	Tiempo (ms)
00-crossing_minimization-9-0	112	37	66.96 %	122.37
01-crossing_minimization-9-0	103	26	74.76 %	139.71
02-crossing_minimization-9-0	86	28	67.44 %	155.79
03-crossing_minimization-9-0	81	37	54.32 %	97.55
04-crossing_minimization-9-0	112	27	75.89 %	87.52
05-crossing_minimization-9-0	98	33	66.33 %	82.70
06-crossing_minimization-9-0	85	32	62.35 %	103.76
07-crossing_minimization-9-0	98	30	69.39 %	93.52
08-crossing_minimization-9-0	110	25	77.27 %	106.01
09-crossing_minimization-9-0	97	23	76.29 %	103.52
10-crossing_minimization-10-0	166	48	71.08 %	162.20
11-crossing_minimization-10-0	128	34	73.44 %	202.11
12-crossing_minimization-10-0	168	49	70.83 %	184.91
13-crossing_minimization-10-0	142	62	56.34 %	143.62
14-crossing_minimization-10-0	142	50	64.79 %	155.43
15-crossing_minimization-10-0	186	51	72.58 %	223.39
16-crossing_minimization-10-0	170	54	68.24 %	161.17
17-crossing_minimization-10-0	175	29	83.43 %	238.59
18-crossing_minimization-10-0	184	48	73.91 %	217.62

Tabla E.3: Análisis detallado de la optimización de cruces (continuación)

Nombre Instancia	Cruces antes	Cruces después	Reducción (%)	Tiempo (ms)
19-crossing_minimization-10-0	141	30	78.72 %	241.88
20-crossing_minimization-8-0	109	26	76.15 %	689.43
21-crossing_minimization-8-0	87	24	72.41 %	267.99
22-crossing_minimization-8-0	91	44	51.65 %	388.79
23-crossing_minimization-8-0	98	21	78.57 %	94.54
24-crossing_minimization-8-0	79	12	84.81 %	257.60
25-crossing_minimization-8-0	73	30	58.90 %	222.13
26-crossing_minimization-8-0	96	31	67.71 %	141.38
27-crossing_minimization-8-0	92	27	70.65 %	118.41
28-crossing_minimization-8-0	81	17	79.01 %	150.67
29-crossing_minimization-8-0	88	31	64.77 %	140.72
30-crossing_minimization-9-0	186	36	80.65 %	209.59
31-crossing_minimization-9-0	153	38	75.16 %	237.78
32-crossing_minimization-9-0	160	44	72.50 %	141.27
33-crossing_minimization-9-0	153	49	67.97 %	185.34
34-crossing_minimization-9-0	153	45	70.59 %	177.89
35-crossing_minimization-9-0	129	57	55.81 %	236.89
36-crossing_minimization-9-0	142	38	73.24 %	248.06
37-crossing_minimization-9-0	156	53	66.03 %	185.64
38-crossing_minimization-9-0	137	39	71.53 %	246.48
39-crossing_minimization-9-0	173	39	77.46 %	975.55
40-crossing_minimization-10-0	249	76	69.48 %	332.56
41-crossing_minimization-10-0	295	64	78.31 %	334.90
42-crossing_minimization-10-0	192	74	61.46 %	202.59
43-crossing_minimization-10-0	270	66	75.56 %	216.14
44-crossing_minimization-10-0	219	95	56.62 %	218.97
45-crossing_minimization-10-0	271	78	71.22 %	218.85
46-crossing_minimization-10-0	249	73	70.68 %	204.74
47-crossing_minimization-10-0	214	102	52.34 %	176.67
48-crossing_minimization-10-0	253	71	71.94 %	216.76
49-crossing_minimization-10-0	237	89	62.45 %	197.19
50-crossing_minimization-8-0	145	30	79.31 %	180.76
51-crossing_minimization-8-0	129	19	85.27 %	305.68
52-crossing_minimization-8-0	153	36	76.47 %	400.91

Tabla E.3: Análisis detallado de la optimización de cruces (continuación)

Nombre Instancia	Cruces antes	Cruces después	Reducción (%)	Tiempo (ms)
53-crossing_minimization-8-0	128	40	68.75 %	212.73
54-crossing_minimization-8-0	131	21	83.97 %	501.05
55-crossing_minimization-8-0	132	38	71.21 %	637.70
56-crossing_minimization-8-0	121	22	81.82 %	336.41
57-crossing_minimization-8-0	115	30	73.91 %	510.49
58-crossing_minimization-8-0	109	44	59.63 %	693.14
59-crossing_minimization-8-0	144	33	77.08 %	389.45
60-crossing_minimization-9-0	219	78	64.38 %	561.24
61-crossing_minimization-9-0	193	81	58.03 %	229.23
62-crossing_minimization-9-0	213	53	75.12 %	351.90
63-crossing_minimization-9-0	208	39	81.25 %	805.67
64-crossing_minimization-9-0	194	60	69.07 %	509.95
65-crossing_minimization-9-0	184	61	66.85 %	511.47
66-crossing_minimization-9-0	187	77	58.82 %	490.25
67-crossing_minimization-9-0	194	88	54.64 %	485.17
68-crossing_minimization-9-0	201	68	66.17 %	372.29
69-crossing_minimization-9-0	188	48	74.47 %	396.21
70-crossing_minimization-8-0	139	61	56.12 %	285.84
71-crossing_minimization-8-0	152	40	73.68 %	275.06
72-crossing_minimization-8-0	150	49	67.33 %	451.81
73-crossing_minimization-8-0	141	33	76.60 %	524.17
74-crossing_minimization-8-0	168	54	67.86 %	546.48
75-crossing_minimization-8-0	163	36	77.91 %	560.41
76-crossing_minimization-8-0	187	58	68.98 %	362.26
77-crossing_minimization-8-0	152	43	71.71 %	290.54
78-crossing_minimization-8-0	155	45	70.97 %	517.75
79-crossing_minimization-8-0	135	40	70.37 %	457.27
80-crossing_minimization-9-0	250	99	60.40 %	471.61
81-crossing_minimization-9-0	239	79	66.95 %	659.74
82-crossing_minimization-9-0	249	81	67.47 %	734.36
83-crossing_minimization-9-0	241	95	60.58 %	667.51
84-crossing_minimization-9-0	229	71	69.00 %	780.75
85-crossing_minimization-9-0	211	93	55.92 %	698.34
86-crossing_minimization-9-0	248	76	69.35 %	1345.51

Tabla E.3: Análisis detallado de la optimización de cruces (continuación)

Nombre Instancia	Cruces antes	Cruces después	Reducción (%)	Tiempo (ms)
87-crossing_minimization-9-0	281	92	67.26 %	1001.44
88-crossing_minimization-9-0	275	91	66.91 %	402.03
89-crossing_minimization-9-0	254	70	72.44 %	893.41
90-crossing_minimization-8-0	147	57	61.22 %	1117.21
91-crossing_minimization-8-0	207	55	73.43 %	587.42
92-crossing_minimization-8-0	215	57	73.49 %	1334.04
93-crossing_minimization-8-0	197	80	59.39 %	875.36
94-crossing_minimization-8-0	163	32	80.37 %	399.94
95-crossing_minimization-8-0	165	51	69.09 %	579.45
96-crossing_minimization-8-0	181	47	74.03 %	518.48
97-crossing_minimization-8-0	190	81	57.37 %	362.04
98-crossing_minimization-8-0	196	57	70.92 %	307.21
99-crossing_minimization-8-0	183	47	74.32 %	560.84

Realizando los cálculos pertinentes sobre los datos de la Tabla E.3, se obtiene que la media del porcentaje de reducción de cruces es del 69.77 % y el tiempo medio de ejecución es de 381.08 ms por instancia. Estos resultados confirman que la implementación de minimización de cruces es efectiva en la mayoría de los casos, logrando reducciones significativas en el número de cruces con tiempos de ejecución razonables para su aplicación en un entorno interactivo.

Además, cabe destacar dos apuntes importantes: el algoritmo de minimización no se ejecuta todo el rato y, en segundo lugar, las instancias de prueba no representan casos reales, sino que están diseñadas para poner a prueba la robustez del algoritmo en situaciones extremas. Por lo tanto, aunque el rendimiento es aceptable en estas instancias, se espera que en casos reales el número de cruces y nodos sea considerablemente menor y, por ende, el tiempo de ejecución también lo sea.

Para ilustrar esta afirmación, nos hemos tomado la molestia de representar cada uno de los DAGs a los que se ha sometido el algoritmo de minimización de cruces, tanto antes como después de su ejecución. La Figura E.1 muestra la visualización de una instancia aleatoriamente seleccionada.

Ordenación de hiperaristas

De manera análoga a la sección anterior, en esta sección se ofrece un desglose más detallado de los resultados obtenidos en la implementación de ordenación de hiperaristas. Además, ofreceremos el pseudocódigo de la función que genera los hipergrafos aleatorios bicapa sobre los que se han realizado las pruebas de rendimiento y validación algorítmica.

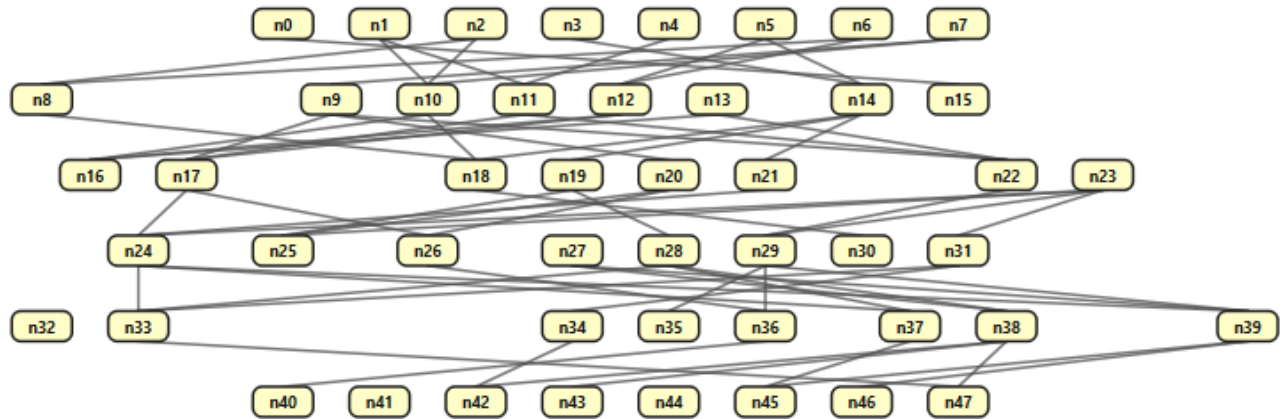


Figura E.1: Visualización de instancia aleatoria de prueba de Global Sifting.

Lo primero de todo, los datos que se exponen en la gráfica 7.3 se han obtenido a partir de la tabla E.4, que se muestra a continuación. Aunque ya se ha comentado en el texto principal, el formato de nombre de cada instancia se corresponde con `hypergraph_n,m`, donde n es el número de nodos en las capas superior e inferior y m el número de hiperaristas.

Tabla E.4: Comparativa de resultados en ordenación de hiperaristas: Implementación MIP vs. Fuerza Bruta (BF)

Instancia	MIP (Cruces)	BF (Cruces)	MIP (ms)	BF (ms)
hypergraph_3, 2	0	0	2.22	0.04
hypergraph_3, 3	1	0	1.46	0.15
hypergraph_3, 4	1	0	1.62	0.96
hypergraph_3, 5	3	1	3.29	7.73
hypergraph_3, 6	3	3	4.54	64.97
hypergraph_3, 7	5	2	5.55	611.16
hypergraph_3, 8	6	3	6.99	6387.59
hypergraph_3, 9	4	2	8.67	100298.13
hypergraph_4, 2	0	0	3.68	0.03
hypergraph_4, 3	0	0	4.46	0.17
hypergraph_4, 4	2	2	1.84	0.94
hypergraph_4, 5	5	3	4.64	7.93
hypergraph_4, 6	11	8	4.12	71.21
hypergraph_4, 7	17	14	6.20	676.85
hypergraph_4, 8	22	22	7.52	6856.37
hypergraph_4, 9	21	12	8.70	76569.82
hypergraph_5, 2	1	0	1.62	0.03

Tabla E.4: Comparativa de resultados (continuación)

Instancia	MIP (Cruces)	BF (Cruces)	MIP (ms)	BF (ms)
hypergraph_5, 3	1	1	1.27	0.15
hypergraph_5, 4	4	4	2.02	1.00
hypergraph_5, 5	8	8	2.31	7.95
hypergraph_5, 6	11	6	5.00	67.27
hypergraph_5, 7	8	8	5.38	643.66
hypergraph_5, 8	16	15	7.33	7194.03
hypergraph_5, 9	21	20	12.75	78432.83
hypergraph_6, 2	1	1	1.95	0.04
hypergraph_6, 3	2	2	3.52	0.17
hypergraph_6, 4	6	5	1.65	1.12
hypergraph_6, 5	8	7	4.27	8.61
hypergraph_6, 6	14	13	4.86	75.60
hypergraph_6, 7	19	17	6.36	724.10
hypergraph_6, 8	20	17	6.69	7338.14
hypergraph_6, 9	29	25	8.32	91586.04
hypergraph_7, 2	1	1	3.32	0.03
hypergraph_7, 3	5	4	2.12	0.21
hypergraph_7, 4	6	5	3.46	1.46
hypergraph_7, 5	6	5	6.23	10.28
hypergraph_7, 6	10	10	7.20	94.50
hypergraph_7, 7	16	15	8.55	1051.68
hypergraph_7, 8	20	19	15.67	10041.26
hypergraph_7, 9	28	25	16.37	115603.57
hypergraph_8, 2	0	0	4.96	0.03
hypergraph_8, 3	5	5	2.19	0.19
hypergraph_8, 4	7	7	6.20	1.24
hypergraph_8, 5	14	14	3.28	10.05
hypergraph_8, 6	25	25	3.62	95.06
hypergraph_8, 7	30	30	9.67	863.15
hypergraph_8, 8	36	36	9.64	8779.66
hypergraph_8, 9	42	39	10.06	102195.66
hypergraph_9, 2	1	1	2.24	0.05
hypergraph_9, 3	6	6	2.03	0.20
hypergraph_9, 4	11	10	2.36	1.28

Tabla E.4: Comparativa de resultados (continuación)

Instancia	MIP (Cruces)	BF (Cruces)	MIP (ms)	BF (ms)
hypergraph_9, 5	16	16	4.37	9.97
hypergraph_9, 6	26	25	5.38	93.18
hypergraph_9, 7	45	45	8.00	912.89
hypergraph_9, 8	53	53	8.22	9397.46
hypergraph_9, 9	65	64	10.50	109720.71
hypergraph_10, 2	0	0	1.90	0.03
hypergraph_10, 3	1	1	1.71	0.18
hypergraph_10, 4	5	5	3.94	1.19
hypergraph_10, 5	14	12	3.81	11.15
hypergraph_10, 6	23	21	17.14	88.42
hypergraph_10, 7	29	26	6.17	879.82
hypergraph_10, 8	45	42	10.16	8896.46
hypergraph_10, 9	47	44	9.49	100075.23

De nuevo, realizando los cálculos pertinentes sobre los datos de la Tabla E.4, se obtiene que el tiempo medio de ejecución de la implementación MIP es de 5.64 ms por instancia, mientras que el tiempo medio de ejecución de la fuerza bruta es de 13226.11 ms por instancia. Además, la diferencia entre la solución óptima y el MIP es, en promedio, de 1.19 cruces. Esto viene a confirmar que la implementación MIP no solo es altamente eficiente en términos de tiempo de ejecución, sino que también ofrece soluciones de alta calidad, prácticamente óptimas, en tiempos ridículamente bajos.

Por todo ello, en un paso tan determinante para la legibilidad del diagrama, optar por una implementación MIP es claramente la mejor opción.

Por último, a continuación se muestra el pseudocódigo de la función que genera los hipergrafos aleatorios bicapa sobre los que se han realizado las pruebas de rendimiento y validación algorítmica.

Input: Número de nodos de la capa superior n_u e inferior n_l , número de hiperaristas m , rangos de cardinalidad para fuentes $[src_{\min}, src_{\max}]$ y destinos $[tgt_{\min}, tgt_{\max}]$.

Output: Hipergrafo bicapa aleatorio H con exactamente m hiperaristas únicas y sin conexiones redundantes entre pares (fuente, destino).

```

1  Crear  $n_u$  nodos en la capa 0:  $U = [u_0, \dots, u_{n_u-1}]$ ;
2  Crear  $n_l$  nodos en la capa 1:  $L = [l_0, \dots, l_{n_l-1}]$ ;

3   $connected[u] \leftarrow \emptyset \quad \forall u \in U$ ;           // pares (fuente, destino) ya usados
4   $used \leftarrow \emptyset$ ;                         // claves de aristas ya creadas
5   $created \leftarrow 0$ ;

6  while  $created < m$  do
7       $n_s \leftarrow \text{randInt}(src_{\min}, src_{\max})$ ;
8       $S \leftarrow \text{chooseRandomly}(U, n_s)$ ;
9       $forbidden \leftarrow \bigcup_{u \in S} connected[u]$ ;
10      $avail \leftarrow L \setminus forbidden$ ;
11     if  $|avail| < tgt_{\min}$  then
12         continue;           // no hay suficientes destinos disponibles
13     end
14      $n_t \leftarrow \text{randInt}(tgt_{\min}, \min(tgt_{\max}, |avail|))$ ;
15      $T \leftarrow \text{chooseRandomly}(avail, n_t)$ ;
16     if  $(\text{sort}(S), \text{sort}(T)) \in used$  then
17         continue
18     end
19      $used \leftarrow used \cup \{k\}$ ;

    // Registrar las nuevas conexiones y crear la hiperarista
20     foreach  $u \in S, v \in T$  do
21          $connected[u] \leftarrow connected[u] \cup \{v\}$ ;
22     end
23      $e \leftarrow \text{addConnection}(S[0], T[0])$ ;
24     foreach  $u \in S \setminus \{S[0]\}$  do
25          $\text{addSourceToEdge}(e, u)$ 
26     end
27     foreach  $v \in T \setminus \{T[0]\}$  do
28          $\text{addTargetToEdge}(e, v)$ 
29     end
30      $created \leftarrow created + 1$ ;
31 end
32 return  $H$ ;

```

Algoritmo E.1: Generación aleatoria de hipergrafo bicapa sin conexiones redundantes.

E.3. Encuesta de usabilidad

Esta sección presenta los resultados y enunciados de la encuesta de usabilidad realizada a los potenciales usuarios de la aplicación. Como ya se ha detallado en la sección 7.3, se ha utilizado el cuestionario *System Usability Scale* (SUS) en su versión validada al castellano, complementado con preguntas adicionales específicas sobre la funcionalidad de la aplicación y preguntas abiertas para recoger impresiones cualitativas. En total, se recogieron respuestas de ocho participantes, algunos de ellos relacionados con el ámbito de la arqueología y otros relacionados con el ámbito del desarrollo de software, con el objetivo de obtener una perspectiva diversa sobre la usabilidad de la aplicación.

Bloque 1: Ejecución de tareas

El enunciado de la tarea guiada es el siguiente:

Como arqueólogo, se te ha pedido registrar las relaciones estratigráficas de cinco unidades descubiertas en una excavación. Las relaciones son las siguientes:

- *La capa A está por encima de las capas B y C.*
- *La capa B está por encima de la capa D.*
- *Las capas B y C comparten la misma relación con la capa D.*
- *La capa D está por encima de la capa E.*

Pasos a realizar:

- 1. Crea un nuevo diagrama en la aplicación.*
- 2. Añade cinco nodos y asígnale las etiquetas A, B, C, D y E.*
- 3. Establece las conexiones entre nodos según las relaciones descritas. Ten en cuenta que B y C comparten la misma arista hacia D.*
- 4. Renombra el nodo A como Capa superficial.*
- 5. Elimina la conexión directa entre B y D y comprueba que el diagrama se actualiza correctamente.*
- 6. Deshaz la eliminación anterior con Ctrl+Z.*
- 7. Guarda el proyecto.*

Cuadro E.1: Enunciado de la tarea guiada proporcionada a los participantes.

Esta tarea prueba las funcionalidades básicas de creación, edición y gestión de nodos y conexiones, así como la función de deshacer y el guardado de proyectos. Sobre todo, la parte en la que se fuerza que las capas B y C compartan la misma hiperarista, evalúa que el usuario entiende la dinámica de conexiones, que entraña algo de complejidad.

Una vez completada, cada participante registró sus impresiones mediante una pregunta abierta. Las respuestas obtenidas se recogen en la Tabla E.5.

Tabla E.5: Impresiones de los participantes tras la ejecución de la tarea.

P	Respuesta
P1	—
P2	El programa es sencillo de entender e intuitivo. Al borrar la conexión directa entre B y D, el sistema reubica automáticamente al nodo dependiente en el nivel que le corresponde según la jerarquía del diagrama, lo cual es útil.
P3	Bastante intuitivo y sencillo de seguir; las funciones para ordenar el diagrama lo hacen bastante funcional.
P4	La aplicación me dejó buenas sensaciones en general. Me pareció bastante cómoda de usar y más intuitiva de lo que esperaba. Se hace bastante fácil trabajar con los nodos y las conexiones.
P5	Intuitivo y sencillo; responde rápido y ofrece muchas opciones para corregir errores, algo que resultó especialmente útil al establecer las conexiones entre los nodos.
P6	El manejo general es sencillo una vez comprendida la lógica de las asociaciones entre nodos. La interfaz, aunque sencilla, es bastante intuitiva y fácil de manejar.
P7	He tenido alguna dificultad para entender la lógica de funcionamiento, quizás porque me aproximo desde una perspectiva estratigráfica-sedimentaria y la herramienta está concebida como un diagrama de nodos y aristas. No obstante, tras familiarizarme, funciona correctamente. Considero que el tutorial debería incorporar explicaciones más detalladas del significado de las distintas posibilidades gráficas para perfiles arqueológicos.
P8	Buenas sensaciones. Todas las acciones se realizaban con pocos clics y de manera intuitiva.

De aquí, la opinión más relevante es la del participante P7, que toma una aproximación más arqueológica y señala que la herramienta, aunque funcional, podría beneficiarse de una explicación más detallada de su lógica de funcionamiento, especialmente en lo que respecta a la representación gráfica de perfiles arqueológicos.

Bloque 2: Cuestionario SUS

El cuestionario SUS adaptado, consta de diecisiete ítems con afirmaciones alternamente positivas y negativas evaluadas en una escala Likert de 1 (totalmente en desacuerdo) a 5 (totalmente de acuerdo). La puntuación global SUS se obtiene transformando las respuestas de forma que los ítems positivos contribuyen con $v_i - 1$ y los negativos con $5 - v_i$, sumando todos los valores registrados y escalando al rango $[0, 100]$.

En otro orden de cosas, este cuestionario se divide en dos partes diferenciadas: los doce primeros ítems corresponden a la escala SUS propiamente dicha, con preguntas de carácter general sobre la usabilidad de la aplicación; mientras que los cinco ítems adicionales se han diseñado específicamente para evaluar aspectos relacionados con la funcionalidad de la aplicación.

A continuación se presentan los resultados de los primeros doce ítems.

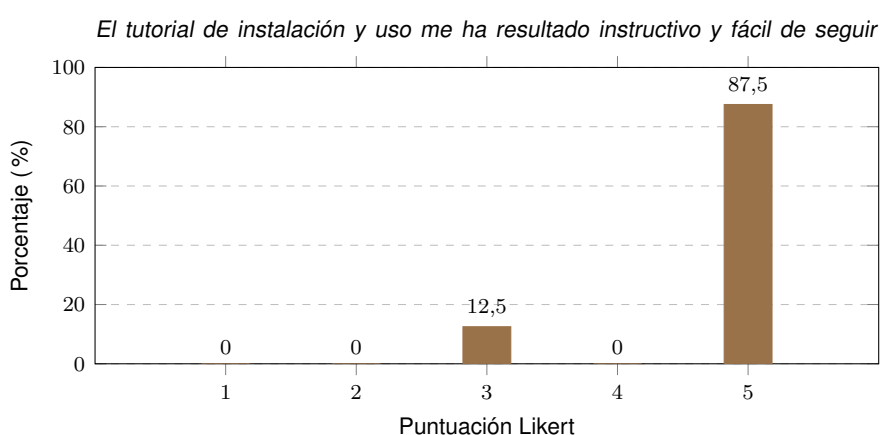


Figura E.2: SUS ítem 1: tutorial ($\bar{x} = 4,75$, $n = 8$).

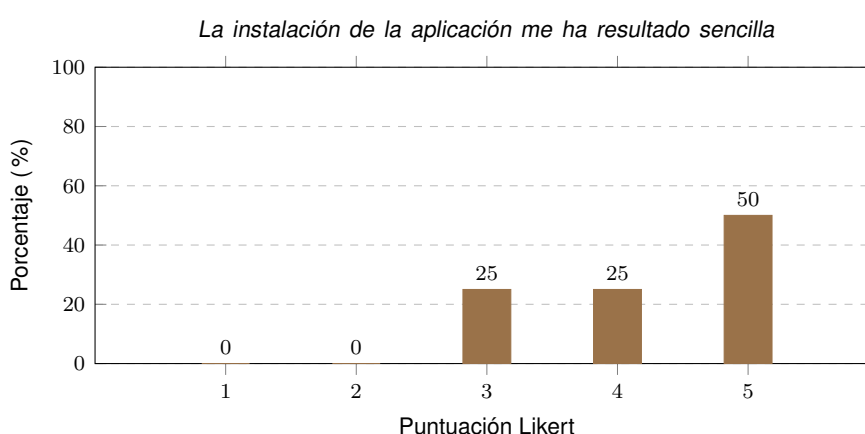


Figura E.3: SUS ítem 2: instalación ($\bar{x} = 4,25$, $n = 8$).

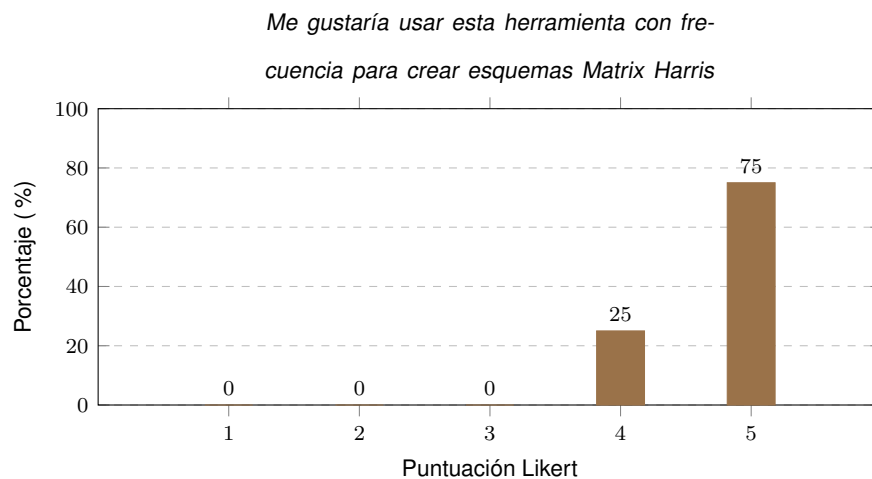


Figura E.4: SUS ítem 3: frecuencia de uso ($\bar{x} = 4,75$, $n = 8$).

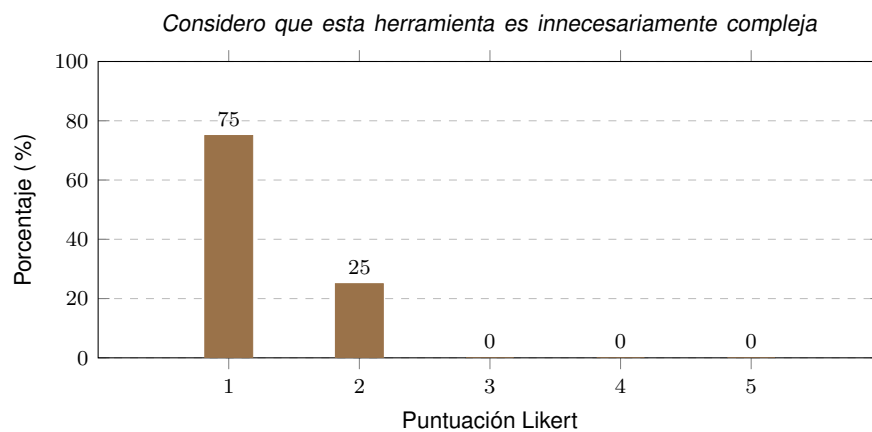


Figura E.5: SUS ítem 4 — ítem negativo: complejidad ($\bar{x} = 1,25$, $n = 8$).

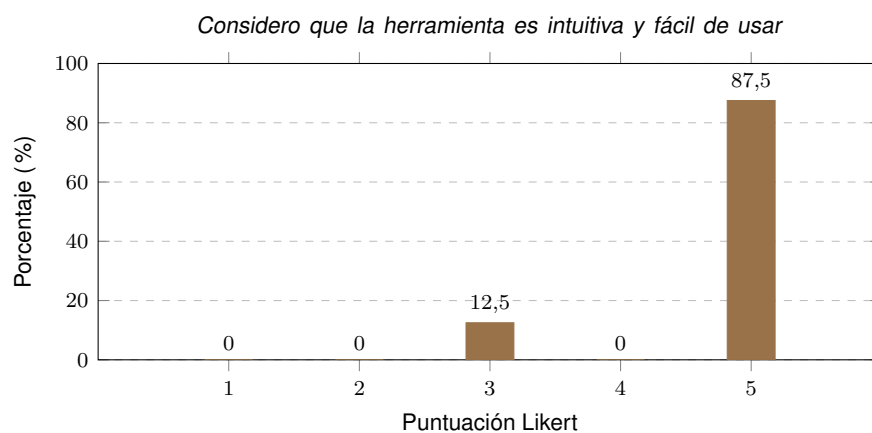


Figura E.6: SUS ítem 5: facilidad de uso ($\bar{x} = 4,75$, $n = 8$).

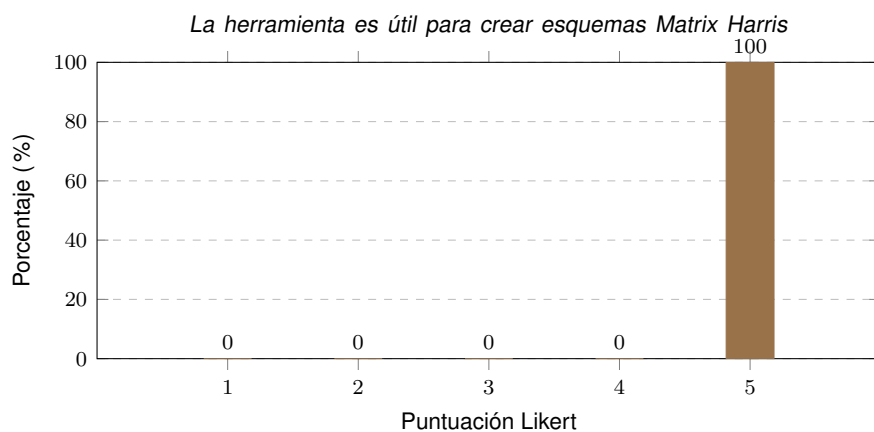


Figura E.7: SUS ítem 6: utilidad ($\bar{x} = 5,00$, $n = 8$).

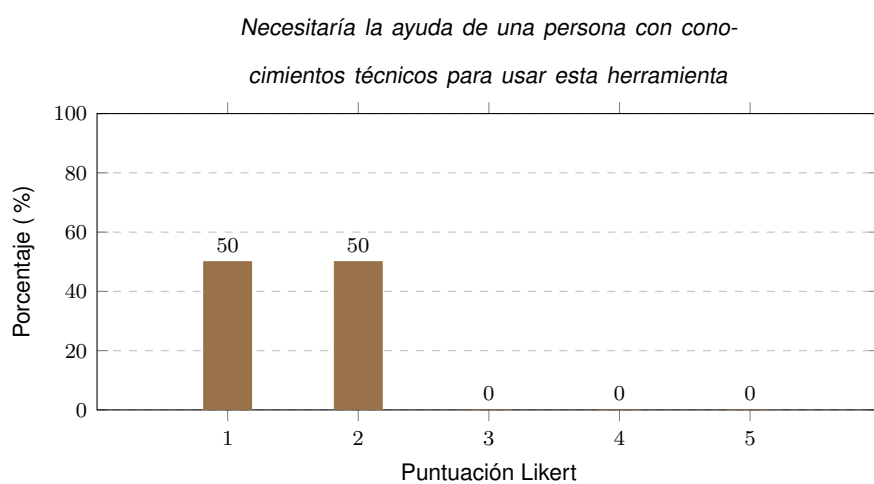


Figura E.8: SUS ítem 7 — ítem negativo: necesidad de ayuda técnica ($\bar{x} = 1,5$, $n = 8$).

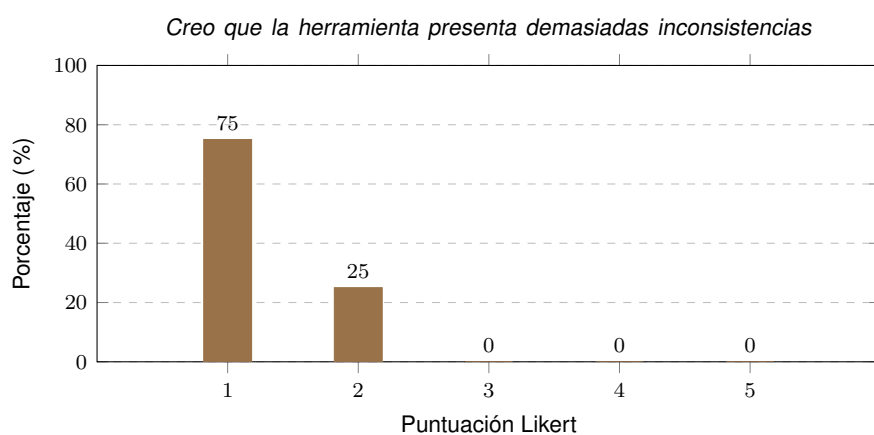


Figura E.9: SUS ítem 8 — ítem negativo: inconsistencias ($\bar{x} = 1,25$, $n = 8$).

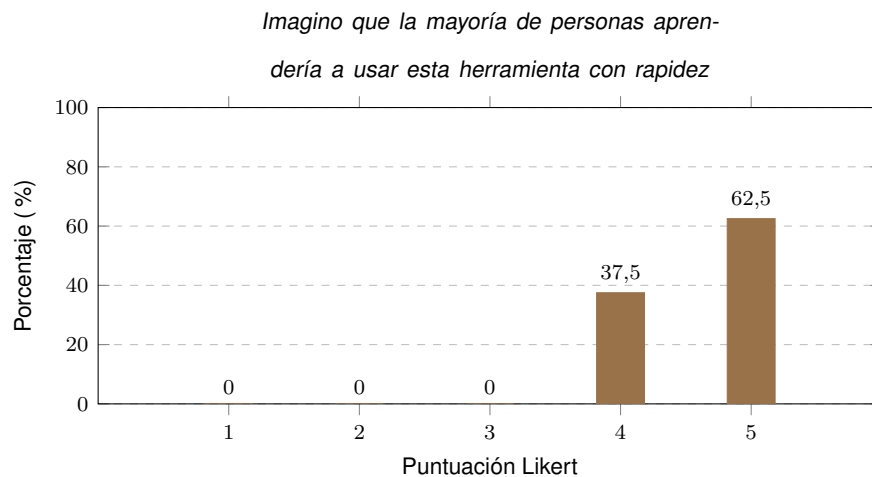


Figura E.10: SUS ítem 9: facilidad de aprendizaje ($\bar{x} = 4,625$, $n = 8$).

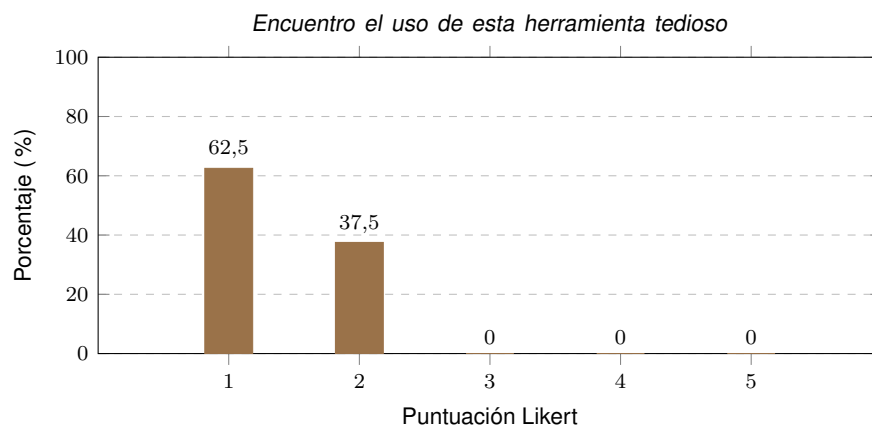


Figura E.11: SUS ítem 10 — ítem negativo: tedio ($\bar{x} = 1,375$, $n = 8$).

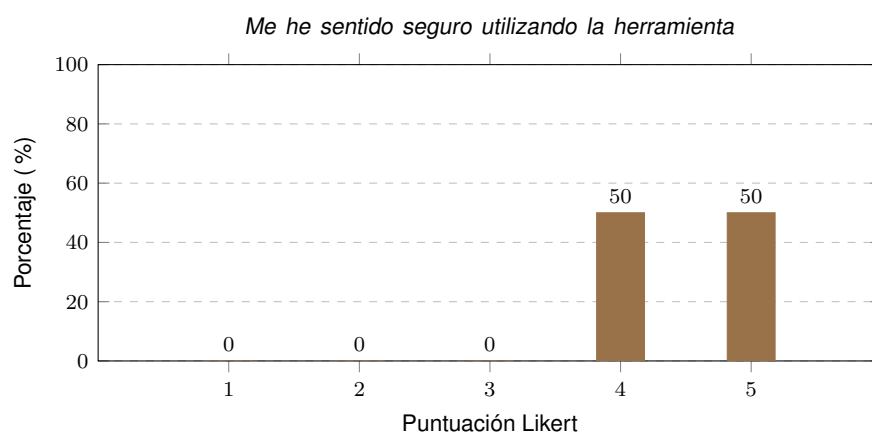


Figura E.12: SUS ítem 11: seguridad de uso ($\bar{x} = 4,50$, $n = 8$).

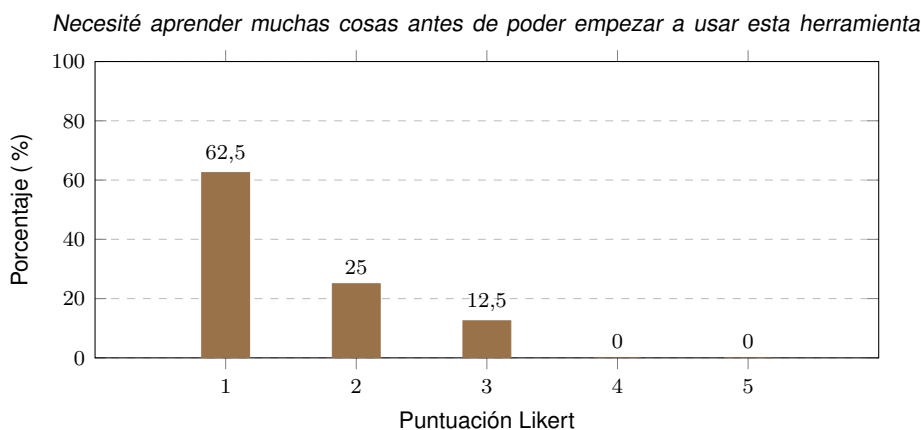


Figura E.13: SUS ítem 12 — ítem negativo: curva de aprendizaje ($\bar{x} = 1,50$, $n = 8$).

La puntuación SUS global media obtenida para esta primera parte es de **85,21 puntos**, lo que sitúa a la aplicación por encima del umbral de 68 considerado como referencia de usabilidad aceptable [8]. Los ítems negativos (4, 7, 8, 10 y 12) recibieron puntuaciones muy bajas ($\bar{x} \leq 1,50$), indicando que los participantes en general no percibieron complejidad innecesaria, inconsistencias, ni necesidad de conocimientos técnicos previos. El ítem positivos más valorados fue el ítem 6 (*utilidad*, $\bar{x} = 5,00$), con una puntuación máxima unánime, lo que indica que todos los participantes consideraron la herramienta útil para su propósito específico de crear esquemas Matrix Harris. En general, estos resultados sugieren una percepción muy positiva de la usabilidad de la aplicación entre los participantes.

A continuación se los obtenidos para los cinco ítems adicionales diseñados específicamente para evaluar aspectos relacionados con la funcionalidad de la aplicación.

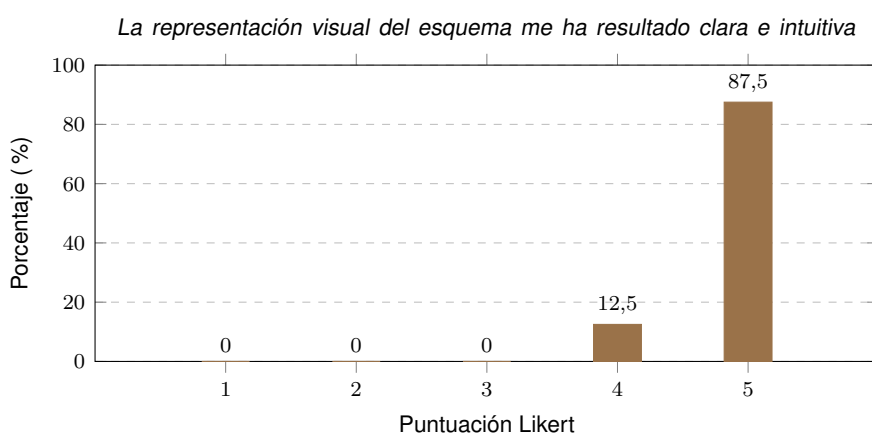


Figura E.14: Ítem adicional 1: claridad visual ($\bar{x} = 4,875$, $n = 8$).

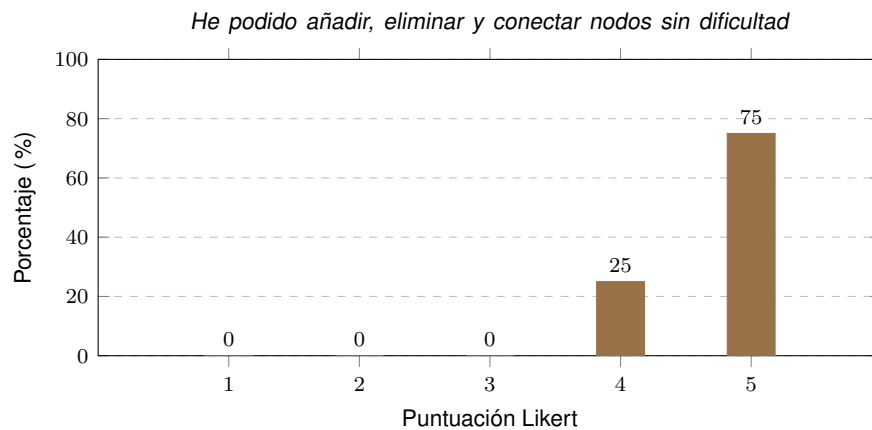


Figura E.15: Ítem adicional 2: gestión de nodos ($\bar{x} = 4,75$, $n = 8$).

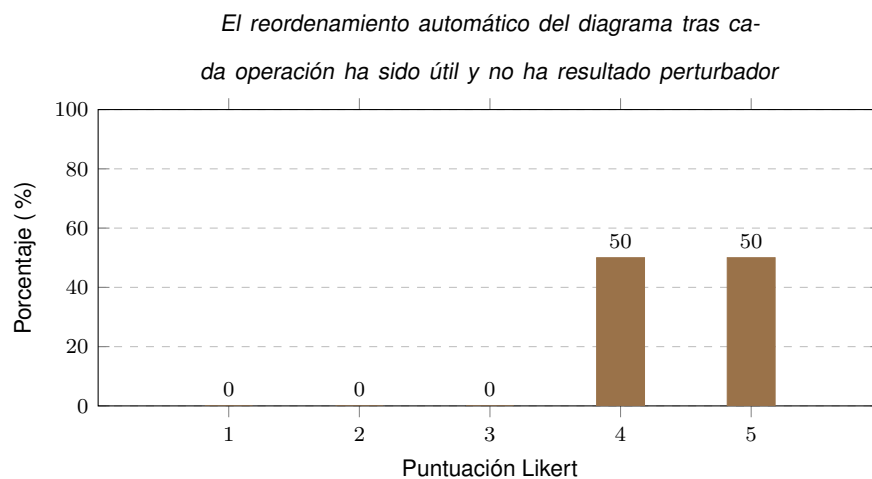


Figura E.16: Ítem adicional 3: reordenamiento automático ($\bar{x} = 4,50$, $n = 8$).

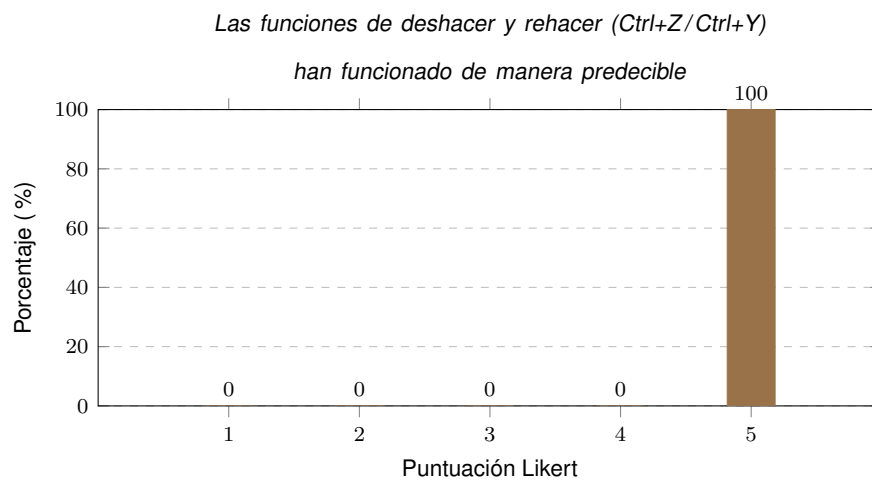


Figura E.17: Ítem adicional 4: deshacer y rehacer ($\bar{x} = 5,00$, $n = 8$).

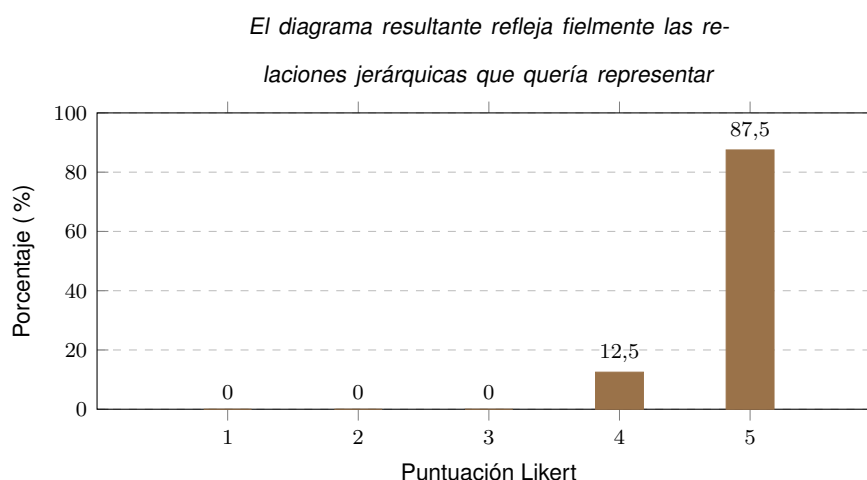


Figura E.18: Ítem adicional 5: fidelidad jerárquica ($\bar{x} = 4,875$, $n = 12$).

Los resultados de los ítems adicionales son especialmente relevantes porque evalúan directamente los objetivos técnicos del proyecto. De aquí, el más requisito más importante es el ítem EXT1 (*claridad visual*, $\bar{x} = 4,875$), pues el requisito en el que el cliente puso mayor énfasis. Dada la puntuación, se puede afirmar que la aplicación cumple con este requisito de manera satisfactoria. En general, la puntuación global obtenida para estos ítems adicionales es de **96,0 puntos**, lo que indica una valoración muy positiva de la funcionalidad específica de la aplicación, superando ampliamente el umbral de usabilidad aceptable.

Bloque 3: Preguntas abiertas

El cuestionario se cerró con dos preguntas abiertas destinadas a capturar información cualitativa complementaria a la escala SUS.

Aspecto más útil o destacable.

Se preguntó a los participantes qué aspecto de la herramienta les había resultado más útil o destacable. Las respuestas se recogen en la Tabla E.6.

Tabla E.6: Respuesta a la pregunta de aspecto más útil o destacable.

P	Respuesta
P1	La ordenación automática.
P2	Los comandos son muy intuitivos.
P3	La forma de organización y la opción de que el diagrama se ordene solo; resulta especialmente útil cuando la complejidad aumenta al añadir nodos.

Tabla E.6 (continuación)

P	Respuesta
P4	Lo más útil fue lo visual que es la herramienta: poder ver los nodos y las conexiones de forma clara ayuda a entender el esquema y a organizarlo bien. También resultó destacable la rapidez con la que se aplican los cambios y la facilidad para corregir errores.
P5	La manera en que los grafos se reordenan es increíble.
P6	La facilidad de uso y la capacidad para manejar y editar los esquemas sin problema.
P7	El manejo general es sencillo una vez comprendida la lógica de las asociaciones.
P8	La reordenación de los grafos resultó especialmente elegante, aunque no siempre produjo el resultado esperado.

Sugerencias de mejora.

Se preguntó qué mejorarían o echarían en falta. Las respuestas se recogen en la Tabla E.7.

Tabla E.7: Sugerencias de mejora

P	Respuesta
P1	Una interfaz más bonita.
P2	Que al dejar el cursor encima de un comando, sin pulsarlo, se mostrase una previsualización de cómo quedaría el diagrama.
P3	Un acceso más directo para añadir nodos por jerarquías; por ejemplo, un menú siempre visible en lugar de tener que hacer clic derecho.
P4	Quizás más colores o una interfaz más llamativa con algún tipo de animación; aunque para una aplicación orientada a la productividad, es perfecta para su fin.
P5	Un botón de ayuda con algunas claves por si el usuario se queda atascado.
P6	Mejoras en el <i>frontend</i> relacionadas con el aspecto visual; en cualquier caso, la aplicación es eficiente y coherente, que es lo necesario.
P7	Una adaptación a la nomenclatura arqueológica, dado que ese será su contexto de uso principal. Con esa adaptación, la herramienta resultaría más intuitiva para estos profesionales.
P8	A nivel técnico nada. Si acaso igual la posibilidad de editar los diferentes nodos visualmente (colores o formas) para crear tipos de cosas.

El aspecto más destacado es la sugerencia de P7, adaptar la nomenclatura y la representación gráfica a la terminología y convenciones propias de la arqueología, lo que podría mejorar la intuitividad de la herramienta para su público objetivo. Otras sugerencias, también serán tenidas en cuenta

para futuras mejoras, como la incorporación de una función de previsualización o un botón de ayuda contextual.



Universidad Autónoma
de Madrid